ESCUELA POLITÉCNICA SUPERIOR DE CÓRDOBA
Universidad de Córdoba

EPSC

TRABAJO FIN DE MÁSTER

Máster en Ingeniería Industrial

CONTROL REMOTO DE UN VEHÍCULO NO TRIPULADO AGRÍCOLA

Autor: Antonio Serrano Corral
Director: Rafael Rubén Sola Guirado

Septiembre, 2021

UNIVERSIDAD D CÓRDOBA

Dedicatoria

A mis padres, gracias a ellos he llegado hasta aquí

Agradecimientos

En primer lugar, me gustaría expresar mi agradecimiento a mi director del TFM D. Rafael Rubén Sola Guirado, quién me ha brindado la posibilidad de llevar en práctica muchas de las competencias adquirida durante mi formación como Ingeniero Industrial. También, me gustaría resaltar el interés continuo que ha tenido durante mi desarrollo de mi TFM, así como la cantidad de consejos que me ha dado para encauzar de la mejor manera posible el mismo.

También, agradecer a mis compañeros de laboratorio D. Sergio Bayano Tejero, D. Juan Pérez Moya y D. Fernando Aragón Rodríguez por solucionarme la gran cantidad de dudas diarias que me han surgido durante el desarrollo de mi TFM, tras ser contratado como ingeniero a cargo a proyecto durante cuatro meses en la Universidad de Córdoba.

Por último, agradezco la financiación recibida a través del Proyecto Innolivar, -Compra Pública de Innovación en su modalidad de Precomercial-, según lo establecido en el Convenio entre el antiguo Ministerio de Economía, Industria y Competitividad (actual Ministerio de Ciencia, Innovación) y la Universidad de Córdoba, cofinanciado en un 80% por fondos FEDER, dentro del Programa Operativo Plurirregional de España 2014-2020. También agradecemos el apoyo económico prestado por la Organización Interprofesional de la Aceituna de Mesa y el Aceite de Oliva, España https://interaceituna.com/.

Resumen

En este Trabajo Fin de Máster (TFM) ha pretendido diseñar y desarrollar un sistema de control inalámbrico para un vehículo no tripulado agrícola multipropósito perteneciente a la Universidad de Córdoba. Se trata por lo tanto de un paso intermedio entre los vehículos agrícolas manejados in situ por operario y los futuros vehículos agrícolas autónomos.

En primer lugar, se ha programado en C++ y diseñado el prototipo electrónico adecuado tanto para el emisor como para el receptor.

En segundo lugar, una vez definidos los prototipos emisor y receptor, se ha diseñado una PCB para el emisor y se ha desarrollado una placa electrónica en protoboard soldable para el receptor con el objetivo de asegurar un conexionado fiable.

En tercer lugar, se ha diseñado y fabricado mediante impresión 3d la carcasa del mando de control remoto que incorpora todo aquel hardware que permite la traslación, dirección y nivelación del vehículo, entre otras funciones.

En cuarto lugar, se ha diseñado, fabricado y programado un robot con el objetivo de simular todos los movimientos del vehículo real en el laboratorio y asegurar que todo vaya correctamente.

Por último, se han realizado varios ensayos estáticos y dinámicos en el vehículo hasta que se consiguió el control remoto adecuado del mismo.

Abstract

In this Master's Thesis (TFM), the aim was to design and develop a wireless control system for a multipurpose unmanned agricultural vehicle belonging to the University of Cordoba. It is therefore an intermediate step between agricultural vehicles operated in situ by an operator and future autonomous agricultural vehicles.

Firstly, the appropriate electronic prototype for both the transmitter and the receiver has been programmed in C++ and designed.

Secondly, once the transmitter and receiver prototypes have been defined, a PCB has been designed for the transmitter and a solderable protoboard electronic board has been designed for the receiver in order to ensure a reliable connection.

Thirdly, the remote control housing has been designed and manufactured using 3D printing, which incorporates all the hardware that allows the vehicle to be moved, steered and levelled, among other functions.

Fourthly, a robot has been designed, manufactured and programmed in order to simulate all the movements of the real vehicle in the laboratory and ensure that everything runs correctly.

Finally, several static and dynamic tests were carried out on the vehicle until adequate remote control of the vehicle was achieved.

Capítulo 1: MEMORIA

Índice de contenido de la memoria

1. Introducción 1
2. Objeto y Alcance 2
3. Antecedentes 3
3.1 Vehículos agrícolas autónomos y con sistemas de control remoto 3
3.2 SAE J3016: Niveles de Automatización en la conducción y la conducción remota 5
3.3 Comunicaciones 5
3.3.1 Tecnologías cableadas 5
3.3.1.1 Bus CAN 6
3.3.1.2 Bus SPI 9
3.3.1.3 Comunicación Asíncrona UART 10
3.3.2 Tecnologías inalámbricas 10
3.3.2.1 Radiofrecuencia 11
3.3.2.2 Bluetooth 12
3.4 Microcontroladores 13
3.4.1 Arduino 13
3.5 Placas de circuito impreso (PCB) 15
4 Técnicas de diseño y desarrollo de soluciones 15
4.1 Diseño asistido por ordenador (CAD) 15
4.2 Técnicas de prototipado rápido 16
4.3 Softwares utilizados 18
5 Disposiciones legales y normas aplicadas 18
6 Análisis de soluciones 19
6.1 Emisor 19
6.1.1 Programación y prototipado electrónico del emisor 20
6.1.1.1 Configuración del hardware 24
6.1.1.2 Tramas CAN BUS emisor 26
6.1.1.3 Programación del emisor 28
6.1.1.4 Resultados del prototipo emisor 44
6.1.2 Diseño de la PCB 45
6.1.3 Diseño CAD del mando 51
6.1.4 Impresión 3d del mando 60
6.1.5 Diseño del sistema de alimentación 62
6.1.6 Diseño de pantalla 63
6.1.6.1 Editor Nextion 64
6.1.6.2 Programación de Nextion en Arduino 66

6.1.7 Extras 66
6.1.7.1 Sensor de Batería 66
6.1.7.2 Módulo de conexión bluetooth 67
6.2 Receptor 68
6.2.1 Programación y prototipado electrónico del receptor 69
6.2.1.1 Tramas CAN BUS del receptor 69
6.2.1.2 Programación del receptor 69
6.2.1.3 Resultados del prototipo receptor 76
6.2.2 Diseño de receptor sobre protoboard soldable 76
6.3 Robot de simulación 77
7. Resultados finales 82
8. Conclusiones 86
9. Líneas futuras 86
10. Bibliografía 87

Índice de figuras

Figura 1. Movimientos principales del vehículo a controlar (Elaboración propia) 2
Figura 2. Esquema visual del objetivo del TFM. (Elaboración propia) 3
Figura 3. Tractor autónomo del fabricante Kubota (Web oficial de Kubota, 2020) 4
Figura 4. Tractores autónomos de los fabricantes CASE IH y John Deere (John Deere, 2019) 4
Figura 5. Vehículo agrícola controlado por radio del fabricante Reform (Web oficial de Reform, 2021) 4
Figura 6. Trasmisión de señales en forma de tensión diferencial (Wikipedia, 2021) 7
Figura 7. Tramas de datos CAN con formato identificador estándar de 11 bits (Wikipedia, 2021) 8
Figura 8. Controlador CAN mcp2515 de Microchip (Elaboración propia) 8
Figura 9. Señales del bus SPI que permiten la sincronización entre dispositivos y la transmisión de datos (Leens, 2009) 9
Figura 10. Formato de trama de datos UART (Wang & Song, 2011) 10
Figura 11. Clasificación de diferentes tipos de redes inalámbricas y sus tecnologías asociadas (Andreu, 2011) 11
Figura 12. Módulos inalámbricos RF. El NRF24L01 con antena zigzag a la izquierda y el NRF24L01 con amplificador y antena externa a la derecha. (Elaboración propia) 12
Figura 13. Módulo bluetooth HC-05 (ITead Studio, 2010) 13
Figura 14. Placa Arduino MEGA 2560 basada en el ATmega2560 (Web oficial de Arduino, 2021a) 14
Figura 15. Placa Arduino NANO basada en ATmega328 (Web oficial de Arduino, 2021b) 14
Figura 16. Placa de circuito impreso o PCB (Del VillarFernández, 2015) 15
Figura 17. Fases de diseño en la que interviene el CAD (Sola, 2021) 16
Figura 18. Ejemplo de carcasa para Raspberry Pi impresa mediante tecnología FDM (Cults3d, 2020) 17
Figura 19. Diagrama de Gant que representa como se ha planificado las distintas tareas necesarias para alcanzar el objetivo del TFM (Elaboración Propia) 19
Figura 20. Eje Y del joystick de 3 ejes para el avance y retroceso del vehículo. (Elaboración propia) 20
Figura 21. Eje X del joystick de 3 ejes para el giro de las ruedas direccionales. (Elaboración propia) 20
Figura 22. Eje Z del joystick de 3 ejes para la nivelación manual del vehículo. (Elaboración propia) 21
Figura 23. Inclinómetro instalado en el chasis del vehículo. (Elaboración propia) 21
Figura 24. Interruptor bipolar de tipo palanca (Electronica Caballero, 2021) 21
Figura 25. Potenciómetro corredera para variar las r.p.m del motor (Electronica Caballero, 2021) 21
Figura 26. Interruptor NC para el arranque del vehículo (Electronica Caballero, 2021) 22
Figura 27. Interruptor NA para la bocina del vehículo (Electronica Caballero, 2021) 22
Figura 28. Interruptor tipo basculante utilizado para activar el modo autónomo del vehículo (Electronica Caballero, 2021) 22
Figura 29. Interruptor basculante ON-OFF para desactivar las funciones del mando (Electronica Caballero, 2021) 23
Figura 30. Pulsador con enclavamiento para activar la parada de emergencia del vehículo (Electronica Caballero, 2021) 23

Figura 31. Selector de 3 posiciones para los distintos modos de conducción (Electronica Caballero, 2021) 23
Figura 32. Modos de giro: ruedas delanteras (izquierda), ruedas delanteras y traseras en distintos sentidos (central) y ruedas delanteras y traseras en distintos sentidos (derecha) (Elaboración propia) 24
Figura 33. Asignación de entradas analógicas y digitales en Arduino de las distintas funciones del mando inalámbrico más las que requiere el NRF24L01 (Elaboración propia) 24
Figura 34. Configuración de resistencia Pull Down (Elaboración propia) 25
Figura 35. Configuración de resistencia Pull Up (Elaboración propia) 25
Figura 36. Configuración de potenciómetros (Elaboración propia) 26
Figura 37. Configuración para el selector de 3 posiciones (Elaboración propia) 26
Figura 38. Librerías utilizadas en el código del emisor (Elaboración propia) 28
Figura 39. Creación de objeto para el NRF24L01 (Elaboración propia) 29
Figura 40. Inicialización del objeto creado para el NRF24L01 (Elaboración propia) 29
Figura 41. Canal de comunicación de escritura del NRF24L01 (Elaboración propia) 29
Figura 42. Dirección de canal para el canal de comunicación de escritura del NRF24L01 (Elaboración propia) 29
Figura 43. Función del NRF24L01 que permite el envío de datos por el canal definido (Elaboración propia) 30
Figura 44. Declaración de variables: entradas analógicas y digitales del Arduino MEGA correspondientes a los distintos componentes físicos del mando inalámbrico (Elaboración propia) 30
Figura 45. Declaración de variables: cinco primeros bytes de las tramas CAN (Elaboración propia) 30
Figura 46. Declaración de funciones correspondientes al tratamiento de variables analógicas y digitales y las funciones que permiten el envío de tramas CAN (Elaboración propia) 31
Figura 47. Declaración de variables: variables que permiten guardar el último valor de una entrada analógica o digital (Elaboración propia) 31
Figura 48. Función ReadAnalogValue: permite escalar los valores leídos de una entrada analógica y ajustar de cuanto en cuanto quiere que varíe el valor (Elaboración propia) 32
Figura 49. Función CalcularValorPasos: posibilita ajustar de cuanto en cuanto quiere que se varíe el valor escalado de la entrada analógica (Elaboración propia) 32
Figura 50. Función ReadDigitalValue indica si una entrada digital se encuentra en estado alto o en bajo (Elaboración propia) 33
Figura 51. Función LeerEnviarValorNivelacion permite leer el eje z del joystick y enviar la trama CAN correspondiente con el objetivo de conseguir la nivelación manual del vehículo (Elaboración propia) 34
Figura 52. Función LeerEnviarValorVelocidad permite leer el eje y del joystick y enviar la trama CAN correspondiente con el objetivo de conseguir el avance y retroceso del vehículo (Elaboración propia) 35
Figura 53. Primera parte de la Función LeerEnviarValorGiro que permite leer el eje x del joystick y enviar la trama CAN correspondiente con el objetivo de conseguir el giro del vehículo (Elaboración propia) 36
Figura 54. Segunda parte la Función LeerEnviarValorGiro que permite leer el eje x del joystick y enviar la trama CAN correspondiente con el objetivo de conseguir el giro del vehículo (Elaboración propia) 37

Figura 55. Función LeerEnviarValorAceleracion permite leer el potenciómetro de corredera utilizado para la aceleración del motor y enviar la trama CAN correspondiente (Elaboración propia) 38
Figura 56. Primera parte de la función LeerEnviarValorLuces permite leer el modo de luces que se encuentra el emisor y enviar la trama CAN correspondiente (Elaboración propia) 39
Figura 57. Segunda parte de la función LeerEnviarValorLuces permite leer el modo de luces que se encuentra el emisor y enviar la trama CAN correspondiente (Elaboración propia) 40
Figura 58. Función LeerEnviarValorBocina permite leer el estado de la entrada digital asociada la bocina y enviar la trama CAN correspondiente al receptor (Elaboración propia) 41
Figura 59. Función LeerEnviarValorSetaEmergencia permite leer el estado de la entrada digital correspondiente a la seta de emergencia y también envía la trama CAN correspondiente al receptor (Elaboración propia) 42
Figura 60. Función EnviarEstoyVivo permite enviar una trama constante cada 100 ms con el objetivo de que el receptor sepa si el emisor sigue vivo (Elaboración propia) 42
Figura 61. Función LeeEnviarValores es la encargada de gestionar cuál de las funciones anteriores son prioritarias (salvo la función EnviarEstoyVivo) y bajo qué condiciones se enviarán (Elaboración propia) 43
Figura 62. Programación de la función set up () del emisor(Elaboración propia) 44
Figura 63. Programación de la función loop() del emisor (Elaboración propia) 44
Figura 64. Aspecto físico del mando inalámbrico prototipo (Elaboración propia) 45
Figura 65. Interior del mando inalámbrico prototipo donde se aprecia que los componentes electrónicos están pinchados en protoboard (Elaboración propia) 45
Figura 66. Schematic Library del Arduino Mega Parte A (Elaboración propia) 46
Figura 67. Schematic Library del Arduino Mega Parte B (Elaboración propia) 46
Figura 68. PCB library del Arduino Mega (Elaboración propia) 47
Figura 69. Footprint del Arduino Mega 2560 (Web oficial de Arduino, 2021c) 47
Figura 70. Schematic Library del NRF24L01 (Elaboración propia) 47
Figura 71. PCB library del NRF24L01 (Elaboración propia) 47
Figura 72. CAD del NRF24L01 asociado al PCB Library del mismo módulo (Elaboración propia) 48
Figura 73. Configuración de un puente de diodos (Vázquez, 2019) 48
Figura 74. Routeado manual de pistas de la PCB objetivo en 2 niveles distintos (Elaboración propia) 49
Figura 75. Modelo 3d de la PCB objetivo (Elaboración propia) 50
Figura 76. PCB sin componentes soldados tras su fabricación (Elaboración propia) 50
Figura 77. Labores de soldadura de los distintos componentes electrónicos a la PCB (Elaboración propia) 51
Figura 78. Sustitución de cables pinchados en la protoboard por la PCB diseñad (Elaboración propia) 51
Figura 79. Mandos comerciales elegidos como inspiración para el diseño CAD del mando objetivo (Saioacontrol, 2021) 52
Figura 80. Diseño Conceptual del mando inalámbrico (Elaboración propia) 52
Figura 81. Diseño 3d preliminar del mando inalámbrico basado en el boceto o Diseño Conceptual (Elaboración propia) 53
Figura 82. Croquis de la base de la parte de abajo del mando (Elaboración propia) 56
Figura 83. Diseño de tetones, plataforma y huecos para atornillar y ajustar el Arduino MEGA, Holders de baterias de litio, BMS y Módulo Bluetooth a la parte de abajo de la carcasa del mando (Elaboración propia) 56

Figura 84. Diseño de huecos para alojar el interruptor de encendido/apagado del mando, el usb-b de montaje sobre panel y Jack de carga (Elaboración propia)..... 57
Figura 85. Acoples para la unión y guías de encaje con la parte de arriba del mando (Elaboración propia)..... 57
Figura 86. Fragmento inclinado de la parte de arriba de la carcasa del mando (Elaboración propia)..... 58
Figura 87. Fragmento plano de la parte de arriba de la carcasa del mando (Elaboración propia) 58
Figura 88. Acoples para la unión, guías de encaje con la parte inferior del mando y agarres para la banda que sustenta el mando (Elaboración propia) 59
Figura 89. Primer ensamblaje de la parte inferior del mando (Elaboración propia) 59
Figura 90. Segundo ensamblaje de la parte inferior del mando (Elaboración propia) 60
Figura 91. Mando inalámbrico totalmente ensamblado (Elaboración propia)..... 60
Figura 92. Esquema del procedimiento a seguir para imprimir una pieza desde un diseño CAD (Elaboración propia)..... 61
Figura 93. Fases de impresión de la parte inferior del mando (Elaboración propia)..... 61
Figura 94. Fases de impresión del fragmento plano de la parte de arriba del mando (Elaboración propia)..... 61
Figura 95. Fases de impresión del fragmento inclinado de la parte arriba del mando (Elaboración propia)..... 62
Figura 96. Ensamblaje de los distintos componentes en las partes impresas en 3d del mando (Elaboración propia)..... 62
Figura 97. Esquema de conexionado del sistema de alimentación del mando (Elaboración propia)..... 63
Figura 98. Identificación de los componentes del sistema de alimentación dentro de la carcasa inferior del mando (Elaboración propia)..... 63
Figura 99. Page 0 y page 1 de la interfaz de la pantalla (Elaboración propia) 64
Figura 100. Configuración de la page 2 de la interfaz de la pantalla utilizando los componentes de la barra de herramientas adecuados para componente del mando (Elaboración propia).... 65
Figura 101. Page 3 de la interfaz de la pantalla (Elaboración propia)..... 65
Figura 102. Esquema del sensor de batería (Elaboración propia) 67
Figura 103. Nivel del batería del mando al 91% (Elaboración propia)..... 67
Figura 104. Incorporación del módulo bluetooth HC-05 con el objetivo de incorporar TIC´s en el vehículo (Elaboración propia) 68
Figura 105. Utilización de App Bluetooth Electronics para la monitorización en tiempo real de algunas variables de funciones del vehículo (Elaboración propia) 68
Figura 106. Librerías utilizadas en el código del receptor (Elaboración propia)..... 70
Figura 107. Canal de comunicación de lectura del NRF24L01 (Elaboración propia) 70
Figura 108. Función que permite escuchar los canales definidos como lectura (Elaboración propia)..... 70
Figura 109. Funciones que permiten comprobar si existen datos en el canal de escritura y leerlos en caso afirmativo (Elaboración propia) 71
Figura 110. Creación del objeto para el mcp2515 (Elaboración propia) 71
Figura 111. Configuración de la velocidad de transmisión del bus CAN (Elaboración propia) ... 71
Figura 112. Configuración del modo de trabajo del mcp2515 (Elaboración propia)..... 71
Figura 113. Configuración de la id, longitud e información de una trama CAN (Elaboración propia)..... 72

Figura 114. Función LeerDatosRF que permite comprobar si existen datos disponibles enviados por el emisor y analizar si se trata de un tipo de dato interno o no (Elaboración propia) 73
Figura 115. Función SendCAN permite enviar los datos provenientes del emisor al bus CAN del vehículo (Elaboración propia) 74
Figura 116. Función EnviarErrores, comprueba si se está danto algún tipo de error y consecuentemente envía una respuesta al bus CAN de si existe algún tipo de error presente (Elaboración propia) 75
Figura 117. Función SplitULong, permite convertir una variable tipo unsigned long de 32 bits a cuatro variables de un solo byte (Elaboración propia) 75
Figura 118. Programación de la función set up() y loop() del receptor (Elaboración propia) 76
Figura 119. Componentes electrónicos pinchados en protoboard del prototipo receptor (Elaboración propia) 76
Figura 120. Labores de soldadura de los componentes electrónicos del receptor en protoboard soldable (Elaboración propia) 77
Figura 121. Placa soldada del receptor incorporada en caja para carril DIN (Elaboración propia) 77
Figura 122. El objetivo de la simulación con robot en laboratorio: aseguramiento de la llegada de tramas CAN BUS (Elaboración propia) 78
Figura 123. Componentes electrónicos del robot pinchados sobre protoboard (Elaboración propia) 79
Figura 124. Diseño de placa en protoboard soldable, ordenación de cableado y disposición optimizada de los componentes electrónicos (Elaboración propia) 80
Figura 125. Diseño CAD del caparazón inferior y superior donde se alojan los componentes electrónicos del robot, ejes de dirección y elementos de unión de los servos de nivelación y dirección (Elaboración propia) 80
Figura 126. Ensamblaje de las piezas impresas en 3D junto con todos los componentes electrónicos del robot (Elaboración propia) 81
Figura 127. Pruebas en laboratorio de la lectura correcta de trama CAN BUS y la correspondiente acción programada del robot ante la llegada de la trama (Elaboración propia) 81
Figura 128. Esquema de conexión de los actuadores y electroválvulas de control del vehículo (Elaboración propia) 82
Figura 129. Esquema general de instalación del sistema de control en el vehículo real (Elaboración propia) 83
Figura 130. Incorporación del receptor al bus CAN del vehículo dentro del cuadro eléctrico del mismo (Elaboración propia) 83
Figura 131. Ensayo de nivelación automática del vehículo real en estático (Elaboración propia) 84
Figura 132. Primer ensayo de nivelación manual y automática, dirección, traslación en dinámico (Elaboración propia) 84
Figura 133. Segundo ensayo en dinámico donde se comprobó que los módulos de radiofrecuencia tenían poco alcance (Elaboración propia) 85
Figura 134. Sustitución en emisor y receptor del módulo NRF24l01 por el módulo NRF24l01+PA+LNA de mayor alcance (Elaboración propia) 85
Figura 135. Tercer ensayo dinámico donde se comprobó el largo alcance entre el emisor y el receptor (Elaboración propia) 86
Figura 136. Navegación por SLAM para la conducción autónoma de robots (OpenSourceTeam, 2018) 87

Índice de tablas

Tabla 1. Funciones presentes y futuras que se deben incorparar en el mando (Elaboración propia) 2
Tabla 2. Especificaciones técnicas del mcp2515 (Elaboración propia) 8
Tabla 3. Bandas de frecuencia del espectro radioeléctrico (Elaboración propia)..... 11
Tabla 4. Especificaciones técnicas del NRF24L01 (Elaboración propia)..... 12
Tabla 5. Especificaciones técnicas del HC-05 (Elaboración propia) 13
Tabla 6. Especificaciones técnicas del Arduino MEGA 2560 (Elaboración propia) 14
Tabla 7. Especificaciones técnicas del Arduino NANO (Elaboración propia) 14
Tabla 8. Definición de las Tramas CAN BUS que serán enviadas del emisor al receptor (Elaboración propia)..... 27
Tabla 9. Diseños CAD de componentes del mando inalámbrico descargados en GRABCAD (Elaboración propia)..... 53
Tabla 10. Diseños CAD de componentes del mando inalámbrico que se han tenido que crear (Elaboración propia)..... 55
Tabla 11. Definición de las Tramas CAN BUS que recibirá el receptor (Elaboración propia)..... 69
Tabla 12. Componentes electrónicos del robot simulador (Elaboración propia) 78

Acrónimos

BMS: Battery Management System

CAD: Diseño Asistido por Ordenador

CAN: Controller Area Network

CJP: Color Jet Printing

FDM: Fused Deposition Modeling o Modelado por Deposición Fundida

IA: Inteligencia Artificial

IoT: Internet of Things

LIDAR: Light Detection and Ranging

PCB: Printed Circuit Board

PTH: Plated Through Holes

ROS: Robot Operating System

SAE: Society of Automotive Engineers

SLA: Stereolithography o Estereolitografía

SLAM: Simultaneous Localization and Mapping

SMD: Surface Mounted Device

SPI: Serial Peripheral Interface

TIC: Tecnología de Información y Comunicación

UART: Universal Asynchronous Reciever Transmitter

WAN: Wireless Personal Area Network

1. Introducción

La robotización es uno de los principales motores de esta cuarta revolución industrial, conocida como **Industria 4.0**. Los robots se utilizan ampliamente en diferentes industrias y se aplican de diversas formas, lo que hace que sus consecuencias económicas y sociales sean de gran alcance (Cséfalvay & Gkotsis, 2020).

La robotización en la industria manufactura (sector de alimentación, sector del automóvil etc) se lleva implantando desde los años 70 y se caracteriza por ser una industria más madura en cuanto a robotización que en la robótica de servicios, donde se incluyen los robots agrícolas. El diseño de estos robots destinados a la agricultura es complejo, ya que intervienen sobre un entorno dinámico como es el campo (Barrientos & del Cerro, 2016). A pesar de todo esto, hoy en día existen varias soluciones comerciales y experimentales de robots agrícolas que realizan tareas específicas de cultivos, recolección, y agricultura de precisión (Misset, 2021).

En este sentido, actualmente está apareciendo el concepto de **Agricultura 5.0**, que se trata de un concepto que integra avances en la agricultura relacionados con la utilización de vehículos robots que utilizan equipos que involucran operaciones no tripuladas y sistemas autónomos de apoyo a las decisiones mediante la incorporación de técnicas de inteligencia artificial (Saiz-Rubio & Rovira-Más, 2020).

El **Ingeniero Industrial** juega un papel fundamental en la robotización de este tipo de industrias, ya que posee los conocimientos y competencias necesarias para el diseño mecatrónico y programación de algoritmos necesarios para el correcto funcionamiento de estos robots.

La Universidad de Córdoba dispone de un vehículo agrícola multipropósito que tiene como misión ser robotizado en estos próximos años. El vehículo se ha diseñado con un sistema hidrostático propulsado por un motor de gasoil. Dispone de dos motores rueda en la parte delantera que permiten la traslación hacia adelante y hacia atrás del vehículo, cinco cilindros hidráulicos, donde uno de ellos permite la dirección delantera del vehículo y los cuatro restantes la nivelación del mismo. También, incorpora un inclinómetro que va a permitir la nivelación automática de este.

En este sentido, el problema ingenieril que se plantea para este proyecto es controlar remotamente los movimientos principales este vehículo (Figura 1). Para ello, se diseñará un mando a distancia que permitirá estos movimientos principales del vehículo de forma remota y será un paso previo al futuro objetivo de desarrollar un vehículo completamente autónomo. Además, se le debe dotar al mando de otras funciones que un futuro se incorporará en el vehículo (Tabla 1).



Figura 1. Movimientos principales del vehículo a controlar (Elaboración propia)

Tabla 1. Funciones presentes y futuras que se deben incorporar en el mando (Elaboración propia)

Funciones presentes que debe realizar el vehículo mediante mando remoto	Dirección en ambos sentidos del eje delantero
	Traslación hacia adelante y hacia atrás
	Nivelación manual
	Nivelación automática mediante inclinómetro
	Paro de emergencia
	Configuración de distintos modos de conducción (lento, normal y rápido)
Funciones futuras que realizará el vehículo mediante mando remoto	Activación de Bocina
	Activación de luces
	Activación de bomba auxiliar para apero
	Arranque del vehículo
	Aceleración del motor propulsor
	Activación de modo autónomo
	Configuración de distintos modos de giro (solo dos ruedas, cuatro ruedas, modo cangrejo)

2. Objeto y Alcance

El objeto de este Trabajo Fin de Máster es desarrollar un sistema de control remoto para un vehículo agrícola multipropósito experimental con un sistema hidrostático perteneciente a la Universidad de Córdoba. Se pretende desarrollar un mando inalámbrico emisor y enviar ordenes vía radiofrecuencia a un receptor instalado en el propio vehículo. Cabe destacar, que este proyecto es un paso previo al futuro objetivo de desarrollar un vehículo completamente autónomo.

Para el envío de datos desde el emisor de las distintas entradas analógicas y digitales del mando, se utilizará un Arduino MEGA 2560 (se ha elegido este modelo ya que se trata de un microcontrolador potente con una amplia comunidad de seguidores y trabajos útiles hechos) junto con un módulo de radiofrecuencia, concretamente el NRF24L01 configurándolo como emisor. Por otro lado, para la recepción de estos, se utilizará un receptor que incorporará un Arduino NANO junto al NRF24L01 configurado como receptor y módulo CAN mcp2515 que

permitirá cruzar datos con el PLC del vehículo mediante protocolo CAN BUS. El PLC ya instalado, es el encargado del control de las distintas electroválvulas que incorpora tanto el distribuidor hidráulico encargado de repartir el aceite a los cilindros que intervienen en la nivelación y dirección del vehículo y la bomba hidráulica encargada de mover los motores ruedas que permiten el avance y el retroceso de este (Figura 2).

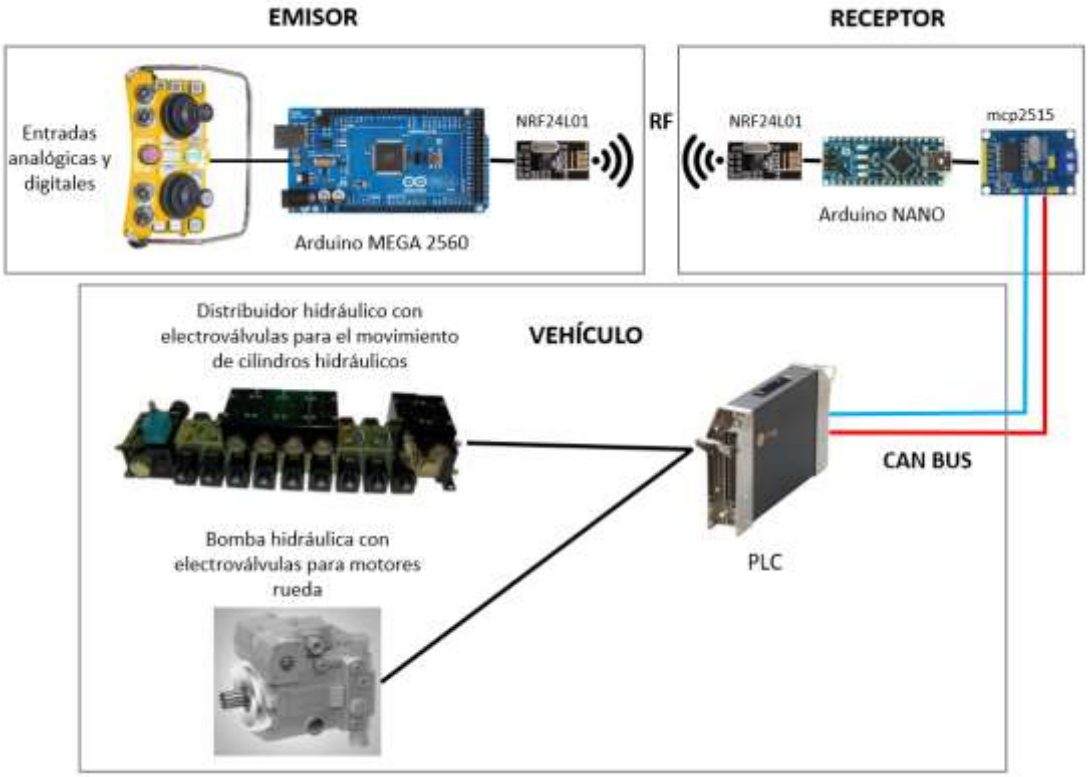


Figura 2. Esquema visual del objetivo del TFM. (Elaboración propia)

El alcance de este proyecto es desarrollar todo el diseño completo del mando inalámbrico emisor, es decir, desde el diseño y desarrollo electrónico (programación del microcontrolador, el diseño y fabricación de la PCB, diseño del sistema de alimentación, programación de pantalla y extras incluidos) hasta el diseño CAD de la carcasa del mando e impresión 3d de la misma. Del mismo modo, se hará de una forma similar lo mismo para el receptor instalado en el vehículo y se implantará la comunicación de este con el PLC que gestiona el vehículo.

También se diseñará un robot de simulación que permita simular el vehículo real en el laboratorio permitiendo poder experimentar sobre este y comprobar si realmente llegan adecuadamente las tramas CAN objetivo.

Por último, una vez ensayadas todas las funcionalidades del mando en el laboratorio se pasará a hacer los ensayos sobre el vehículo real ubicado en el área de mecanización del departamento de ingeniería rural de la ETSIAM.

3. Antecedentes

3.1 Vehículos agrícolas autónomos y con sistemas de control remoto

A lo largo de las últimas décadas, se ha ido investigando en la incorporación de vehículos autónomos en la agricultura. Centros de investigación como el instituto de investigación del Ministerio de Agricultura, Silvicultura y Pesca (MAFF) ha desarrollado un tractor forrajero

autónomo (Torii, 2000) y conocidos fabricantes como Kubota Co. Ltd ha desarrollado un tractor autónomo completamente eléctrico que incorpora paneles solares en la parte superior del carenado para la generación de energía independiente que sirva como complemento a la demanda por el motor eléctrico propulsor (Figura 3). Otros fabricantes como CASE IH y John Deere también han apostado por desarrollar este tipo de vehículos. En el caso del tractor de CASE IH incorpora un motor propulsor de gasoil y en el caso del tractor John Deere un motor propulsor eléctrico (Figura 4). Todos estos vehículos irán dotados de una gran cantidad de sensores (GPS RTK, LIDAR) y cámaras encargadas de generar información que será procesada por las distintas computadoras incluidas en el tractor. Estos necesitarán de la implantación de algoritmos que permitan el reconocimiento de escenas, la planificación de las rutas y el control del vehículo (Kato, 2015).



Figura 3. Tractor autónomo del fabricante Kubota (Web oficial de Kubota, 2020)



Figura 4. Tractores autónomos de los fabricantes CASE IH y John Deere (John Deere, 2019)

Por otro lado, empresas como Valtra y Elisa presentaron en la sala de exposiciones de exposiciones de Elisa 5G en Helsinki, Finlandia un tractor controlado remotamente mediante conexión 5G y una cámara 360 ° montada en el techo del tractor (Infoagro, 2020). También la empresa Reform ha comercializado el Metron P48 RC que se trata del primer vehículo agrícola controlado por radio con una unidad híbrida real (Figura 5). El automatizar las máquinas agrícolas, supone aumentar la productividad, mejorar la seguridad y reducir los costos de muchas operaciones agrícolas (Stentz et al., 2002).



Figura 5. Vehículo agrícola controlado por radio del fabricante Reform (Web oficial de Reform, 2021)

3.2 SAE J3016: Niveles de Automatización en la conducción y la conducción remota

Según la *Society of Automotive Engineers* (SAE) en su estándar J3016 define seis niveles de automatización de conducción. Estos niveles van desde un nivel 0 donde existe una ausencia de automatización en la conducción, hasta un nivel 5 donde la automatización de la conducción es completa. A los niveles 1 y 2 se les denomina "Sistemas de apoyo o asistencia al conductor" mientras que por otro lado a los niveles del 3 al 5 se les nombra como "Sistemas de conducción automatizada". Cada uno de estos sistemas tiene una serie de funciones que se van a ir describiendo una a una a continuación (SAE International, 2021).

Nivel 0 SAE: estas funciones se limitan a proporcionar avisos y asistencia momentánea, algunos ejemplos serían, el frenado automático o de emergencia, aviso de ángulo muerto o aviso de salida del carril.

Nivel 1 SAE: en este caso, proporcionan apoyo a la dirección o al freno/aceleración del conductor. El centrado de carril o el control de crucero adaptativo podrían ser algunos ejemplos.

Nivel 2 SAE: el nivel 2 SAE es prácticamente igual al nivel 1 SAE con la particularidad que puede simultanear variar funciones a la vez, es decir, que podría tener activado el centrado de carril y el control de crucero adaptativo al mismo tiempo.

Nivel 3 SAE: estas funciones pueden conducir el vehículo bajo condiciones limitadas y no funcionarán a menos que se cumplan todas las condiciones requeridas. Será necesario la presencia del conductor.

Nivel 4 SAE: es un nivel muy parecido al nivel 3 SAE, pero con la particularidad de que los pedales y volante pueden estar o no instalados. Los taxis sin conductor entrarían dentro de este nivel SAE.

Nivel 5 SAE: esta función puede conducir el vehículo en todas las direcciones y en todas las condiciones.

En la revisión de abril del estándar SAE J3016 también se incluyen conceptos como el asistente remoto y el conductor remoto. Un conductor remoto, por ejemplo, podría ser alguien que supervisa de que el vehículo estaciona de forma remota y asegura que este responda bien ante los obstáculos y los peatones. Sin embargo, un asistente remoto podría ser alguien que ofrece una colaboración remota a un vehículo a un vehículo equipado con sistema de condición automatizado (Blanco, 2021).

3.3 Comunicaciones

En este apartado 3.3, se van a tratar las distintas tecnologías de transmisión de información que se han utilizado en este proyecto. Se realizado una clasificación de las mismas en función de si precisa o prescinde de cables para la comunicación.

3.3.1 Tecnologías cableadas

Se estudiarán tres tecnologías de comunicación por cable muy utilizas en la transmisión de datos, la comunicación por bus CAN que necesitará un par de cables trenzados, la comunicación por bus SPI que precisará mínimo cuatro cables para la comunicación (MISO, MOSI, reloj y Select) y la comunicación UART hará uso de dos cables para establecer la comunicación (Rx y Tx).

3.3.1.1 Bus CAN

CAN (*Controller Area Network*) es un protocolo de comunicaciones definido por las normas ISO 11898 muy utilizado para el intercambio de información entre microcontroladores. Entre las ventajas más importantes que ofrece:

- Simplificación del cableado al utilizar una red multiplexada a diferencia de las conexiones punto a punto.
- Alta inmunidad a interferencias (transmisión de señales en forma de tensión diferencial)
- Pueden conectarse al bus CAN hasta 110 nodos (Requena, 2017).

El bus CAN funciona como un bus de transmisión, es decir, un nodo prioritario emite un mensaje con identificador encapsulado en forma de trama y todos los nodos acoplados al bus escuchan el mensaje, pero solo lo recibirán aquellos nodos que les interese.

Del modelo OSI de interconexión de sistemas abiertos, los estándares vinculados al bus CAN especifican la capa física y la capa de enlace de datos. La capa física define los niveles eléctricos, las diferentes tipologías del bus (velocidades máximas, cableado y terminaciones) y conectores utilizados, mientras que la capa de enlace de datos define tanto el método de acceso al medio (arbitraje) como la tipología de tramas existentes para el envío de mensajes.

Existen dos tipologías de bus CAN definidas por los estándares, el CAN de alta velocidad, que según la ISO 11898-2 permite alcanzar velocidades del bus de hasta 1 Mbit/s. Utiliza un par de cables trenzados en cuyas terminaciones se le cola una resistencia de un valor de 120 Ω con el objetivo de que el bus obtenga los niveles de tensión correctos y elimine los reflejos de la señal al final del bus (KVASER, 2021). El otro tipo, según la ISO 11898-3, es el llamado CAN de baja velocidad. Este puede alcanzar velocidades de transmisión más bajas, máximo 125 kbit/s pero tiene la peculiaridad de que es tolerante a fallos, permitiendo incluso trasmitir información si un cable de bus está cortado, en corto con tierra etc. Además, los estándares no definen un tipo de conector específico para el bus CAN, sino que cada aplicación puede tener un conector específico.

Normalmente, se utiliza un par de cables trenzados para la transmisión de señales. Las dos señales que aparecen en el bus son CAN_H o nivel de tensión alto y CAN_L o nivel de tensión bajo y pueden encontrarse en dos estados distintos:

- Estado recesivo: el nivel de tensión de los cables del bus es el mismo. Se corresponde con el estado inactivo del bus (un bit recesivo equivale al valor lógico 1).
- Estado dominante: existe al menos una diferencia de tensión entre CAN_H y CAN_L de 1.5 V. Se corresponde con el estado activo del bus (un bit dominante equivale al valor lógico 0) (Figura 6).

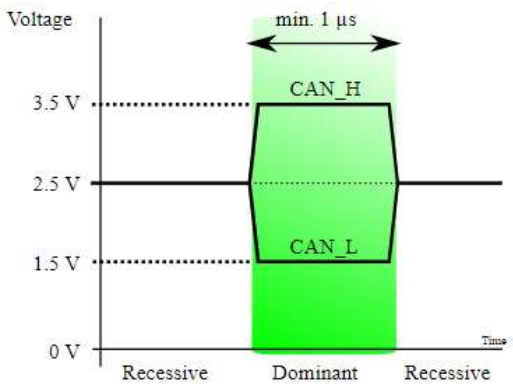


Figura 6. Trasmisión de señales en forma de tensión diferencial (Wikipedia, 2021)

La forma de transmitir las señales en forma de tensión diferencial permite una mayor protección frente a interferencias que en la transmisión de señales en forma de tensiones absolutas.

El arbitraje de mensajes, es decir, el proceso en el que dos o más controladores CAN se ponen de acuerdo sobre quién utilizará el bus, es de gran importancia para el ancho de banda realmente disponible para la transmisión de datos. De esta forma, se usa una regla que dice que las tramas CAN con identificadores más bajos prevalecerán sobre las que tengan un identificador más alto. Esto permitirá asignarle identificadores más bajos a funciones de mayor peso, como podría ser la activación de la parada de emergencia.

Para el protocolo CAN se definen cuatro tipos de tramas distintas, la trama de datos (se hablará de esta en profundidad a continuación), la trama remota (tiene como propósito solicitar la transmisión de una determinada trama), la trama de error (se transmite cuando un nodo detecta un mensaje erróneo) y la trama de sobrecarga (se transmite cuando un nodo está muy ocupado.

La trama de datos es la trama que se utiliza para enviar un mensaje con determinada información desde un nodo emisor a un nodo receptor. Consta principalmente de las siguientes partes:

- El campo de arbitraje. Dentro de este se definen los identificadores de las tramas (números más bajos se corresponden con un mayor peso). Existen dos tipos de formatos con distintas longitudes de identificador, el formato estándar con identificador de 11 bits y el formato extendido con identificador de 29 bits.
- El campo de datos. Es el campo que realmente lleva la información útil para el nodo objetivo. El tamaño máximo es de 8 bytes. Previamente se destinan 4 bits para el control de la longitud de datos.
- El campo CRC. Estos 15 bits de la trama son utilizados para la detección de errores. Permitiendo verificar que la transmisión de datos ha sido correcta.

Además, existen otros bits en la trama destinados al inicio y fin de la misma, acuse de recibo etc (Figura 7).

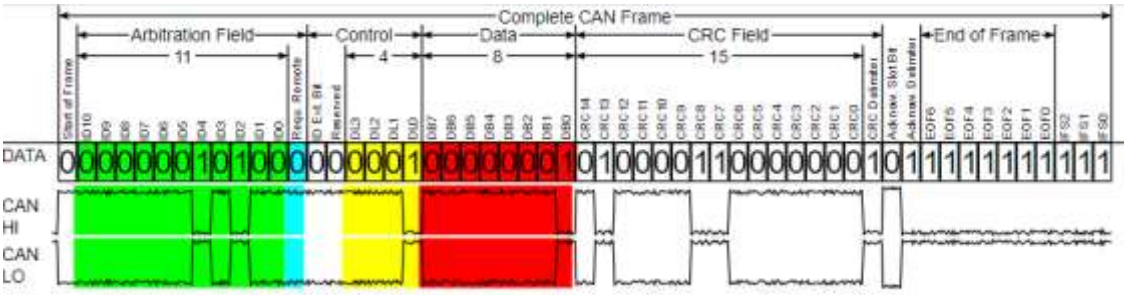


Figura 7. Tramas de datos CAN con formato identificador estándar de 11 bits (Wikipedia, 2021)

Para finalizar, se debe tener en cuenta que todos los nodos de un bus CAN deben estar sincronizados (frecuencias de reloj adecuadas) con el objetivo de poder recibir y leer de forma adecuada los mensajes que se transmitan por el bus.

Mcp 2515

El mcp 2515 (Figura 8) es un controlador autónomo CAN capaz de transmitir y recibir tramas de datos con identificador estándar (CAN 2.0A) y extendido (CAN 2.0B), además de tramas remotas. Se comunica con microcontroladores como el Arduino mediante bus SPI (Microchip, 2021).

-



Figura 8. Controlador CAN mcp2515 de Microchip (Elaboración propia)

Como se observa en la figura 8, además de los pines necesarios para la comunicación por bus SPI y los pines de CAN_H y CAN_L, existen los pines Jumpers que al unirlos con un cable simulamos el terminal con una resistencia de 120 Ω propio de un bus CAN.

En la tabla 2, se recogen las especificaciones técnicas más importantes del mcp2515.

Tabla 2. Especificaciones técnicas del mcp2515 (Elaboración propia)

Parámetros	Valores con unidades
Velocidades de Operación	5 V
Consumo máximo de Corriente	500 uA
Banda de frecuencia	2400 a 2525 MHz
Velocidad máxima	10 Mbit/s (a 12 metros)
Longitud máxima de alcance	1200 m (a 100 kbit/s)

3.3.1.2 Bus SPI

El bus SPI (*Serial Peripheral Interface*) es un estándar de comunicaciones muy utilizado para la comunicación de dispositivos electrónicos, normalmente entre microcontroladores y sus periféricos (Oudjida et al., 2010).

La arquitectura que acompaña al bus SPI es de tipo maestro-esclavo (master-slave). El maestro permite iniciar la comunicación, enviar y/o recibir datos de uno o diferentes dispositivos esclavos. Funciones que no se permiten entre dispositivos esclavos.

Otra de peculiaridades de la comunicación del bus SPI, es que es Full Duplex, es decir, el maestro es capaz de enviar y recibir datos al mismo tiempo (Toporski, 2016). Esto es posible, porque existen dos líneas de comunicaciones diferentes, una de maestro a esclavo y viceversa.

El bus SPI es un bus síncrono, esto significa que garantiza una buena transmisión de datos, ya que el maestro ofrece una señal de reloj que permite que todos los esclavos se encuentren sincronizados.

Las señales del bus SPI que permiten la sincronización y transmisión de datos son (Figura 9)(Leens, 2009):

SCK (Clock): señal de reloj que permite la sincronización de dispositivos y es proporcionada por el maestro.

MOSI (Master out, Slave in): los datos son transmitidos del maestro hacia el esclavo.

MISO (Master in, Slave out): en este caso, los datos son trasmitidos del esclavo hacia el maestro.

SS (Slave Select): señal que permite seleccionar el esclavo que va a intercambiar información con el maestro.

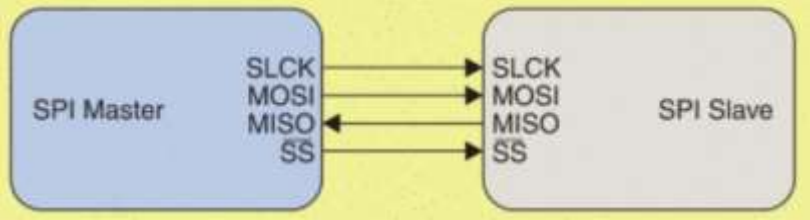


Figura 9. Señales del bus SPI que permiten la sincronización entre dispositivos y la transmisión de datos (Leens, 2009)

Entre las ventajas más importantes del bus SPI se encuentran:

- Comunicación Full Duplex, por lo que una mayor velocidad de transmisión de datos (con respecto a I2C por ejemplo)
- Permite enviar cualquier tipo de secuencia de bits (sin límites de tamaño).
- No existe la necesidad de un transceptor como en el caso del bus CAN.

Por otro lado, los inconvenientes más comunes son:

- Se requiere un cable Select por cada esclavo conectado, además de las señales SCK, MOSI y MISO
- Solo para comunicación entre dispositivos cercanos
- No dispone de la capacidad para asegurar si se ha recibido correctamente el mensaje o si se ha provocado un cierto error en el envió.

3.3.1.3 Comunicación Asíncrona UART

UART (*Universal Asynchronous Receiver Transmitter*) se emplea frecuentemente en sistemas integrados. Además es un protocolo de transmisión de datos en serie muy usado, debido a que puede realizar la transmisión de datos de velocidad baja o del sistema de información (Dannenberg, 2006).

En la figura 10, se muestra el formato de datos que utiliza el protocolo de transmisión y recepción de datos UART. Por lo común incorporan bit de inicio, bits de datos, bit de paridad, bit de parada y estado inactivo. La transmisión de datos comienza con el bit de inicio, que se trata de un cero lógico con un ancho de tiempo igual a un ciclo de reloj de velocidad en baudios. Sucesivo a este bit de inicio le acompañan los bits de datos, que pueden ser un array de 5 a 8 bits. Ahora es el bit menos significativo (LSB) el que comienza enviándose (Wang & Song, 2011). Seguidamente se le agrega un bit de paridad utilizado en las comunicaciones serie con el objetivo de comprobar si el dispositivo remoto ha recibido adecuadamente los datos (IBM, 2021). El último bit (o bits) del formato de datos es el bit de parada que se trata de una señal lógica de "1". El bit de inicio y de parada son los encargados de adoptar este formato de trama con el objetivo de lograr la sincronización de caracteres. UART dispone de un registro interno de configuración (Shao-wei, 2006).

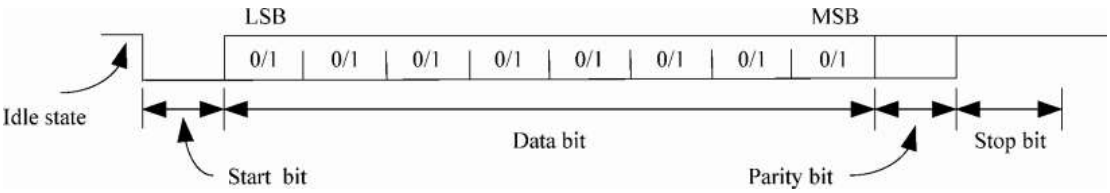


Figura 10. Formato de trama de datos UART (Wang & Song, 2011)

3.3.2 Tecnologías inalámbricas

Antes de hablar de las tecnologías inalámbricas que existen, se debe de conocer qué es una red inalámbrica wireless y los distintos tipos que existen.

Una red inalámbrica es una red con ausencia de cables que se comunica por medios no guiados a través de ondas electromagnéticas. Las antenas son los elementos encargados de la transmisión y recepción de estas ondas.

Dependiendo del alcance, existen varios tipos de redes (Figura 11) (Andreu, 2011):

WPAN (*Wireless Personal Area Network*): se trata de una red inalámbrica que permite interconectar periféricos a distancias cercanas y a velocidades de transmisión inferiores al megabit por segundo. Las tecnologías inalámbricas más conocidas que utilizan redes WPAN son Bluetooth, Bluetooth ULP, HomeRF2, ZigBee, VWB, NFC, RFID, INSTEON, IrDA etc.

WLAN (*Wireless Local Area Network*): se trata de la red inalámbrica que se suele instalar en un edifico teniendo un rango óptimo de 100 m y hasta un máximo de 450m. La tecnología inalámbrica más conocida que implementa este tipo de red WLAN es la tecnología Wifi.

WMAN (*Wireless Metropolitan Area Network*): este tipo de redes se instalan a nivel de barrio, urbanización o municipio pequeño. Las tecnologías que utilizan este tipo de redes son WiMax, WiBro, son conocidas como tecnologías Inalámbricas de Banda Ancha. Estas tecnologías pueden alcanzar una distancia de hasta 54 km en condiciones favorables y 22 km con condiciones climatológicas adversas.

WWAN (*Wireless Wide Area Network*): se tratan de redes inalámbricas globales fundamentadas en tecnologías que utilizan conexión satélite (vSAT), soluciones vía telefonía móvil (3G, 4G, el actual 5G).

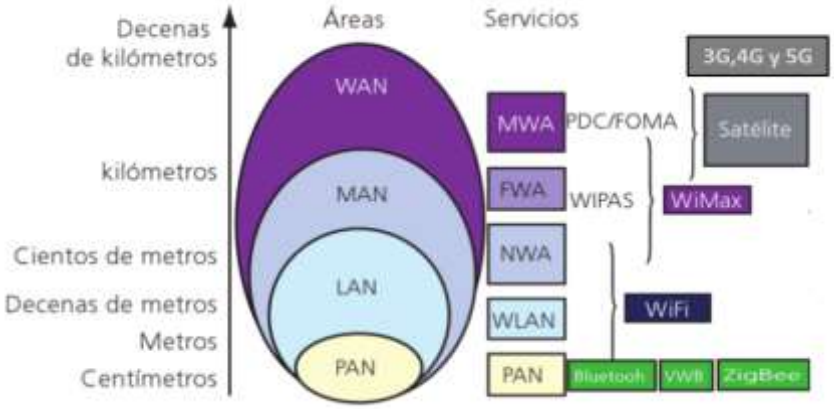


Figura 11. Clasificación de diferentes tipos de redes inalámbricas y sus tecnologías asociadas (Andreu, 2011)

3.3.2.1 Radiofrecuencia

El espectro de radiofrecuencia se trata de uno de los fragmentos menos energéticos del espectro electromagnético, es decir, se refiere a aquellas señales transmitidas desde los 3 Hz (incluso menor) hasta los 3000 GHz (Esopo, 2021).

El espectro radioeléctrico, a su vez, se ramifica en trece bandas de frecuencias. En la tabla 3, se ha realizado una clasificación de bandas de menor a mayor energía (Subtel, 2005).

Tabla 3. Bandas de frecuencia del espectro radioeléctrico (Elaboración propia)

Símbolos (en inglés)	**Nombre (en inglés)**	**Gama de frecuencias (incluido límite inferior y superior)**
SELF	Super Extremely Low Frequency	< 3 Hz
ELF	Extremely Low Frequency	3 a 30 Hz
SLF	Super Low Frequency	30-300 Hz
ULF	Ultra Low Frequency	300-3000 Hz
VLF	Very low frequency	3 a 30 kHz
LF	Low frequency	30 a 300 kHz
MF	Medium frequency	300 a 3000 kHz
HF	High frequency	3 a 30 MHz
VHF	Very high frequency	30 a 300 MHz
UHF	**Ultra high frequency**	**300 a 3000 MHz**
SHF	Super high frequency	3 a 30 GHz
EHF	Extremely high frequency	30 a 300 GHz
SHEF	Super Extremely High Frequency	300 a 3000 GHz

NRF24L01

Es un transceptor de radio (transmisor y receptor) de un solo chip para la banda ISM mundial de 2,4 a 2,5 GHz (se trata de una UHF). Permite mediante interfaz SPI la programación de la potencia de salida , los canales de frecuencia y la configuración del protocolo que incorpora (NORDIC SEMICONDUCTOR, 2021).

En la tabla 4, se recogen una serie de especificaciones técnicas del módulo inalámbrico.

Tabla 4. Especificaciones técnicas del NRF24L01 (Elaboración propia)

Parámetros	Valores con unidades
Velocidades de transmisión	250 Kbps, 1Mbps y 2 Mbps
Tensión de alimentación	1.9 a 3.6 V
Banda de frecuencia	2400 a 2525 MHz
Canales de operación	125
Consumo de corriente NRF24L01	12.3 mA
Consumo de corriente NRF24L01+PA+LNA	115 mA

Hay dos versiones del NRF24L01, el que incorpora una antena en forma de zigzag (Figura 12 izquierda) permitiendo un alcance máximo de 20-30 metros o el módulo (NRF24L01 +PA+LNA) que monta un amplificador con una antena externa , que permite alcanzar un máximo de 700-1000 metros (Figura 12 derecha).

Este módulo inalámbrico es compatible con Arduino y con cualquier microcontrolador con comunicación SPI.



Figura 12. Módulos inalámbricos RF. El NRF24L01 con antena zigzag a la izquierda y el NRF24L01 con amplificador y antena externa a la derecha. (Elaboración propia)

3.3.2.2 Bluetooth

Bluetooth es una especificación industrial para WPAN que permite la transmisión de datos por radiofrecuencia entre dispositivos. Cada dispositivo debe contar con chip transceptor que permite la transmisión de datos en la frecuencia de 2.4 GHz (Castellano, 2012).

Existen unos estándares diseñados para la compatibilidad entre dispositivos. Cada estándar que aparece nuevo cubre las versiones anteriores. El último estándar de compatibilidad es el **Bluetooth v5.2** del 6 de enero del 2020.

HC-05

Se trata de un transceptor que permite enviar y recibir datos en la banda de frecuencia de 2,4 GHz utilizando tecnología bluetooth (Figura 13). Este es compatible con Arduino o con cualquier microcontrolador que disponga de puerto UART (ITead Studio, 2010).

Esta versión permite poder configurarlo como maestro o como esclavo mediante comandos AT por el puerto serie. En la tabla 5, se encuentran las especificaciones técnicas más importantes.

Tabla 5. Especificaciones técnicas del HC-05 (Elaboración propia)

Parámetros	Valores con unidades
Velocidades de transmisión	1200 bps hasta 1.3 Mbps
Voltaje de operación	3.6- 6V
Alcance máximo	10 m
Consumo de corriente	50 mA



Figura 13. Módulo bluetooth HC-05 (ITead Studio, 2010)

3.4 Microcontroladores

Un microcontrolador es un circuito integrado programable que permite ejecutar órdenes previamente registradas en su memoria. Se compone de ciertos bloques funcionales encargados de realizar tareas específicas. Las tres unidades funcionales principales del microcontrolador son: memoria, periféricos E/S y unidad de procesamiento (Montañez Ruiz, 2017).

3.4.1 Arduino

Se trata de una plataforma electrónica de código abierto cuyo hardware y software no se caracteriza por tener una gran dificultad a la hora de su utilización. Las placas Arduino permiten leer entradas analógicas y digitales y convertirlo en salidas mediante la programación de un conjunto de instrucciones en el microcontrolador de la placa. Para llevar a cabo esta programación se utiliza el lenguaje de programación Arduino basado en Wiring (Web oficial de Arduino, 2021d).

Arduino MEGA 2560

Se trata de una placa microcontrolador Arduino de 8 bits basada en el ATmega2560 (Figura 14). Dispone de 54 pines E/S digitales donde 15 de ellos se permiten utilizar como salidas PWM, 16 entradas analógicas, 4 puertos serie UART hardware, conexión USB y alimentación, un oscilador de cristal de 16 MHz y un encabezado ICSP. También cuenta con comunicación SPI e I2C.



Figura 14. Placa Arduino MEGA 2560 basada en el ATmega2560 (Web oficial de Arduino, 2021a)

En la tabla 6 se recogen otras especificaciones técnicas del Arduino MEGA 2560 (Web oficial de Arduino, 2021a).

Tabla 6. Especificaciones técnicas del Arduino MEGA 2560 (Elaboración propia)

Tensión de funcionamiento	5 V
Voltaje de entrada (recomendado)	7-12 V
Voltaje de entrada (límite)	6-20 V
Corriente CC por pin de E/S	20 mA
Corriente CC por pin de 3.3 V	50 mA
LED_BUILTIN	13
Largo	101,53 mm
Ancho	53,3 mm

Arduino NANO

Al igual que el Arduino MEGA 2560, el Arduino NANO (Figura 15) se trata de una placa microcontrolador de 8 bits, pero en este caso basada en el ATmega328. Dispone de 22 pines E/S digitales donde 6 de ellos se permiten utilizar como salidas PWM, 8 entradas analógicas, 1 puerto serie UART hardware, conexión USB, y un oscilador de cristal de 8 MHz. También cuenta con comunicación SPI e I2C.

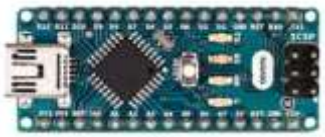


Figura 15. Placa Arduino NANO basada en ATmega328 (Web oficial de Arduino, 2021b)

En la tabla 7, se muestran otras características técnicas del Arduino NANO (Web oficial de Arduino, 2021b).

Tabla 7. Especificaciones técnicas del Arduino NANO (Elaboración propia)

Tensión de funcionamiento	5 V
Voltaje de entrada (recomendado)	7-12 V
Corriente CC por pin de E/S	40 mA
El consumo de energía	19 mA
Largo	45 mm
Ancho	18 mm

3.5 Placas de circuito impreso (PCB)

Se tratan de superficies compuestas por pistas de un material conductor sobre una base no conductora (Figura 16). A estas pistas se le sueldan los diversos componentes electrónicos (resistencias, condensadores…) necesarios para la aplicación objetivo de la PCB. La utilización de pistas permite ahorrar una gran cantidad de cableado y permite un conexionado entre componentes mucho más sólido. El material de las pistas normalmente es cobre y la base se fabrica en resinas de vidrio reforzada (Del VillarFernández, 2015).

Para intentar conseguir un tamaño de placa reducido y las pistas no se corten entre sí a la hora de diseñarlas, la PCB se diseña en distintas capas o layers. Se hace uso de las vías para pasar de una capa a otra. También, los softwares de diseño de PCB te permiten el grabado de esta para identificar los distintos componentes electrónicos.

La mayoría de los componentes electrónicos que se montan en la PCB, pueden ser componentes que se montan introduciéndolos en el agujero, también conocidos como *Plated Through Holes* (PTH) o de montaje superficial o *Surface Mounted Device* (SMD) sin necesidad de ninguna perforación. Una gran diferencia entre ambos es que los PTH pueden interconectar varias capas y sin embargo los SMD solo intervienen en la capa superficial.



Figura 16. Placa de circuito impreso o PCB (Del VillarFernández, 2015)

4 Técnicas de diseño y desarrollo de soluciones

En este apartado, se va a describir en que consiste el diseño CAD y qué se conoce por técnicas de prototipado rápido y cuáles son las más utilizadas hoy en día, así como los softwares utilizados para el desarrollo del TFM.

4.1 Diseño asistido por ordenador (CAD)

El diseño asistido por ordenador se basa en el uso de softwares que permiten la creación, el análisis y la documentación de representaciones gráficas en 2D y 3D de objetos físicos. Es decir, se trata de una alternativa optimizada y mejorada a los borradores manuales (Siemens Digital Industries Software, 2021).

El CAD interviene durante todo el proceso ingenieril de un producto, es decir, pasa desde la primera fase de diseño conceptual hasta la determinación del proceso de fabricación pasando previamente por otras fases como el análisis de ensamblajes (Figura 17). Si nos referimos a un producto, antes de su diseño CAD se deben de tener en cuenta otras fases como la fase de generación de ideas o identificación de oportunidades, la evaluación y selección de esas ideas, así como la especificación del nuevo producto (Sola, 2021).

Entre las ventajas más importantes del CAD se encuentran:

- Agilización del proceso de diseño y mejora de la visualización del mismo
- Reducción de errores gracias a una mayor precisión
- Obtención de una documentación sólida que incluye geometrías, lista de materiales y dimensiones
- Permite reutilizar otros diseños realizados previamente

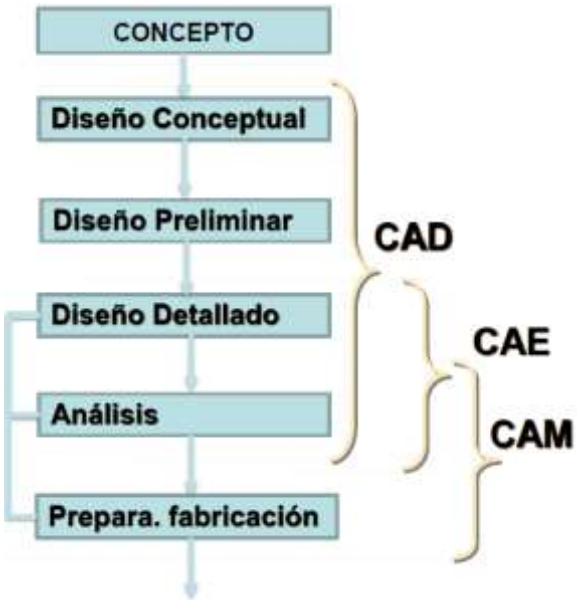


Figura 17. Fases de diseño en la que interviene el CAD (Sola, 2021)

4.2 Técnicas de prototipado rápido

Las técnicas de prototipado rápido abarcan una cantidad de procesos basados en agregar material de forma selectiva mediante la superposición de capas sucesivas permitiendo crear un objeto a partir de un diseño CAD. Una de las diferencias más importantes respecto a otros procesos de fabricación es que aquí se añade material en vez de eliminarlo.

Existen una gran variedad de procesos de fabricación aditiva y distintas técnicas asociadas a ellos (Guerrero & Rodríguez, 2020):

- **Extrusión del material**
  Mediante un orificio es dispensado el material selectivamente. La tecnología más utilizada asociada a este proceso de fabricación es el **Modelado por Deposición Fundida (FDM)** (Figura 18). Se trata de inyectar un filamento de un termoplástico mediante un cabezal extrusor en distintas capas a alta precisión. Los materiales más usados para este tipo de técnica son el poliácido láctico (PLA), el poliacrilonitrilo butadieno estireno (ABS) o el policarbonato (PC) entre otros muchos.



Figura 18. Ejemplo de carcasa para Raspberry Pi impresa mediante tecnología FDM (Cults3d, 2020)

- **Fotopolimerización en tanque**
  Consiste en curar selectivamente un fotopolímero líquido mediante la polimerización activa por luz. Una tecnología muy usada es la **Estereolitografía (SLA)** que consiste en curar una resina fotosensible inmersa en una cubeta mediante una luz ultravioleta.
- **Fusión de lecho de polvo**
  Se trata de fundir selectivamente un lecho en polvo mediante energía térmica.
  Para este proceso la tecnología más usada es el **Sinterizado por Láser Selectivo (SLS)** que consiste en fundir un material base de polvo con un láser.
- **Laminación de hojas**
  Se basa en unir diferentes láminas de material hasta formar el objeto deseado. La tecnología asociada a este proceso es la Fabricación por **Laminado de Capas (LOM)**. En esta técnica se van pegado distintas capas de papel con adhesivo y posteriormente se le da forma al objeto haciendo uso de una cuchilla o láser.
- **Inyección del material**
  El objeto se conforma mediante gotas de material depositadas. Entre las tecnologías más destacadas se encuentran el **Polyjet, MJM** etc.
- **Inyección del aglutinante**
  Un adhesivo líquido se deposita de forma selectiva con el objetivo de unir partículas de polvo. La técnica que utiliza este proceso se le conoce como ***Color Jet Printing*** **(CJP)**. Esta técnica consta de un dispositivo con una o múltiples boquillas y un depósito de polvo.

Todas estas técnicas comparten una característica común de requerir un mínimo número de fases para el proceso de fabricación, es decir, desde el que el diseñador desarrolla la idea hasta que se obtiene el producto acabado (Calzado Jimenez et al., 2013):

- Desarrollo de la idea concepto
- Diseño CAD del modelo mediante software
- Se genera un archivo. stl del modelo CAD para que la impresora 3d pueda entender esa información geométrica.
- Disposición correcta del modelo, configuración de impresora y material en el software adecuado para la impresión y generación de código de CN por parte del equipo de fabricación aditiva.
- Impresión física del objeto deseado
- Eliminación de soportes y limpieza de pieza

- Postprocesado con el objetivo de un mejor acabado

4.3 Softwares utilizados

A continuación, se explican breventemente los softwares utilizados para el desarrollo TFM:

PlatformIO en Microsoft Visual Studio Code (VSCode)

Es una herramienta profesional multiplataforma de arquitectura cruzada, marco simple para ingenieros de sistemas integrados y para desarrolladores de software que escriben aplicaciones para productos integrados. En definitiva, se trata de un entorno de desarrollo pensado para aplicaciones de *Internet of Things* (IoT).

Se puede instalar como una extensión nativa de PlatformIO IDE en el editor Microsoft VSCode.

Altium Designer

Se trata de un paquete software desarrollado por la empresa australiana Altium Limited enfocado en la automatización de diseño electrónico y placas de circuito impreso (PCB).

Solidworks

Es un software de diseño asistido por computadora (CAD) que te permite crear piezas, ensamblajes de piezas y planos de estas. Además, dispone de complementos de cálculo de elementos finitos (Simulation), de simulación de movimiento (SOLIDWORKS MOTION), de renderizado para generar imágenes fotorrealistas (PhotoView360) etc.

Ultimaker Cura

Es un software de impresión 3d que permite a partir transformar un archivo stl de un diseño CAD en un archivo Gcode con las instrucciones necesarias para que las entienda una impresora 3D.

5 Disposiciones legales y normas aplicadas

- **Real Decreto 1644/2008, de 10 de octubre**. Se establecen las normas para la comercialización y puesta en servicio de las máquinas. Concretamente es importante tener en cuenta el **ANEXO I:** Requisitos esenciales de seguridad y de salud, relativos al diseño y la fabricación de máquinas. Tras la lectura de este Real Decreto, se han tenido en cuenta una serie de requisitos de seguridad (incorporación de seta de emergencia) y de ergonomía a la hora del diseño del mando emisor.
- **Norma ISO 11898.** Estándares que definen el protocolo CAN, cómo ocurre la comunicación, cómo se configura el cableado y cómo se construyen los mensajes, entre otras cosas. Ha sido necesario revisar esta norma para entender correctamente el protocolo CAN, protocolo que será nombrada multitud de ocasiones en este Proyecto.
- **Orden IET/1311/2013, de 9 de julio.** Se aprueba el Reglamento de uso del dominio público radioeléctrico por radioaficionados. Se ha indagado en esta Orden, ya que la comunicación radio entre el emisor y el receptor se realiza en la frecuencia de 2,4 GHz.
- **UNE-EN ISO/ASTM 52910:2020.** Fabricación aditiva. Diseño. Requisitos, directrices y recomendaciones. Se ha seguido esta norma para adquirir recomendaciones técnicas tanto para el diseño como la impresión del mando emisor como del robot de simulación

- **Norma IPC 2221B.** Norma genérica sobre diseño de circuito impreso. Ha sido útil para entender cuál es el proceso de diseño de la placa de circuito impreso del emisor, así como recomendaciones relacionadas con el diseño de la misma.

6 Análisis de soluciones

En este apartado 6, se va a abordar las distintas soluciones de diseño adoptadas tanto para el emisor como para el receptor del mando. Además, se ha diseñado un robot para simular las distintas funciones del mando emisor en el laboratorio. Una vez que se ha diseñado y desarrollado el mando inalámbrico en el laboratorio, se probarán el control remoto en el vehículo real ubicado en el taller de mecanización del departamento de ingeniería rural de la ETSIAM.

En la figura 19, se recoge un **diagrama de Gant** que representa como se han planificado las distintas tareas necesarias para alcanzar el objetivo del TFM.

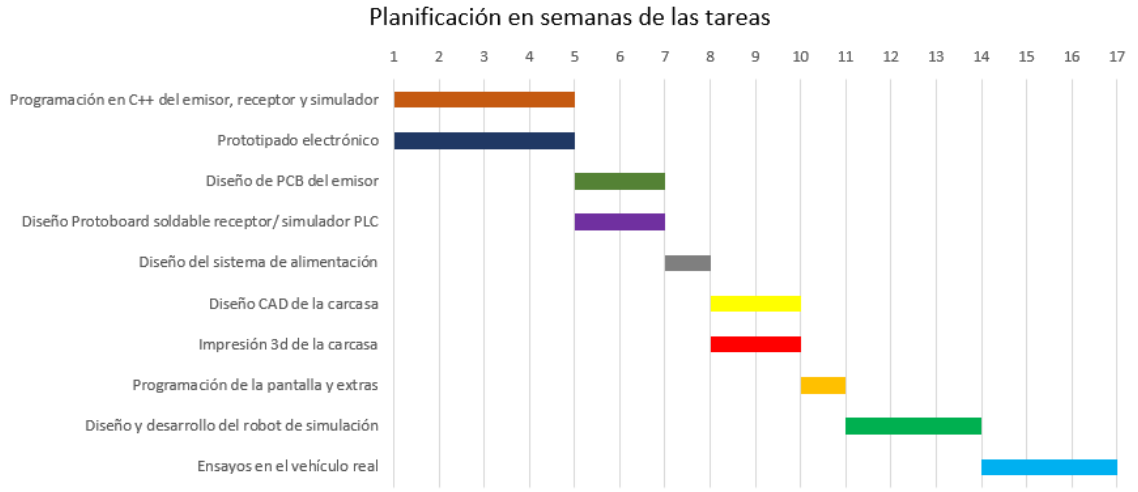


Figura 19. Diagrama de Gant que representa como se ha planificado las distintas tareas necesarias para alcanzar el objetivo del TFM (Elaboración Propia)

6.1 Emisor

El objetivo del emisor es enviar paquetes de datos de una forma particular al receptor por vía radiofrecuencia. Este paquete de datos se ajusta al modelo de trama que se usa en el protocolo Bus CAN con el objetivo de tener estos datos tratados para cuando se tenga en enviar por el bus CAN.

Además, se ha elegido la radiofrecuencia como tecnología inalámbrica candidata frente a otras tecnologías inalámbricas como el wifi o el bluetooth porque permite una mayor distancia de comunicación entre el emisor y el receptor, ya que el módulo de radiofrecuencia NRF24L01 +PA +LNA (con antena) permite llegar en condiciones favorables hasta 1 kilómetro de distancia a diferencia del módulo Wifi para Arduino que permite un máximo de 200 metros y en el caso del Bluetooth unos 40 metros.

Las tareas objetivo vinculadas con el emisor de la planificación en semanas presentada anteriormente son las que se muestran a continuación:

- Programación en C++ del emisor y prototipado electrónico

- Diseño de la PCB del emisor
- Diseño CAD de la carcasa del mando
- Impresión 3d de la carcasa del mando
- Diseño del sistema de alimentación del emisor
- Diseño de la pantalla HMI

6.1.1 Programación y prototipado electrónico del emisor

En este apartado, se abordará detalladamente la programación en C++ y el conexionado hardware de las distintas entradas analógicas y digitales que constituyen la parte física del mando inalámbrico (pulsadores, interruptores, potenciómetros y selectores).

Para conocer el número exacto de componentes físicos que va a contar el mando se deben conocer las distintas funciones que dispondrá el mismo. Las **funciones** son las siguientes:

- **Avance y retroceso:** permite la traslación del vehículo en ambos sentidos. Se utiliza el eje Y de un joystick de 3 ejes (Figura 20). Se trata de un potenciómetro de 5KΩ y se programa en el microcontrolador como una entrada analógica.



Figura 20. Eje Y del joystick de 3 ejes para el avance y retroceso del vehículo. (Elaboración propia)

- **Giro de ruedas**: permiten el giro izquierdo y derecho de las ruedas direccionales. En este caso se utiliza el eje X del mismo joystick de 3 ejes (Figura 21). Al igual que para el eje Y, se trata de un potenciómetro de 5 kΩ y se programa en el microcontrolador como otra entrada analógica.



Figura 21. Eje X del joystick de 3 ejes para el giro de las ruedas direccionales. (Elaboración propia)

- **Nivelación Manual**: mediante el eje Z del joystick de 3 ejes (Figura 22) se posibilita la nivelación manual del vehículo permitiendo ajustar a deseo del operario el ángulo de inclinación del chasis del vehículo con respecto al terreno. De la misma forma que para los otros casos, al tratarse de un potenciómetro de 5kΩ se programa como una entrada analógica.



Figura 22. Eje Z del joystick de 3 ejes para la nivelación manual del vehículo. (Elaboración propia)

- **Nivelación Automática**: el objetivo de activar este interruptor es conseguir que el chasis del vehículo se nivele automáticamente a medida que varíe el terreno. Esto lo consigue mediante el uso de un inclinómetro instalado en el chasis del vehículo (Figura 23). Como se observa en la figura 24 el interruptor bipolar que se utiliza es de tipo palanca y se configura en el microcontrolador como una entrada digital.



Figura 23. Inclinómetro instalado en el chasis del vehículo. (Elaboración propia)



Figura 24. Interruptor bipolar de tipo palanca (Electronica Caballero, 2021)

- **Aceleración del motor**: el objetivo de este potenciómetro corredera (Figura 25) es controlar las r.p.m del motor con la finalidad de aumentar o disminuir las mismas cuando sea necesario. Se reserva también una entrada analógica del microcontrolador para este potenciómetro.



Figura 25. Potenciómetro corredera para variar las r.p.m del motor (Electronica Caballero, 2021)

- **Arranque de vehículo:** mediante el uso de un pulsador normalmente cerrado (Figura 26) se permite arrancar el vehículo desde el mando inalámbrico activando el motor de arranque del vehículo. Se destina un pin digital del microcontrolador para este interruptor.



-

Figura 26. Interruptor NC para el arranque del vehículo (Electronica Caballero, 2021)

- **Activación de bocina**: por medio de un interruptor normalmente abierto (Figura 27) se activa la bocina del vehículo y se configura como otra entrada digital en el Arduino.



Figura 27. Interruptor NA para la bocina del vehículo (Electronica Caballero, 2021)

- **Manual/Autónomo**: se ha instalado un interruptor tipo basculante (Figura 28) que permite activar el modo autónomo del vehículo cuando el usuario lo deseé. Cuando se active este modo, se desactivan todas las funciones del mando salvo la seta de emergencia. También este interruptor se configura como una entrada digital en el Arduino.



Figura 28. Interruptor tipo basculante utilizado para activar el modo autónomo del vehículo (Electronica Caballero, 2021)

- **Desactivación de las funciones del mando**: al igual que el caso anterior, se ha utilizado un interruptor tipo basculante (Figura 29) que permite desactivar todas las funciones del mando (interruptores, pulsadores, potenciómetros) salvo la seta de emergencia. Se reserva un pin digital en el microcontrolador para esta función.



Figura 29. Interruptor basculante ON-OFF para desactivar las funciones del mando (Electronica Caballero, 2021)

- **Seta de emergencia**: se ha utilizado un pulsador con enclavamiento para la parada de emergencia del vehículo (Figura 30). Este tiene como función anular las demás funcionalidades del mando. Este pulsador se configura en el Arduino como una entrada digital.



Figura 30. Pulsador con enclavamiento para activar la parada de emergencia del vehículo (Electronica Caballero, 2021)

- **Modos de conducción**: la función de este selector de tres posiciones (Figura 31) es tener a disposición tres modos de conducción distintos, lento, medio y rápido. Para evitar ocupar tres entradas digitales del microcontrolador se ha utilizado una sola entrada analógica. Más adelante en la explicación del código de programación se profundizará detalladamente como se ha configurado esta entrada analógica.



Figura 31. Selector de 3 posiciones para los distintos modos de conducción (Electronica Caballero, 2021)

- **Modos de giro**: al igual que en los modos de conducción, se utiliza un selector de tres posiciones para el giro de las distintas ruedas del vehículo. Existen tres modos, uno donde solo giren las ruedas delanteras (Figura 32 Izquierda), otro para que giren las ruedas delanteras y traseras en distintos sentidos y poder así efectuar un giro más cerrado (Figura 32 Central) y un último modo que permite girar las ruedas delanteras y traseras en el mismo sentido (Figura 32 Derecha) con el objetivo de poder superar un obstáculo sin hacer demasiadas maniobras. Se reserva una entrada analógica para estas funciones.

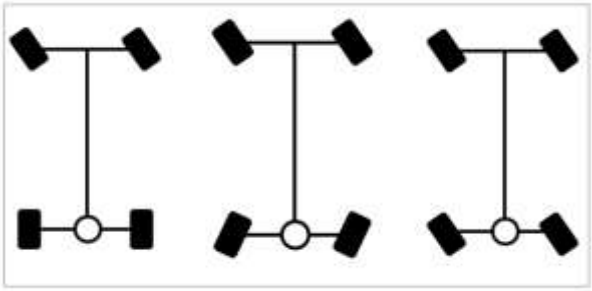


Figura 32. Modos de giro: ruedas delanteras (izquierda), ruedas delanteras y traseras en distintos sentidos (central) y ruedas delanteras y traseras en distintos sentidos (derecha) (Elaboración propia)

- **Encendido de luces**: también se utiliza un selector de tres posiciones para el encendido de luces. En principio, se destina una posición para el apagado de luces, otra para las luces delanteras y una última posición para el encendido de luces delanteras y traseras del vehículo. Se necesita de otra entra analógica para poder configurar estas funciones.
- **Bomba auxiliar:** mediante la incorporación de un potenciómetro se controla de forma progresiva la bomba auxiliar instalada en el vehículo. Esta bomba servirá para suministrar presión y caudal al apero auxiliar que se coloque en el vehículo. De este modo que es necesario reservar una entrada analógica en Arduino para esta función.

En la figura 33, se muestra cómo se han distribuido en los pines del Arduino las distintas entradas analógicas y digitales correspondientes a las distintas funciones comentadas anteriormente. También se aprecian los pines ocupados por el módulo de radiofrecuencia NRF24L01.

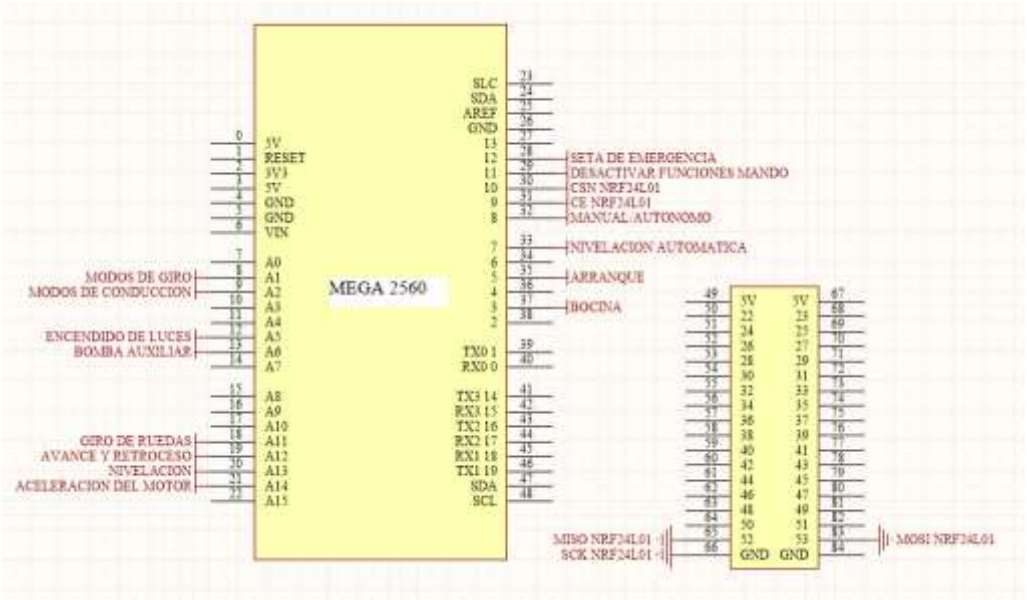


Figura 33. Asignación de entradas analógicas y digitales en Arduino de las distintas funciones del mando inalámbrico más las que requiere el NRF24L01 (Elaboración propia)

6.1.1.1 Configuración del hardware

En este apartado, se van a ir explicando mediante esquemas como se han ido conectando los diferentes componentes físicos del mando inalámbrico a la protoboard y al Arduino.

Para los interruptores y pulsadores que se han usado para las funciones bocina, arranque, nivelación automática, desactivación de las funciones del mando y modo autónomo se ha elegido una configuración de resistencia **PULL DOWN**. En esta configuración, cuando el circuito

está en reposo como se aprecia en la figura 34 (el interruptor o pulsador abierto), la caída de tensión en la resistencia es 0V (LOW), en cambio sí cerramos el interruptor o apretamos el pulsador, dejará pasar la corriente y existirá una diferencia de potencial de 5V (HIGH)(Hernández, 2019).

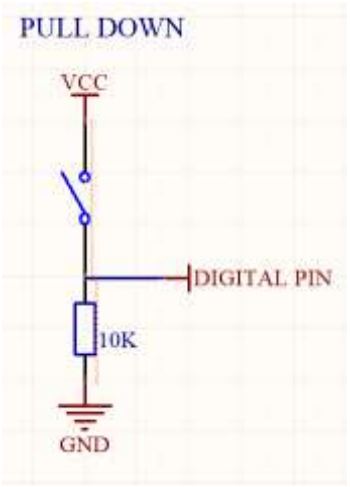


Figura 34. Configuración de resistencia Pull Down (Elaboración propia)

Por otro lado, para la seta de emergencia, se ha adoptado una configuración de resistencia **PULL UP**. Cuando el circuito está en reposo como se muestra en la figura 35 (pulsador con enclavamiento sin apretar), la caída de tensión es de 5V(HIGH), en cambio cuando apretamos el pulsador se deriva toda la corriente a masa y la caída de tensión es 0V (LOW)(Hernández, 2019).

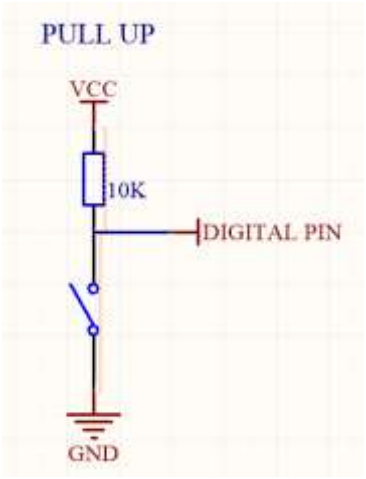


Figura 35. Configuración de resistencia Pull Up (Elaboración propia)

A la hora de la programación de estas entradas digitales, se tendrá en cuenta que tipo de configuración de resistencia se ha asignado y si el componente es normalmente abierto o cerrado.

Para los potenciómetros utilizados en los diferentes ejes del joystick, el acelerador del motor y la bomba auxiliar se conectan acorde al esquema que se muestra en la figura 36. Se le ha insertado un condensador de 100pF para estabilizar la señal y evitar ruidos.

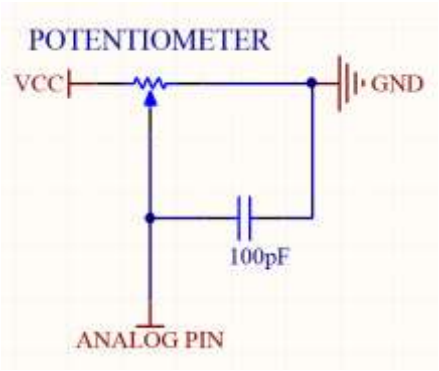


Figura 36. Configuración de potenciómetros (Elaboración propia)

Para los selectores de tres posiciones utilizados para el encendido de luces, modos de conducción y modos de giro se adopta la configuración de la figura 37. El objetivo de esta configuración es utilizar rangos de valores entre 0 y 1023 de una entrada analógica del Arduino en lugar de utilizar tres pines digitales diferentes del mismo.

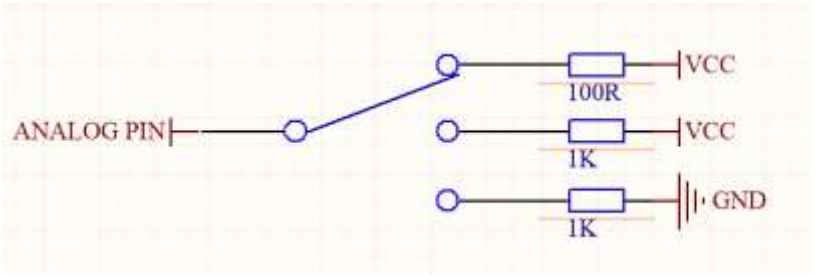


Figura 37. Configuración para el selector de 3 posiciones (Elaboración propia)

6.1.1.2 Tramas CAN BUS emisor

Uno de los requisitos primordiales que se exigía en este Trabajo Fin de Máster (TFM) era que los datos que recibiera el PLC que gestiona el vehículo se ajustaran al protocolo Bus CAN, ya que este dispone de salidas Bus CAN. Además, como se ha mencionado en los antecedentes del proyecto, se trata de un protocolo muy adecuado para aplicaciones automotrices, en este caso para maquinaria agrícola debido a su robustez y su sencillez de cableado.

En la tabla 8, se han definido las tramas de datos que se envían desde el emisor al receptor de acuerdo con el protocolo Bus CAN. Pero antes de definir las mismas, se deben tener en cuenta las siguientes **consideraciones**:

1- El avance de las ruedas se indica en porcentaje positivo (avance) o negativo (retroceso). Se usa un único byte para cada rueda, siendo este byte con signo (valores máximos del byte 127/-127). Por lo tanto, cualquier valor superior a 100 se considera como 100 (o -100 en caso negativo).
2- El giro de las ruedas se indica en porcentaje positivo (giro hacia la derecha) o negativo (giro hacia la izquierda). Se usa un único byte para cada rueda, siendo este byte con signo (valores máximos del byte 127/-127). Por lo tanto, cualquier valor superior a 100 se considera como 100 (o -100 en caso negativo).
3- La nivelación ruedas se indica en porcentaje positivo (el cilindro se extiende) o negativo (el cilindro se contrae). Se usa un único byte para cada rueda, siendo este byte con signo (valores máximos del byte 127/-127). Por lo tanto, cualquier valor superior a 100 se considera como 100 (o -100 en caso negativo). Como en este vehículo el eje delantero

y trasero es único para las dos ruedas, es decir, no se pueden subir o bajar las dos ruedas si no que si una se sube la otra debe bajar y por eso no se usan los valores negativos. Por el contrario, solo se envía el valor positivo al lado a subir. Para subir el lado izquierdo se envía el mismo valor en los valores X1 y X3, siendo X2 y X4 igual a cero. Por el contrario, si se desea subir el lado derecho se envía el mismo valor en X2 y X4, siendo X1 y X3 igual a 0.

4- La señal "Estoy Vivo" se debe enviar cada cierto tiempo de forma regular, evitando que pase más tiempo del configurado entre un envío y otro. El resto de las señales pueden enviarse únicamente cuando el valor de alguna entrada analógica/digital cambie respecto a su estado anterior, evitando así la saturación del canal.

5- Los datos cuyo B0 sean igual a 255 no se enviarán por CAN, se usa **exclusivamente para uso interno** entre el receptor y el transmisor.

Tabla 8. Definición de las Tramas CAN BUS que serán enviadas del emisor al receptor (Elaboración propia)

Bytes de ID CAN				**Bytes de Datos CAN**								
B0	**B1**	**B2**	**B3**	**B4**	**B5**	**B6**	**B7**	**B8**	**B9**	**B10**	**B11**	
0	0	0	101	2	X1	X2	X3	X4	0	0	0	Avance / Retroceso vehículo X1-X4 – velocidad de cada rueda en porcentaje, según la lectura del joystick. En principio los 4 valores deben ser iguales. El valor máximo depende de los modos de conducción.
0	0	0	101	3	X1	X2	X3	X4	0	0	0	Giro de las ruedas. X1-X4 – Cantidad de giro de cada una de las ruedas en porcentaje, según los valores del joystick. Los valores depende del modo de giro elegido.
0	0	0	101	5	X1	X2	X3	X4	0	0	0	Nivelación de las ruedas. Indica la velocidad con la que avanzan los cilindros de nivelación en porcentaje.
0	0	0	101	10	X1	0	0	0	0	0	0	Aceleración del Motor Diesel X1 – Valor en porcentaje (0-100%)
0	0	0	101	50	0 255	0	0	0	0	0	0	Arranque del vehículo (motor Diesel) 0 – para el vehículo 255 – arranca el vehículo
0	0	0	101	51	0 255	0	0	0	0	0	0	Nivelación 0 – Manual 255 - Automática
0	0	0	101	60	X1	X2	X3	X4	X5	X6	X7	Activación Bombas auxiliares

												X1-X7 – Bombas a activar en porcentaje según caudal (0/ 100 %)
0	0	0	101	100	L1 0 255	L2 0 255	L3 0 255	L4 0 255	0	0	0	Indica el encendido de luces. Solo admite 0 (apagadas) / 255 (encendidas). L1 – Luz delantera izquierda L2 – Luz delantera derecha L3 – Luz trasera izquierda L4 – Luz trasera derecha
0	0	0	101	102	0 255	0	0	0	0	0	0	Activación bocina 0 – Apagada / 255 - Encendida
0	0	0	101	200	0 255	0	0	0	0	0	0	0 – Modo Manual 255 – Modo autónomo
255	0	0	1	0	0	0	0	0	0	0	0	Señal de “Estoy Vivo”
255	0	0	2	0 255	0	0	0	0	0	0	0	Se activa o desactiva el control del mando inalámbrico.
255	255	255	255	0 255	0	0	0	0	0	0	0	“Seta” de emergencia 0 – Liberada 255 – Presionada

6.1.1.3 Programación del emisor

En este apartado, se analizan detalladamente el código de programación del emisor. Es decir, se explican todas aquellas variables, funciones y librerías que intervienen en el mismo.

El código se ha desarrollado sobre PlatformIO, que se trata de un entorno de desarrollo pensado para aplicaciones de IoT y se ejecuta sobre el editor Visual Studio Code. Asimismo, el lenguaje utilizado para programar el código ha sido C++.

En primer lugar, se va a explicar las librerías que se han incluido en el sketch del emisor. Es decir, trozos de código desarrollados por terceros que facilitan la programación y permiten que se desarrollen programas más sencillos(Jecrespom, 2017). En la figura 38, se muestra como se ha hecho uso de tres librerías distintas. La primera es Arduino.h que permite trabajar en PlatformIO de la misma forma que con el IDE de Arduino pero de un forma mucho más clara y esquematizada. La segunda es SPI.h, esta permite que exista comunicación (envio y recibo de datos) entre el Arduino y el NRF24L01 por el bus SPI. La última librería utilizada es RF24.h (incluye a nRf24L01.h y RF24_config.h) y se encarga de gestionar el módulo de radiofrecuencia utilizado NRF24L01.

```
#include <Arduino.h>
#include <SPI.h>
#include <nRF24L01.h>
#include <RF24.h>
#include <RF24_config.h>
```

Figura 38. Librerías utilizadas en el código del emisor (Elaboración propia)

A continuación, se explica las funciones de la librería RF24.h utilizadas en el código del emisor:

- **uint8_t CE PIN, uint8_t CSN PIN:** son las funciones constructor, crean un nuevo objeto (instancia) del NRF24L01. La instancia se establece con los pines de comunicación SPI. Es necesario especificar que pines digitales del Arduino están conectados al módulo (Figura 39).

```
//NRF24L01
const uint8_t CE_PIN = 9;
const uint8_t CSN_PIN = 10;
RF24 radio(CE_PIN, CSN_PIN);//creamos el objeto radio (NRF24L01)
```

Figura 39. Creación de objeto para el NRF24L01 (Elaboración propia)

- **radio.begin() :** Inicializa el objeto creado, normalmente se llama a esta función en el set up () del sketch(Figura 40).

```
void setup() {

 //NRF24L01
   radio.begin();  //inicializamos el NRF24L01
```

Figura 40. Inicialización del objeto creado para el NRF24L01 (Elaboración propia)

- **radio.openWritingPipe(dirección):** Abre un canal de comunicación de escritura (Figura 41).

```
void setup() {

//NRF24L01
  radio.begin();  //inicializamos el NRF24L01
  radio.openWritingPipe(RFPipe); //Abrimos un canal de escritura
```

Figura 41. Canal de comunicación de escritura del NRF24L01 (Elaboración propia)

Este canal de comunicación de escritura lleva asociada una dirección de canal (Figura 42) que se asigna a través de una matriz de 5 bytes de longitud.

```
//NRF24L01
const uint8_t CE_PIN = 9;
const uint8_t CSN_PIN = 10;
RF24 radio(CE_PIN, CSN_PIN);//creamos el objeto radio (NRF24L01)
//Variable con la dirección del canal por donde se va a transmitir
byte RFPipe[5] ={0x60, 0x59, 0x70, 0x54, 0x01};
```

Figura 42. Dirección de canal para el canal de comunicación de escritura del NRF24L01 (Elaboración propia)

- **radio.write(datos, sizeof(datos)):** envía los datos por el canal definido en openWritingPipe(). En sizeof(datos) se puede ajustar el tamaño de los datos a enviar teniendo en cuenta que el límite máximo de datos por cada envío es de 32 bytes (Figura 43).

```
radio.write(datos, sizeof(datos));  //Se envian los datos por RF
```

Figura 43. Función del NRF24L01 que permite el envío de datos por el canal definido (Elaboración propia)

En segundo lugar, se van a exponer todas las variables y funciones declaradas justo antes del set up (). Estas variables y funciones aparecerán a medida que se vayan explicando el código de forma detallada de cada función del mando inalámbrico.

Las primeras variables declaradas han sido todas aquellas entradas analógicas y digitales correspondientes a las distintas funciones (componentes físicos) del mando inalámbrico (Figura 44). Teniendo en cuenta la distribución elegida de los componentes físicos en la carcasa del mando y la distribución de los pines del Arduino MEGA se le ha intentado asignar una enumeración lógica a cada una de las entradas analógicas y digitales con el objetivo de evitar que se crucen los cables de conexión.

El tipo de dato utilizado para todas las entradas digitales y analógicas ha sido uint8_t, que se trata de un número entero sin signo de 1 byte (de 0 a 255).

```
//Entradas Analógicas
const uint8_t PinAceleracion= A14;    //Pin Analogico donde está conectado el acelerador del motor (r.p.m)
const uint8_t PinNivelacion = A13;    //Pin Analogico donde está conectado el potenciometro para la nivelacion
const uint8_t PinGiro = A12;          //Pin Analogico donde está conectado el potenciometro para el giro
const uint8_t PinVelocidad = A11;     //Pin Analogico donde está conectado el potenciometro para la velocidad
const uint8_t PinBombaAux= A6;        //Pin Analógico donde está conectada el potenciometro de la bomba auxiliar
const uint8_t PinModoLuces = A5;      //Pin Analogico para leer el modo de las luces
const uint8_t PinModoConduccion = A2; //Pin Analogico para el modo de conducción (Eco, Normal o Sport)
const uint8_t PinModoGiro = A1;       //Pin Analogico para los modos de giro

//Entradas Digitales
const uint8_t Bocina= 3;              //Pin digital donde está conectado el cable de control (modo PULL DOWN) de la bocina
const uint8_t Arranque= 5;            //Pin digital donde está conectado el cable de control (modo PULL DOWN) del arranque
const uint8_t NivAuto=7;              //Nivelación automática del vehículo
const uint8_t Autonomo=8;             //Pin digital para activar el modo autónomo/manual
const uint8_t Interruptor0F= 11;      //Pin digital que desactiva todas las funciones del mando inalámbrico salvo la seta de emergencia
const uint8_t SetaEmergencia= 12;     //Pin digital Seta de Emergencia
```

Figura 44. Declaración de variables: entradas analógicas y digitales del Arduino MEGA correspondientes a los distintos componentes físicos del mando inalámbrico (Elaboración propia)

Una vez definidas las variables anteriores, se han declarado las variables correspondientes a los cinco primeros bytes (del byte 0 al byte 4) de las tramas CAN definidas anteriormente para el emisor. Correspondiéndose los cuatro primeros bytes (del byte 0 al byte 3) a la parte de identificación de las tramas y el byte 4 como identificador propio de cada función del emisor (Figura 45).

También, se ha utilizado el tipo de dato uint_8 para la declaración de estas.

```
//Bytes de ID CAN (Byte 0,1,2 y 3) de las Tramas CAN definidas
const uint8_t idModulo[4] = {0, 0, 0, 101};
//Byte 4 de las Tramas CAN definidas
const uint8_t idNivelacion = 5;
const uint8_t idVelocidad = 2;
const uint8_t idAceleracion= 10;
const uint8_t idGiro= 3;
const uint8_t idBocina= 102;
const uint8_t idArranque= 50;
const uint8_t idNivAuto= 51;
const uint8_t idAutonomo= 200;
const uint8_t idLuces= 100;
const uint8_t idBombaAux= 60;
```

Figura 45. Declaración de variables: cinco primeros bytes de las tramas CAN (Elaboración propia)

En la figura 46, se muestra cómo se han definido cada una de las funciones utilizadas en el sketch que después se irán viendo detalladamente una a una. Las primeras funciones son de tratamiento de variables analógicas y digitales y las demás, son funciones que permiten enviar los datos de acuerdo con las tramas CAN definidas correspondientes a los componentes físicos del mando.

```
//Funciones tratamiento de entrada analógica
int8_t ReadAnalogValue(uint8_t port, int portmin, int portmax, int8_t outmin, int8_t outmax, uint8_t steps);
int8_t CalcularValorPasos(int8_t value, uint8_t steps);

//Función tratamiento de entrada digital
bool ReadDigitalValue(uint8_t port);

//Funciones de envío de tramas CAN
void EnviarEstoyVivo();
void LeeEnviarValores();
void LeerEnviarValorNivelacion();
void LeerEnviarValorVelocidad();
void LeerEnviarValorGiro();
void LeerEnviarValorBocina();
void LeerEnviarValorArranque();
void LeerEnviarValorSetaEmergencia();
void LeerEnviarValorRF();
void LeerEnviarValorBombaAux();
void LeerEnviarValorNivAuto();
void LeerEnviarValorAceleracion();
void LeerEnviarValorAutonomo();
void LeerEnviarValorLuces();
```

Figura 46. Declaración de funciones correspondientes al tratamiento de variables analógicas y digitales y las funciones que permiten el envío de tramas CAN (Elaboración propia)

También, en la figura 47, se observa cómo se han declarado variables que permiten guardar el último valor de una entrada analógica o digital. Estas variables servirán para no saturar el canal de transmisión de datos. Se han utilizado datos del tipo unsigned long, que se trata de una variable larga sin signo y ocupa 32 bytes (de 0 a $2^{32} - 1$), del tipo int8_t, que es un número entero con signo de 1 byte (de -128 a 127), del tipo int, dato que almacena un valor de 2 bytes (de -32768 a +32767), del tipo uint8_t y de tipo bool, que ocupa 1 byte de memoria y puede ser o true o false.

```
//Último valor de la variable
unsigned long lastTimeEstoyVivo = 0;
 int8_t lastValueNivelacion = 0;
 int8_t lastValueVelocidad = 0;
 int8_t lastValueGiro = 0;
 int8_t lastValueAceleracion =0;
 int8_t lastValueBombAux =0;
 uint8_t lastValueModoConduccion = 0;
 uint8_t lastValueModoGiro = 0;
 int lastvalueluces=0;
 bool lastValueBocina =false;
 bool lastValueArranque =false;
 bool lastValueSetaEmergencia= false;
 bool lastValueRF= false;
 bool lastValueNivAuto= false;
 bool lastValueAutonomo= false;
```

Figura 47. Declaración de variables: variables que permiten guardar el último valor de una entrada analógica o digital (Elaboración propia)

En tercer lugar, se va a describir detalladamente cada una de las funciones comentadas anteriormente en la declaración de variables.

Se comenzará por explicar la función **ReadAnalogValue** (Figura 48), se emplea esta función para leer una entrada analógica indicando cual es valor mínimo y máximo de la misma (0 y 1023 ya que las entradas analógicas del Arduino MEGA disponen de 10 bits de resolución, lo que proporciona 1024 niveles digitales (Llamas, 2014) y escalarla a un valor mínimo y máximo indicado. Además, permite ajustar de cuanto en cuanto quiere que varíe el valor escalado indicándole el valor del salto. Ha sido necesario llamar a la función **CalcularValorPasos** para que se lleve a cabo esta variación del valor escalado (Figura 49).

Además, la función ReadAnalogValue se ha programado para cuando el valor escalado se encuentre entre ± un valor (ValueZero), el valor leído del puerto analógico sea cero. El objetivo de estas dos particularidades es evitar que las electroválvulas del distribuidor estén constantemente abriéndose y cerrándose en el caso de tener un joystick (o potenciómetros) de gran sensibilidad.

```
int8_t ReadAnalogValue(uint8_t port, int portmin, int portmax, int8_t outmin, int8_t outmax, uint8_t steps) {

  int valor = analogRead(port); //Leemos el puerto analogico

  //Convertimos el valor leido al rango que necesitamos
  int8_t conv = map(valor, portmin, portmax, outmin, outmax);

  //Si el valor se encuentra entre +/- ValueZero, el valor leido del puerto analógico se considera cero.
  if (conv < ValueZero && conv>-ValueZero){
    conv = 0;
  }
  //Convertimos el valor leido al salto más proximo según la configuración.
  conv = CalcularValorPasos(conv, steps);

  return conv;// En este tipo de funciones si retornamos
}
```

Figura 48. Función ReadAnalogValue: permite escalar los valores leídos de una entrada analógica y ajustar de cuanto en cuanto quiere que varíe el valor (Elaboración propia)

```
int8_t CalcularValorPasos(int8_t value, uint8_t steps) {

  int8_t value1 = round(value / steps);

  return (value1 * steps);
}
```

Figura 49. Función CalcularValorPasos: posibilita ajustar de cuanto en cuanto quiere que se varíe el valor escalado de la entrada analógica (Elaboración propia)

Una vez aclarada la función que permite el tratamiento de una variable analógica se va a hacer lo mismo con una variable digital. La función **ReadDigitalValue** (Figura 50), permite devolver si una entrada digital se encuentra true (o estado alto) o false (en estado bajo).

```
bool ReadDigitalValue(uint8_t port) {

  int valor = digitalRead(port); //Leemos el puerto digital

  if (valor>0) return true;
  else return false;

}
```

Figura 50. Función ReadDigitalValue indica si una entrada digital se encuentra en estado alto o en bajo (Elaboración propia)

A continuación, se va a explicar las funciones encargadas del envío de tramas CAN. Se van a ir describiendo de forma ordenada de acuerdo con la semejanza entre ellas.

La primera función para analizar es **LeerEnviarValorNivelacion**, se trata de una función que permite leer el eje z del joystick y enviar la trama CAN correspondiente con el objetivo de conseguir la nivelación manual del vehículo (Figura 51). El código correspondiente a esta función está estructurado de la siguiente manera:

- Se declara el array de bytes que ocupa una trama CAN (12 bytes del byte 0 al byte 11).
- Se hace uso de la función ReadAnalogValue para obtener el valor del potenciómetro z de forma escalada.
- Mediante el uso de un condicional if comprobamos si el valor actual de nivelación leído es distinto al valor anterior guardado con el objetivo de no enviar la misma trama CAN y no saturar el canal de radiofrecuencia.
- Se definen los bytes ID CAN y el byte 4 correspondiente a la nivelación.
- Haciendo uso del condicional if, se especifican las consideraciones respectivas a la nivelación que se definieron en el apartado de Tramas CAN BUS.
- Por medio de la función radio.write(datos, sizeof(datos)) de la librería NRF24L01 se envía la trama correspondiente a la nivelación por el canal adecuado con el objetivo que llegue al receptor.
- Por último, se guarda el valor de la variable actual en la variable correspondiente al último valor guardado con el objetivo de tener esta actualizada para no enviar la misma trama CAN y no saturar el canal.

```
void LeerEnviarValorNivelacion() {

  byte datos[12]; // Las tramas CAN tienen 12 bytes que van desde el byte 0 al 11

  //Leemos el potenciometro de la nivelación de forma escalada utilizando la función RealAnalogValue
  int8_t currentValueNivelacion = ReadAnalogValue(PinNivelacion, 0, 1023, -100, 100, ValueSteps);

  if(currentValueNivelacion != lastValueNivelacion) {  //Si el valor es distinto, envia el nuevo valor y setea el valor actual como el ultimo.

    datos[0] = idModulo[0];
    datos[1] = idModulo[1];
    datos[2] = idModulo[2];
    datos[3] = idModulo[3];
    datos[4] = idNivelacion;

    if (currentValueNivelacion < 0) {
      //Si la nivelación es negativa se entiende que es el lado derecho el que debe subir (pistones 2 y 4)
      datos[5] = 0; //piston 1
      datos[6] = -currentValueNivelacion; //piston 2
      datos[7] = 0; //piston 3
      datos[8] = -currentValueNivelacion; //piston 4
    }
    else {
      //Si la nivelación es positiva se entiende que es el lado izquierdo el que debe subir (pistones 1 y 3)
      datos[5] = currentValueNivelacion; //piston 1
      datos[6] = 0; //piston 2
      datos[7] = currentValueNivelacion; //piston 3
      datos[8] = 0; //piston 4
    }
    datos[9] = 0;
    datos[10] = 0;
    datos[11] = 0;

    radio.write(datos, sizeof(datos));  //Se envian los datos por RF

    lastValueNivelacion = currentValueNivelacion;  //Se establece la nueva medicion como la ultima.
  }
```

Figura 51. Función LeerEnviarValorNivelacion permite leer el eje z del joystick y enviar la trama CAN correspondiente con el objetivo de conseguir la nivelación manual del vehículo (Elaboración propia)

La segunda función para estudiar es **LeerEnviarValorVelocidad**. Esta se emplea cuando se quiere leer los valores del eje y del joystick y enviar la trama CAN correspondiente para que el vehículo avance o retroceda (Figura 52). De esta forma, la función queda estructurada de la siguiente manera:

- Se declara el array de bytes que ocupa una trama CAN (12 bytes del byte 0 al byte 11).
- Se hace uso de la función ReadAnalogValue para obtener el valor del potenciómetro y de forma escalada.
- Mediante el uso de un condicional if y de la entrada analógica designada a la variable PinModoConduccion se comprueba en qué modo de conducción (lento, normal o rápido) se encuentra el selector de tres posiciones destinado para esta función. El PLC cuando recibe la trama CAN correspondiente a el avance y retroceso interpreta el +100 como un máximo y el -100 como un mínimo lo que significa que se correspondería con el modo rápido del selector. El modo normal y el modo lento no deja de ser un 50% y 25% del modo rápido.
- También, mediante el uso de un condicional, se comprueba si el valor actual de avance/retroceso leído es distinto al valor anterior guardado con el objetivo de no enviar la misma trama CAN y no saturar el canal de radiofrecuencia.
- Se definen los bytes correspondientes al avance/retroceso que se definieron en el apartado de Tramas CAN BUS.
- Por medio de la función radio.write(datos, sizeof(datos)) de la librería NRF24L01 se envía la trama correspondiente a el avance/retroceso por el canal adecuado con el objetivo que llegue al receptor.
- Por último, se guarda el valor de la variable actual en la variable correspondiente al último valor guardado con el objetivo de tener esta actualizada para no enviar la misma trama CAN y no saturar el canal.

```
void LeerEnviarValorVelocidad() {

  byte datos[12]; // Las tramas CAN tienen 12 bytes que van desde el byte 0 al 11

 //Leemos el potenciometro del avance/retroceso de forma escalada utilizando la función RealAnalogValue
  int8_t currentValueVelocidad = ReadAnalogValue(PinVelocidad, 0, 1023, -100, 100, ValueSteps);

  //Comprobamos en que modo de conducción estamos (lento,normal o rápido) leyendo el pin A0
  int value = analogRead(PinModoConduccion);
  uint8_t factor = 1;

  if (value>=450 && value<=550) {
    factor= FactorSport;
  } else if (value>=850 && value<=950){
    factor= FactorEco;
  } else {
    factor= FactorNormal;
  }
  if (lastValueModoConduccion!=factor) {
    lastValueModoConduccion = factor;
  }
  currentValueVelocidad = currentValueVelocidad/factor;

  if(currentValueVelocidad != lastValueVelocidad) {  //Si el valor es distinto, envia el nuevo valor y setea el valor actual como el ultimo.

    datos[0] = idModulo[0];
    datos[1] = idModulo[1];
    datos[2] = idModulo[2];
    datos[3] = idModulo[3];
    datos[4] = idVelocidad;
    datos[5] = currentValueVelocidad; //DELANTERA
    datos[6] = currentValueVelocidad; //DELANTERA
    datos[7] = 0; //TRASERA
    datos[8] = 0; //TRASERA
    datos[9] = 0;
    datos[10] = 0;
    datos[11] = 0;

    radio.write(datos, sizeof(datos));  //Se envian los datos por RF

    lastValueVelocidad = currentValueVelocidad;  //Se establece la nueva medicion como la ultima.
  }
}
```

Figura 52. Función LeerEnviarValorVelocidad permite leer el eje y del joystick y enviar la trama CAN correspondiente con el objetivo de conseguir el avance y retroceso del vehículo (Elaboración propia)

La tercera función para examinar es **LeerEnviarValorGiro**. Se hace uso de esta, cuando se quiere leer los valores del eje x del joystick y enviar la trama CAN correspondiente para que el vehículo gire en la dirección indicada (Figura 53 y Figura 54). De esta forma, el esqueleto de esta función queda tal que así:

- Se declara el array de bytes que ocupa una trama CAN
- Se hace uso de la función ReadAnalogValue para obtener el valor del potenciómetro x de forma escalada.
- Haciendo uso de un condicional if y de la entrada analógica designada a la variable analógica PinModoGiro se comprueba en qué modo de giro (solo ruedas delanteras, ruedas delanteras y traseras en el mismo sentido o ruedas delanteras y traseras en distinto sentido) se encuentra el selector de tres posiciones destinado para esta función.
- También, mediante el uso de un condicional, comprobamos si el valor actual del giro o el modo de giro leído es distinto al valor anterior guardado con el objetivo de no enviar la misma trama CAN y no saturar el canal de radiofrecuencia.
- Se definen los bytes ID CAN y el byte 4 correspondiente a el giro del vehículo.
- Mediante el uso del condicional if y teniendo en cuenta el modo de giro elegido, se especifican las consideraciones respectivas el giro que se definieron en el apartado de Tramas CAN BUS. Además, se añade un condicional anidado que imponga que el si el valor de giro está por debajo de un valor mínimo exigido, el valor de giro será cero. El objetivo de este condicional es evitar el giro de las ruedas cuando se realice operaciones de nivelación o de avance/retroceso exclusivas. Este escenario inapropiado sucede por la alta sensibilidad del joystick de 3 ejes utilizado.

- Por medio de la función radio.write(datos, sizeof(datos)) de la librería NRF24L01 se envía la trama correspondiente a el giro por el canal adecuado con el objetivo que llegue al receptor.
- Por último, se guarda el valor de la variable actual en la variable correspondiente al último valor guardado con el objetivo de tener esta actualizada para no enviar la misma trama CAN y no saturar el canal. Del mismo modo se hace lo mismo con la variable correspondiente al modo de giro.

```
void LeerEnviarValorGiro() {

  byte datos[12]; // Desde el byte 0 al 11

  //Leemos el potenciometro de giro
  int8_t currentValueGiro = ReadAnalogValue(PinGiro, 0, 1023, -100, 100, ValueSteps);

  //MODOS DE GIRO
  int currentAnalogModoGiro = analogRead(PinModoGiro);
  uint8_t currentModoGiro;

  if (currentAnalogModoGiro >= 450 && currentAnalogModoGiro <= 550){
    currentModoGiro = 1;
  }
  else if (currentAnalogModoGiro >= 850 && currentAnalogModoGiro <= 950) {
    currentModoGiro = 0;
  }
  else {
    currentModoGiro = 2;
  }

  if (lastValueModoGiro != currentModoGiro || lastValueGiro != currentValueGiro) {

    datos[0] = idModulo[0];
    datos[1] = idModulo[1];
    datos[2] = idModulo[2];
    datos[3] = idModulo[3];
    datos[4] = idGiro;

    if (currentModoGiro == 1) {

      //RUEDAS DELANTERAS Y TRASERAS EN EL MISMO SENTIDO
      if (currentValueGiro < 10 and currentValueGiro > -10) {
        //No gira debido a que el valor de giro no es el minimo exigido (para evitar perturbaciones)
        datos[5] = 0;
        datos[6] = 0;
        datos[7] = 0;
        datos[8] = 0;
      }
      else {
        datos[5] = currentValueGiro;
        datos[6] = currentValueGiro;
        datos[7] = currentValueGiro;
        datos[8] = currentValueGiro;
      }
```

Figura 53. Primera parte de la Función LeerEnviarValorGiro que permite leer el eje x del joystick y enviar la trama CAN correspondiente con el objetivo de conseguir el giro del vehículo (Elaboración propia)

```
    else if (currentModoGiro == 2) {

      //RUEDAS DELANTERAS Y TRASERAS EN DISTINTO SENTIDO
      if (currentValueGiro < 10 and currentValueGiro > -10) {
        //No gira debido a que el valor de giro no es el minimo exigido (para evitar perturbaciones)
        datos[5] = 0;
        datos[6] = 0;
        datos[7] = 0;
        datos[8] = 0;
      }
      else {
        datos[5] = currentValueGiro;
        datos[6] = currentValueGiro;
        datos[7] = -currentValueGiro;
        datos[8] = -currentValueGiro;
      }
    }
    else {

      //SOLO RUEDAS DELANTERAS
      if (currentValueGiro < 10 and currentValueGiro > -10) {
        //No gira debido a que el valor de giro no es el minimo exigido (para evitar perturbaciones)
        datos[5] = 0;
        datos[6] = 0;
        datos[7] = 0;
        datos[8] = 0;

      }
      else {
        datos[5] = currentValueGiro;
        datos[6] = currentValueGiro;
        datos[7] = 0;
        datos[8] = 0;
      }
    }
    datos[9] = 0;
    datos[10] = 0;
    datos[11] = 0;

    radio.write(datos, sizeof(datos));  //Se envian los datos por RF

    lastValueModoGiro = currentModoGiro;
    lastValueGiro = currentValueGiro;
  }
}
```

Figura 54. Segunda parte la Función LeerEnviarValorGiro que permite leer el eje x del joystick y enviar la trama CAN correspondiente con el objetivo de conseguir el giro del vehículo (Elaboración propia)

La cuarta y la quinta función para analizar son muy similares, se trata de la función **LeerEnviarValorAceleración** y la función **LeerEnviarValorBombAux**. La primera permite leer el potenciómetro corredera utilizado para la aceleración del motor y enviar la trama CAN correspondiente (Figura 55) y la segunda posibilita la lectura del potenciómetro utilizado para regular el caudal de la bomba auxiliar destinada al apero y envía también la trama CAN correspondiente a esta operación. Ambas funciones siguen la siguiente estructura:

- Se declara el array de bytes que ocupa una trama CAN (12 bytes del byte 0 al byte 11).
- Se hace uso de la función ReadAnalogValue para obtener el valor del potenciómetro (ya sea el del acelerador del motor o el de la bomba auxiliar) de forma escalada.
- Mediante el uso de un condicional, se comprueba si el valor actual de aceleración (porcentaje de apertura de caudal de la bomba) es distinto al valor anterior guardado con el objetivo de no enviar la misma trama CAN y no saturar el canal de radiofrecuencia.
- Se definen los bytes correspondientes a la aceleración o la bomba auxiliar que se definieron en el apartado de Tramas CAN BUS.

- Por medio de la función radio.write(datos, sizeof(datos)) de la librería NRF24L01 se envía la trama correspondiente a la aceleración o la bomba auxiliar por el canal adecuado con el objetivo que llegue al receptor.
- Por último, se guarda el valor de la variable actual en la variable correspondiente al último valor guardado con el objetivo de tener esta actualizada para no enviar la misma trama CAN y no saturar el canal.

```
void LeerEnviarValorAceleracion() {

 byte datos[12];

//Leemos el potenciometro de la aceleración del motor de forma escalada utilizando la función RealAnalogValue
 int8_t currentValueAceleracion = ReadAnalogValue(PinAceleracion, 0, 1023, 0, 100, ValueSteps);

 //Si el valor es distinto, envia el nuevo valor y setea el valor actual como el ultimo.
 if(currentValueAceleracion != lastValueAceleracion) {

   datos[0] = idModulo[0];
   datos[1] = idModulo[1];
   datos[2] = idModulo[2];
   datos[3] = idModulo[3];
   datos[4] = idAceleracion;
   datos[5] = currentValueAceleracion;
   datos[6] = 0;
   datos[7] = 0;
   datos[8] = 0;
   datos[9] = 0;
   datos[10] = 0;
   datos[11] = 0;

   radio.write(datos, sizeof(datos));  //Se envian los datos por RF

   lastValueAceleracion = currentValueAceleracion;  //Se establece la nueva medicion como la ultima.

 }
}
```

Figura 55. Función LeerEnviarValorAceleracion permite leer el potenciómetro de corredera utilizado para la aceleración del motor y enviar la trama CAN correspondiente (Elaboración propia)

La sexta función que se va a estudiar es **LeerEnviarValorLuces**. Esta función permite leer la posición de un selector de tres posiciones mediante una variable analógica y posteriormente dependiendo de la posición que se encuentre se enviará la trama CAN correspondiente para que en el vehículo se enciendan solo las luces delanteras, todas (delanteras y traseras) o ninguna (Figura 56 y Figura 57).

Como se ha hecho con las otras funciones, se va a presentar como se ha estructurado la misma:

- Se declara el array de bytes que ocupa una trama CAN
- Se procede a la lectura del valor de la entrada analógica que va a determinar en qué modo se encuentra las luces.
- Se comprueba mediante un condicional if en qué modo se encuentran las luces, es decir si la entrada analógica se encuentra entre un rango u otro determinado, pertenecerá a un modo u otro (solo luces delanteras, delanteras y traseras o apagadas).
- Una vez dentro del modo seleccionado, haciendo uso de otro condicional if se comprueba si el valor actual de las luces es distinto al valor anterior guardado con el objetivo de no enviar la misma trama CAN y no saturar el canal de radiofrecuencia.
- Se definen los bytes correspondientes a las luces (dependiendo del modo seleccionado) que se definieron en el apartado de Tramas CAN BUS.

- Por medio de la función radio.write(datos, sizeof(datos)) de la librería NRF24L01 se envía la trama correspondiente a las luces por el canal adecuado con el objetivo que llegue al receptor.
- Por último, se guarda el valor de la variable actual en la variable correspondiente al último valor guardado con el objetivo de tener esta actualizada para no enviar la misma trama CAN y no saturar el canal.

```
void LeerEnviarValorLuces() {

  byte datos[12];

  int value = analogRead(PinModoLuces);

  int lucesapagadas=11;
  int lucesdelanteras=12;
  int lucestodas=13;

  if (value>=450 && value<=550){

    if(lucesapagadas != lastvalueluces){

      datos[0] = idModulo[0];
      datos[1] = idModulo[1];
      datos[2] = idModulo[2];
      datos[3] = idModulo[3];
      datos[4] = idLuces;
      datos[5] = 255;
      datos[6] = 255;
      datos[7] = 255;
      datos[8] = 255;
      datos[9] = 0;
      datos[10] = 0;
      datos[11] = 0;
      radio.write(datos, sizeof(datos));

      lastvalueluces = lucesapagadas;
    }
  }
```

Figura 56. Primera parte de la función LeerEnviarValorLuces permite leer el modo de luces que se encuentra el emisor y enviar la trama CAN correspondiente (Elaboración propia)

```
  else if (value>=850 && value<=950){
    if(lucestodas != lastvalueluces){

      datos[0] = idModulo[0];
      datos[1] = idModulo[1];
      datos[2] = idModulo[2];
      datos[3] = idModulo[3];
      datos[4] = idLuces;
      datos[5] = 0;
      datos[6] = 0;
      datos[7] = 0;
      datos[8] = 0;
      datos[9] = 0;
      datos[10] = 0;
      datos[11] = 0;

      radio.write(datos, sizeof(datos));
    }
    lastvalueluces=lucestodas;
  }
  else {
    if(lucesdelanteras != lastvalueluces){

      datos[0] = idModulo[0];
      datos[1] = idModulo[1];
      datos[2] = idModulo[2];
      datos[3] = idModulo[3];
      datos[4] = idLuces;
      datos[5] = 255;
      datos[6] = 255;
      datos[7] = 0;
      datos[8] = 0;
      datos[9] = 0;
      datos[10] = 0;
      datos[11] = 0;
      radio.write(datos, sizeof(datos));
    }
    lastvalueluces=lucesdelanteras;
  }
}
```

Figura 57. Segunda parte de la función LeerEnviarValorLuces permite leer el modo de luces que se encuentra el emisor y enviar la trama CAN correspondiente (Elaboración propia)

La séptima, octava, novena y décima función se programan de la misma forma. Se trata de las funciones **LeerEnviarValorBocina**, **LeerEnviarValorArranque**, **LeerEnviarValorNivAuto** y **LeerEnviarValorAutonomo** que son las encargadas de leer el estado (alto o bajo) del pulsador normalmente abierto bocina (Figura 58), del pulsador normalmente cerrado para el arranque, del interruptor nivelación automática y del interruptor modo autónomo y posteriormente enviar la trama CAN correspondiente a cada función.

El esqueleto fundamental que siguen estas funciones se presenta a continuación:

- Se declara el array de bytes que ocupa una trama CAN
- Se hace uso de la función ReadDigitalValue para obtener el estado que se encuentra el pulsador o el interruptor
- Mediante el uso de un condicional, se comprueba si el estado actual (true o false) es distinto al valor anterior guardado (variable booleana true o false) con el objetivo de no enviar la misma trama CAN y no saturar el canal de radiofrecuencia.

- Se definen los bytes correspondientes a la bocina, arranque, nivelación automática y modo autónomo que se definieron en el apartado de Tramas CAN BUS. Se activa la bocina, la nivelación y el modo automático cuando la entrada digital esté en estado alto (true). Sin embargo, el arranque se activa cuando la entrada digital se encuentre en estado bajo ya que el pulsador utilizado es normalmente cerrado. Se ha elegido un pulsador normalmente cerrado para el arranque porque debido a la escasez de stock de la tienda local del pulsador normalmente abierto utilizado en la bocina.
- Por medio de la función radio.write(datos, sizeof(datos)) de la librería NRF24L01 se envian las tramas correspondientes a la bocina, arranque, nivelación automática y modo automático por el canal adecuado con el objetivo que llegue al receptor.
- Por último, se guarda el valor de la variable actual en la variable correspondiente al último valor guardado con el objetivo de tener esta actualizada para no enviar la misma trama CAN y no saturar el canal.

```
void LeerEnviarValorBocina() {

  byte datos[12];

 //Leemos el pulsador normalmente abierto utilizado para la bocina utilizando la función RealDigitalValue
  bool sinobocina = ReadDigitalValue(Bocina);

  if(sinobocina != lastValueBocina) {  //Si el valor es distinto, envia el nuevo valor y setea el valor actual como el ultimo.

    datos[0] = idModulo[0];
    datos[1] = idModulo[1];
    datos[2] = idModulo[2];
    datos[3] = idModulo[3];
    datos[4] = idBocina;

    if(sinobocina == true){ //Como el pulsador es normalmente abierto
      datos[5] = 255; //Bocina activada
    }
    else {
      datos[5] = 0; //Bocina desactivada
    }

    datos[6] = 0;
    datos[7] = 0;
    datos[8] = 0;
    datos[9] = 0;
    datos[10] = 0;
    datos[11] = 0;

    radio.write(datos, sizeof(datos));  //Se envian los datos por RF

    lastValueBocina = sinobocina;  //Se establece la nueva medicion como la ultima.
  }
}
```

Figura 58. Función LeerEnviarValorBocina permite leer el estado de la entrada digital asociada la bocina y enviar la trama CAN correspondiente al receptor (Elaboración propia)

La undécima, duodécima y decimotercera función, es decir, las funciones **LeerEnviarValorSetaEmergencia**, **LeerEnviarValorRF** y **EnviarEstoyVivo** se encargan de enviar tramas CAN que se usan exclusivamente para uso interno entre el receptor y el transmisor, a diferencia de las anteriores que las tramas enviadas por el emisor tenían que llegar al Bus CAN que está conectado el PLC del vehículo.

Las funciones LeerEnviarValorSetaEmergencia (Figura 59) y LeerEnviarValorRF son funciones muy similares de programar a la función LeerEnviarValorBocina pero teniendo que los bytes de identificación de trama CAN cambian (consultar los bytes que se definieron en el apartado de Tramas CAN BUS). El objetivo de estas funciones es leer el estado de la entrada digital asociada a la seta de emergencia (o el interruptor de desactivación de funciones) y enviar la trama CAN

correspondiente al receptor. Tramas que serán manipuladas en la programación del receptor ya que se tratan de variables internas.

```
void LeerEnviarValorSetaEmergencia() {

  byte datos[12]; // Desde el byte 0 al 11

//Leemos el interruptor seta de emergencia utilizando la función RealDigitalValue
  bool sinosetaemergencia = ReadDigitalValue(SetaEmergencia);

  if(sinosetaemergencia != lastValueSetaEmergencia) {

    datos[0] = 255;
    datos[1] = 255;
    datos[2] = 255;
    datos[3] = 255;

    if(sinosetaemergencia == false) {
      datos[4] = 0; // Seta de emergencia desactivada
    } else {
      datos[4] = 255; //Seta de emergencia activada
    }
    datos[5] = 0;
    datos[6] = 0;
    datos[7] = 0;
    datos[8] = 0;
    datos[9] = 0;
    datos[10] = 0;
    datos[11] = 0;

    radio.write(datos, sizeof(datos));

    lastValueSetaEmergencia = sinosetaemergencia;
  }
}
```

Figura 59. Función LeerEnviarValorSetaEmergencia permite leer el estado de la entrada digital correspondiente a la seta de emergencia y también envía la trama CAN correspondiente al receptor (Elaboración propia)

Como ya se ha comentado, la decimotercera función **EnviarEstoyVivo** enviarán tramas CAN que se usaran exclusivamente para uso interno entre el receptor y el transmisor y tiene como objetivo enviar una trama constante cada 100 milisegundo (se trata de la variable EstoyVivoPeriod declarada en la parte de declaración de variables del sketch) con la intención de que el receptor sepa si el emisor ha perdido la señal de radiofrecuencia o está demasiado lejos (Figura 60).

```
void EnviarEstoyVivo() {
  if ((millis()-lastTimeEstoyVivo) >= EstoyVivoPeriod) {    //Enviamos la señal de ESTOY VIVO cada 100 ms

    byte datos[12] = {255, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0};

    radio.write(datos, sizeof(datos));

    lastTimeEstoyVivo = millis();
  }
}
```

Figura 60. Función EnviarEstoyVivo permite enviar una trama constante cada 100 ms con el objetivo de que el receptor sepa si el emisor sigue vivo (Elaboración propia)

La última función para analizar es la función **LeeEnviarValores** es la encargada de gestionar cuál de las funciones anteriores son prioritarias (salvo la función EnviarEstoyVivo) y bajo qué condiciones se enviarán.

Las funciones LeerEnviarSetaEmergencia, LeerEnviarValorRF y LeerEnviarValorAutónomo siempre se deben enviar. El resto de las funciones solo se envian si la seta de emergencia está desactivada, si está desactivado el interruptor que inhabilita las funciones manuales del mando y si el modo autónomo está desactivado (Figura 61).

```
void LeeEnviarValores() {

  //Leer RF
  LeerEnviarValorRF();

  //Seta de emergencia
  LeerEnviarValorSetaEmergencia();

  //Modo autónomo/manual
  LeerEnviarValorAutonomo();

   if (lastValueSetaEmergencia == false && lastValueRF == true && lastValueAutonomo == false) {
    //Nivelacion
    LeerEnviarValorNivelacion();

    //Velocidad
    LeerEnviarValorVelocidad();

    //Giro
    LeerEnviarValorGiro();

    //Aceleracion del motor
    LeerEnviarValorAceleracion();

    //Bocina
    LeerEnviarValorBocina();

    //Arranque
    LeerEnviarValorArranque();

    //Bomba auxiliar
    LeerEnviarValorBombaAux();

    //Nivelación Automática
    LeerEnviarValorNivAuto();

    //Luces
    LeerEnviarValorLuces();

    //Bomba Auxiliar
    LeerEnviarValorBombaAux();
   }
}
```

Figura 61. Función LeeEnviarValores es la encargada de gestionar cuál de las funciones anteriores son prioritarias (salvo la función EnviarEstoyVivo) y bajo qué condiciones se enviarán (Elaboración propia)

En cuarto lugar, se describe la programación del **set up ()** del Arduino. El set up es la primera función en ejecutarse dentro de un programa de Arduino. Aquí es donde se establecen algunos criterios que requieren una ejecución única (García Gonzalez, 2015). Para el caso del emisor, primero, como ya se comentó cuándo se habló de las librerías utilizadas, se inicializó el NRF24L01 y se abrió el canal de envío de datos. Seguidamente, se ha configurado el puerto serie a velocidad de 115200 baudios con el objetivo de poder depurar el código cuando sea necesario. Más tarde, se han declarado los pines digitales utilizados del Arduino como entradas digitales (no es necesario esta declaración para las entradas analógicas). Posteriormente, se declara la función millis() de la librería Arduino que devuelve el número de milisegundos transcurridos desde que la placa Arduino comienza a ejecutar el programa con el objetivo de utilizarla en la función EstoyVivoPeriod definida anteriormente (Web oficial de Arduino, 2020). Para finalizar, se han inicializado todas las funciones relacionadas con los componentes físicos del mando (pulsadores, interruptores, potenciómetros) con la intención de inicializar (al encender el mando) los valores de las variables y los estados de los distintos pulsadores, interruptores y potenciómetros de la última vez que se manipularon (Figura 62).

```
void setup() {

//NRF24L01
  radio.begin();  //inicializamos el NRF24L01
  radio.openWritingPipe(RFPipe); //Abrimos un canal de escritura

//Configuramos el puerto serie si tenemos la depuracion activa
  #if DEBUG == true
    Serial.begin(115200);
  #endif

  //Pines digitales
  pinMode(Bocina,INPUT);
  pinMode(Arranque,INPUT);
  pinMode(SetaEmergencia,INPUT);
  pinMode(InterruptorRF,INPUT);
  pinMode(NivAuto,INPUT);
  pinMode(Autonomo,INPUT);

  lastTimeEstoyVivo = millis();

  //Leer todas las variables relacionadas con las funciones del mando inalámbrico
  LeeEnviarValores();

}
```

Figura 62. Programación de la función set up () del emisor(Elaboración propia)

En último lugar, se ha programado la función **loop()** del Arduino. La función loop () es la que se ejecuta un número infinito de veces. Al encenderse el Arduino se ejecuta el código del set up() y luego entra al loop(), el cual se repite de forma indefinida hasta que se apague o se reinicie el microcontrolador (García Gonzalez, 2015).

Se han intentado programar la función loop() de forma optimizada y es por eso que solo se llaman a las funciones LeeEnviarValores y EnviarEstoyVivo (Figura 63).

```
void loop(){

  //Leemos los valores y enviamos los datos.
  LeeEnviarValores();

  //Enviamos la señal de Estoy Vivo.
  EnviarEstoyVivo();

}
```

Figura 63. Programación de la función loop() del emisor (Elaboración propia)

6.1.1.4 Resultados del prototipo emisor

En las figuras 64 y 65, se muestran el resultado del mando prototipo que se ha fabricado recortando una caja de plástico comercial, a la cual se le han ido añadiendo los componentes electrónicos necesarios para cubrir todas las funciones detalladas anteriormente. Como se aprecian en las figuras mencionadas, todos los componentes electrónicos están pinchados sobre protoboard y ninguno de ellos soldado. Se debe comentar que el sistema de alimentación utilizado en ese momento era una pila de 9V.

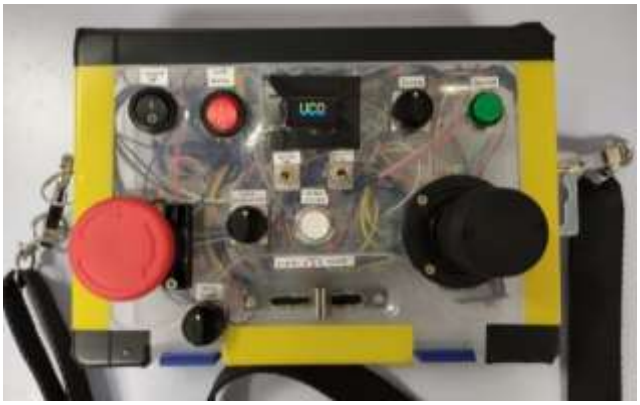


Figura 64. Aspecto físico del mando inalámbrico prototipo (Elaboración propia)

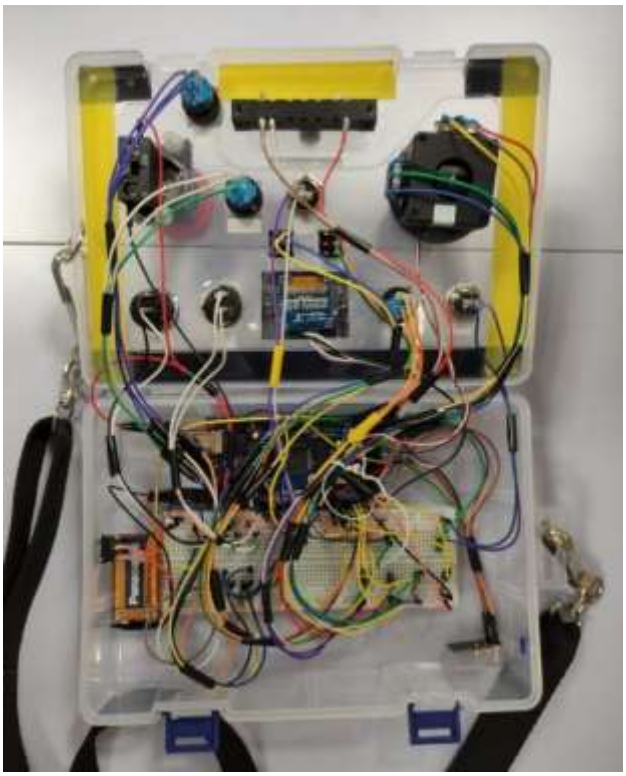


Figura 65. Interior del mando inalámbrico prototipo donde se aprecia que los componentes electrónicos están pinchados en protoboard (Elaboración propia)

6.1.2 Diseño de la PCB

Como se puede apreciar en las figuras 66 y 67 del apartado de Resultados del prototipo existen una gran cantidad de cables que coexisten dentro del mando inalámbrico. Además, estos cables van pinchados sobre la protoboard, es decir, existe una alta probabilidad de que se suelte algún cable y el mando deje de estar operativo. Es por estas razones por las que se ha decidido diseñar e imprimir una Placa de Circuito Impreso o Printed Circuit Board (PCB). El objetivo de diseñar una PCB es ahorrar una gran cantidad de cable y garantizar una mayor fiabilidad de no fallo a el mando, ya que los cables irán soldados sobre la PCB.

El software que se va a utilizar para el diseño de la PCB es Altium Designer. El objetivo a la hora de crear un nuevo proyecto en este software es hacer un esquema de circuito de todos los componentes electrónicos en la ventana de Schematic y posteriormente cargar estos

componentes en la ventana de PCB y poder unirlos de forma lógica (ya sea manualmente o con el autoroute) mediante pistas a distintos niveles, vías o pads.

En primer lugar, se debe crear una librería propia con los distintos componentes que van a aparecer en el esquema del circuito electrónico y en la ventana de la PCB. De cada componente se deben de crear su Schematic Library y footprint asociado (PCB library). Seguidamente, se van a describir detalladamente los componentes más importantes y mencionar los demás utilizados.

- **Arduino Mega 2560**

Se trata del microcontrolador del circuito. En las figuras 65 y 66, muestran cómo se han recogido todos los pines (alimentación, comunicaciones, entradas/salidas digitales…) que recoge esta placa.

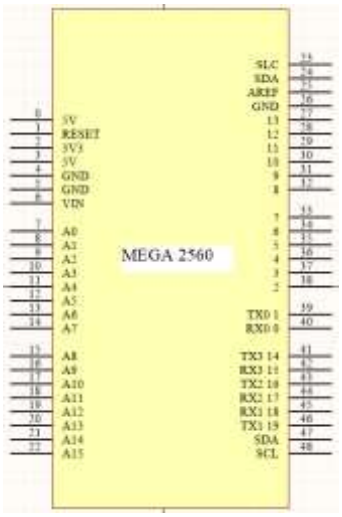


Figura 66. Schematic Library del Arduino Mega Parte A (Elaboración propia)



Figura 67. Schematic Library del Arduino Mega Parte B (Elaboración propia)

Seguidamente, se ha dibujado su footprint asociado (su aspecto dimensional en la PCB)(Figura 68), es decir, las distancias que existen entre sus pines. Para conocer las mismas se ha descargado una imagen footprint del ArduinoMega (Figura 69).

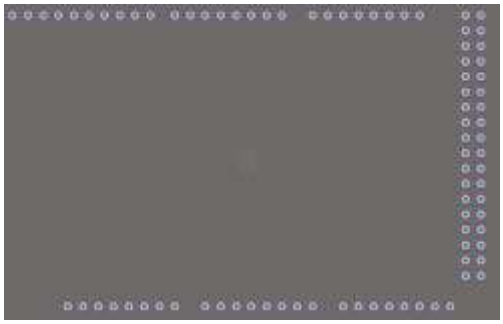


Figura 68. PCB library del Arduino Mega (Elaboración propia)



Figura 69. Footprint del Arduino Mega 2560 (Web oficial de Arduino, 2021c)

- **NRF24L01**

Se trata del módulo de radiofrecuencia utilizado para la comunicación entre el emisor y el receptor. En la figura 70, se expone como se han dibujado los todos los pines que utiliza este módulo de radiofrecuencia.

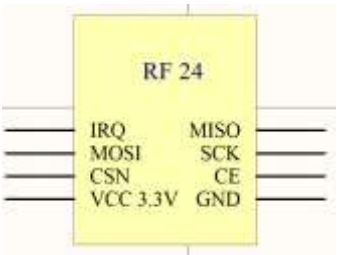


Figura 70. Schematic Library del NRF24L01 (Elaboración propia)

A continuación, se le añade su footprint asociado (Figura 71) teniendo en cuenta las dimensiones del módulo y las distancias que existen entre sus pines.



Figura 71. PCB library del NRF24L01 (Elaboración propia)

Además, se le ha añadido un archivo CAD 3D al componente para ajustar las dimensiones exactas del mismo y poder verlo de manera tridimensional en la PCB objetivo (Figura 72).

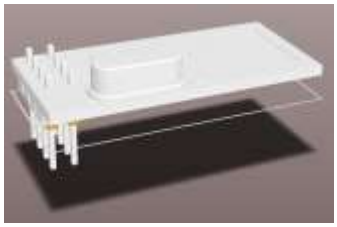


Figura 72. CAD del NRF24L01 asociado al PCB Library del mismo módulo (Elaboración propia)

- **Regulador de Tensión**

El objetivo de este componente es regular la tensión de 12 V proveniente de las baterías y sacar una tensión de salida de 5V para la alimentación del microcontrolador.

- **Conectores hembra (headers)**

Se tratan de los elementos que permiten la conexión segura de los cables procedentes de los componentes físicos del mando (interruptores, pulsadores, potenciómetros…). Se han creado conectores de 2,4 y 6 vías.

- **Resistencias SMD**

Se han elegido resistencias de tipo SMD, ya que son resistencias que ocupan menor espacio que las de encapsulado comunes. Estas tienen la particularidad que son de montaje superficial, y es necesario soldarlas en la capa de soldado (Top Solder) de la PCB.

- **Condensadores SMD**

Muy similar a las resistencias SMD, se ha hecho lo mismo con los condensadores SMD, que tienen por objetivo filtrar y estabilizar las señales que llevan demasiado ruido.

- **Puente de diodos (Bridge of diodes)**

Se ha incluido un puente de diodos a la entrada el regulador de tensión con el objetivo de evitar equivocarse de cable a la hora de alimentar el circuito. Este componente permite conectar el positivo o el negativo en el agujero que se quiera del conector de alimentación ya que por la configuración de los diodos las tensiones de salida siempre seguirán el mismo recorrido (Figura 73).

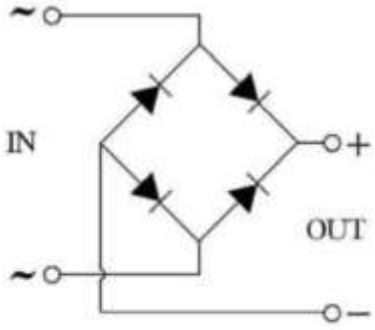


Figura 73. Configuración de un puente de diodos (Vázquez, 2019)

- **Conector de alimentación**

Se trata del conector donde se unen los cables de alimentación procedentes de las baterías.

En segundo lugar, después de crear la librería con todos los componentes electrónicos que se van a utilizar, se ha realizado el esquema de circuito de la lógica de conexionado de todos estos componentes electrónicos. Este esquema se recoge con el título de "**PLANO N°1:** Esquema de conexionado de los componentes electrónicos del emisor" en el Capítulo 3: PLANOS. Se debe comentar que en este esquema se han tenido en cuenta todas las configuraciones de hardware de los diferentes componentes del mando inalámbrico que se hablaron en el apartado *6.1.1.1 Configuración de hardware*. Además, en este esquema, se puede apreciar el conexionado entre los pines del Arduino y los pines de la pantalla Nextion, el módulo Bluetooth HC-05 y el sistema de alimentación del mando a los que se le dedicará su apartado propio para hablar en detalle de ellos.

En tercer lugar, una vez realizado el esquema de conexionado de los componentes electrónicos del mando. Este se carga en la ventana PCB y todos los footprints asociados a los componen tes del esquema se colocan de forma lógica y funcional dentro del límite máximo definido en la capa Botton Soldier de la PCB (se ha dado unas dimensiones similares a la placa del Arduino MEGA sobre la que irá pinchada). Seguidamente, se dibujan las pistas en diferentes capas (en este caso se han utilizado dos niveles: Top Layer en rojo y Botton Layer en azul) y las vías necesarias para pasar de una capa a otra para que las pistas no se crucen. Altium dispone de la función de Auto Route que te permite crear las pistas de conexionado de forma automática, pero en este caso, se ha preferido routear las pistas de forma manual para entender mejor por dónde va el conexionado y en caso de fallo a la hora de comprobarlo poder detectarlo de una forma más sencilla (Figura 74). También, es importante resaltar que para las pistas que sustituyen a cables de señales de control se le ha dado un ancho de pista de 10 milésimas de pulgada y para las pistas correspondientes a los cables de alimentación un ancho de 20 o 30 milésimas de pulgada dependiendo del espacio disponible donde fueran ubicadas.

Se ha de resaltar, que en la capa TopOverlay (en amarillo) de la PCB se ha indicado las referencias de donde va pinchado cada componente en la placa y la enumeración normalizada de las distintas resistencias y condensadores utilizada en el esquema de conexionado de componentes.

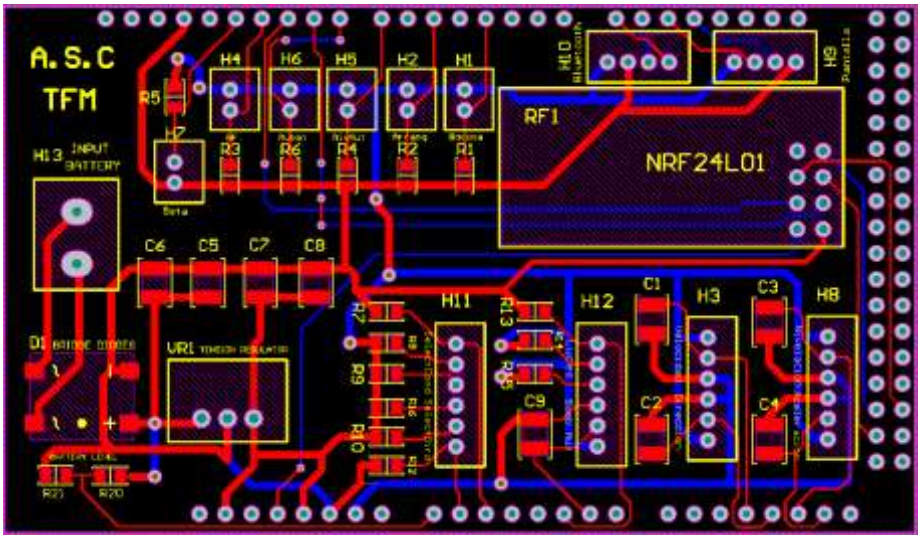


Figura 74. Routeado manual de pistas de la PCB objetivo en 2 niveles distintos (Elaboración propia)

En la figura 75, se muestra el modelo 3D que te genera el propio Altium a partir de la PCB 2D diseñada. Este modelo se puede guardar como archivo .SLDPRT para posteriormente poder abrirlo con SOLIDWORKS (software de diseño 3D que se usará para el diseño CAD del mando inalámbrico).

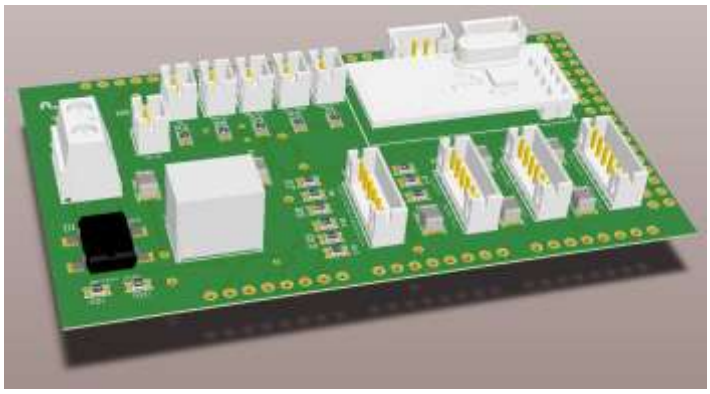


Figura 75. Modelo 3d de la PCB objetivo (Elaboración propia)

Una vez diseñada la PCB, se mandó a fabricar y para ello es necesario enviar los archivos Gerber, es decir, los archivos requeridos para fabricar la placa, que incluyen las pistas en las distintas capas, agujeros de las vías, rotulación y la dimensión de la PCB. En la figura 76, se muestra el resultado de la PCB sin componentes soldados tras su fabricación.

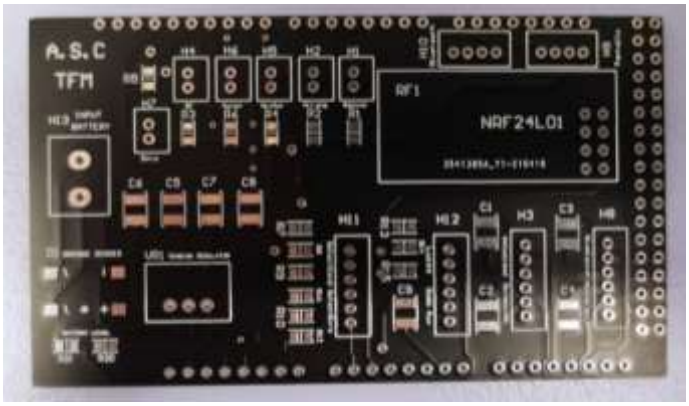


Figura 76. PCB sin componentes soldados tras su fabricación (Elaboración propia)

En la figura 77, se observa una imagen soldando los distintos componentes electrónicos a la PCB fabricada.



Figura 77. Labores de soldadura de los distintos componentes electrónicos a la PCB (Elaboración propia)

Seguidamente, como se aprecia en la figura 78, se sustituyó una gran cantidad de cables pinchados en la protoboard por la PCB. Esto permitió simplificar el cableado, asegurar mejor las conexiones y ahorrar espacio en la carcasa (dato importante a la hora del diseño CAD).

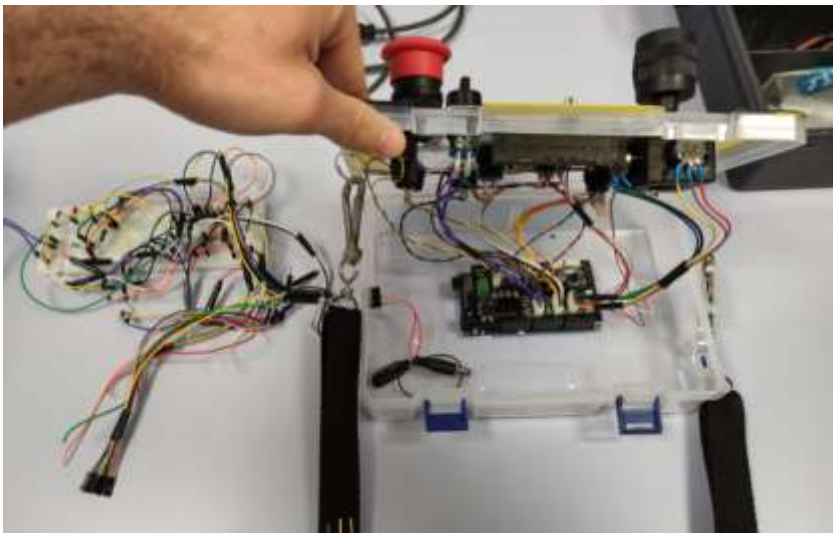


Figura 78. Sustitución de cables pinchados en la protoboard por la PCB diseñad (Elaboración propia)

6.1.3 Diseño CAD del mando

Una vez llegado a este punto, el mando inalámbrico cumplía con los objetivos que se le habían marcado para este TFM, pero disponía de una apariencia física mísera, ya que los componentes se sujetaban en una caja de plástico. Entonces, se pensó en diseñar una carcasa para el mando basada en una comercial y fabricarla mediante una impresora 3D.

Para el diseño CAD de la carcasa del mando se han seguido una serie de **fases de diseño** hasta llegar al diseño final objetivo de la carcasa:

1. **Generación de ideas**

El objetivo de esta fase es adquirir ideas haciendo un estudio sobre productos que existentes. En este caso se ha realizado una búsqueda somera de los diferentes aspectos de mandos inalámbricos que existen en el mercado, intentado seleccionar un modelo estético y funcional

que incluya pantalla en una posición inclinada y a doble altura. Entre los candidatos se encuentran recogidos en la figura 79.



Figura 79. Mandos comerciales elegidos como inspiración para el diseño CAD del mando objetivo (Saioacontrol, 2021)

2. Diseño Conceptual

Se trata de la fase de bocetado, es decir la fase donde se dibuja a mano alzada la forma del mando a diseñar. En esta fase se intentó plasmar a mano alzada utilizando bolígrafo y papel el aspecto del mando objetivo teniendo en cuenta la revisión realizada en la fase de generación de ideas de los diferentes mandos inalámbricos comerciales existentes (Figura 80).

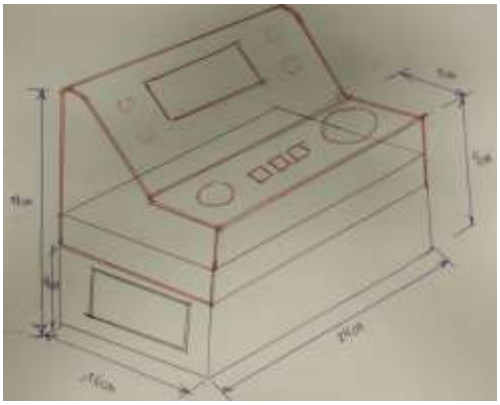


Figura 80. Diseño Conceptual del mando inalámbrico (Elaboración propia)

3. Diseño Preliminar

En esta fase, mediante el uso del software SOLIDWORKS se ha realizado el diseño CAD del mando boceto dibujado a mano alzada en la fase de Diseño Conceptual con el objetivo de ver como quedaba estética y funcionalmente en formato 3D. En un principio se pensó en un mando que incluyera una pantalla LCD de 16x2 y con una forma cuadrada que fuese fácil de imprimir en 3D (Figura 81).

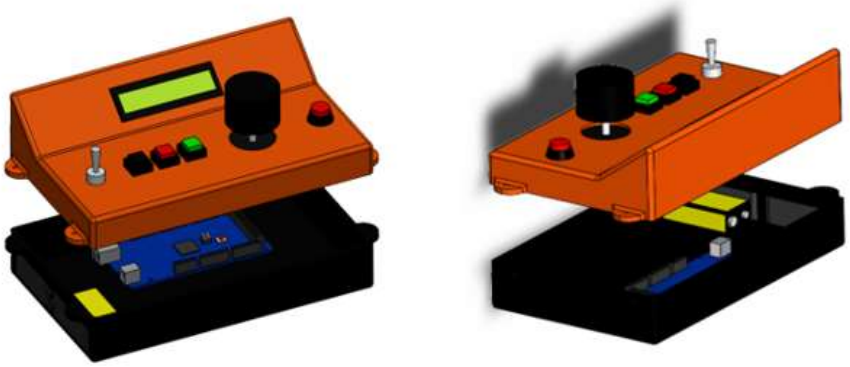


Figura 81. Diseño 3d preliminar del mando inalámbrico basado en el boceto o Diseño Conceptual (Elaboración propia)

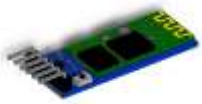
4. Diseño Detallado

En esta última fase del diseño, se ha intentado buscar un diseño más elaborado que el diseño presentado en las fases anteriores. Además, se ha sustituido el display LCD de 16x2 por una pantalla Nextion txt de 3.5 inch a la que se le dedicará un capítulo más adelante. Además, en este diseño si se han considerado todos los componentes funcionales (pulsadores, interruptores y potenciómetros) descritos en el apartado *6.1.1 Programación y prototipado electrónico,* componentes internos (microcontrolador, PCB, sistema de alimentación, módulo bluetooth) y tomas de salida (jack de 5,5 mm para la carga, usb-b para la programación e interruptor de corte de alimentación).

En primer lugar, haciendo uso de GRABCAD, que se trata de un entorno de colaboración gratuito basado en la nube para compartir archivos CAD, se han intentado descargar el mayor número de componentes que constituyen el mando (Tabla 9). Hay que recalcar que cada componente descargado se ha ido ajustando a las dimensiones exactas reales de los distintos componentes comerciales adquiridos.

Tabla 9. Diseños CAD de componentes del mando inalámbrico descargados en GRABCAD (Elaboración propia)

Componente	Diseño CAD
Joystick de 3 ejes	
Arduino MEGA 2560	

Pantalla Nextion 3.5 inch	
Celda de litio 18650	
Holder para una celda de litio 18650	
Battery Management System (BMS) 3S	
Módulo Bluetooth HC-05	
Usb-b hembra para montaje sobre panel para la programación del mando	
Jack cargador hembra de 5.5mm	
Interruptor ON/OFF del mando	
Interruptor Modo Autónomo	
Interruptor que desactiva las funciones manuales de control	
Potenciómetro de 10K para la bomba auxiliar	
Interruptor de palanca para la nivelación automática	

No todos los componentes se han podido descargar y adaptar. En la tabla 10, se han recogido aquellos componentes del mando inalámbrico que **se han tenido que diseñar** de acuerdo con las dimensiones reales exactas de los componentes comerciales adquiridos.

Tabla 10. Diseños CAD de componentes del mando inalámbrico que se han tenido que crear (Elaboración propia)

Componente	Diseño CAD
Seta de Emergencia	
Selector de 3 posiciones AS6-Y para los modos de conducción, giro y luces	
Pulsador NA para Bocina	
Pulsador NC para el Arranque	
Potenciómetro de corredera para el Acelerador del Motor	

En segundo lugar, una vez conocidas las dimensiones de los componentes que va a albergar el mando, se va a proceder a explicar cómo se ha diseñado la carcasa definitiva del mando en sí.

El mando definitivo que se va a diseñar consta de dos partes, una parte inferior donde van albergados los componentes internos como el microcontrolador con la PCB pinchada, 3 baterías de litio 18650 junto a sus holders, el BMS, el módulo bluetooth HC-05, el Usb-b hembra para montaje sobre panel, el Jack de 5.5 mm hembra para la carga de baterías y el interruptor de encendido/apagado del mando inalámbrico y una parte de arriba (que se dividirá en dos pensando en la futura fabricación 3d) donde van albergados todos interruptores, pulsadores, potenciómetros que tienen que ver con las funciones del mando , así como la pantalla donde se visualizarán los valores y estados de estos componentes.

Primero, se va a describir cómo se ha realizado la **parte inferior del mando**. En la figura 82, se muestra como se ha realizado el croquis de la base del mando. Para ello se han combinado círculos tangentes y rectángulos buscando una forma similar a los mandos comerciales. Además, se han dado unas dimensiones de tal forma que cojan todos los componentes de forma espaciosa, estética y ordenada tanto en la parte de arriba como en la parte de abajo del mando. El grosor de las paredes es de 2,5 mm, grosor suficiente para que resista tras la impresión 3D.

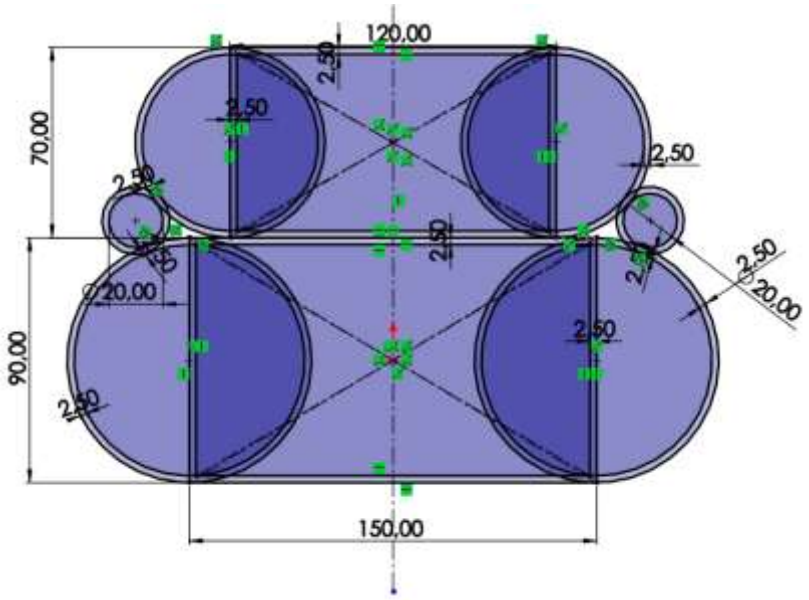


Figura 82. Croquis de la base de la parte de abajo del mando (Elaboración propia)

En la figura 83 (en azul), se muestra cómo se han incorporado unos tetones donde irá asentado y atornillado el Arduino MEGA 2560. Se ha tenido que recurrir al footprint del microcontrolador para saber dónde están ubicados sus taladros. También, se ha levantado una plataforma (en rojo) de 4 mm de altura donde irán asentados y atornillados los holders de las baterías de litio 18650. Además, se ha diseñado dos huecos para ubicar y restringir los movimientos del BMS (en amarillo) y módulo bluetooth HC-05 (en verde).

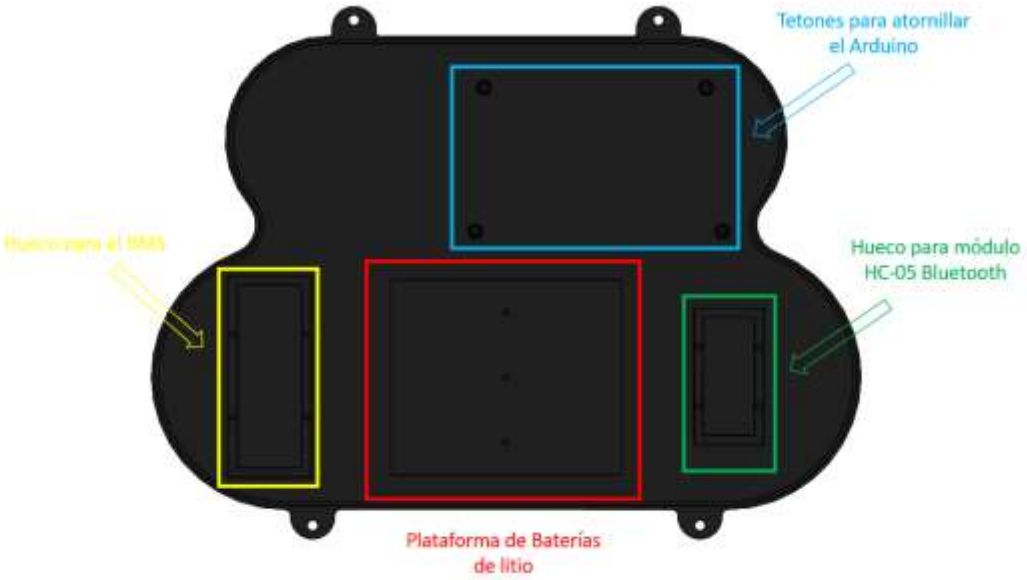


Figura 83. Diseño de tetones, plataforma y huecos para atornillar y ajustar el Arduino MEGA, Holders de baterias de litio, BMS y Módulo Bluetooth a la parte de abajo de la carcasa del mando (Elaboración propia)

Asimismo, en la figura 84, se han tenido en cuenta los huecos para alojar el interruptor de encendido y apagado del mando inalámbrico (en rojo), el usb-b hembra de montaje sobre panel (en azul) y el jack hembra de 5.5 m para cargar el mando (en amarillo).



Figura 84. Diseño de huecos para alojar el interruptor de encendido/apagado del mando, el usb-b de montaje sobre panel y Jack de carga (Elaboración propia)

Además, se le han diseñado unos acoples que permitirán la unión con la parte de arriba de la carcasa del mando y unas guías de 1.25 mm de espesor para conseguir el encaje perfecto con la parte superior del mando (Figura 85). Las dimensiones de la parte inferior del mando se encuentran en el **PLANO N°8** del Capítulo 3: PLANOS.

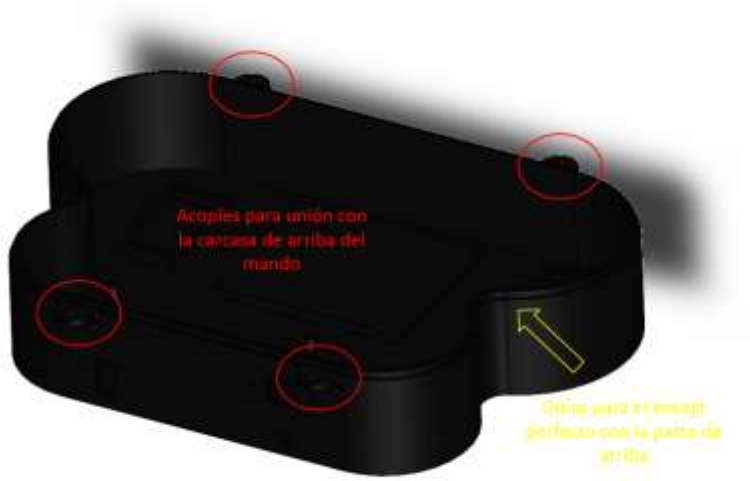


Figura 85. Acoples para la unión y guías de encaje con la parte de arriba del mando (Elaboración propia)

Segundo, una vez presentado el diseño de la parte inferior de la carcasa del mando, se va a hacer lo mismo con la parte de arriba. Como ya se ha mencionado anteriormente, la parte de arriba se ha diseñado en dos fragmentos pensando en el ahorro de tiempo en la futura fabricación 3d de los mismos.

El diseño de parte de arriba se ha realizado en dos alturas, que cada una de ellas se corresponden con cada uno de los fragmentos. El primer fragmento es donde se ubica la pantalla Nextion de 3.5 inch, el interruptor de modo autónomo, interruptor de desactivación de las funciones manuales del mando, el pulsador NA de la bocina y el selector de luces al que se le ha dado una inclinación de 18.6° para una mejor visión de la pantalla por parte del operario (Figura 86). Las dimensiones de la parte superior inclinada del mando se encuentran en el **PLANO N°10** del Capítulo 3: PLANOS.

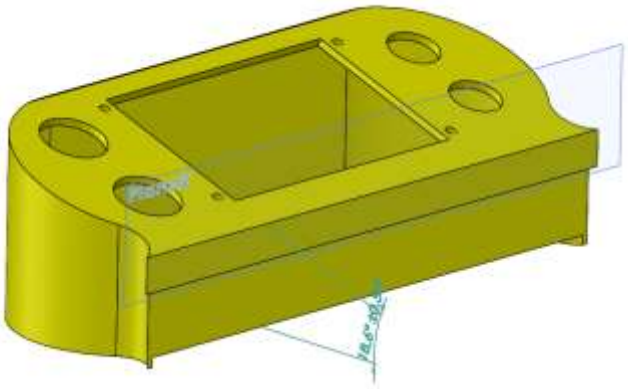


Figura 86. Fragmento inclinado de la parte de arriba de la carcasa del mando (Elaboración propia)

Por otro lado, en el fragmento plano (Figura 87) se instala el joystick, la seta de emergencia, los selectores de modo de conducción y de giro, el potenciómetro de corredera, el pulsado NC para el arranque, el interruptor de palanca para la nivelación automática y el potenciómetro de la bomba auxiliar.

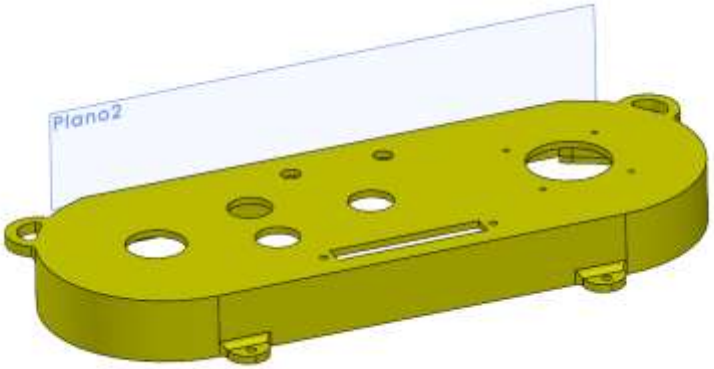


Figura 87. Fragmento plano de la parte de arriba de la carcasa del mando (Elaboración propia)

En la figura 88 se aprecia en rojo como se han incluido acoples que permiten la unión con la parte de abajo de la carcasa del mando. En negro, los agarraderos donde van introducidos los ganchos de la banda que se cuelga el operario para sustentar el mando. Además, en verde, se señalan las guías de 1.25 mm de ancho que permiten un encaje perfecto con la parte inferior del mando.

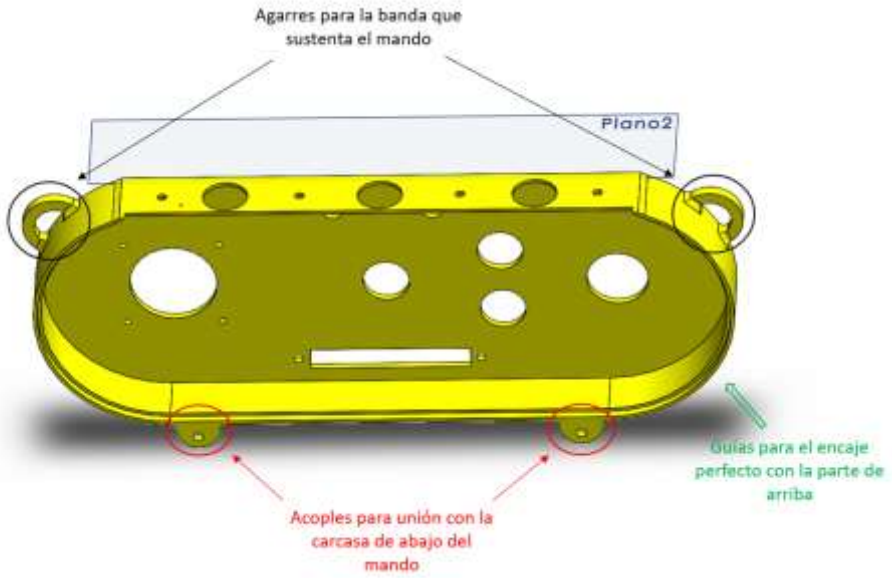


Figura 88. Acoples para la unión, guías de encaje con la parte inferior del mando y agarres para la banda que sustenta el mando (Elaboración propia)

Las dimensiones de la parte superior plana del mando se encuentran en el **PLANO N°9** del Capítulo 3: PLANOS.

En tercer lugar, se va a proceder al **ensamblaje** de los distintos componentes en sus respectivos lugares asociados de las distintas partes que componen el mando.

En la figura 89, se aprecia el ensamblaje mediante tornillería DIN 7981 de la parte inferior del mando con el microcontrolador, los tres holders que albergan las tres pilas en serie de litio 18650), el BMS y el módulo bluetooth (para estos dos últimos se han fabricado unas piezas pequeñas que pasan por encima de los módulos quedando atrapados estos en sus huecos correspondientes. Además, se ha montado el conector usb-b hembra, se ha roscado el jack 5.5 mm y se encajado el interruptor de encendido y apagado del mando (Figura 90). El ensamblaje de la parte inferior del mando se encuentra en el **PLANO N°5** recogido en el Capítulo 3: PLANOS.



Figura 89. Primer ensamblaje de la parte inferior del mando (Elaboración propia)



Figura 90. Segundo ensamblaje de la parte inferior del mando (Elaboración propia)

Para el ensamblaje de ambas partes superiores del mando se ha utilizado adhesivo. Además, se han montado los componentes destinados a estas partes, se ha fijado el joystick, el potenciómetro de corredera y la pantalla con tornillos métrica 3. Por último, se han unido ambas partes del mando (inferior y superior) utilizando esta misma tornillería (Figura 91). El ensamblaje de la parte superior plana e inclinada del mando, se encuentran en el **PLANO N°6 y PLANO N°7** recogidos en el Capítulo 3: PLANOS.



Figura 91. Mando inalámbrico totalmente ensamblado (Elaboración propia)

6.1.4 Impresión 3d del mando

Como ya se vio en el apartado de antecedentes, la impresión 3d es una de las técnicas más utilizadas para el prototipado rápido, ya que requiere una inversión inicial baja y puede ofrecer prototipos muy interesantes a partir de un diseño CAD realizado previamente. Estos son los motivos por los que se ha utilizado esta técnica de fabricación.

El tipo de impresora que se ha utilizado para la fabricación de las diferentes partes del mando inalámbrico es de tipo FDM, concretamente la Artillery Sidewinder X1, que dispone de dimensiones adecuadas (300x300x400) para imprimir las partes del mando de forma cómoda. El material que se utilizará es PLA de 1.75 mm de grosor. A pesar de no ser el material imprimible con las mejores propiedades mecánicas, es barato y suficiente para esta aplicación.

El procedimiento que se ha seguido para poder imprimir las piezas es el que se muestra en el esquema de la figura 92.

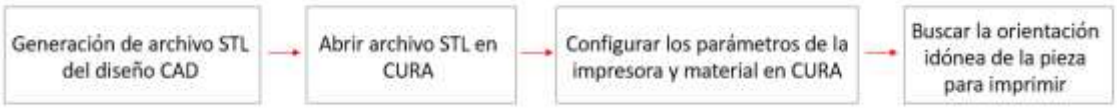


Figura 92. Esquema del procedimiento a seguir para imprimir una pieza desde un diseño CAD (Elaboración propia)

Hay que destacar que el último paso de "Buscar la orientación idónea de la pieza a imprimir" se refiere a intentar buscar la posición óptima de la pieza que ahorre el mayor tiempo de impresión y que se necesiten la menor cantidad de estructuras soportes o al menos que no tengan formas difíciles de llevarlas a la impresión. No se debe olvidar que imprimir en 3d es un arte y hay que diseñar pensando en la fabricación.

En la figura 93, se presenta distintas fases de la impresión de la parte inferior del mando inalámbrico: preview en CURA (imagen izquierda), fase inicial de impresión (imagen central) y fase final de impresión (imagen derecha).



Figura 93. Fases de impresión de la parte inferior del mando (Elaboración propia)

En la figura 94, se ha hecho lo mismo para el fragmento plano de la parte superior del mando.



Figura 94. Fases de impresión del fragmento plano de la parte de arriba del mando (Elaboración propia)

Del mismo modo, se ha hecho lo mismo para el fragmento inclinado de la parte superior del mando. En este caso, se ha comprobado que la mejor orientación de la pieza para ahorrar tiempo y soportes es colocar la parte inclinada en la cama de la impresora (Figura 95).



Figura 95. Fases de impresión del fragmento inclinado de la parte arriba del mando (Elaboración propia)

Una vez impresas todas las piezas, se han ido ensamblado en su posición exacta cada uno de los componentes del mando (Figura 96).



Figura 96. Ensamblaje de los distintos componentes en las partes impresas en 3d del mando (Elaboración propia)

6.1.5 Diseño del sistema de alimentación

En este apartado, se va a explicar el sistema de alimentación que alimenta todos los componentes electrónicos del mando (microcontrolador, pantalla, HC-05 etc).

En las primeras pruebas de envío de datos del mando se utilizó una pila de 9V no recargable (en ese momento no existía pantalla). Tras el agotamiento de varias pilas de este tipo debido al gran consumo a la hora de enviar datos, se pensó en buscar una solución al problema. Como alternativa, se optó por pilas de litio 18650 de 3.7V que gozan de una mayor capacidad (2600 mAh) que pila de 9V en proporción al tamaño y tienen la particularidad de que son recargables. Como el voltaje input recomendado para el Arduino MEGA 2560 está entre 7 a 12 V se va a tener que hacer una conexión serie de tres pilas de litio. Además, se va a tener que instalar un sistema de gestión de baterías o Battery Management System (BMS) que permita cargar y descargar las celdas hasta un límite, todas por igual, para evitar daños en las mismas. Además, se hará uso de un jack hembra 5.5 mm para la recarga de las pilas y un interruptor ON/OFF para el encendido y apagado del mando. En la figura 97, se muestra el esquema que recoge el conexionado de los

distintos componentes del sistema de alimentación del mando inalámbrico. Este esquema también se recoge con el título de "**PLANO N°3:** Esquema de conexionado del sistema de alimentación" en el Capítulo 3: PLANOS.

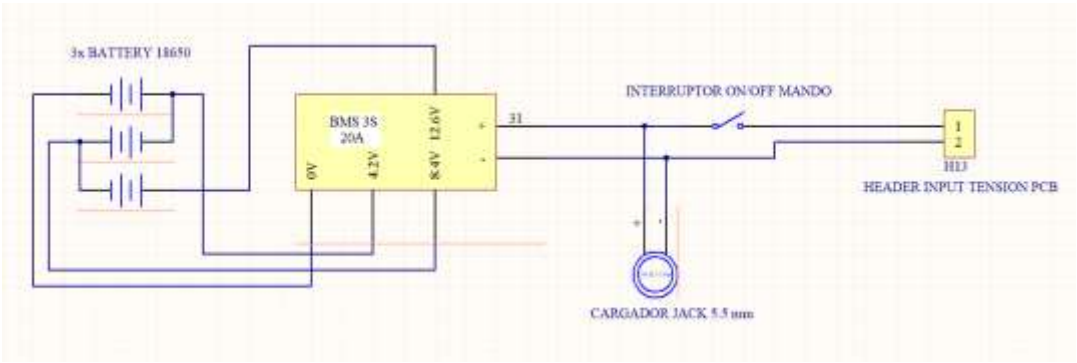


Figura 97. Esquema de conexionado del sistema de alimentación del mando (Elaboración propia)

En la figura 98, se identifican cada uno de los componentes relacionados con el sistema de alimentación mencionados anteriormente dentro de la parte inferior de la carcasa del mando.

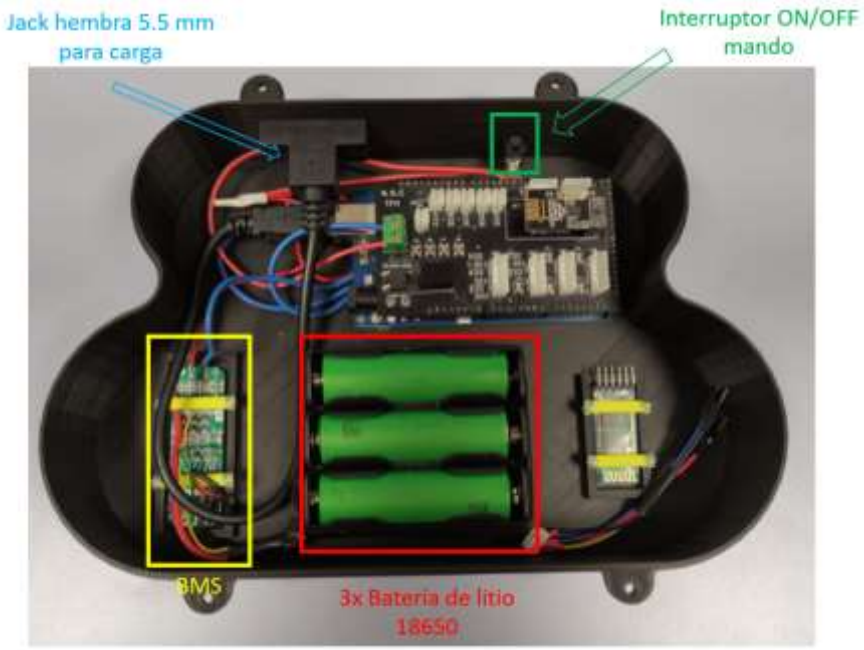


Figura 98. Identificación de los componentes del sistema de alimentación dentro de la carcasa inferior del mando (Elaboración propia)

6.1.6 Diseño de pantalla

Desde un primer momento, como se mencionó en las primeras fases de diseño, se le ha querido instalar una pantalla que indique los valores y estados de los distintos potenciómetros, interruptores, pulsadores y selectores (modos de conducción, de giro etc) que componen el mando.

Primero se probó con una pantalla LCD de 16x2, pero el número de caracteres alfanuméricos era muy limitado, necesitaba una gran cantidad de pines ya que va por conexión paralelo, no tiene color, una gran cantidad de desventajas que llevó a probar con otro modelo. El siguiente modelo a utilizar fue una pantalla OLED monocromo de 0.96 inch. Esta tenía la gran ventaja

respecto a la anterior que iba por comunicación i2c y reducía mucho el número de pines utilizados en el microcontrolador, pero tenía la gran desventaja que disponía de unas dimensiones muy reducidas y no se podía ver a golpe de vista todas las variables y estados de los componentes del mando. Seguidamente a esto, se pensó en la idea de instalar una **pantalla txt de 3.5 inch del fabricante Nextion**, concretamente el modelo NX4832K035. Este gozaba de una gran cantidad de ventajas sobre las anteriores, el tamaño era importante, tenía su propio microprocesador lo que permitirá una reducción de código en el Arduino, va a color, se programa mediante la UART, entre otras muchas otras que no nos servirán en este proyecto.

La lógica de programación de pantalla Nextion es la siguiente:

- Se crea la interfaz de la misma mediante su propio Editor Nextion
- Se carga la interfaz en la Nextion mediante un adaptador de usb a ttl.
- Se crea y se carga el programa Arduino que enlace los componentes creados en la interfaz de la pantalla con los distintos componentes físicos del mando pinchados en los pines del microcontrolador.
- Se unen los cables de la pantalla con el header de 4 pines reservado en la PCB para la pantalla. No olvidar que para que exista comunicación de datos, el TX debe ir con el RX y viceversa.

6.1.6.1 Editor Nextion

Este software ha permitido configurar cada uno de los objetos que se visualizan en la pantalla.

Para entender a manejar el editor, se ha usado una guía del editor que ofrece el propio fabricante (Nextion.tech, 2021).

En primer lugar, se ha creado un nuevo proyecto y se ha configurado el modelo de pantalla NX4832K035 y la orientación de la misma. Seguidamente, en la page 0 y page 1 se han cargado dos imágenes una del logo de la universidad y otra del vehículo a controlar remotamente (Figura 99). Imágenes que han sido cargadas previamente en el panel de recursos de imagen.

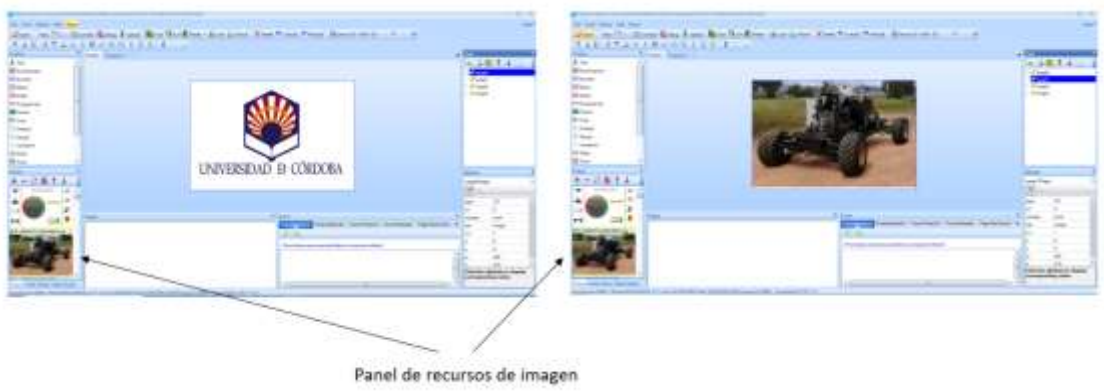


Figura 99. Page 0 y page 1 de la interfaz de la pantalla (Elaboración propia)

Posteriormente, se ha pasado a la page 2, es decir a la pantalla donde aparecerán la mayoría de los testigos y contadores de los estados y valores asociados a los distintos componentes del mando. Para crear todo esto, se ha utilizado los siguientes componentes del panel de la caja de herramientas del editor Nextion:

- **Text:** es el componente texto, tiene como nombre del objeto la letra t y un número asociado (ej: t0). A la componente de texto se le asigna una fuente previamente cargada

en el panel de recursos de fuente. Este recurso ha servido para sacar por pantalla los valores numéricos (convirtiéndoles en caracteres previamente en el Arduino) de variables como la nivelación, avance/retroceso, nivel de batería.

- **Picture:** es el componente imagen, tiene como nombre del objeto la letra p y un número asociado (ej: p0). A cada componente imagen, se le puede asociar varias imágenes cargadas en el panel de recursos de imagen previamente. Esto ha permitido que un testigo cambie al variar un estado (p.ej: si la bocina está activa aparece con un símbolo verde y si está desactivada con un símbolo rojo). Este componente, se ha utilizado para los testigos de la bocina, modos de conducción, modos de giro, modo autónomo, desactivación de las funciones del mando, nivelación automática, modos de luces y diferentes imágenes para los distintos niveles de batería.
- **Gauge:** es componente que permite simular el movimiento de una aguja a medida que cambia una variable analógica. Tiene como nombre del objeto la letra z y un número asociado (ej: z0). Este componente se ha utilizado para simular el velocímetro del acelerador del motor.

En la figura 100, se muestra como ha quedado la configuración de la page 2 de la interfaz de la pantalla.



Figura 100. Configuración de la page 2 de la interfaz de la pantalla utilizando los componentes de la barra de herramientas adecuados para componente del mando (Elaboración propia)

Cuando se active la seta de emergencia, aparecerá la page 3 (Figura 101) y cuando se desactive volverá a la page 2.



Figura 101. Page 3 de la interfaz de la pantalla (Elaboración propia)

6.1.6.2 Programación de Nextion en Arduino

Para la programación en Arduino de la Nextion se ha utilizado una librería no oficial descargada de la plataforma de desarrollo colaborativo Github.

Antes de nada, para poder utilizar esta librería en el Arduino MEGA es necesario hacer una serie de modificaciones (relacionadas con el hardware serial) que aparecen en al archivo Readme.mp de la librería.

En este apartado, no se pretende explicar cada parte del código utilizado relacionado con la pantalla sino aquellas funciones que se han utilizado en el mismo. En el "**ANEXO 1:** Código de programación del emisor" recogido en el Capítulo 5: ANEXOS se puede consultar todo el código de programación del emisor donde aparece el relacionado con la pantalla.

myNextion.Init(): esta función permite inicializar la pantalla Nextion a la page0.

myNextion.sendCommand("page 1"): esta función permite cambiar de la página que se encuentre a la página indicada.

myNextion.setComponentText("page2.t0"," MANUAL"): posibilita escribir el texto indicado (e.j: MANUAL) en el objeto indicado (t0) de la página señalada (page2).

String valorstringpotenciometro =String(valorpotenciometro): esta función a pesar de no ser propia de la librería permite pasar un valor numérico a una cadena String. Se hará uso del valor String generado en la siguiente función.

myNextion.setComponentText("page2.t0"," valorstringpotenciometro"): se trata de la misma función Nextion anterior, pero con el detalle que en vez de escribir texto tal cual, se asocia una variable String que se corresponde con un valor de un potenciómetro. Esta función permite sacar valores numéricos por pantalla utilizando una función propia de texto.

myNextion.setComponentValue("page.2. z0"," lecturaAnalógicaparaGauge"): esta función permite leer un valor analógico y reproducirlo en la componente Gauge definida en el Editor de Nextion.

myNextion.setPictureValue("page2.p6",23): se trata de una función que se le ha tenido que añadir a la librería, teniendo en cuenta que se trata de lenguaje orientado a objetos. Esta función permite asociar a las distintas componentes imagen (e.j p6) las distintas imagen cargadas en el panel de recursos de imagen, ya que cada una de estas tienen su propia id asociada (e.j 23), posibilitando que un testigo cambie al variar un estado (p.ej: si la bocina está activa aparece con un símbolo verde y si está desactivada con un símbolo rojo).

6.1.7 Extras

Se ha decidido dedicar un apartado a ciertos extras que se le ha incorporado al mando, es decir, se le ha incluido un sensor de batería y un módulo de conexión bluetooth que le dan valor añadido al mando.

6.1.7.1 Sensor de Batería

El objetivo de este sensor es monitorizar en cada momento el nivel de batería del mando pudiendo así cargarlo cuando sea necesario y tenerlo siempre operativo.

Este sensor se ha realizado diseñando un divisor de tensión dentro de la PCB que utiliza dos resistencias comerciales, una de 2.2kΩ y otra de 1kΩ. Como entrada al divisor se une la tensión

proveniente del BMS de las baterías y como salida se le ha asociado un pin analógico del Arduino (A0) para que pueda leer tensión (Figura 102).

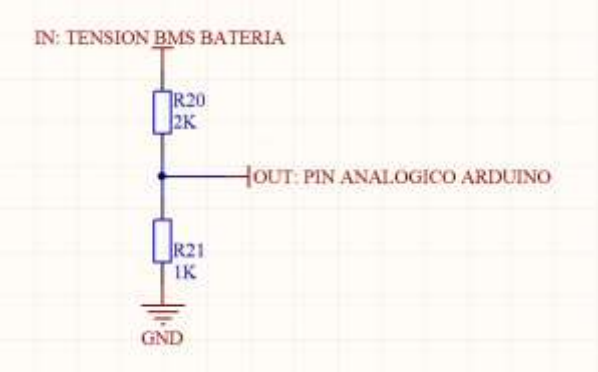


Figura 102. Esquema del sensor de batería (Elaboración propia)

Mediante la programación de la Ecuación 1 se permite leer el voltaje exacto al que se encuentran las baterías.

$$Level\ Batteries = \left(\left(\frac{ValuePinA0\ x\ 5}{1023}\right) x(3.208)\right) + 1.35 \qquad (1)$$

De este modo, mediante la adecuada programación de la pantalla Nextion y el Arduino, se puede conocer en cada momento el porcentaje en pantalla que queda disponible en cada momento (Figura 103). En el "**ANEXO 1:** Código de programación del emisor" recogido en el Capítulo 5: ANEXOS se puede consultar todo el código de programación del emisor donde aparece el relacionado con el sensor de batería.



Figura 103. Nivel del batería del mando al 91% (Elaboración propia)

6.1.7.2 Módulo de conexión bluetooth

En este apartado, se va a explicar otro extra que se le ha incorporado al mando inalámbrico, concretamente el módulo HC-05 que va a permitir que el mando emisor disponga de conectividad bluetooth. El objetivo de este extra es dotar al mando de una tecnología de Información y Comunicación (TIC) muy utilizada hoy en día por su versatilidad.

Como se observa en la figura 104, el módulo se ha incorporado en el lado derecho de la carcasa inferior del mando y se ha conectado de acuerdo al "**PLANO N°1:** Esquema de conexionado de los componentes electrónicos del emisor" recogido en el Capítulo 3: PLANOS.

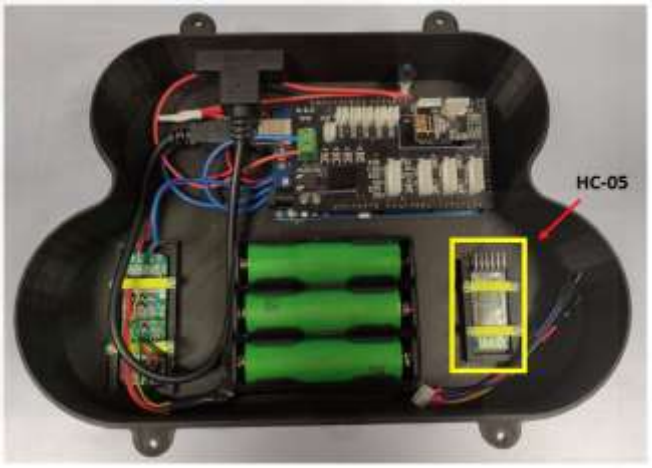


Figura 104. Incorporación del módulo bluetooth HC-05 con el objetivo de incorporar TIC´s en el vehículo (Elaboración propia)

Como prueba de esta TIC, se ha descargado la App BlueTooth Electronics donde se ha creado un panel de visualización para monitorizar a tiempo real los valores de algunas funciones específicas del emisor tales como si el vehículo va hacia adelante o hacia atrás, si gira derecha o izquierda, cuantos grados se encuentra nivelado o a que revoluciones se encuentra el motor (Figura 105). En el apartado *9. Líneas futuras*, se propondrán otras futuras aplicaciones útiles de esta tecnología para el vehículo.

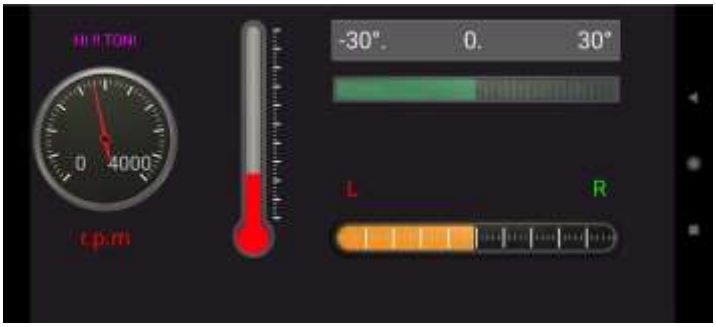


Figura 105. Utilización de App Bluetooth Electronics para la monitorización en tiempo real de algunas variables de funciones del vehículo (Elaboración propia)

6.2 Receptor

El objetivo del receptor es recibir paquetes de datos provenientes del emisor por vía radiofrecuencia (utilizando el módulo NRF24L01) y seguidamente, adecuar los datos recibidos al protocolo bus CAN (usando el módulo mcp2515) para poder interactuar con el PLC que gestiona el vehículo mediante este protocolo. De esta forma, el receptor solo cuenta con dos terminales de salida, el CAN_H y el CAN_L que irán conectados bus CAN del vehículo.

Las tareas objetivo vinculadas al receptor presentadas al principio del capítulo 6 son las que se muestran a continuación:

- Programación en C++ del receptor y prototipado electrónico
- Diseño del receptor sobre protoboard soldable

6.2.1 Programación y prototipado electrónico del receptor

En este capítulo, se va a abordar como se ha programado el receptor mediante lenguaje C++ haciendo uso del entorno de desarrollo PlatformIO, al igual que se hizo con el emisor.

6.2.1.1 Tramas CAN BUS del receptor

Se comenzará definiendo las tramas que el receptor va a recibir del emisor y las que enviará al bus CAN del vehículo (Tabla 11). Pero antes de definir las mismas, se deben tener en cuenta las siguientes **consideraciones**:

1. El receptor debe convertir a CAN todas las señales que le lleguen del emisor sin alterar ni procesar ningún dato. El emisor debe ser el encargado de filtrar y procesar los datos antes de enviarlos. Por lo tanto, el receptor será totalmente transparente en cuanto al envío de datos por el bus CAN.
2. Se han utilizado números de ID más bajos para las funciones más prioritarias
3. Si se sobrepasa el tiempo de espera de la señal "Estoy Vivo", se presiona la seta de emergencia o se desactivan las funciones del control inalámbrico, el receptor debe enviar una trama con ID 11 (trama error) que será leída más tarde por el módulo WatchDog instalado en el bus CAN del vehículo. El módulo WatchDog es un dispositivo encargado de comprobar constantemente las tramas enviadas al bus CAN del vehículo y provocará la parada de este, cuando detecte un tipo de trama específica. Solo el módulo WatchDog puede liberar por completo el vehículo de nuevo al comprobar que no existen errores en todo el conjunto del sistema. En este sentido, es necesario cuando se libere la seta de emergencia, enviar una trama de limpieza de errores.

Tabla 11. Definición de las Tramas CAN BUS que recibirá el receptor (Elaboración propia)

ID CAN	B0	B1	B2	B3	B4	B5	B6	B7	Aclaraciones
101	X	x	x	x	x	0	0	0	Todas las tramas de las distintas funciones del emisor. El Byte 0 de la tabla 11, se corresponde con el Byte 4 definido en la tabla 7
11	0	0	0	0	0	0	0	0	Limpieza de errores en el sistema. Indica que todos los sistemas del módulo inalámbrico funcionan correctamente.
11	255	0	0	0	0	0	0	0	Trama de Error – Se ha presionado la "seta" de emergencia.
11	1	B0	B1	B2	B3	0	0	0	Trama de Error que indica que se sobrepasó el tiempo de espera para recibir la señal "Estoy Vivo" y que por lo tanto se ha perdido la comunicación con el emisor. B0-B3 – Indica el tiempo máximo en milisegundos establecido.
11	2	255	0	0	0	0	0	0	Trama de Error. El control inalámbrico está desactivado.

6.2.1.2 Programación del receptor

En primer lugar, una vez definidas las tramas, se va a explicar las librerías que se han incluido en el sketch del receptor. En la figura 106, se muestra como se ha hecho uso de cuatro librerías

distintas. La primera es Arduino.h que permite trabajar en PlatformIO de la misma forma que con el IDE de Arduino pero de un forma mucho más clara y esquematizada. La segunda es SPI.h, esta permite que exista comunicación (envió y recibo de datos) entre el Arduino y el NRF24L01 por el bus SPI. Al igual que el emisor, también se ha utilizado RF24.h (incluye a nRf24L01.h y RF24_config.h) que se encarga de gestionar el módulo de radiofrecuencia utilizado (NRF24L01). Por último, se ha incluido la librería mcp2515.h, que permite el envío de datos mediante protocolo CAN BUS.

```
#include <Arduino.h>
#include <SPI.h>
#include <RF24.h>
#include <nRF24L01.h>
#include <RF24_config.h>
#include <mcp2515.h>
```

Figura 106. Librerías utilizadas en el código del receptor (Elaboración propia)

Seguidamente, se va a definir aquellas funciones de la librería RF24.h que no se definieron para el emisor, pero que si son necesarias en el receptor para que exista intercambio de datos entre ambos:

- **radio.openReadingPipe(dirección):** Abre un canal de comunicación de lectura. Como ya se vio en el emisor, la dirección es una matriz de 5 byte de longitud. Pueden existir 6 canales de lectura distintos (Figura 107).

```
//Variable con la dirección del canal por donde se va a transmitir
byte RFPipe[5] ={0x60, 0x59, 0x70, 0x54, 0x01};

void setup() {
  radio.openReadingPipe(1, RFPipe);
```

Figura 107. Canal de comunicación de lectura del NRF24L01 (Elaboración propia)

- **radio.startListening ():** esta función permite empezar a escuchar los canales definidos como lectura. Se debe tener en cuenta, que, tras llamar esta función, no podemos hacer escrituras (Figura 108).

```
void setup() {

  //Inicializamos el módulo de RF
  radio.begin();
  radio.openReadingPipe(1, RFPipe);
  radio.startListening();
```

Figura 108. Función que permite escuchar los canales definidos como lectura (Elaboración propia)

- **radio.available ():** esta función permite comprobar si existen datos disponibles en el canal de escritura para ser leídos.
- **radio.read(datos, sizeof(datos)):** esta función posibilita la leer datos existentes en el canal de lectura (Figura 109).

```
if (radio.available() ) {           //Comprobamos si hay datos que leer

  radio.read(datos,sizeof(datos)); //Leemos los datos disponibles
```

Figura 109. Funciones que permiten comprobar si existen datos en el canal de escritura y leerlos en caso afirmativo (Elaboración propia)

De la misma forma, se va a explicar algunas las funciones más importantes utilizadas de la librería mcp2515.h necesaria para que exista comunicación mediante protocolo CAN BUS entre el receptor y el PLC del vehículo. Estas funciones son las siguientes:

- **MCP2515 mcp2515(PinCS_MCP2515):** es la función constructor, crea un nuevo objeto (instancia) del mcp2515. La instancia se establece con el pin Select (CS) del módulo mcp2515 (Figura 110).

```
const uint8_t PinCS_MCP2515 = 8;
MCP2515 mcp2515(PinCS_MCP2515);
```

Figura 110. Creación del objeto para el mcp2515 (Elaboración propia)

- **mcp2515.setBitrate(canspeed):** esta función permite regular la velocidad de transmisión del bus CAN. La librería permite configurar un gran número de velocidades de transmisión, destacando la de 250 KBPS y 500 KBPS utilizadas en las pruebas de este proyecto (Figura 111).

```
const CAN_SPEED canspeed = CAN_500KBPS;

void setup() {

  mcp2515.reset();
  mcp2515.setBitrate(canspeed);
```

Figura 111. Configuración de la velocidad de transmisión del bus CAN (Elaboración propia)

- **mcp2515.setXMode():** esta función permite configurar los distintos modos de trabajo que incorpora la librería del módulo, donde cada uno de ellos está programado para que haga una tarea distinta. Entre los que se han probado durante los ensayos se encuentran: setNormalMode, setListenOnlyMode, setSleepMode (Figura 112).

```
void setup() {

  mcp2515.reset();
  mcp2515.setBitrate(canspeed);
  mcp2515.setNormalMode();
```

Figura 112. Configuración del modo de trabajo del mcp2515 (Elaboración propia)

- **canMsg.can_id = data:** permite definir la ID de la trama que identifica a la misma. La propia librería comprueba si se trata de una ID Estándar (identificador de 11 bits) o una ID Extendida (identificador de 29 bits).
- **canMsg.can_dlc=X:** mediante el uso de la misma, se fija el número de bytes del campo de datos de las tramas CAN definidas.
- **canMsg.data[X]= data[X]:** se trata de los mensajes asignados a las distintas funcionalidades del mando, entre otros mensajes.
- **mcp2515.sendMessage(&canMsg):** esta función envía la trama completa, es decir el identificador, la longitud de la trama, así como la información de la misma (Figura 113).

```
canMsg.can_id =* (unsigned long*)(&array_id);

canMsg.can_dlc = 8;

canMsg.data[0] = data[4];
canMsg.data[1] = data[5];
canMsg.data[2] = data[6];
canMsg.data[3] = data[7];
canMsg.data[4] = data[8];
canMsg.data[5] = data[9];
canMsg.data[6] = data[10];
canMsg.data[7] = data[11];
mcp2515.sendMessage(&canMsg);
```

Figura 113. Configuración de la id, longitud e información de una trama CAN (Elaboración propia)

En segundo lugar, se van a explicar detalladamente cada una de las funciones utilizadas en el sketch del receptor. Todas estas funciones deben ser declaradas previamente justo antes del set up().

La primera función por justificar es **LeerDatosRF**, esta función permite leer los datos provenientes del emisor vía radio (Figura 114). De la siguiente forma, quedaría estructurada esta función:

- Se declara el array de bytes que ocupa una trama CAN
- Se comprueba por medio de la función radio.available() de la librería RF24.h si existen datos disponibles por leer y en caso de que existan, se procede a su lectura mediante la función radio.read(datos,sizeof(datos)).
- Mediante el uso de un condicional if se verifica si el primer byte (datos [0]) de las tramas recibidas es distinto de doscientos cincuenta y cinco , es decir, si se trata de un dato interno entre el emisor y el receptor o no. En caso de no ser un dato interno, se envía el array de bytes a el bus CAN, comprobando antes que el módulo se encuentra habilitado (función **CheckEnable**) y en caso contrario, se identifica que tipo de dato interno se trata, es decir, se comprueba si se activado o desactivado la seta de emergencia o el control del mando o se verifica si se ha perdido la señal de control. Dependiendo de una situación u otra, se le asignarán un estado u otro a variables previamente programadas antes del set up ().

º

```
void LeerDatosRF() {
  uint8_t datos[12];
  if (radio.available() ) {          //Comprobamos si hay datos que leer

    radio.read(datos,sizeof(datos));  //Leemos los datos disponibles

    //Si el primer dato es distinto a 255 se trata de una trama CAN y debe ser enviada a traves del módulo CAN.
    if (datos[0] != 255) {

      //Solo enviamos ordenes si el modulo está habilitado
      if (CheckEnable()) SendCAN(datos);
    }
    else {

      //Si el primer dato es 255 se trata de unos datos internos y deben ser procesados según corresponda.

      if (datos[0] == 255 && datos [1] == 0 && datos[2] == 0 && datos[3] == 1) {
          //Es la señal de Estoy Vivo
          LastTimeEstoyVivo = millis(); //Establecemos la hora actual como la ultima a la que se recibio la señal
      }
      else if (datos[0] == 255 && datos [1] == 0 && datos[2] == 0 && datos[3] == 2) {
          //Activacion / Desactivacion control inalambrico
          if (datos[4] == 255) errorControlDesactivado = true;
          else errorControlDesactivado = false;
      }
      else if (datos[0] == 255 && datos [1] == 255 && datos[2] == 255 && datos[3] == 255) {
          //Seta de emergencia
          if (datos[4] == 255) errorSetaEmergencia = true;
          else errorSetaEmergencia = false;
      }
    }
  }
}
```

Figura 114. Función LeerDatosRF que permite comprobar si existen datos disponibles enviados por el emisor y analizar si se trata de un tipo de dato interno o no (Elaboración propia)

La segunda función por analizar es **SendCAN**, esta función posibilita el envío de tramas al bus CAN del vehículo haciendo uso de las funciones descritas anteriormente de la librería mcp2515.h . Se debe tener en cuenta, que la función canMsg.can_id (el identificador de trama) trabaja con variables **unsigned long**, es decir, variables largas sin signo que ocupan 32 bits (4 bytes) y que su rango va de 0 a $2^{32} - 1$. De esta manera, se deben convertir los 4 bytes correspondiente a la identificación de trama recibidos vía radio a una variable unsigned long de 32 bits (Figura 115).

```
void SendCAN(byte * data) {

  struct can_frame canMsg;

  byte array_id[4];

  array_id[0] = data[3];
  array_id[1] = data[2];
  array_id[2] = data[1];
  array_id[3] = data[0];

  canMsg.can_id =* (unsigned long*)(&array_id);

  canMsg.can_dlc = 8;

  canMsg.data[0] = data[4];
  canMsg.data[1] = data[5];
  canMsg.data[2] = data[6];
  canMsg.data[3] = data[7];
  canMsg.data[4] = data[8];
  canMsg.data[5] = data[9];
  canMsg.data[6] = data[10];
  canMsg.data[7] = data[11];

  mcp2515.sendMessage(&canMsg);
}
```

Figura 115. Función SendCAN permite enviar los datos provenientes del emisor al bus CAN del vehículo (Elaboración propia)

La tercera función a estudiar es **EnviarErrores**, esta función comprueba si se está dando algún tipo de error y le envía al bus CAN una orden correspondiente (Figura 116). Se pueden dar cuatro situaciones distintas:

1. Que esté presionada la seta de emergencia. En este caso, se le llama a la función **EnviarErrorSetaEmergencia** que contiene la trama de error relacionada con la activación de seta de emergencia (Tabla 11).
2. Que esté desactivado el control inalámbrico. En consecuencia, se hará uso de la función **EnviarErrorControlDesactivado** que contiene la trama de error relacionada con la desactivación del control inalámbrico (Tabla 11).
3. Que se haya superado límite de espera para recibir la señal de estoy vivo enviada por el emisor. De esta manera, se utiliza la función **EnviarErrorEstoyVivo** que contiene la trama de error relacionada con que se sobrepasó de tiempo de espera para recibir la señal “Estoy Vivo” y que por lo tanto se ha perdido la comunicación con el emisor. Dentro de esta función, se debe de hacer uso de la función **SplitULong** que permite convertir una variable unsigned log de 32 bits en cuatro variables independientes de un solo byte (Figura 117). Esta conversión es necesaria para poder enviar los datos correctamente de acuerdo al protocolo CAN BUS.
4. Que no se dé ninguno de los errores anteriores. En este sentido, cada cierto periodo (valor de la variable periodNoErrors) se envía la función **EnviarNoErrores** que contiene la trama de limpieza de errores que indica que todos los sistemas del módulo inalámbrico funcionan correctamente (Tabla 11).

En el "**ANEXO 2:** Código de programación del receptor" recogido en el Capítulo 5: ANEXOS se puede consultar todas las funciones no detalladas anteriormente.

```
void EnviarErrores() {

  //Si hay error en la seta de emergencia (está presionada) se envia por CAN
  if (errorSetaEmergencia) {
    EnviarErrorSetaEmergencia();
  }
  //Si hay error en la activación del control inalambrico (está desactivado) se envia por CAN
  if (errorControlDesactivado) {
    EnviarErrorControlDesactivado();
  }
  //Si se ha superado el tiempo limite de espera para recibir una señal de Estoy Vivo, se envia por CAN
  if (errorEstoyVivoTimeLimit) {
    EnviarErrorEstoyVivo();
  }
  //Si no hay errores se envia por CAN para indicar que el modulo receptor funciona correctamente
  if (!errorControlDesactivado && !errorEstoyVivoTimeLimit && !errorSetaEmergencia) {

    //Comprobamos el tiempo desde el ultimo envio. Solo se envia cada cierto periodo para evitar saturar el canal.
    if ((millis() - LastTimeNoErrors) >= periodNoErrors) {
      EnviarNoErrores();
      LastTimeNoErrors = millis();
    }
  }
}
```

Figura 116. Función EnviarErrores, comprueba si se está danto algún tipo de error y consecuentemente envía una respuesta al bus CAN de si existe algún tipo de error presente (Elaboración propia)

```
void SplitULong(unsigned long value, uint8_t *b0, uint8_t *b1, uint8_t *b2, uint8_t *b3) {

    uint8_t len = sizeof(unsigned long);
    uint8_t data[len];
    memcpy(data, &value, len);

    *b0 = data[0];
    *b1 = data[1];
    *b2 = data[2];
    *b3 = data[3];
}
```

Figura 117. Función SplitULong, permite convertir una variable tipo unsigned long de 32 bits a cuatro variables de un solo byte (Elaboración propia)

En último lugar, se describe la programación del **set up ()** y del **loop ()** del receptor (Figura 118). En el set up() se llaman a algunas funciones respectivas a las librerías RF24.h y mcp2515.h así como la declaración de alguna variable digital como entrada (PinEnable) y la inicialización de variables como LastTimeEstoyVivo atribuyéndole la función millis (). Por otro lado, en el loop (), se leen y se procesan los datos que existen disponibles por el canal de RF llamando a la función LeerDatosRF y se comprueba el tiempo desde la última señal de EstoyVivo enviada por el emisor y por último se envía cualquier tipo de error haciendo uso de la función Enviar Errores en caso de que existan los mismos.

```
void setup() {

  //Inicializamos el módulo de RF
  radio.begin();
  radio.openReadingPipe(1, RFPipe);
  radio.startListening();
  //Inicializamos el modulo CAN
  mcp2515.reset();
  mcp2515.setBitrate(canspeed);
  mcp2515.setNormalMode();
  pinMode(PinEnable, INPUT);
  LastTimeEstoyVivo = millis();
}
void loop() {

  //Se leen y procesan los datos que existan disponibles por el canal de RF
  LeerDatosRF();
  //Leemos los datos disponibles en CAN y los procesamos
  LeerDatosCAN();
  //Se comprueba el tiempo desde la ultima señal EstoyVivo y en caso necesario envia un error.
  if (!CheckTimeEstoyVivo()) errorEstoyVivoTimeLimit = true;
  else errorEstoyVivoTimeLimit = false;
  //Se envian los errores si los hay
  EnviarErrores();
}
```

Figura 118. Programación de la función set up() y loop() del receptor (Elaboración propia)

6.2.1.3 Resultados del prototipo receptor

En la figura 119, se muestran el resultado del prototipo electrónico del receptor, donde se aprecia en azul el módulo de radiofrecuencia NRF24L01, en rojo el módulo mcp2515 que permite la comunicación CAN BUS y en verde el microcontrolador Arduino Nano. Además, al tratarse de un prototipo inicial todos los componentes electrónicos están pinchados sobre protoboard y ninguno de ellos soldado.

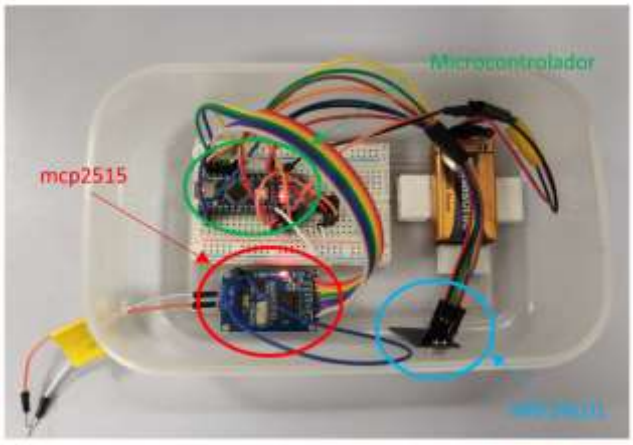


Figura 119. Componentes electrónicos pinchados en protoboard del prototipo receptor (Elaboración propia)

6.2.2 Diseño de receptor sobre protoboard soldable

Al igual que ocurría con el prototipo inicial del emisor, los cables pinchados directamente en protoboard tienen una alta probabilidad de soltarse y dejar inoperativo el receptor. Es por este motivo por el que se ha decido diseñar un receptor sobre una protoboard soldable lo que permite una mayor fiabilidad en cuanto operatividad frente al prototipo del apartado *6.2.1.3 Resultados del prototipo receptor*.

Para evitar equivocaciones a la hora de soldar los componentes electrónicos en la protoboard, se ha realizado mediante el software Altium un esquema electrónico del receptor. Este esquema se recoge con el título de "**PLANO N°2:** Esquema de conexionado de los componentes electrónicos del receptor" en el Capítulo 3: PLANOS". En la figura 120, se puede apreciar cómo se han llevado a cabo las labores de soldadura del receptor en la protoboard soldable.

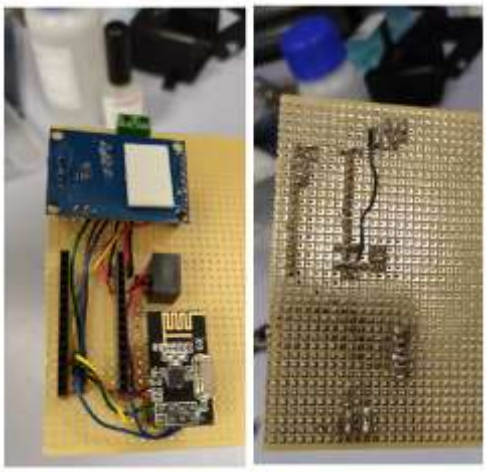


Figura 120. Labores de soldadura de los componentes electrónicos del receptor en protoboard soldable (Elaboración propia)

Una vez terminada la placa soldada se ha montado sobre una caja para carril DIN con el objetivo de fijarla adecuadamente al armario eléctrico del vehículo (Figura 121).



Figura 121. Placa soldada del receptor incorporada en caja para carril DIN (Elaboración propia)

6.3 Robot de simulación

Una vez diseñado y desarrollado el emisor y el receptor, es decir, el control inalámbrico por completo, es necesario realizar pruebas en laboratorio para validar que el sistema funciona y de esta manera poderlo implementar a posteriori en el vehículo real. Es por este motivo, por el que se ha diseñado y desarrollado un robot de laboratorio que simula todos los movimientos y funciones del vehículo real cuando se emite una orden desde el mando emisor. De esta manera el objetivo de esta simulación es asegurar que las tramas CAN BUS llegan al robot de forma correcta y a una velocidad considerable (Figura 122).

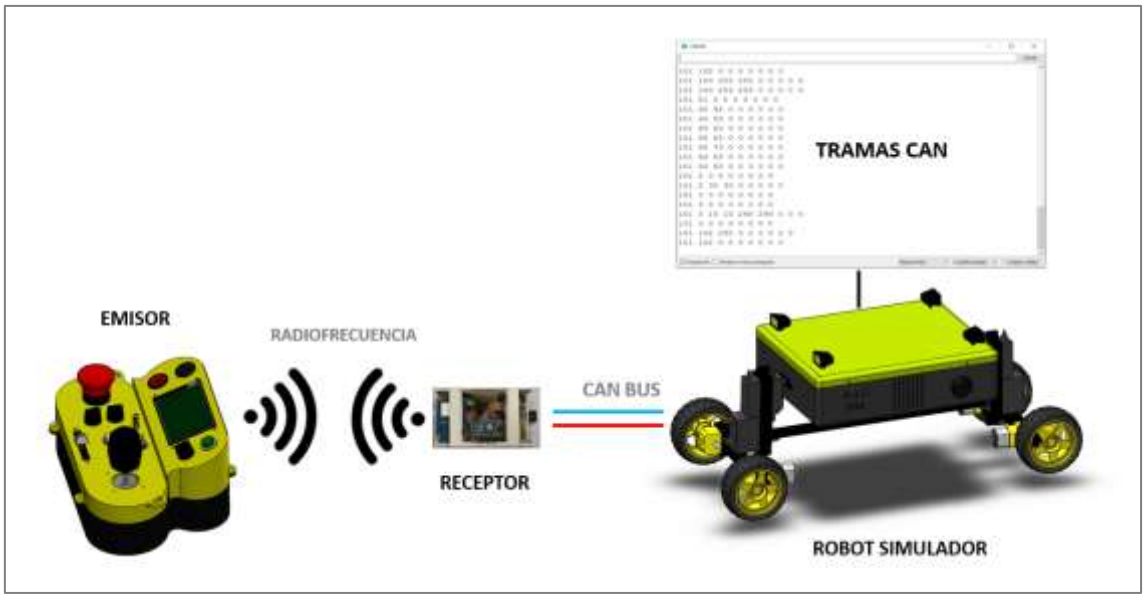


Figura 122. El objetivo de la simulación con robot en laboratorio: aseguramiento de la llegada de tramas CAN BUS (Elaboración propia)

Para la construcción del robot de simulación, se deben conocer la lista de componentes electrónicos que se han utilizado para poder realizar todas aquellas órdenes emitidas por el mando emisor, es decir, todas aquellas funcionalidades descritas en el apartado *6.1.1 Programación y prototipado electrónico del emisor* (Tabla 12).

Tabla 12. Componentes electrónicos del robot simulador (Elaboración propia)

Componentes	Función	Figura
4x Motor DC 5V	Permiten la traslación en ambos sentidos del robot	
1x Controlador de motor DC L298N	Mediante dos puentes H se permite controlar la velocidad y la dirección de los motores DC	
5x Servomotor 6V	Permiten la dirección (los tres modos de giro) y nivelación del robot. También van a permitir simular el apero del vehículo.	
2x Regulador de tensión hasta 8A	El objetivo de este componente es convertir los 12V del sistema alimentación a los 5V requeridos para los motores DC y a 6V para los servomotores. Es necesario este componente, ya que el	

	Arduino tiene un límite de salida de corriente y no satisface la demanda de estos componentes	
Inclinómetro de 3 ejes GY-291	Posibilita la nivelación automática del robot	
Buzzer Activo 5V	Permite simular la bocina del vehículo	
Lámparas RC	Van a simular los distintos modos de luces programados en el emisor	

Adicional a los componentes mencionados en la tabla 11 se ha utilizado un Arduino MEGA 2560 para el control del robot, el módulo mcp2515 para que el microcontrolador pueda recibir los mensajes CAN del receptor, el receptor presentado en el apartado *6.2.2 Diseño de receptor sobre protoboard soldable* y un sistema de alimentación similar al del apartado *6.1.5 Diseño del sistema de alimentación*.

En la figura 123, se aprecia como una vez que se adquirieron todos los componentes electrónicos, estos se unieron utilizando una protoboard teniendo en cuenta el "**PLANO N°4:** Esquema de conexionado de los componentes electrónicos del robot" recogido en el Capítulo 3: PLANOS".

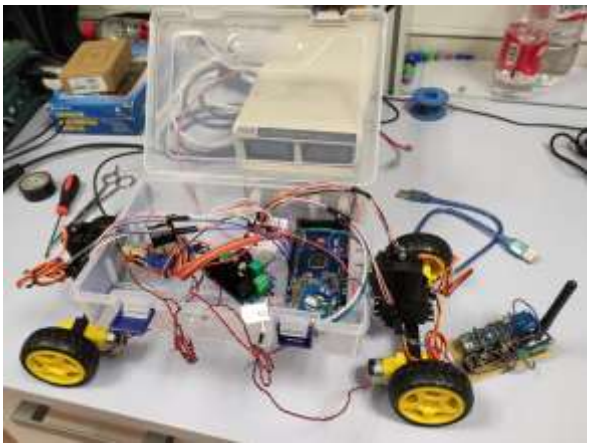


Figura 123. Componentes electrónicos del robot pinchados sobre protoboard (Elaboración propia)

Al igual que se ha hecho con los componentes de receptor, se ha diseñado y soldado una placa electrónica con el objetivo de evitar que se suelte el cableado y proporcionar así una mayor fiabilidad al robot. En la figura 124, se aprecia como se ha ordenado el cableado del robot y se ha adoptado una disposición optimizada de los componentes electrónicos.

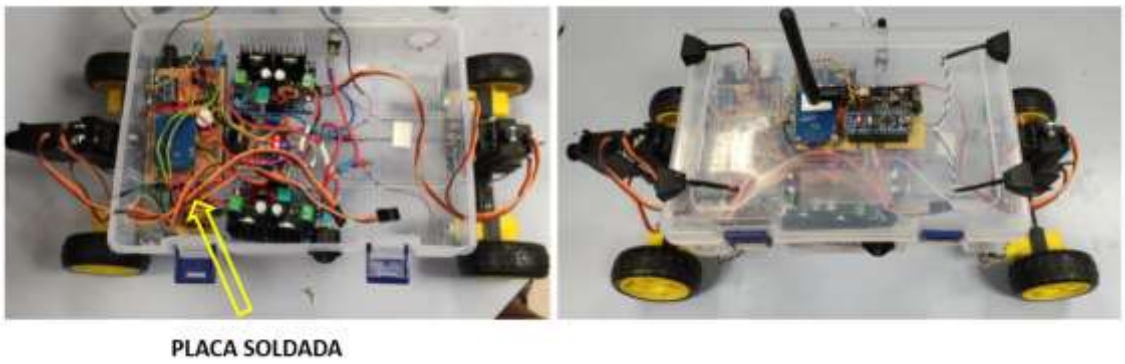


Figura 124. Diseño de placa en protoboard soldable, ordenación de cableado y disposición optimizada de los componentes electrónicos (Elaboración propia)

Una vez obtenida la disposición optimizada de todos los componentes electrónicos del robot, se ha realizado un diseño CAD del caparazón inferior y superior donde van alojados todos los componentes electrónicos con el objetivo de sustituir la caja de plástico utilizada hasta el momento, los ejes de dirección y los elementos de unión entre los servos de nivelación y los destinados a la dirección del vehículo (Figura 125). Las dimensiones de las piezas que componente al robot, así como su ensamblaje, se encuentran desde el **PLANO 11 al PLANO 15** dentro del Capítulo 3: PLANOS.

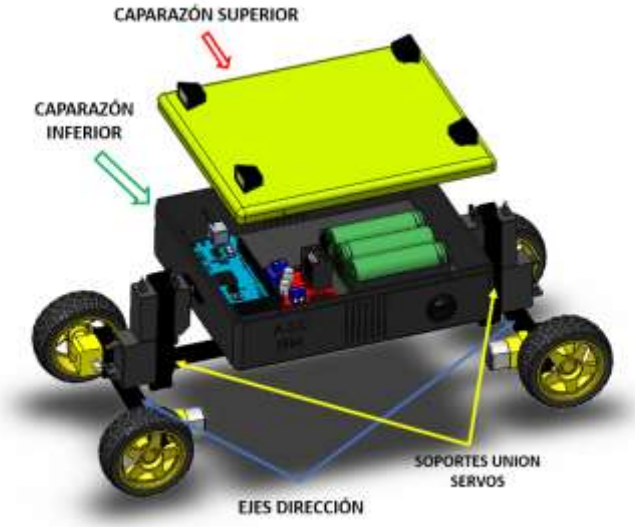


Figura 125. Diseño CAD del caparazón inferior y superior donde se alojan los componentes electrónicos del robot, ejes de dirección y elementos de unión de los servos de nivelación y dirección (Elaboración propia)

Todos estos archivos CAD se han guardado en formato .stl y mediante el software Ultimaker CURA se ha pasado a código máquina (GCode) con el objetivo de poder fabricarlos mediante impresión 3D. En la figura 126, se muestra el ensamblaje de todas piezas impresas en 3D junto con todos los componentes electrónicos del robot simulador.



Figura 126. Ensamblaje de las piezas impresas en 3D junto con todos los componentes electrónicos del robot (Elaboración propia)

Una vez diseñado y ensambladas todas las piezas del robot, se ha procedido a la programación del mismo. Al igual que el emisor y el receptor, el robot se ha programado en el entorno de desarrollo PlatformIO en lenguaje C++. El objetivo principal del código es leer las tramas CAN BUS que le llegan del receptor y transformarlas en el movimiento de los motores DC, servos o en la activación de bocina etc. En resumen, en hacer una demostración práctica de las distintas órdenes emitidas por el emisor. En el "**ANEXO 3:** Código de programación del robot simulador" recogido en el Capítulo 5: ANEXOS se puede consultar todo el código de programación del robot.

El siguiente paso, una vez programado, ha sido realizar pruebas en laboratorio. Se han ensayado con éxito todas las funciones definidas para el mando, observando que le llega su correspondiente trama CAN BUS al microcontrolador del robot y también se ha comprobado que el robot responde con el movimiento o acción física asociada a la trama CAN BUS que le ha llegado. En la figura 127, se observa cómo se ha realizado un ensayo de nivelación donde se puede leer la trama de nivelación correspondiente definidas en el apartado *6.1.1.2 Tramas Bus CAN emisor*.



Figura 127. Pruebas en laboratorio de la lectura correcta de trama CAN BUS y la correspondiente acción programada del robot ante la llegada de la trama (Elaboración propia)

7. Resultados finales

Una vez probado el sistema de control remoto en el laboratorio, se ha procedido a probarlo en el vehículo real ubicado en el taller de mecanización del departamento de Ingeniería Rural de la ETSIAM. Puesto que el vehículo es un prototipo reciente, no dispone de todas las funciones programadas en el emisor. A continuación, se describen los movimientos físicos y acciones permitidas hasta ahora en el vehículo:

- **Traslación:** permite tanto la traslación hacia adelante y como hacia atrás, gracias a los motores rueda hidrostáticos instalados en las ruedas delanteras. A estos motores hidráulicos les suministra la presión necesaria una bomba hidráulica solidaria al motor propulsor del vehículo. Esta bomba incorpora un par de electroválvulas conectadas al PLC, que van a permitir el control del aceite a presión necesario para poner en funcionamiento los motores rueda (Figura 128).
- **Dirección:** hasta ahora solo existe dirección delantera que es posible gracias a un cilindro hidráulico instalado en una rueda y una barra rígida que copia el movimiento del cilindro y se lo transmite a la otra rueda.
- **Nivelación manual:** permite la nivelación hacia la derecha y hacia la izquierda gracias a cuatro cilindros hidráulicos que actúan de en pares.
  El aceite a presión necesario para mover los cilindros encargados de la dirección y la nivelación es gobernado gracias a un distribuidor hidráulico que incorpora electroválvulas. Al igual que en la traslación, estas electroválvulas están conectadas PLC, permitiendo este el control de las mismas. La presión necesaria para mover los cilindros hidráulicos es suministrada por otra bomba hidrostática solidaria al motor del vehículo y en serie a la bomba utilizada para los motores rueda (Figura 128).
- **Nivelación automática:** es posible gracias al uso de un inclinómetro y su respectiva programación el el PLC del vehículo.
- **Seta de emergencia:** dispone de Seta de Emergencia física y su respectiva acción programada en el PLC del vehículo.

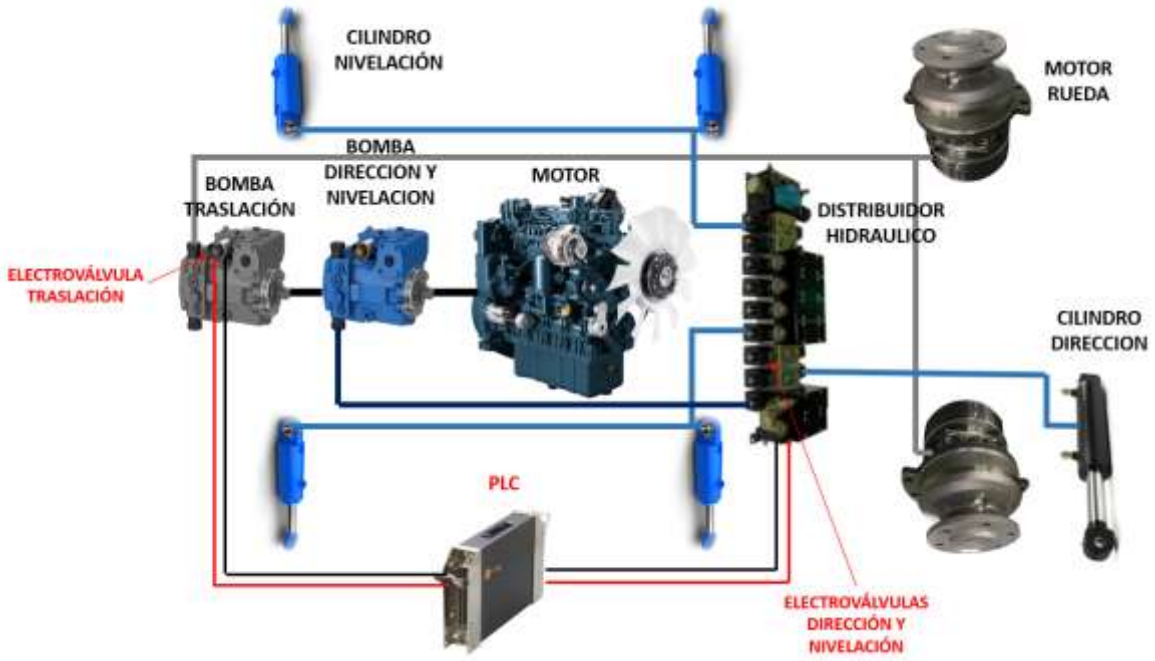


Figura 128. Esquema de conexión de los actuadores y electroválvulas de control del vehículo (Elaboración propia)

En este apartado, el objetivo es conectar el sistema de control remoto de acuerdo a la figura 129, con el objetivo de verificar los movimientos y acciones programadas hasta ahora en el vehículo.

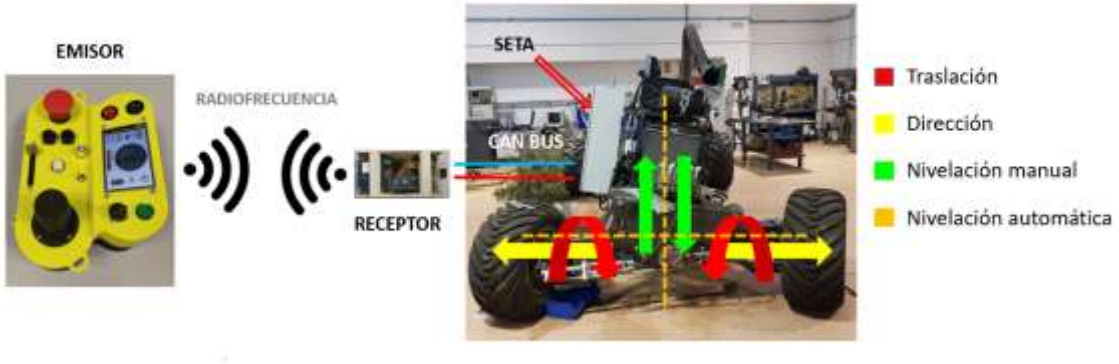


Figura 129. Esquema general de instalación del sistema de control en el vehículo real (Elaboración propia)

En este sentido, se ha instalado el receptor al bus CAN del vehículo ubicado dentro del cuadro eléctrico del vehículo (Figura 130). En el interior de este cuadro eléctrico también se pueden contemplar dispositivos como el PLC del vehículo, conexiones de entrada al bus CAN, módulo E/S del PLC y todo tipo de protecciones (fusibles, magnetotérmicos, salvamotor) necesarias para la protección eléctrica del vehículo.

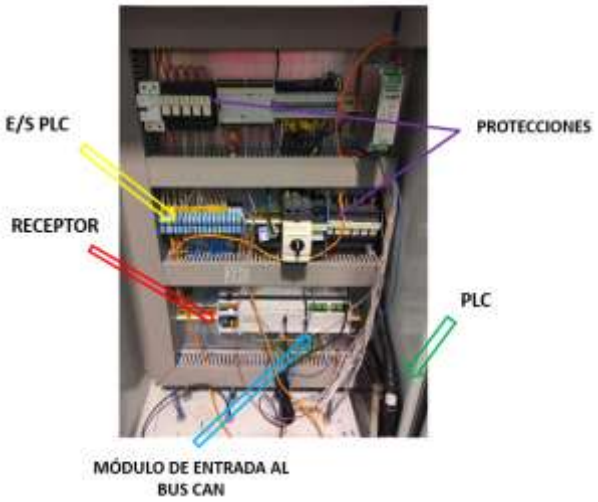


Figura 130. Incorporación del receptor al bus CAN del vehículo dentro del cuadro eléctrico del mismo (Elaboración propia)

Una vez todo listo, se han realizado varios ensayos tanto en estático como en dinámico entre los que destacan:

- **Ensayo en estático:** mediante el uso de una carretilla elevadora se ha simulado un terreno con diferentes pendientes con el objetivo comprobar que el vehículo nivela tanto manual como automáticamente en estático manteniendo el chasis siempre en un plano horizontal (Figura 131).



Figura 131. Ensayo de nivelación automática del vehículo real en estático (Elaboración propia)

- **Primer en ensayo en dinámico:** para este ensayo fue necesario sacar el vehículo fuera del taller porque hubo que hacer maniobras de dirección y movimientos hacia adelante y hacia atrás. De esta forma, entre otras se comprobó que la dirección y la traslación funcionaban adecuadamente. Una vez en el exterior, se buscó una plataforma donde poder probar la nivelación automática en dinámico, resultando con éxito el ensayo (Figura 132).



Figura 132. Primer ensayo de nivelación manual y automática, dirección, traslación en dinámico (Elaboración propia)

- **Segundo ensayo en dinámico:** esta vez, se llevó el vehículo a las naves de aperos ubicadas cerca de la finca experimental de rabanales y en este ensayo se corrigió ciertos detalles observados en el primer ensayo dinámico relacionados con la sensibilidad del joystick, que afectaban en la apertura innecesaria de algunas válvulas del vehículo y que provocaban traslaciones o giros indebidos. Tras estas modificaciones, el vehículo se trasladaba, giraba y nivelaba correctamente, pero necesitaba que el emisor estuviera muy cerca del receptor ya que los módulos de radiofrecuencia utilizados (NRF24L01) eran de corto alcance (Figura 133).



Figura 133. Segundo ensayo en dinámico donde se comprobó que los módulos de radiofrecuencia tenían poco alcance (Elaboración propia)

- **Tercer ensayo dinámico:** tras el problema del alcance detectado en el segundo ensayo dinámico, se sustituyó el modelo NRF24L01 del emisor y del receptor por el modelo NRF24L01+PA+LNA que incorpora antena y permite un alcance teórico según sus características técnicas de hasta un kilómetro de distancia (Figura 134). Seguidamente, se realizó un tercer ensayo donde se comprobó de nuevo que el vehículo realizaba correctamente todos los movimientos físicos y acciones permitas hasta el momento (traslación, dirección etc) pero esta vez el emisor se encontraba a un alcance mayor del receptor (Figura 135).

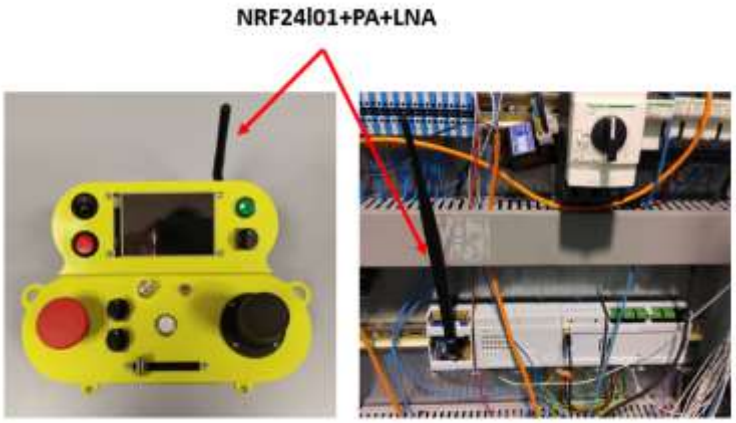


Figura 134. Sustitución en emisor y receptor del módulo NRF24l01 por el módulo NRF24l01+PA+LNA de mayor alcance (Elaboración propia)



Figura 135. Tercer ensayo dinámico donde se comprobó el largo alcance entre el emisor y el receptor (Elaboración propia)

8. Conclusiones

Tras los ensayos estáticos y dinámicos en campo del vehículo, se comprueba que el sistema de control remoto objetivo funciona correctamente y realiza los movimientos de traslación, dirección y nivelación (manual y automática) del vehículo. Además, se observa que para lograr un mayor alcance entre el mando emisor y el receptor instalado en el vehículo, ha sido necesario incorporar un módulo inalámbrico con antena (NRF24L01+PA+LNA) tanto en el emisor como en el receptor.

La realización de este TFM me ha permitido, aprender a programar microcontroladores de una forma más optimizada, diseñar y desarrollar circuitos electrónicos más complejos, aprender a diseñar una PCB y mejorar mis conocimientos en diseño CAD e impresión 3D. También, me ha permitido fabricar un robot con el que he podido mejorar mediante prueba-error el sistema de control remoto en el laboratorio. Por último y más importante, he podido hacer funcionar correctamente un sistema de control remoto diseñado y probado en laboratorio sobre un vehículo real en el que surgen una gran cantidad de situaciones impredecibles.

Como conclusión final, el desarrollo de este TFM me ha permitido alcanzar esa formación polivalente de carácter generalista y multidisciplinar que se busca en el perfil de un Ingeniero Industrial, combinando conocimientos mecánicos, eléctricos, electrónicos, robóticos etc teniendo en cuenta siempre la normativa vigente.

9. Líneas futuras

Se espera que la robótica desempeñe un papel importante en el campo agrícola (Albani et al., 2017), es por este motivo, por el que se debería continuar con este proyecto y alcanzar el objetivo de desarrollar un vehículo agrícola totalmente autónomo (Nivel 5 SAE).

Para poder alcanzar la autonomía del vehículo, se utilizará la navegación por SLAM (*Simultaneus Localization And Mapping)*. Se trata de una técnicas de navegación que permite a un robot o vehículo, construir un mapa del entorno y al mismo tiempo navegar por él, gracias a la información que captan sus sensores en tiempo real (Figura 136)(Martínez, 2020).

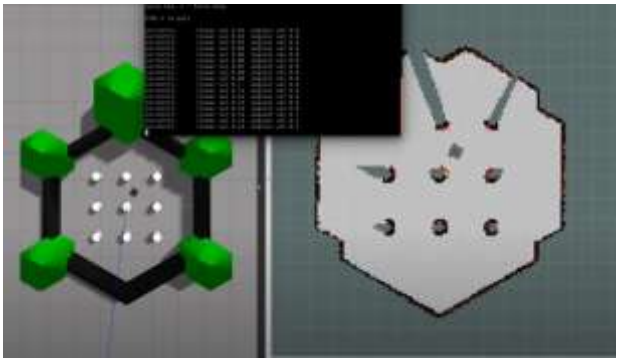


Figura 136. Navegación por SLAM para la conducción autónoma de robots (OpenSourceTeam, 2018)

Entre los sensores que se utilizan para hacer SLAM, se encuentran el **LIDAR** (*Light Detection and Ranging*), que se trata de un dispositivo que, mediante un haz láser pulsado, determina la distancia desde un láser emisor a un objeto y cámaras de tipo **RGB-D** que se tratan de dispositivos de detección de profundidad que funcionan en asociación con una cámara RGB permitiendo hacer lo que se conocer como **Visual SLAM.** Este concepto es una variante del SLAM que utiliza como sensor una cámara y la compresión del entorno requiere técnicas de **visión artificial**.

Para el tratamiento de los algoritmos SLAM, se utilizará **ROS** (Robot Operating System), que se trata de un framework flexible para escribir software de robot. Es una colección de herramientas, bibliotecas y convenciones que tiene como objetivo simplificar la tarea de crear un comportamiento robótico complejo y robusto en una amplia variedad de plataformas robóticas(ROS, 2021).

Antes de probar prototipos, se realizará simulaciones en software como **Gazebo**, que se trata de un simulador bien diseñado que permite probar rápidamente algoritmos, diseñar robots, realizar pruebas de regresión y entrenar el sistema de **IA (Inteligencia Artificial)** utilizando escenarios realistas (Web oficial de Gazebo, 2021).

Por otro lado, como otra línea futura, se buscará una aplicabilidad más productiva a la TIC incorporada al mando emisor (módulo bluetooth). Se pretenderá que en el momento que el vehículo requiera de algún tipo de mantenimiento predictivo o preventivo (cambio de aceite cada cierta hora, sustitución de filtros etc) se envíe algún tipo de notificación al smartphone del operario y que mediante alguna app le salte algún tipo de alarma o aviso.

10. Bibliografía

1. Albani, D., IJsselmuiden, J., Haken, R., & Trianni, V. (2017). Monitoring and mapping with robot swarms for agricultural applications. *2017 14th IEEE International Conference on Advanced Video and Signal Based Surveillance (AVSS)*, 1–6. https://doi.org/10.1109 / AVSS.2017.8078478
2. Barrientos, A., & del Cerro, J. (2016). *El uso de robots en tareas agrícolas*. https://www.interempresas.net/Horticola/Articulos/151745-El-uso-de-robots-en-tareas-agricolas.html. Consultado el 31/08/2021.
3. Blanco, S. (2021). *Actualizaciones de SAE, mejora la tabla de niveles de "conducción autónoma."* https://www.caranddriver.com/news/a36364986/sae-updates-refines-autonomous-driving-levels-chart/. Consultado el 03/05/2021.
4. Calzado Jimenez, M., Porras Galán, J., A. Domingues, I., Romero, L., & del Mar Espinosa, M. (2013). La fabricación aditiva. La evidencia de una necesidad. In *Interempresas Industria*

Metalmecánica (Vol. 235, pp. 64–82).
5. Castellano, A. R. (2012). *Bluetooth. Introduccion a su funcionamiento*. 1–16.
6. Cséfalvay, Z., & Gkotsis, P. (2020). *Global race for robotisation-Looking at the entire robotisation chain*. https://doi.org/10.2760/60257
7. Dannenberg, A. (2006). *UART Implementation Using the High-End Timer*. *June*, 1–9.
8. Del VillarFernández, I. (2015). *Diseño e implementación en PCB de un robot auto-balanceado mediante Arduino con módulo inalámbrico Grado en Ingeniería Eléctrica y Electrónica*. 103.
9. Electronica Caballero. (2021). *Tienda Online Electrónica Caballero*. https://electronicacaballero.com/.Consultado el 28/05/2021.
10. Esopo. (2021). *Espectro Radioeléctrico*. https://iie.fing.edu.uy/proyectos/esopo/espectro-radioelectrico/. Consultado el 07/05/2021.
11. García Gonzalez, A. (2015). *El setup y el loop en Arduino*. http://panamahitek.com/el-setup-y-el-loop-en-arduino/. Consultado el 02/06/2021.
12. Guerrero, G., & Rodríguez, Ó. (2020). *Técnicas Avanzadas de Fabricación*.
13. Hernández, L. del V. (2019). *Resistencia pull up y pull down con Arduino, para qué sirven*. https://programarfacil.com/blog/arduino-blog/resistencia-pull-up-y-pull-down/. Consultado el 03/06/2021.
14. IBM. (2021). *Bits de paridad*. https://www.ibm.com/docs/es/aix/7.2?topic=parameters-parity-bits. Consultado el 05/06/2021.
15. Infoagro. (2020). *Valtra y Elisa presentan el tractor a control remoto: un gran paso hacia la conducción autónoma*. https://www.infoagro.com/noticias/2020/valtra_y_elisa_presentan_el_tractor_a_control_remoto__un_gran_paso_hac.asp. Consultado el 04/05/2021.
16. ITead Studio. (2010). HC - 05 - Bluetooth to Serial Port Module (Datasheet). *Datasheet*, 1.
17. Jecrespom. (2017). *Manejar Librerías Arduino*. https://aprendiendoarduino.wordpress.com/2016/11/16/librerias-arduino-2/. Consultado el 07/06/2021.
18. John Deere. (2019). *El futuro de la tecnología agrícola*. https://www.deere.es/es/agricultura/el-futuro-de-la-agricultura/. Consultado el 05/05/2021.
19. Kato, S. (2015). *An open approach to autonomous vehicles*. https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=7368032. Consultado el 03/05/2021.
20. KVASER. (2021). *The CAN Protocol Tutorial*. https://www.kvaser.com/can-protocol-tutorial/. Consultado el 13/05/2021.
21. Leens, F. (2009). An introduction to I2C and SPI protocols. *IEEE Instrumentation and Measurement Magazine*, *12*(1), 8–13. https://doi.org/10.1109/MIM.2009.4762946
22. Llamas, L. (2014). *Entradas analógicas en Arduino*. https://www.luisllamas.es/entradas-analogicas-en-arduino/ Consultado el 17/05/2021.
23. Martínez, J. L. (2020). *Navegación SLAM: robots que construyen mapas*. https://clem.es/noticia/navegacion-slam-robots-que-construyen-mapas-514. Consultado el 07/07/2021.
24. Microchip. (2021). *MCP2515. Stand-Alone CAN Controller with SPI Interface*. 1–24. http://ww1.microchip.com/downloads/en/DeviceDoc/MCP2515-Stand-Alone-CAN-Controller-with-SPI-20001801J.pdf. Consultado el 19/05/2021.
25. Misset. (2021). *Future Farming Field Robots Catalogue*. https://misset.com/field-robots/field-robots/ Consultado el 01/09/2021.
26. Montañez Ruiz, J. (2017). *Sistema Basado En Microcontrolador Para La Automatización De Un Acuario*. 64. https://riunet.upv.es/bitstream/handle/10251/86108/MONTAÑEZ - DESARROLLO DE SISTEMA BASADO EN MICROCONTROLADOR PARA LA AUTOMATIZACIÓN DE UN ACUARIO..pdf?sequence=1 Consultado el 24/05/2021.

27. Nextion.tech. (2021). *La guía del editor de Nextion*. https://nextion.tech/editor_guide/. Consultado el 22/06/2021.
28. NORDIC SEMICONDUCTOR. (2021). *Single chip 2.4 GHz Transceiver nRF24L01*.
29. OpenSourceTeam, R. (2018). *Chapter 11 SLAM and Navigation*. https://www.youtube.com/watch?v=IH_n9bfy-nM. Consultado el 08/07/2021.
30. Oudjida, A. K., Berrandjia, M. L., Liacha, A., Tiar, R., Tahraoui, K., & Alhoumays, Y. N. (2010). Design and test of general-purpose SPI master/slave IPs on OPB bus. *2010 7th International Multi-Conference on Systems, Signals and Devices, SSD-10*, 0–5. https://doi.org/10.1109/SSD.2010.5585592
31. Requena, A. M. (2017). Introducción a CAN bus: Descripción, ejemplos y aplicaciones de tiempo real. *Universidad Politecnica De Madrid*, *15*(7), 1–98.
32. ROS, W. O. de. (2021). *About ROS*. https://www.ros.org/ Consultado el 08/07/2021.
33. SAE International. (2021). *SURFACE VEHICLE RECOMMENDED PRACTICE*.
34. Saioacontrol. (2021). *Mandos de Radiocontrol para Grúas Torre*. https://saioacontrol.com/mando-radio-control-gruas-torre-industrial Consultado el 18/06/2021.
35. Saiz-Rubio, V., & Rovira-Más, F. (2020). From smart farming towards agriculture 5.0: A review on crop data management. In *Agronomy* (Vol. 10, Issue 2, p. 207). MDPI AG. https://doi.org/10.3390/agronomy10020207
36. Siemens Digital Industries Software. (2021). *Diseño asistido por ordenador (CAD)*. https://www.plm.automation.siemens.com/global/es/our-story/glossary/computer-aided-design-cad/12507. Consultado el 17/06/2021.
37. Sola, R. (2021). *Análisis teórico del diseño de máquinas. Diseño Avanzado en Ingeniería Mecánica*.
38. Stentz, A., Dima, C., Wellington, C., Herman, H., & Stager, D. (2002). A System for Semi-Autonomous Tractor Operations. *Autonomous Robots 2002 13:1*, *13*(1), 87–104. https://doi.org/10.1023/A:1015634322857
39. Subtel. (2005). *ANEXO C Volumen II Estudio relativo a la actualización del Plan General de Uso del Espectro Radioeléctrico Actualización Marco Regulatorio y Evolución Sector de Telecomunicaciones*.
40. Toporski, T. (2016). Analog Applications Journal. *Texas Instruments*.
41. Torii, T. (2000). *Research in autonomous agriculture vehicles in*. *25*, 133–153.
42. Vázquez, J. (2019). *Para que sirve un puente rectificador*. https://uelectronics.com/para-que-sirve-un-puente-rectificador/. Consultado el 21/06/2021.
43. Wang, Y., & Song, K. (2011). *A new approach to realize UART*.
44. Web oficial de Arduino. (2020). *millis ()*. https://www.arduino.cc/reference/en/language/functions/time/millis/. Consultado el 09/06/2021.
45. Web oficial de Arduino. (2021a). *Arduino Mega 2560*. https://store.arduino.cc/arduino-mega-2560-rev3?queryID=d3b3b84c0addbb810d82813027c1852c. Consultado el 11/06/2021.
46. Web oficial de Arduino. (2021b). *Arduino Nano*. https://store.arduino.cc/arduino-nano. Consultado el 11/06/2021.
47. Web oficial de Arduino. (2021c). *Nice drawings of the Arduino UNO and Mega 2560*. Consultado el 12/06/2021.
48. https://blog.arduino.cc/2011/01/05/nice-drawings-of-the-arduino-uno-and-mega-2560/. Consultado el 12/06/2021.
49. Web oficial de Arduino. (2021d). *What is Arduino?* https://www.arduino.cc/en/Guide/Introduction. Consultado el 10/06/2021.
50. Web oficial de Gazebo. (2021). *Why Gazebo ?* http://gazebosim.org/. Consultado el 09/07/2021.

51. Web oficial de Kubota. (2020). *Kubota presenta un "tractor de ensueño."* https://www.kubota.com/news/2020/20200115_1.html. Consultado el 05/05/2021.
52. Web oficial de Reform. (2021). *Metron P48 RC. El revolucionario porta-implementos*. https://www.reform.at/produkte/metron/metron-p48. Consultado el 06/05/2021.
53. Wikipedia. (2021). *Bus CAN*. https://es.wikipedia.org/wiki/Bus_CAN. Consultado el 19/05/2021.

Capítulo 2: MEDICIONES Y PRESUPUESTO

El **Estado de Mediciones** constituye uno de los documentos básicos del Proyecto. Tiene como misión definir y determinar las unidades de cada partida o unidad de obra que configuran la totalidad del producto, obre, instalación, servicio o software objeto del Proyecto.

El **Presupuesto** constituye también uno de los documentos básicos del Proyecto. Tiene como misión determinar el coste económico del objeto del Proyecto.

Índice de las Mediciones y el Presupuesto

1. Mediciones y presupuesto desglosado de los materiales del emisor, receptor y robot simulador
2. Mediciones y presupuesto desglosado de las horas de trabajo del emisor, receptor, robot simulador y ensayos
3. Presupuesto resumen de los costes generales del Proyecto

1. Mediciones y presupuesto desglosado de los materiales del emisor, receptor y robot simulador

	COMPONENTE	UNIDADES	PRECIO/UNIDAD	TOTAL POR COMPONENTE
EMISOR	ARDUINO MEGA 2560	1	9,12 €	9,12 €
	PCB	1	6,11 €	6,11 €
	NRF24L01	1	1,87 €	1,87 €
	NRF24L01+PA+LNA	1	4,01 €	4,01 €
	HEADER 4 VIAS	4	0,24 €	0,96 €
	HEADER 6 VIAS	8	0,20 €	1,60 €
	HEADER 2 VIAS	12	0,15 €	1,80 €
	REGULADOR DE TENSIÓN 12V A 5V	1	2,31 €	2,31 €
	PUENTE DE DIODOS	1	0,74 €	0,74 €
	BMS 3S	1	5,69 €	5,69 €
	HOLDER BATERÍA 18650	3	0,70 €	2,10 €
	BATERÍA 18650	3	5,10 €	15,30 €
	HC-05 BLUETOOTH	1	6,85 €	6,85 €
	ALARGADOR USB-B	1	9,38 €	9,38 €
	INTERRUPTOR UNIPORLAR	1	0,80 €	0,80 €
	JACK CARGADOR 5.5mm	1	0,95 €	0,95 €
	JOYSTICK 3 EJES	1	10,34 €	10,34 €
	SETA EMERGENCIA	1	3,65 €	3,65 €
	POTENCIOMETRO CORREDERA	1	1,52 €	1,52 €
	PULSADOR EMPOTRAR	2	1,20 €	2,40 €
	INTERUPTOR BASCULANTE	2	1,00 €	2,00 €
	INTERRUPTOR PALANCA	1	1,10 €	1,10 €
	SELECTOR 3 POSICIONES	3	3,60 €	10,80 €
	POTENCIOMETRO 10K	1	2,10 €	2,10 €
	ALARGADOR ANTENA	1	5,12 €	5,12 €
	NEXTION ENHANCED 3.5 INCH	1	28,64 €	28,64 €
	PIEZAS PLA 1.75 MM NEGRO	0,196	16,49 €	3,23 €
	PIEZAS PLA 1.75MM AMARILLO	0,152	18,30 €	2,78 €
			TOTAL (SIN I.V.A)	143,26 €
			TOTAL (CON I.V.A)	173,35 €

	COMPONENTE	UNIDADES	PRECIO/UNIDAD	TOTAL POR COMPONENTE
RECEPTOR	PROTOBOARD SOLDABLE	1	3,60 €	3,60 €
	NRF24L01	1	1,87 €	1,87 €
	NRF24L01+PA+LNA	1	4,01 €	4,01 €
	ARDUINO NANO	1	2,88 €	2,88 €
	MCP2515	1	0,83 €	0,83 €
	PILA 9V	1	2,50 €	2,50 €
	CARCASA PARA CARRIL DIN	1	5,1798	5,18 €
			TOTAL (SIN I.V.A)	20,87 €
			TOTAL (CON I.V.A)	25,25 €

	COMPONENTE	UNIDADES	PRECIO/UNIDAD	TOTAL POR COMPONENTE
ROBOT	PROTOBOARD SOLDABLE	1	3,60 €	3,60 €
	ARDUINO MEGA 2560	1	9,12 €	9,12 €
	MCP2515	1	0,83 €	0,83 €
	KIT 4 MOTORES DC + L298N	1	17,79 €	17,79 €
	REGULADOR DE TENSIÓN XH-M401	2	6,22 €	12,44 €
	HOLDER BATERÍA 18650	3	0,70 €	2,10 €
	BATERÍA 18650	3	5,60 €	16,80 €
	SERVO MG996R	5	4,45 €	22,25 €
	JACK CARGADOR 5.5mm	1	0,95 €	0,95 €
	BMS 3S	1	5,69 €	5,69 €
	BUZZER ACTIVO	1	1,07 €	1,07 €
	ACELEROMETRO 3 EJES GY 291	1	2,02 €	2,02 €
	INTERUPTOR BASCULANTE	1	1,00 €	1,00 €
	PIEZAS PLA 1.75 MM NEGRO	0,248	16,49 €	4,09 €
	PIEZAS PLA 1.75MM AMARILLO	0,09	18,30 €	1,65 €
			TOTAL (SIN I.V.A)	101,40 €
			TOTAL (CON I.V.A)	122,69 €

2. Mediciones y presupuesto desglosado de las horas de trabajo del emisor, receptor, robot simulador y ensayos

	TAREA	HORAS	PRECIO/HORA	TOTAL POR TAREA
EMISOR	PROGRAMACIÓN	50	20,00 €	1.000,00 €
	DISEÑO Y DESARROLLO ELECTRÓNICO	20	20,00 €	400,00 €
	DISEÑO 3D	15	20,00 €	300,00 €
	IMPRESIÓN 3D	32	2,00 €	64,00 €
			TOTAL (SIN I.V.A)	1.764,00 €
			TOTAL (CON I.V.A)	2.134,44 €

	TAREA	HORAS	PRECIO/HORA	TOTAL POR TAREA
RECEPTOR	PROGRAMACIÓN	20	20,00 €	400,00 €
	DISEÑO Y DESARROLLO ELECTRÓNICO	10	20,00 €	200,00 €
			TOTAL (SIN I.V.A)	600,00 €
			TOTAL (CON I.V.A)	726,00 €

	TAREA	HORAS	PRECIO/HORA	TOTAL POR TAREA
ROBOT	PROGRAMACIÓN	20	20,00 €	400,00 €
	DISEÑO Y DESARROLLO ELECTRÓNICO	20	20,00 €	400,00 €
	DISEÑO 3D	10	20,00 €	200,00 €
	IMPRESIÓN 3D	28	2,00 €	56,00 €
			TOTAL (SIN I.V.A)	1.056,00 €
			TOTAL (CON I.V.A)	1.277,76 €

	TAREA	HORAS	PRECIO/HORA	TOTAL POR TAREA
ENSAYOS	LABORATORIO	4	20,00 €	80,00 €
	TALLER Y CALLE	25	20,00 €	500,00 €
			TOTAL (SIN I.V.A)	580,00 €
			TOTAL (CON I.V.A)	701,80 €

3. Presupuesto resumen de los costes generales del Proyecto

	TOTAL SIN I.V.A	**TOTAL CON I.V.A (21%)**
MATERIALES	164,13 €	198,60 €
HORAS DE TRABAJO	2.944,00 €	3.562,24 €
TOTAL EMISOR+RECEPTOR	3.108,13 €	3.760,84 €

El Presupuesto Total de la **solución emisor más receptor** asciende a la cantidad de tres mil setecientos sesenta euros y ochenta y cuatro céntimos (3.760,84 €).

	TOTAL SIN I.V.A	**TOTAL CON I.V.A (21%)**
MATERIALES	265,52 €	321,28 €
HORAS DE TRABAJO	4.000,00 €	4.840,00 €
TOTAL EMISOR + RECEPTOR + ROBOT	4.265,52 €	5.161,28 €

El Presupuesto Total de la **solución emisor, receptor más robot de simulación** asciende a la cantidad de cinco mil ciento sesenta y un euros y veintiocho céntimos (5.161,28 €).

Capítulo 3: PLANOS

Los **planos** son documentos contractuales y constituyen uno de los documentos básicos del Proyecto y son esenciales para su materialización. Tienen como misión, junto con la Memoria, definir de forma unívoca el objeto del Proyecto.

A continuación, se presenta un índice con los distintos planos utilizados en este presente proyecto. Los cuatro primeros son planos de esquema de conexionado electrónico realizados mediante el sofware Altium y desde el quinto plano hasta el último son planos de acotación de los diferentes diseños CAD realizados mediante el software SOLIDWORKS.

Índice de planos

PLANO N°1: Esquema de conexionado de los componentes electrónicos del emisor

PLANO N°2: Esquema de conexionado de los componentes electrónicos del receptor

PLANO N°3: Esquema de conexionado del sistema de alimentación

PLANO N°4: Esquema de conexionado de los componentes electrónicos del robot

PLANO N°5: Ensamblaje de la parte inferior del mando

PLANO N°6: Ensamblaje de la parte superior plana del mando

PLANO N°7: Ensamblaje de la parte superior inclinada del mando

PLANO N°8: Dimensiones parte inferior del mando emisor

PLANO N°9: Dimensiones parte superior plana del mando emisor

PLANO N°10: Dimensiones parte superior inclinada del mando emisor

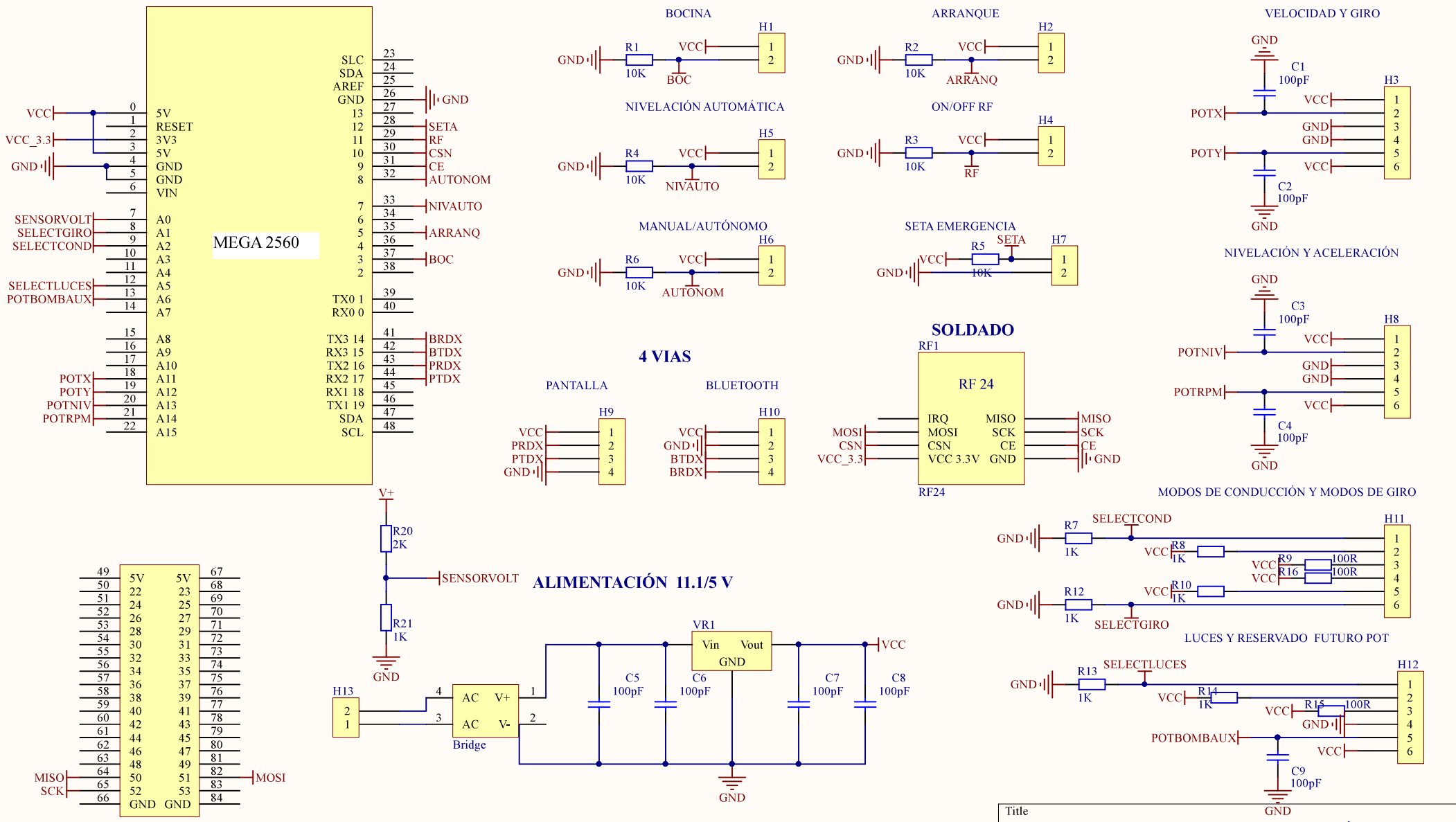
PLANO N°11: Ensamblaje del robot de simulación

PLANO N°12: Dimensiones caparazón inferior robot

PLANO N°13: Dimensiones caparazón superior robot

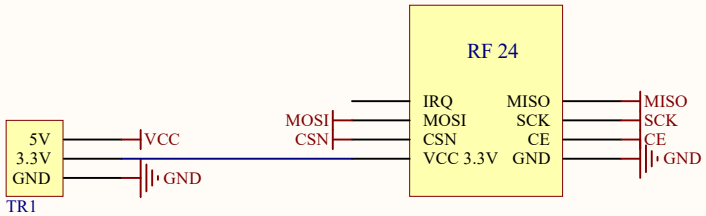
PLANO N°14: Dimensiones dirección robot

PLANO N°15: Dimensiones elemento unión servos robot

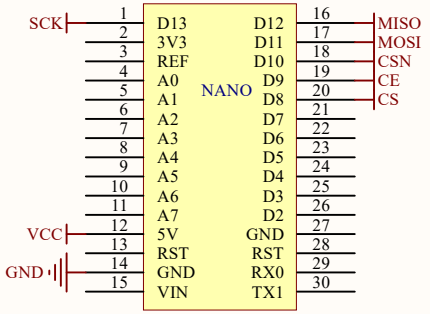


Title	ESQUEMA DE CONEXIONADO DE LOS COMPONENTES ELECTRÓNICOS DEL EMISOR	
Size: A4	Number: PLANO Nº1	Revision: A.S.C
Date: 21/07/2021		Sheet of
File: C:\Users\..\Sheet.SchDoc		Drawn By:

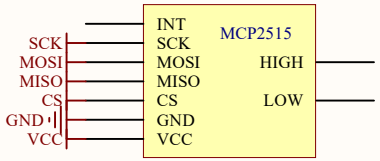
COMUNICACIÓN VIA RADIO



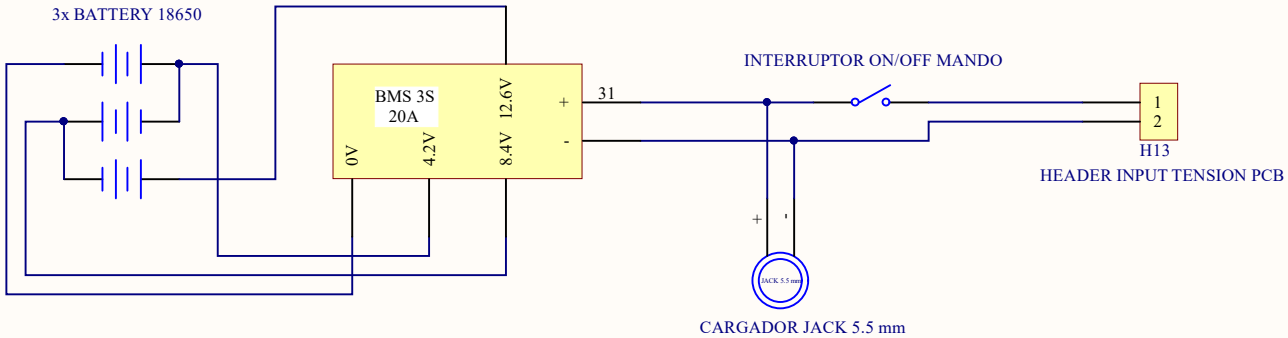
MICROCONTROLADOR



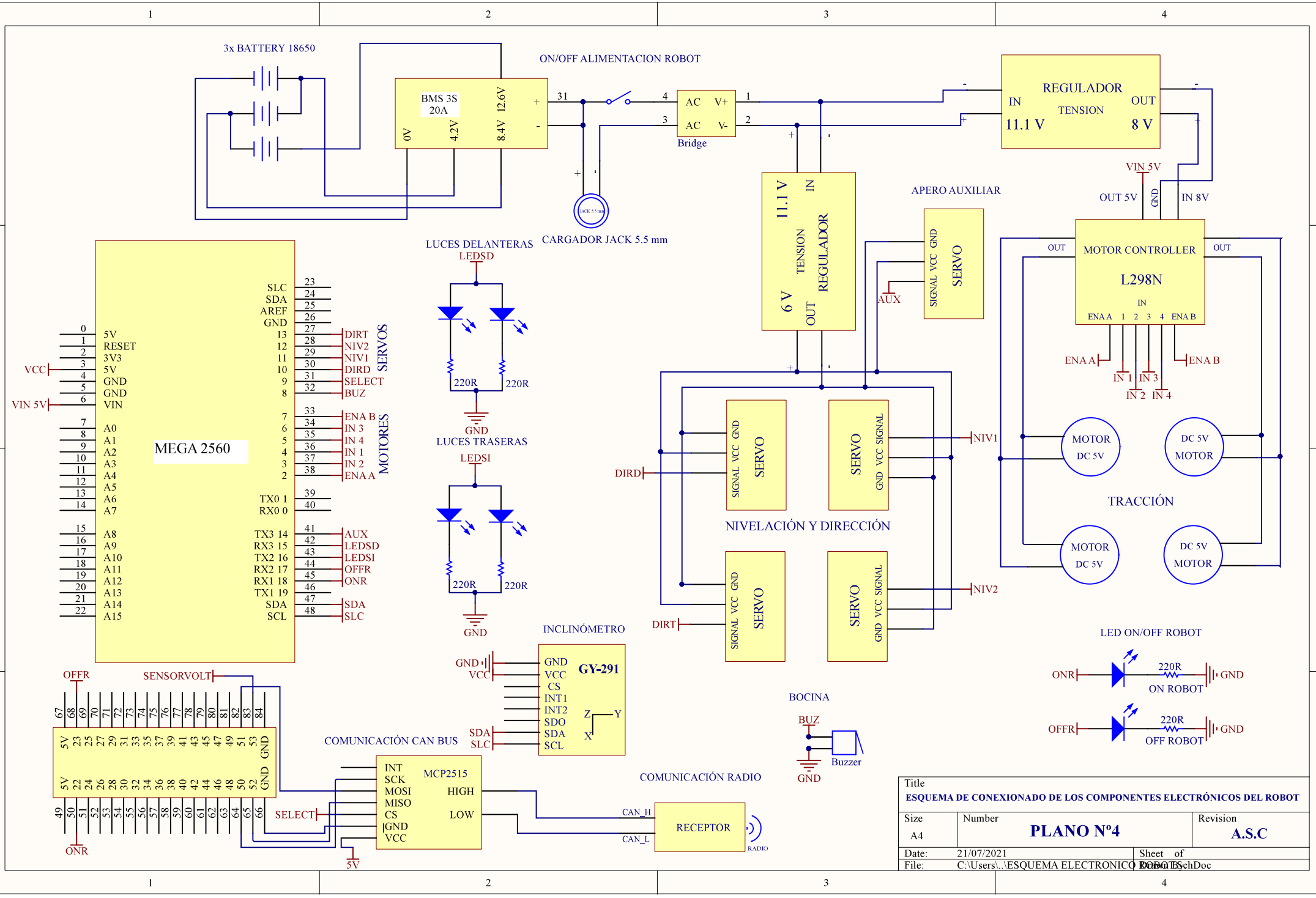
COMUNICACIÓN CAN BUS



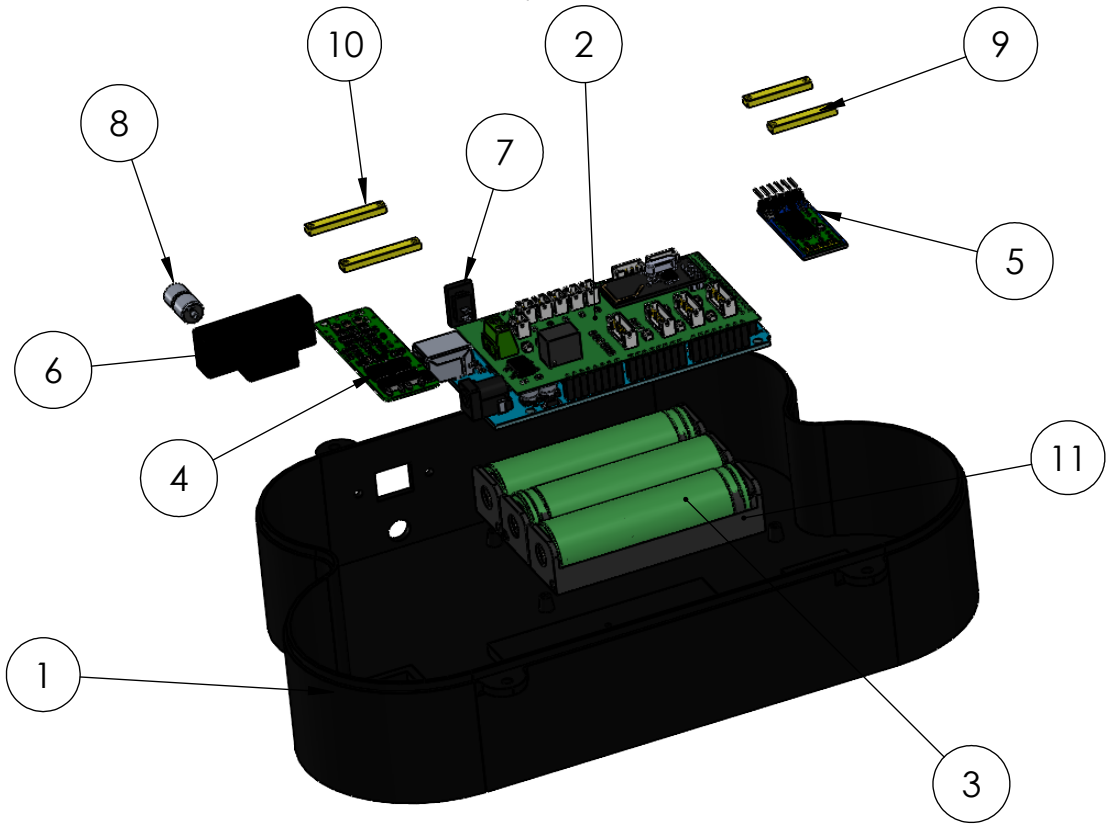
Title: ESQUEMA DE CONEXIONADO DE LOS COMPONENTES ELECTRÓNICOS DEL EMISOR		
Size: A4	Number: PLANO Nº2	Revision: A.S.C
Date: 21/07/2021		Sheet of
File: C:\Users\..\Sheet1.SchDoc		Drawn By:



Title	ESQUEMA DE CONEXIONADO DEL SISTEMA DE ALIMENTACIÓN	
Size: A4	Number: PLANO Nº3	Revision: A.S.C
Date: 21/07/2021		Sheet of
File: C:\Users\...\SISTEMA_ALIMENTACION_BATTERY.SchDoc		Drawn By:



Title	ESQUEMA DE CONEXIONADO DE LOS COMPONENTES ELECTRÓNICOS DEL ROBOT
Size	A4
Number	PLANO Nº4
Revision	A.S.C
Date:	21/07/2021
File:	C:\Users\..\ESQUEMA ELECTRONICO ROBOT.SchDoc
Sheet	of



Nº ELEMENTO	PIEZA	CANTIDAD
1	PARTE INFERIOR MANDO	1
2	ARDUINO MEGA+PCB	1
3	BATERIA 18650	3
4	BMS	1
5	HC-05	1
6	ALARGADOR USB-B A USB-B	1
7	INTERRUPTOR UNIPOLAR	1
8	JACK CARGADOR	1
9	ANCLAJE HC-05	4
10	ANCLAJE BMS	4
11	HOLDER BATERIA	3

ANTONIO SERRANO CORRAL

Uds
mm

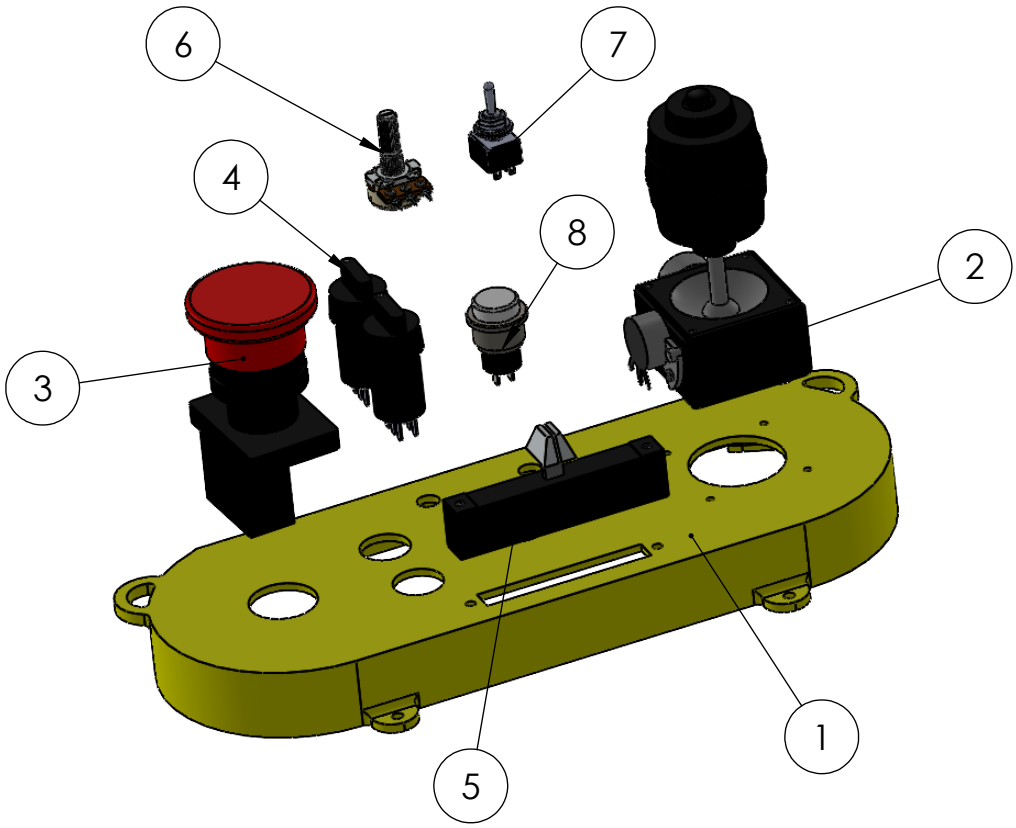
PROCESO DE FABRICACIÓN	**MATERIAL**	**ESCALA**
FABRIACIÓN ADITIVA	PLA	1:2

ESCUELA POLITÉCNICA SUPERIOR DE CÓRDOBA
Universidad de Córdoba
EPSC

PLANO Nº5

ENSAMBLAJE PARA INFERIOR MANDO

Producto SOLIDWORKS Educational. Solo para uso en la enseñanza.

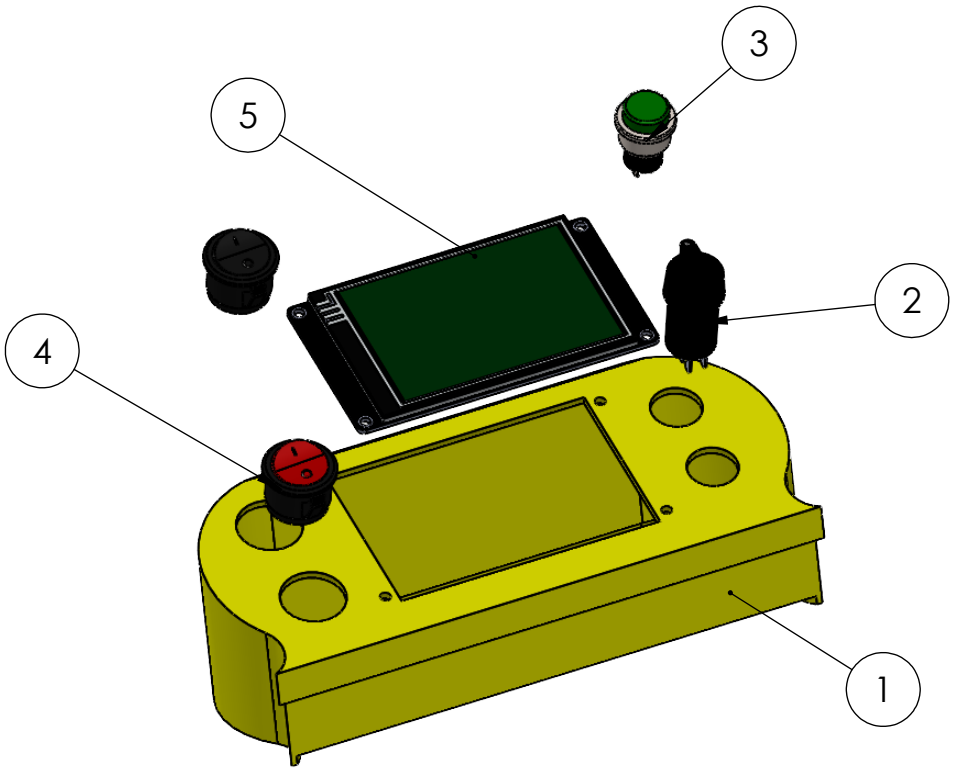


Nº ELEMENTO	COMPONENTE	CANTIDAD
1	PARTE SUPERIOR PLANA MANDO	1
2	JOYSTICK 3 EJES	1
3	SETA DE EMERGENCIA	1
4	SELECTOR 3 POSICIONES	2
5	POTENCIOMETRO CORREDERA	1
6	POTENIOMETRO 10K	1
7	INTERRUPTOR DE PALANCA	1
8	PULSADOR	1

ANTONIO SERRANO CORRAL		**Uds** mm
PROCESO DE FABRICACIÓN FABRIACIÓN ADITIVA	**MATERIAL** PLA	**ESCALA** 1:2
PLANO Nº6 ENSAMBLAJE PARTE SUPERIOR PLANA DEL MANDO		

ESCUELA POLITÉCNICA SUPERIOR DE CÓRDOBA
Universidad de Córdoba

Producto SOLIDWORKS Educational. Solo para uso en la enseñanza.

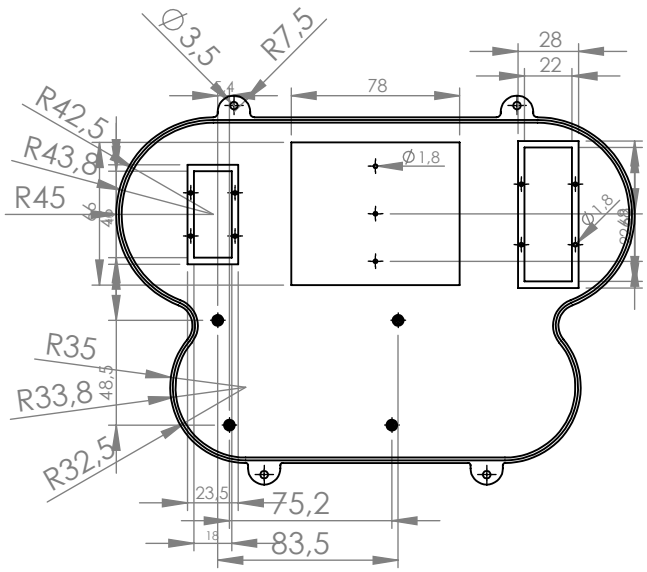


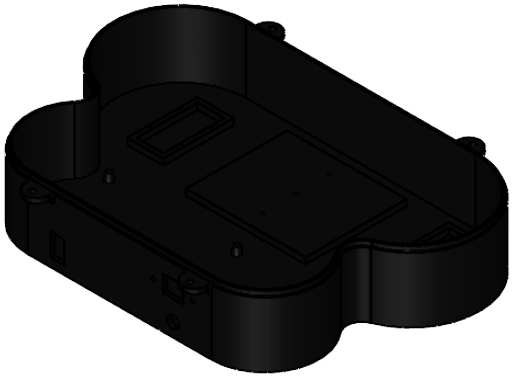
Nº ELEMENTO	COMPONENTE	CANTIDAD
1	PARTE SUPERIOR INCLINADA MANDO	1
2	SELECTOR 3 POSICIONES	1
3	PULSADOR	1
4	INTERRUPTOR BASCULANTE	2
5	PANTALLA NEXTION 3.5 INCH	1

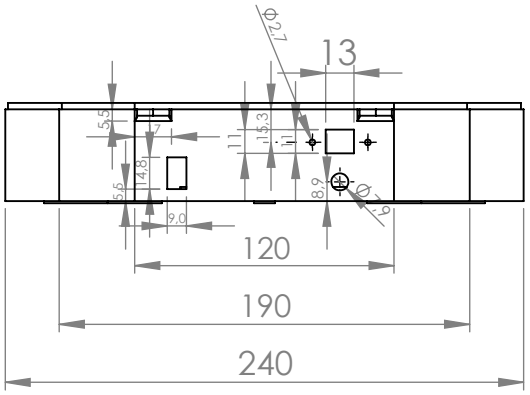
ANTONIO SERRANO CORRAL		**Uds** mm
PROCESO DE FABRICACIÓN FABRIACIÓN ADITIVA	**MATERIAL** PLA	**ESCALA** 1:2
PLANO Nº7 ENSAMBLAJE PARTE SUPERIOR INCLINADA DEL MANDO		

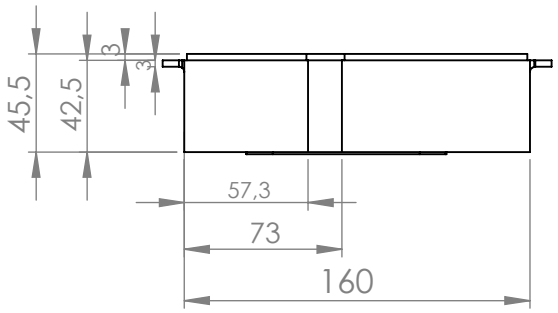
ESCUELA POLITÉCNICA SUPERIOR DE CÓRDOBA
Universidad de Córdoba

Producto SOLIDWORKS Educational. Solo para uso en la enseñanza.









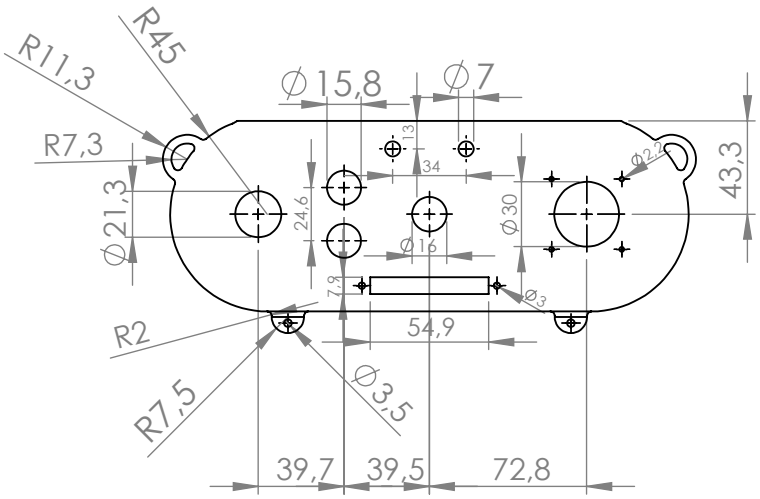
ANTONIO SERRANO CORRAL		**Uds** mm
PROCESO DE FABRICACIÓN FABRIACIÓN ADITIVA	**MATERIAL** PLA	**ESCALA** 1:5

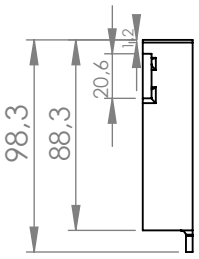
ESCUELA POLITÉCNICA SUPERIOR DE CÓRDOBA
Universidad de Córdoba

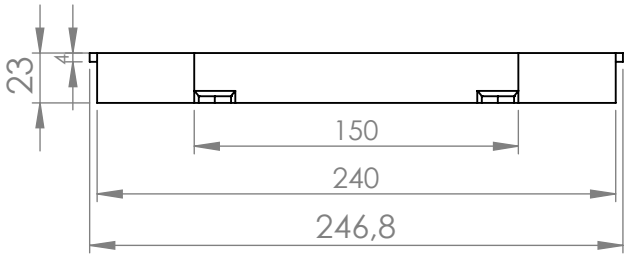
PLANO Nº8

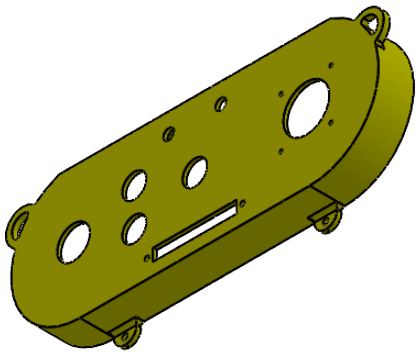
DIMENSIONES PARTE INFERIOR MANDO EMISOR

Producto SOLIDWORKS Educational. Solo para uso en la enseñanza.









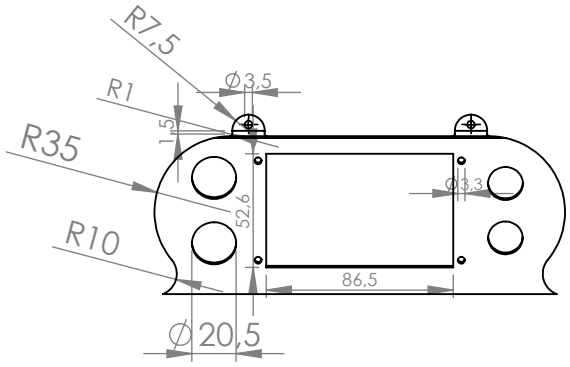
ANTONIO SERRANO CORRAL		**Uds** mm
PROCESO DE FABRICACIÓN FABRIACIÓN ADITIVA	**MATERIAL** PLA	**ESCALA** 1:5

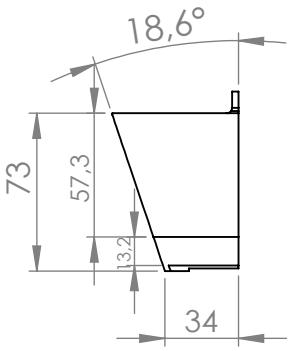
ESCUELA POLITÉCNICA SUPERIOR DE CÓRDOBA
Universidad de Córdoba

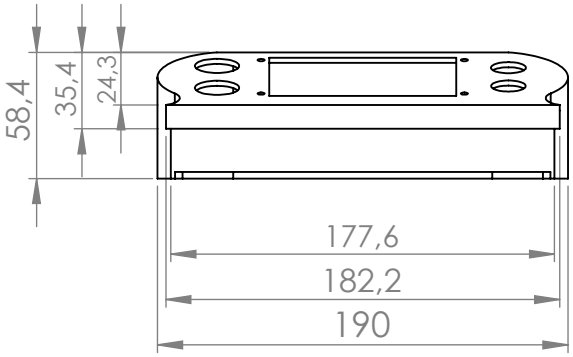
PLANO Nº9

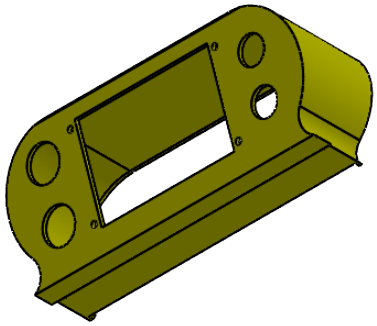
DIMENSIONES PARTE SUPERIOR PLANA MANDO EMISOR

Producto SOLIDWORKS Educational. Solo para uso en la enseñanza.









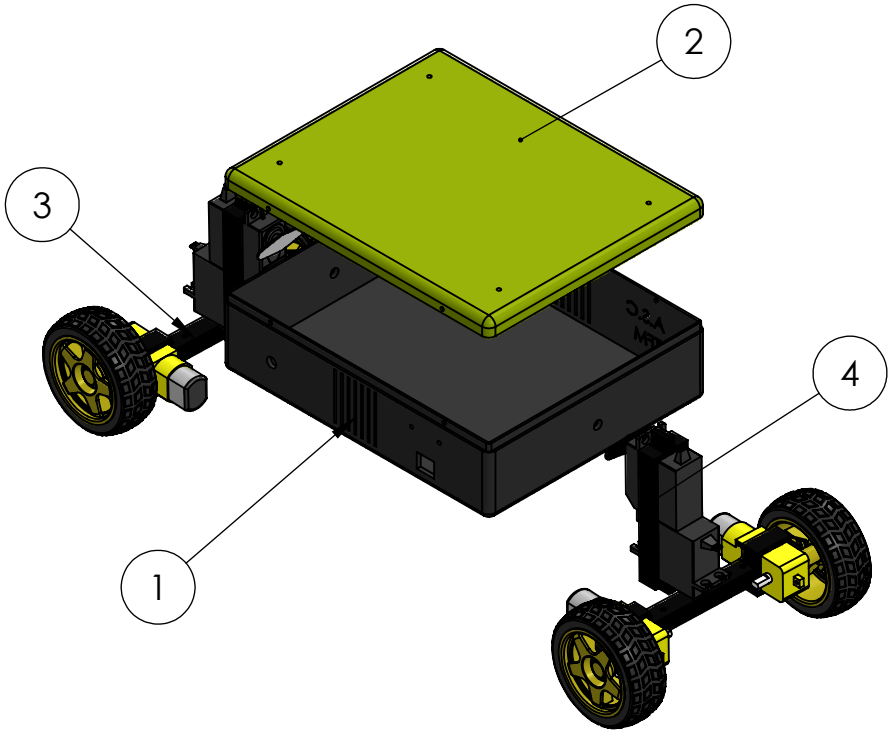
ANTONIO SERRANO CORRAL		**Uds** mm
PROCESO DE FABRICACIÓN FABRIACIÓN ADITIVA	**MATERIAL** PLA	**ESCALA** 1:5

ESCUELA POLITÉCNICA SUPERIOR DE CÓRDOBA
Universidad de Córdoba

PLANO Nº10

DIMENSIONES PARTE SUPERIOR INCLINADA MANDO EMISOR

Producto SOLIDWORKS Educational. Solo para uso en la enseñanza.

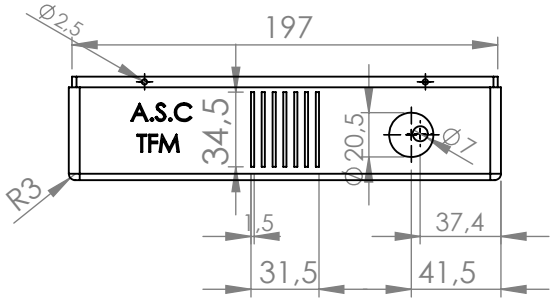


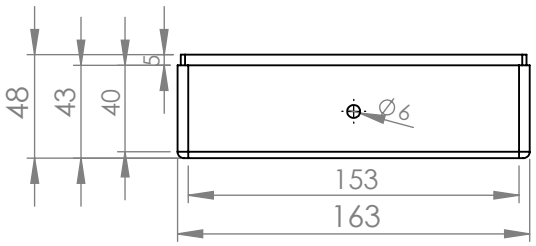
Nº ELEMENTO	COMPONENTE	CANTIDAD
1	CAPARAZON INFERIOR	1
2	CAPARAZON SUPERIOR	1
3	DIRECCION	4
4	UNION SERVOS	4

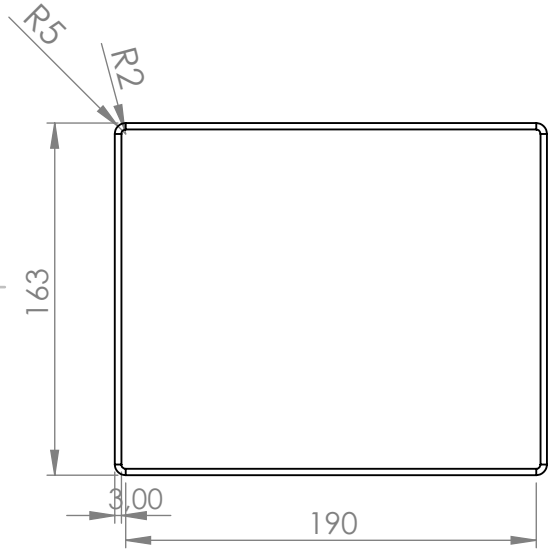
ANTONIO SERRANO CORRAL		**Uds** mm
PROCESO DE FABRICACIÓN FABRIACIÓN ADITIVA	**MATERIAL** PLA	**ESCALA** 1:5
PLANO Nº11 ENSAMBLAJE DE ROBOT DE SIMULACIÓN		

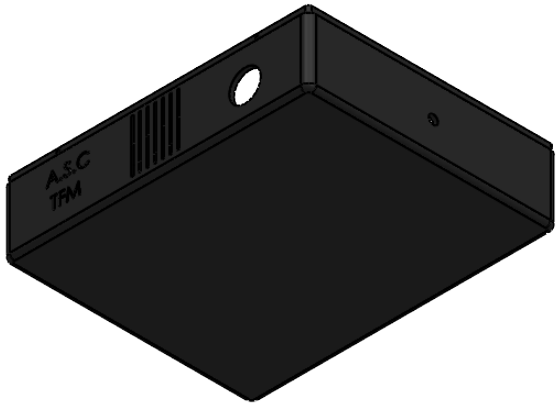
ESCUELA POLITÉCNICA SUPERIOR DE CÓRDOBA
Universidad de Córdoba

Producto SOLIDWORKS Educational. Solo para uso en la enseñanza.









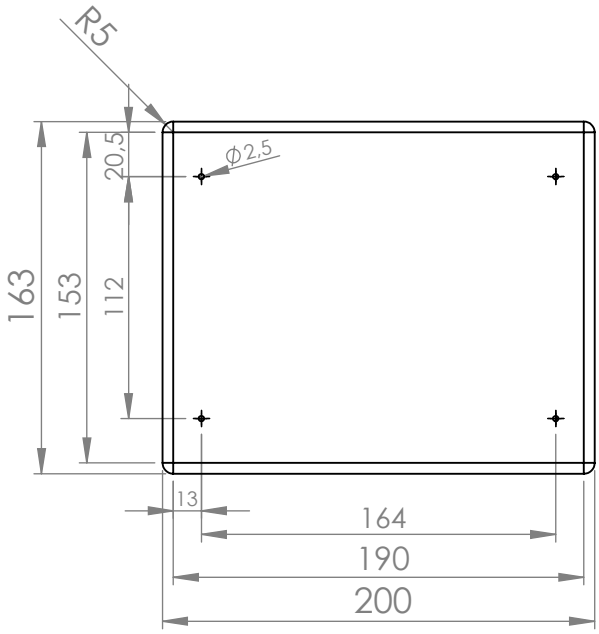
ANTONIO SERRANO CORRAL		**Uds** mm
PROCESO DE FABRICACIÓN FABRIACIÓN ADITIVA	**MATERIAL** PLA	**ESCALA** 1:5

ESCUELA POLITÉCNICA SUPERIOR DE CÓRDOBA
Universidad de Córdoba

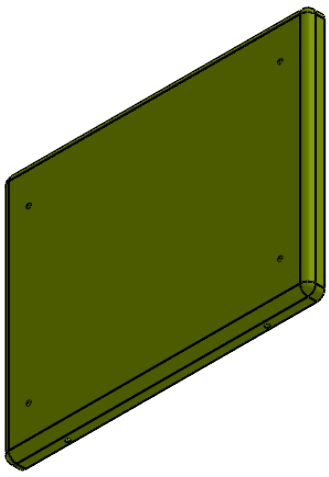
PLANO Nº12

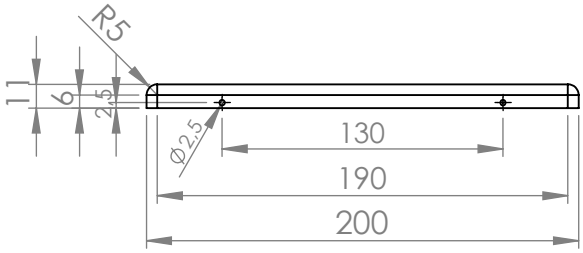
DIMENSIONES CAPARAZÓN INFERIOR ROBOT

Producto SOLIDWORKS Educational. Solo para uso en la enseñanza.









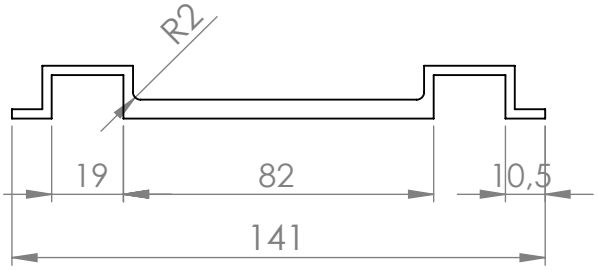
ANTONIO SERRANO CORRAL		**Uds** mm
PROCESO DE FABRICACIÓN FABRIACIÓN ADITIVA	**MATERIAL** PLA	**ESCALA** 1:5

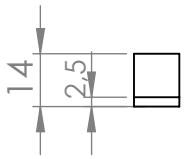
ESCUELA POLITÉCNICA SUPERIOR DE CÓRDOBA
Universidad de Córdoba

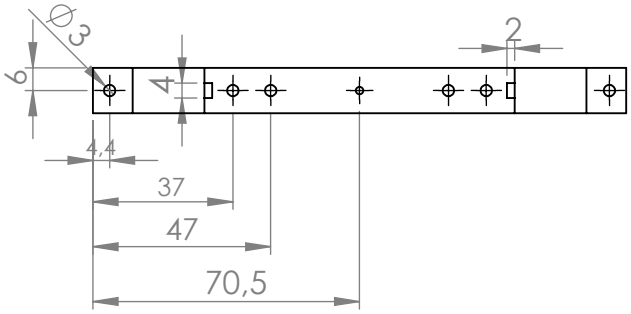
PLANO Nº13

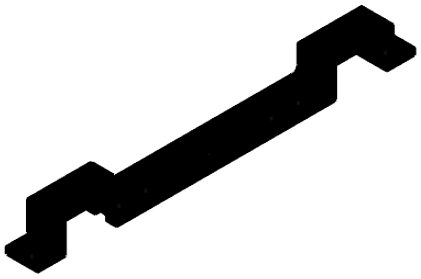
DIMENSIONES CAPARAZÓN SUPERIOR ROBOT

Producto SOLIDWORKS Educational. Solo para uso en la enseñanza.









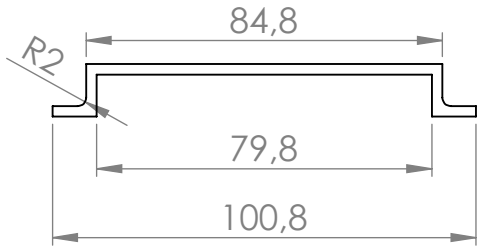
ANTONIO SERRANO CORRAL		**Uds** mm
PROCESO DE FABRICACIÓN FABRIACIÓN ADITIVA	**MATERIAL** PLA	**ESCALA** 1:2

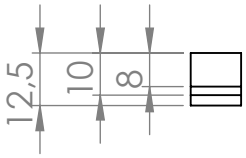
ESCUELA POLITÉCNICA SUPERIOR DE CÓRDOBA
Universidad de Córdoba

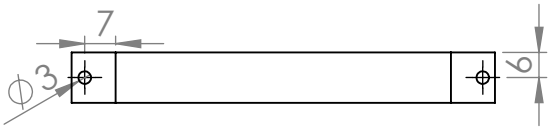
PLANO Nº14

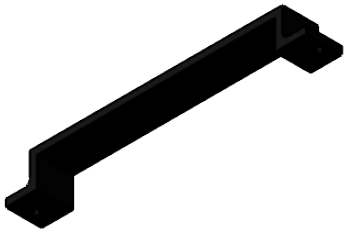
DIMENSIONES DIRECCIÓN ROBOT

Producto SOLIDWORKS Educational. Solo para uso en la enseñanza.









ANTONIO SERRANO CORRAL		**Uds** mm
PROCESO DE FABRICACIÓN FABRIACIÓN ADITIVA	**MATERIAL** PLA	**ESCALA** 1:2

ESCUELA POLITÉCNICA SUPERIOR DE CÓRDOBA
Universidad de Córdoba

PLANO Nº15

DIMENSIONES ELEMENTO UNION SERVOS ROBOT

Producto SOLIDWORKS Educational. Solo para uso en la enseñanza.

Capítulo 4:
PLIEGO DE CONDICIONES

El **pliego de Condiciones** constituye uno de los documentos básicos del Proyecto. Tiene como misión establecer las condiciones técnicas, económicas, administrativas y legales para que el objeto del Proyecto pueda materializarse en las condiciones especificadas, evitando posibles interpretaciones diferentes de las deseadas.

Índice del Pliego de Condiciones

1. Condiciones facultativas 1
1.1 Director de obra 1
1.2 Instalador 1
1.3 Verificación de los documentos del proyecto 1
1.4 Trabajos no estipulados expresamente 1
1.5 Interpretaciones y aclaraciones en los documentos del proyecto 1
1.6 Trabajos defectuosos 1
1.7 Plazo de garantía 1
1.8 Mantenimiento durante el plazo de garantía 2
1.9 Prórroga del plazo de garantía 2
2. Condiciones económicas 2
2.1 Variaciones en los precios unitarios 2
2.2 Pagos a proveedores 2
2.3 Demora de pagos 2
2.4 Segura durante la ejecución 2
3. Condiciones técnicas 2
3.1 Requisitos técnicos del sistema de control remoto 2
3.2 Diseño del mando emisor 3

1. Condiciones facultativas

1.1 Director de obra

Son competencia del director de obra las siguientes tareas:

- Redactar complementos o modificaciones del proyecto.
- Asesorar al promotor en la recepción de la máquina.
- Controlar la correcta ejecución del proyecto.
- Realizar la inspección necesaria en cuestiones de materiales, subcontratas y dispositivos.
- Suscribir el certificado final de la obra.
- Realizar las comprobaciones y mediciones necesarias durante la ejecución

1.2 Instalador

Son competencia del instalador las siguientes tareas:

- Organizar los trabajos respecto a lo indicado en el proyecto.
- Informar al director de obra sobre el proceso y su estado.
- Asegurar la condición de los materiales y herramientas utilizados.
- Realizar la instalación respecto a lo proyectado.

1.3 Verificación de los documentos del proyecto

Previo al comienzo de la instalación, el instalador deberá confirmar por escrito que la documentación aportada es suficiente y comprende la totalidad de la instalación a realizar, solicitando aclaraciones en caso contrario.

1.4 Trabajos no estipulados expresamente

Queda a disposición del instalador la realización de acciones no estipuladas en el proyecto, teniendo como fin la ejecución correcta en funcionalidad y aspecto de la instalación, siempre que no afecte en la medida de lo posible a los presupuestos.

1.5 Interpretaciones y aclaraciones en los documentos del proyecto

Cualquier reinterpretación o aclaración realizada en el proyecto se deberá notificar al instalador, sustituyendo todos los documentos modificados para evitar confusión. Además, el instalador deberá volver a verificar que comprende las modificaciones realizadas y la documentación modificada es suficiente para tal efecto.

1.6 Trabajos defectuosos

Quedará a responsabilidad del instalador la aparición de errores en el funcionamiento o aspecto del producto final producidos por una instalación errónea o deficiente, no siendo el caso para casos de materiales defectuosos. El instalador deberá responsabilizarse en la rectificación de la ejecución, así como los daños materiales causados por esta.

1.7 Plazo de garantía

El producto final tendrá un plazo de garantía de 12 meses desde la finalización de la ejecución y cubrirá cualquier mal funcionamiento o error que se produzca en la misma siempre que este sea provocado por una instalación deficiente, un defecto de fábrica de los componentes comerciales o un defecto previo en los materiales. En ningún caso se cubrirán defectos producidos por mal uso o accidente externo sobre el aparato.

1.8 Mantenimiento durante el plazo de garantía

El mantenimiento durante el plazo de garantía queda a disposición de la empresa cliente. Las pautas y tiempos de revisión de mantenimiento serán tuteladas del instalador al responsable del producto por parte de la empresa.

1.9 Prórroga del plazo de garantía

En caso de ocurrir un mal funcionamiento del producto tras su instalación por las causas cubiertas por la garantía se procederá a la solución del mismo. Tras esto se prorrogará el plazo de garantía durante seis meses tras la solución aportada.

2. Condiciones económicas

2.1 Variaciones en los precios unitarios

En el presente proyecto se aceptarán las variaciones en los precios unitarios que sean subsanables mediante la partida alzada del cinco por ciento del presupuesto dispuesta para tal fin. En caso de superarse dicha partida será necesaria la revisión del presupuesto y un estudio de los proveedores disponibles.

2.2 Pagos a proveedores

El pago a proveedores se realizará en el momento de la confirmación del pedido mediante su correspondiente factura. Así mismo, el abonado del total del presente proyecto deberá realizarse al aceptarse por parte de la empresa promotora el presente documento y condiciones mediante factura.

2.3 Demora de pagos

Un retraso en los pagos por parte del cliente representará un retraso de igual magnitud en el comienzo de la ejecución.

2.4 Segura durante la ejecución

Durante la ejecución del presente proyecto, tanto los materiales como herramientas quedarán a disposición de la empresa promotora y serán amparados por su seguro. En cuanto a los trabajadores presentes en la ejecución serán contratados y estarán cubiertos por el seguro laboral según convenio.

3. Condiciones técnicas

3.1 Requisitos técnicos del sistema de control remoto

El sistema de control remoto objetivo tiene que cumplir los siguientes objetivos:

- El vehículo no tripulado agrícola debe realizar de forma remota los movimientos de traslación (hacia adelante y hacia atrás), dirección en ambos sentidos del eje delantero, nivelación manual y automática (haciendo uso del inclinómetro instalado), configuración de distintos modos de conducción (lento, normal y rápido) y el paro de emergencia en situación de riesgo.
- Se debe incorporar en el mando emisor otras funciones auxiliares que en un futuro realizará el vehículo no tripulado. Estas funciones son:
  - Activación de bocina
  - Activación de luces
  - Activación de bomba auxiliar para el movimiento de un futuro apero
  - Arranque del vehículo

  - Aceleración del motor propulsor
  - Activación del un modo autónomo del vehículo
  - Configuración de distintos modos de giro (dos ruedas, cuatro ruedas, modo cangrejo)
- La comunicación entre el emisor y el receptor se debe realizar vía radio y en una banda de frecuencia libre y de dominio público. Se debe tener en cuenta la **Orden IET/1311/2013, de 9 de Julio**, donde se aprueba el Reglamento de uso del dominio público radioeléctrico por radioaficionados.
- La distancia mínima entre el emisor y el receptor debe ser al menos de 100 metros.
- La comunicación entre el receptor y el PLC del vehículo debe ser acorde al protocolo CAN. Es por ello, que se debe tener en cuenta la **Norma ISO 11898**, ya que es un estándar que define el protocolo CAN.
- El sistema de control debe darle prioridad máxima cuando se active la parada de emergencia del vehículo.
- El sistema de control remoto debe de tener al menos una autonomía de 3 horas.
- El diseño de la PCB del emisor debe cumplir la **norma IPC 2221B** Norma genérica sobre diseño de circuito impreso.

3.2 Diseño del mando emisor

El diseño del mando emisor del sistema de control remoto debe cumplir los siguientes requerimientos:

- El mando debe diseñarse de modo que incluya seta emergencia y estar colocada de forma que sea claramente visible para el operario, de acuerdo al **Real Decreto 1644/2008, de 10 de Octubre**, donde se establecen las normas para la comercialización y puesta en servicio de las máquinas. Concretamente, el **Anexo 1: Requisitos esenciales de seguridad y salud,** relativos al diseño y la fabricación de máquinas.
- El mando debe diseñarse de modo que sea ergonómico para el operario, evitando filos punzantes y cortantes y de un color vistoso para que pueda ser identificado correctamente, de acuerdo al **Real Decreto 1644/2008, de 10 de Octubre**.
- A la hora de fabricar el mando mediante impresión 3D, se debe tener en cuenta la norma **UNE-EN ISO/ASTM 52910:2020. Fabricación aditiva. Diseño. Requisitos, directrices y recomendaciones.**
- Se debe incluir una pantalla tipo HMI donde se indiquen mediante testigos de forma clara el estado de cada una de las funciones del mando.

Capítulo 5: ANEXOS

Los **anexos** constituyen uno de los documentos básicos del Proyecto y está formado por documentos que desarrollan, justifican o aclaran apartados específicos de la memoria u otros documentos básicos del Proyecto.

Índice de anexos

ANEXO N°1: Código de programación del emisor

ANEXO N°2: Código de programación del receptor

ANEXO N°3: Código de programación del robot simulador

ANEXO N°4: Datasheet MCP2515

ANEXO N°5: Datasheet NRF24L01

ANEXO N°1: Código de programación del emisor

```cpp
#include <Arduino.h>
#include <SPI.h>
#include <nRF24L01.h>
#include <RF24.h>
#include <RF24_config.h>
#include <Nextion.h>
#define nextion Serial2
Nextion myNextion(nextion, 9600);

#define DEBUG true
#define OLED false
#define NEXTION true

#if OLED == true

  #include <Wire.h>      // libreria para bus I2C
  #include <Adafruit_GFX.h>
  #include <Adafruit_SSD1306.h>

  //PANTALLA
  #define ANCHO 128     // reemplaza ocurrencia de ANCHO por 128
  #define ALTO 64       // reemplaza ocurrencia de ALTO por 64
  #define OLED_RESET 4      // necesario por la libreria pero no usado
  Adafruit_SSD1306 oled(ANCHO, ALTO, &Wire, OLED_RESET);  // crea objeto

#endif


//CONFIGURACION



//MODOS DE CONDUCCIÓN
const uint8_t FactorEco = 4;
const uint8_t FactorNormal = 2;
const uint8_t FactorSport = 1;

const uint8_t CE_PIN = 9;   //Declaremos los pines CE y el CSN
const uint8_t CSN_PIN = 10;

byte RFPipe[5] ={0x60, 0x59, 0x70, 0x54, 0x01}; //Variable con la dirección del canal por donde se va a transmitir

const int EstoyVivoPeriod = 100;      //Periodo de envio de la señal Estoy Vivo (en milisegundos)

const uint8_t ValueSteps = 5; //Incrementos en los valores leidos por los potenciometros
const int8_t ValueZero = 6;   //Valor a partir del cual la lectura analógica no se considera 0 (incluido este valor).

const uint8_t PinNivelacion = A13;    //Pin Analogico donde está conectado el potenciometro para la nivelacion
const uint8_t PinGiro = A12;          //Pin Analogico donde está conectado el potenciometro para el giro
const uint8_t PinVelocidad = A11;     //Pin Analogico donde está conectado el potenciometro para la velocidad
const uint8_t PinAceleracion= A14;    //Pin Analogico donde está conectado el acelerador del motor (r.p.m)
const uint8_t PinBombAux= A6;         //Queda por hacer
```

```
53 const uint8_t PinModoConduccion = A2; //Pin Analogico para el modo de conducción
   (Eco, Normal o Sport)
54 const uint8_t PinModoLuces = A5;      //Pin Analogico para leer el modo de las
   luces
55 const uint8_t PinModoGiro = A1;       //Pin Analogico para el modo de giro (normal,
   cangrejo, super)
56 const uint8_t Bocina= 3;              //Pin digital donde está conectado el cable
   de control (modo PULL DOWN) de la bocina
57 const uint8_t Arranque= 5;            //Pin digital donde está conectado el cable
   de control (modo PULL DOWN) del arranque
58 const uint8_t InterruptorRF= 11;      //Pin digital que desactiva todas las
   funciones del mando inalámbrico salvo la seta de emergencia
59 const uint8_t SetaEmergencia= 12;     //Pin digital Seta de Emergencia
60 const uint8_t NivAuto=7;              //Futura nivelación automática del vehículo
61 const uint8_t Autonomo=8;             //Pin digital para activar el modo
   autónomo/manual
62
63 const uint8_t idModulo[4] = {0, 0, 0, 101};
64 const uint8_t idNivelacion = 5;
65 const uint8_t idVelocidad = 2;
66 const uint8_t idAceleracion= 10;
67 const uint8_t idGiro= 3;
68 const uint8_t idBocina= 102;
69 const uint8_t idArranque= 50;
70 const uint8_t idNivAuto= 51;
71 const uint8_t idAutonomo= 200;
72 const uint8_t idLuces= 100;
73 const uint8_t idBombaAux= 60;
74 //----------------------------------------
75
76
77
78
79 //DEFINICIONES
80
81 RF24 radio(CE_PIN, CSN_PIN);  //creamos el objeto radio (NRF24L01)
82 int8_t ReadAnalogValue(uint8_t port, int portmin, int portmax, int8_t outmin,
   int8_t outmax, uint8_t steps);
83 bool ReadDigitalValue(uint8_t port);
84 int8_t CalcularValorPasos(int8_t value, uint8_t steps);
85 void EnviarEstoyVivo();
86 void LeeEnviarValores();
87 void LeerEnviarValorNivelacion();
88 void LeerEnviarValorVelocidad();
89 void LeerEnviarValorGiro();
90 void LeerEnviarValorBocina();
91 void LeerEnviarValorArranque();
92 void LeerEnviarValorSetaEmergencia();
93 void LeerEnviarValorRF();
94 void LeerEnviarValorBombaAux();
95 void LeerEnviarValorNivAuto();
96 void LeerEnviarValorAceleracion();
97 void LeerEnviarValorAutonomo();
98 void LeerEnviarValorLuces();
99 void imprimeNextionGauge ();
100 void statusbattery ();
101
102 unsigned long lastTimeEstoyVivo = 0;
103  int8_t lastValueNivelacion = 0;
104  int8_t lastValueVelocidad = 0;
```

```
105  int8_t lastValueGiro = 0;
106  int8_t lastValueAceleracion =0;
107  uint8_t lastValueModoConduccion = 0;
108
109  bool lastValueBocina =false;
110  bool lastValueArranque =false;
111  bool lastValueSetaEmergencia= false;
112  bool lastValueRF= false;
113
114  bool lastValueNivAuto= false;
115  bool lastValueAutonomo= false;
116  uint8_t lastValueModoGiro = 0;
117
118  int lastvalueluces=0;
119  int8_t lastValueBombAux =0;
120
121
122 //----------------------------------------
123
124
125 void setup() {
126
127   //Tiempo para poder cargar el ARDUINO
128   //delay(2000);
129
130   //NEXTION
131   #if NEXTION == true
132   Serial.begin(9600);
133   myNextion.init();
134   delay(2500);
135   myNextion.sendCommand("page 1");
136   delay(2500);
137   myNextion.sendCommand("page 2");
138
139 //Inicializamos valores
140 myNextion.setComponentText("page2.t0","0");
141 myNextion.setComponentText("page2.t1","MANUAL");
142 myNextion.setComponentText("page2.t2","0");
143 myNextion.setPictureValue("page2.p0",3);
144   #endif
145
146
147   // //PANTALLA
148   #if OLED == true
149     Wire.begin();                                  // inicializa bus I2C
150     oled.begin(SSD1306_SWITCHCAPVCC, 0x3C);   // inicializa pantalla con direccion
    0x3C
151   #endif
152
153   //Configuramos el puerto serie si tenemos la depuracion activa
154   #if DEBUG == true
155     Serial.begin(115200);
156   #endif
157
158   radio.begin();  //inicializamos el NRF24L01
159   radio.openWritingPipe(RFPipe); //Abrimos un canal de escritura
160
161   lastTimeEstoyVivo = millis();
162
163   //Pines digitales
```

```
164   pinMode(Bocina,INPUT);
165   pinMode(Arranque,INPUT);
166   pinMode(SetaEmergencia,INPUT);
167   pinMode(InterruptorRF,INPUT);
168   pinMode(NivAuto,INPUT);
169   pinMode(Autonomo,INPUT);
170 
171   LeeEnviarValores();
172 
173   //Leer variables 1 vez para saber en que posición se quedaron (solo en
    interruptores, los pulsadores vuelven a su posición)
174   //Leer todas las variables al principio
175   // LeerEnviarValorNivelacion();
176   // LeerEnviarValorVelocidad();
177   // LeerEnviarValorGiro();
178   // LeerEnviarValorBocina();
179   // LeerEnviarValorArranque();
180   // LeerEnviarValorSetaEmergencia();
181   // LeerEnviarValorRF();
182   // LeerEnviarValorNivAuto();
183   // LeerEnviarValorAceleracion();
184   // LeerEnviarValorAutonomo();
185   // LeerEnviarValorLuces();
186   // LeerEnviarValorBombaAux();
187 
188 }
189 
190 void loop(){
191 
192   //Leemos los valores y enviamos los datos.
193   LeeEnviarValores();
194 
195   //Enviamos la señal de Estoy Vivo.
196   EnviarEstoyVivo();
197 
198 
199 }
200 
201 
202 void LeeEnviarValores() {
203 
204   //Leer RF
205   LeerEnviarValorRF();
206 
207   //Seta de emergencia
208   LeerEnviarValorSetaEmergencia();
209 
210   //Modo autónomo/manual
211    LeerEnviarValorAutonomo();
212 
213    if (lastValueSetaEmergencia == false && lastValueRF == true && lastValueAutonomo
    == false) {
214 
215     //Nivelacion
216     LeerEnviarValorNivelacion();
217 
218     //Velocidad
219     LeerEnviarValorVelocidad();
220 
221     //Giro
```

```
222     LeerEnviarValorGiro();
223 
224     //Aceleracion del motor
225     LeerEnviarValorAceleracion();
226 
227     //Bocina
228     LeerEnviarValorBocina();
229 
230     //Arranque
231     LeerEnviarValorArranque();
232 
233     //Bomba auxiliar
234     LeerEnviarValorBombaAux();
235 
236     //Nivelación Automática
237     LeerEnviarValorNivAuto();
238 
239     //Luces
240     LeerEnviarValorLuces();
241 
242     //Bomba Auxiliar
243     LeerEnviarValorBombaAux();
244 
245     #if NEXTION ==true
246     //Gauge Nextion
247     imprimeNextionGauge();
248     statusbattery();
249     #endif
250 
251 
252    }
253 
254 
255 }
256 #if NEXTION == true
257 void imprimeNextionGauge() {
258 int  leerPosPotenciometro= analogRead(A14);
259   int lecturaAnalogicaNextion= map(leerPosPotenciometro,0,1023,0,242); //PASAR A
    GRADOS
260   if (lecturaAnalogicaNextion <20){
261     lecturaAnalogicaNextion+=315;
262   }
263   else{
264     lecturaAnalogicaNextion-=21;
265   }
266 myNextion.setComponentValue("page2.z0",lecturaAnalogicaNextion);
267 }
268 
269 void statusbattery() {
270 float valueAnalogBattery=analogRead(A0);
271 float volt_battery = ((((valueAnalogBattery*5)/1023)*3.208)+1.35);
272 int volt_battery_porcent = (((volt_battery - 9)*100) / (12.2-9));
273 
274 String datolecturaString_Bat =String(volt_battery_porcent);
275 myNextion.setComponentText("page2.t3",datolecturaString_Bat);
276 
277 if (volt_battery_porcent>75 && volt_battery_porcent<=100){
278   myNextion.setPictureValue("page2.p6",21);
279 }
280 else if (volt_battery_porcent>35 && volt_battery_porcent<=75){
```

```
281   myNextion.setPictureValue("page2.p6",23);
282 }
283 else if (volt_battery_porcent>10 && volt_battery_porcent<=35){
284   myNextion.setPictureValue("page2.p6",22);
285 }
286 else {
287   myNextion.setPictureValue("page2.p6",24);
288 }
289 }
290 #endif
291 
292 void LeerEnviarValorNivelacion() {
293 
294   byte datos[12]; // Desde el byte 0 al 11
295 
296   //Leemos el potenciometro de la nivelación
297   int8_t currentValueNivelacion = ReadAnalogValue(PinNivelacion, 0, 1023, -100,
    100, ValueSteps);
298 
299   #if DEBUG == true
300     Serial.println("Nivelacion :");
301     Serial.println(currentValueNivelacion);
302   #endif
303 
304   if(currentValueNivelacion != lastValueNivelacion) {  //Si el valor es distinto,
    envia el nuevo valor y setea el valor actual como el ultimo.
305     #if NEXTION == true
306     String datolecturaString_Niv_2 =String(currentValueNivelacion);
307     myNextion.setComponentText("page2.t2",datolecturaString_Niv_2);
308     #endif
309 
310     datos[0] = idModulo[0];
311     datos[1] = idModulo[1];
312     datos[2] = idModulo[2];
313     datos[3] = idModulo[3];
314     datos[4] = idNivelacion;
315 
316     if (currentValueNivelacion < 0) {
317 
318       //Si la nivelación es negativa se entiende que es el lado derecho el que debe
    subir (pistones 2 y 4)
319       datos[5] = 0; //piston 1
320       datos[6] = -currentValueNivelacion; //piston 2
321       datos[7] = 0; //piston 3
322       datos[8] = -currentValueNivelacion; //piston 4
323 
324     }
325     else {
326 
327       //Si la nivelación es positiva se entiende que es el lado izquierdo el que
    debe subir (pistones 1 y 3)
328       datos[5] = currentValueNivelacion; //piston 1
329       datos[6] = 0; //piston 2
330       datos[7] = currentValueNivelacion; //piston 3
331       datos[8] = 0; //piston 4
332 
333     }
334 
335     datos[9] = 0;
336     datos[10] = 0;
```

```
337     datos[11] = 0;
338 
339 
340     radio.write(datos, sizeof(datos));  //Se envian los datos por RF
341 
342     lastValueNivelacion = currentValueNivelacion;  //Se establece la nueva medicion
    como la ultima.
343 
344   }
345 
346 }
347 
348 
349 
350 void LeerEnviarValorVelocidad() {
351 
352   byte datos[12]; // Desde el byte 0 al 11
353 
354   //Leemos el potenciometro de velocidad
355   int8_t currentValueVelocidad = ReadAnalogValue(PinVelocidad, 0, 1023, -100, 100,
    ValueSteps);
356   Serial.println(currentValueVelocidad);
357 
358   #if DEBUG == true
359     Serial.println("Velocidad :");
360     Serial.println(currentValueVelocidad);
361   #endif
362 
363   //Leemos el modo de conducción
364   int value = analogRead(PinModoConduccion);
365   uint8_t factor = 1;
366 
367   if (value>=450 && value<=550) {
368     factor= FactorSport;
369   } else if (value>=850 && value<=950){
370     factor= FactorEco;
371   } else {
372     factor= FactorNormal;
373   }
374 
375   if (lastValueModoConduccion!=factor) {
376     #if NEXTION == true
377       if (factor == FactorEco) {
378         myNextion.setPictureValue("page2.p4",13);
379       }
380       else if (factor == FactorSport) {
381         myNextion.setPictureValue("page2.p4",15);
382       }
383       else{
384         myNextion.setPictureValue("page2.p4",14);
385       }
386     #endif
387     #if OLED == true
388 
389       if (factor == FactorEco) {
390 
391         oled.clearDisplay();                  // limpia pantalla
392         oled.setTextColor(WHITE);             // establece color al unico
    disponible (pantalla monocromo)
```

```
393         oled.setCursor(0, 0);                 // ubica cursor en inicio de
    coordenadas 0,0
394         oled.setTextSize(1);                  // establece tamano de texto en 1
395         oled.print(" MODO DE CONDUCCION");    // escribe en pantalla el texto
396         oled.setCursor (10, 30);              // ubica cursor en coordenas 10,30
397         oled.setTextSize(2);                  // establece tamano de texto en 2
398         oled.print("  ECO");                  // escribe en pantalla el texto
399         oled.display();                       // muestra en pantalla todo lo
    establecido anteriormente
400
401       } else if (factor == FactorSport) {
402
403         oled.clearDisplay();                  // limpia pantalla
404         oled.setTextColor(WHITE);             // establece color al unico
    disponible (pantalla monocromo)
405         oled.setCursor(0, 0);                 // ubica cursor en inicio de
    coordenadas 0,0
406         oled.setTextSize(1);                  // establece tamano de texto en 1
407         oled.print(" MODO DE CONDUCCION");    // escribe en pantalla el texto
408         oled.setCursor (10, 30);              // ubica cursor en coordenas 10,30
409         oled.setTextSize(2);                  // establece tamano de texto en 2
410         oled.print("  SPORT");                // escribe en pantalla el texto
411         oled.display();                       // muestra en pantalla todo lo
    establecido anteriormente
412
413       } else {
414
415         oled.clearDisplay();                  // limpia pantalla
416         oled.setTextColor(WHITE);             // establece color al unico
    disponible (pantalla monocromo)
417         oled.setCursor(0, 0);                 // ubica cursor en inicio de
    coordenadas 0,0
418         oled.setTextSize(1);                  // establece tamano de texto en 1
419         oled.print(" MODO DE CONDUCCION");    // escribe en pantalla el texto
420         oled.setCursor (10, 30);              // ubica cursor en coordenas 10,30
421         oled.setTextSize(2);                  // establece tamano de texto en 2
422         oled.print("  NORMAL");               // escribe en pantalla el texto
423         oled.display();                       // muestra en pantalla todo lo
    establecido anteriormente
424
425       }
426
427     #endif
428
429     lastValueModoConduccion = factor;
430
431   }
432
433   currentValueVelocidad = currentValueVelocidad/factor;
434   #if NEXTION == true
435   String datolecturaString_Vel =String(currentValueVelocidad);
436   myNextion.setComponentText("page2.t0",datolecturaString_Vel);
437   #endif
438
439   if(currentValueVelocidad != lastValueVelocidad) {  //Si el valor es distinto,
    envia el nuevo valor y setea el valor actual como el ultimo.
440
441
442
443     datos[0] = idModulo[0];
```

```
444     datos[1] = idModulo[1];
445     datos[2] = idModulo[2];
446     datos[3] = idModulo[3];
447     datos[4] = idVelocidad;
448     datos[5] = currentValueVelocidad; //DELANTERA
449     datos[6] = currentValueVelocidad; //DELANTERA
450     datos[7] = 0; //TRASERA
451     datos[8] = 0; //TRASERA
452     datos[9] = 0;
453     datos[10] = 0;
454     datos[11] = 0;
455
456     radio.write(datos, sizeof(datos));  //Se envian los datos por RF
457
458     lastValueVelocidad = currentValueVelocidad;  //Se establece la nueva medicion
    como la ultima.
459
460   }
461
462 }
463
464
465
466 void LeerEnviarValorGiro() {
467
468
469   byte datos[12]; // Desde el byte 0 al 11
470
471   //Leemos el potenciometro de giro
472   int8_t currentValueGiro = ReadAnalogValue(PinGiro, 0, 1023, -100, 100,
    ValueSteps);
473
474   #if DEBUG == true
475     Serial.println("Giro :");
476     Serial.println(currentValueGiro);
477   #endif
478
479   //Modo de giro
480   int currentAnalogModoGiro = analogRead(PinModoGiro);
481   uint8_t currentModoGiro;
482
483   if (currentAnalogModoGiro >= 450 && currentAnalogModoGiro <= 550){
484
485     currentModoGiro = 1;
486
487   } else if (currentAnalogModoGiro >= 850 && currentAnalogModoGiro <= 950) {
488
489     currentModoGiro = 0;
490
491   } else {
492
493     currentModoGiro = 2;
494
495   }
496
497
498   if (lastValueModoGiro != currentModoGiro || lastValueGiro != currentValueGiro) {
499
500     datos[0] = idModulo[0];
501     datos[1] = idModulo[1];
```

```
502     datos[2] = idModulo[2];
503     datos[3] = idModulo[3];
504     datos[4] = idGiro;
505 
506     if (currentModoGiro == 1) {
507 
508       //MODO CANGREJO
509 
510       if (currentValueGiro < 10 and currentValueGiro > -10) {
511 
512         //No gira debido a que el valor de giro no es el mínimo exigido (para
    evitar perturbaciones)
513         datos[5] = 0;
514         datos[6] = 0;
515         datos[7] = 0;
516         datos[8] = 0;
517 
518       } else {
519 
520         //Proximamente solo habrá que controlar un pistón delantero y uno trasero
521         datos[5] = currentValueGiro;
522         datos[6] = currentValueGiro;
523         datos[7] = currentValueGiro;
524         datos[8] = currentValueGiro;
525 
526          #if NEXTION == true
527          myNextion.setPictureValue("page2.p5",19);
528          #endif
529       }
530 
531 
532 
533     } else if (currentModoGiro == 2) {
534 
535       //MODO GIRO COMPLETO
536 
537       if (currentValueGiro < 10 and currentValueGiro > -10) {
538 
539         //No gira debido a que el valor de giro no es el mínimo exigido (para
    evitar perturbaciones)
540         datos[5] = 0;
541         datos[6] = 0;
542         datos[7] = 0;
543         datos[8] = 0;
544 
545       } else {
546 
547         //Proximamente solo habrá que controlar un pistón delantero y uno trasero
548         datos[5] = currentValueGiro;
549         datos[6] = currentValueGiro;
550         datos[7] = -currentValueGiro;
551         datos[8] = -currentValueGiro;
552 
553          #if NEXTION == true
554          myNextion.setPictureValue("page2.p5",18);
555          #endif
556 
557       }
558 
559 
```

```
560     } else {
561 
562       //MODO NORMAL
563 
564       if (currentValueGiro < 10 and currentValueGiro > -10) {
565 
566         //No gira debido a que el valor de giro no es el mínimo exigido (para
    evitar perturbaciones)
567         datos[5] = 0;
568         datos[6] = 0;
569         datos[7] = 0;
570         datos[8] = 0;
571 
572       } else {
573 
574         //Proximamente solo habrá que controlar un pistón delantero y uno trasero
575         datos[5] = currentValueGiro;
576         datos[6] = currentValueGiro;
577         datos[7] = 0;
578         datos[8] = 0;
579 
580          #if NEXTION == true
581          myNextion.setPictureValue("page2.p5",17);
582          #endif
583 
584       }
585 
586     }
587 
588 
589     datos[9] = 0;
590     datos[10] = 0;
591     datos[11] = 0;
592 
593     radio.write(datos, sizeof(datos));  //Se envian los datos por RF
594 
595     lastValueModoGiro = currentModoGiro;
596     lastValueGiro = currentValueGiro;
597 
598 
599   }
600 
601 
602 }
603 
604 
605 void LeerEnviarValorAceleracion() {
606 
607   byte datos[12]; // Desde el byte 0 al 11
608 
609   //Leemos el potenciometro de aceleración del motor
610   int8_t currentValueAceleracion = ReadAnalogValue(PinAceleracion, 0, 1023, 0, 100,
    ValueSteps);
611 
612   if(currentValueAceleracion != lastValueAceleracion) {  //Si el valor es distinto,
    envia el nuevo valor y setea el valor actual como el ultimo.
613 
614     datos[0] = idModulo[0];
615     datos[1] = idModulo[1];
616     datos[2] = idModulo[2];
```

```
    datos[3] = idModulo[3];
    datos[4] = idAceleracion;
    datos[5] = currentValueAceleracion;
    datos[6] = 0;
    datos[7] = 0;
    datos[8] = 0;
    datos[9] = 0;
    datos[10] = 0;
    datos[11] = 0;

    radio.write(datos, sizeof(datos));  //Se envian los datos por RF

    lastValueAceleracion = currentValueAceleracion;  //Se establece la nueva medicion como la ultima.

  }

}


void LeerEnviarValorLuces() {

  byte datos[12]; // Desde el byte 0 al 11

  int value = analogRead(PinModoLuces);

  int lucesapagadas=11;
  int lucesdelanteras=12;
  int lucestodas=13;

  if (value>=450 && value<=550){

    if(lucesapagadas != lastvalueluces){
    #if NEXTION ==true
    myNextion.setPictureValue("page2.p3",10);
    #endif
      datos[0] = idModulo[0];
      datos[1] = idModulo[1];
      datos[2] = idModulo[2];
      datos[3] = idModulo[3];
      datos[4] = idLuces;
      datos[5] = 255;
      datos[6] = 255;
      datos[7] = 255;
      datos[8] = 255;
      datos[9] = 0;
      datos[10] = 0;
      datos[11] = 0;
      radio.write(datos, sizeof(datos));  //Se envian los datos por RF

      //  oled.clearDisplay();
      //  oled.setTextColor(WHITE);
      //  oled.setCursor(0, 0);
      //  oled.setTextSize(1);
      //  oled.print("      LUCES");
      //  oled.setCursor (10, 30);
      //  oled.setTextSize(2);
      //  oled.print("APAGADAS");
      //  oled.display();
      //  delay(500);
```

```
676       //  oled.clearDisplay();
677
678       lastvalueluces = lucesapagadas;
679
680     }
681
682   }
683   else if (value>=850 && value<=950){
684     if(lucestodas != lastvalueluces){
685       #if NEXTION ==true
686       myNextion.setPictureValue("page2.p3",16);
687       #endif
688
689       datos[0] = idModulo[0];
690       datos[1] = idModulo[1];
691       datos[2] = idModulo[2];
692       datos[3] = idModulo[3];
693       datos[4] = idLuces;
694       datos[5] = 0;
695       datos[6] = 0;
696       datos[7] = 0;
697       datos[8] = 0;
698       datos[9] = 0;
699       datos[10] = 0;
700       datos[11] = 0;
701       radio.write(datos, sizeof(datos));  //Se envian los datos por RF
702
703       //  oled.clearDisplay();
704       //  oled.setTextColor(WHITE);
705       //  oled.setCursor(0, 0);
706       //  oled.setTextSize(1);
707       //  oled.print("      LUCES");
708       //  oled.setCursor (10, 30);
709       //  oled.setTextSize(2);
710       //  oled.print(" TODAS");
711       //  oled.display();
712       //  delay(500);
713       //  oled.clearDisplay();
714     }
715     lastvalueluces=lucestodas;
716
717
718   }
719   else {
720     if(lucesdelanteras != lastvalueluces){
721       #if NEXTION ==true
722       myNextion.setPictureValue("page2.p3",11);
723       #endif
724
725       datos[0] = idModulo[0];
726       datos[1] = idModulo[1];
727       datos[2] = idModulo[2];
728       datos[3] = idModulo[3];
729       datos[4] = idLuces;
730       datos[5] = 255;
731       datos[6] = 255;
732       datos[7] = 0;
733       datos[8] = 0;
734       datos[9] = 0;
735       datos[10] = 0;
```

```
736       datos[11] = 0;
737       radio.write(datos, sizeof(datos));  //Se envian los datos por RF
738 
739       //  oled.clearDisplay();
740       //  oled.setTextColor(WHITE);
741       //  oled.setCursor(0, 0);
742       //  oled.setTextSize(1);
743       //  oled.print("      LUCES");
744       //  oled.setCursor (10, 30);
745       //  oled.setTextSize(2);
746       //  oled.print("DELANTERA");
747       //  oled.display();
748       //  delay(500);
749       //  oled.clearDisplay();
750     }
751     lastvalueluces=lucesdelanteras;
752 
753 
754   }
755 }
756 
757 
758 void LeerEnviarValorBocina() {
759 
760   byte datos[12]; // Desde el byte 0 al 11
761 
762   //Leemos el pulsador
763   bool sinobocina = ReadDigitalValue(Bocina);
764 
765   #if DEBUG == true
766     Serial.println("Bocina :");
767     Serial.println(sinobocina);
768   #endif
769 
770   if(sinobocina != lastValueBocina) {  //Si el valor es distinto, envia el nuevo
    valor y setea el valor actual como el ultimo.
771 
772     datos[0] = idModulo[0];
773     datos[1] = idModulo[1];
774     datos[2] = idModulo[2];
775     datos[3] = idModulo[3];
776     datos[4] = idBocina;
777 
778     if(sinobocina == true){ //PULSADOR NA
779 
780        //Si la bocina está pulsada
781       datos[5] = 255;
782       #if NEXTION == true
783       myNextion.setPictureValue("page2.p0",4);
784       #endif
785 
786       #if OLED == true
787         oled.clearDisplay();      // limpia pantalla
788         oled.setTextColor(WHITE);   // establece color al unico disponible
    (pantalla monocromo)
789         oled.setCursor(0, 0);     // ubica cursor en inicio de coordenadas 0,0
790         oled.setTextSize(1);      // establece tamano de texto en 1
791         oled.print("      ACTIVACION");  // escribe en pantalla el texto
792         oled.setCursor (10, 30);    // ubica cursor en coordenas 10,30
793         oled.setTextSize(2);      // establece tamano de texto en 2
```

```
794         oled.print(" BOCINA");  // escribe en pantalla el texto
795         oled.display();     // muestra en pantalla todo lo establecido
    anteriormente
796         delay(300);
797         oled.clearDisplay();      // limpia pantalla
798       #endif
799
800     } else {
801
802        //Si la bocina NO está pulsada
803       datos[5] = 0;
804       #if NEXTION == true
805       myNextion.setPictureValue("page2.p0",3);
806       #endif
807
808     }
809
810     datos[6] = 0;
811     datos[7] = 0;
812     datos[8] = 0;
813     datos[9] = 0;
814     datos[10] = 0;
815     datos[11] = 0;
816
817     radio.write(datos, sizeof(datos));  //Se envian los datos por RF
818
819     lastValueBocina = sinobocina;  //Se establece la nueva medicion como la ultima.
820
821   }
822
823 }
824
825
826
827
828 void LeerEnviarValorArranque() {
829
830   byte datos[12]; // Desde el byte 0 al 11
831   //Leemos el pulsador
832   bool sinoarranque = ReadDigitalValue(Arranque);
833   Serial.println("Arranque: ");
834   Serial.println(sinoarranque);
835   if(sinoarranque != lastValueArranque) {  //Si el valor es distinto, envia el
    nuevo valor y setea el valor actual como el ultimo.
836
837     datos[0] = idModulo[0];
838     datos[1] = idModulo[1];
839     datos[2] = idModulo[2];
840     datos[3] = idModulo[3];
841     datos[4] = idArranque;
842     if(sinoarranque==true){
843
844       datos[5] = 0;
845       datos[6] = 0;
846       datos[7] = 0;
847       datos[8] = 0;
848     }
849     if(sinoarranque==false){
850
851       datos[5] = 255;
```

```
852       datos[6] = 0;
853       datos[7] = 0;
854       datos[8] = 0;
855       //  oled.clearDisplay();      // limpia pantalla
856       //  oled.setTextColor(WHITE);   // establece color al unico disponible
(pantalla monocromo)
857       //  oled.setCursor(0, 0);     // ubica cursor en inicio de coordenadas 0,0
858       //  oled.setTextSize(1);      // establece tamano de texto en 1
859       //  oled.print("      ACTIVACION");  // escribe en pantalla el texto
860       //  oled.setCursor (10, 30);    // ubica cursor en coordenas 10,30
861       //  oled.setTextSize(2);      // establece tamano de texto en 2
862       //  oled.print(" ARRANQUE");  // escribe en pantalla el texto
863       //  oled.display();     // muestra en pantalla todo lo establecido
anteriormente
864       //  delay(1200);
865       //  oled.clearDisplay();      // limpia pantalla
866     }
867     datos[9] = 0;
868     datos[10] = 0;
869     datos[11] = 0;
870 
871     radio.write(datos, sizeof(datos));  //Se envian los datos por RF
872 
873     lastValueArranque = sinoarranque;  //Se establece la nueva medicion como la
ultima.
874   }
875 }
876 
877 void LeerEnviarValorBombaAux() {
878 byte datos[12]; // Desde el byte 0 al 11
879 
880   //Leemos el potenciometro de Bomba Auxiliar
881   int8_t currentValueBombAux = ReadAnalogValue(PinBombAux, 0, 1023, 0, 100,
ValueSteps);
882 
883   if(currentValueBombAux != lastValueBombAux) {  //Si el valor es distinto, envia
el nuevo valor y setea el valor actual como el ultimo.
884 
885     datos[0] = idModulo[0];
886     datos[1] = idModulo[1];
887     datos[2] = idModulo[2];
888     datos[3] = idModulo[3];
889     datos[4] = idBombaAux;
890     datos[5] = currentValueBombAux;
891     datos[6] = 0;
892     datos[7] = 0;
893     datos[8] = 0;
894     datos[9] = 0;
895     datos[10] = 0;
896     datos[11] = 0;
897 
898     #if NEXTION == true
899     String datolecturaString_Niv =String(currentValueBombAux);
900     myNextion.setComponentText("page0.t10",datolecturaString_Niv);
901     #endif
902 
903     radio.write(datos, sizeof(datos));  //Se envian los datos por RF
904 
905     lastValueBombAux = currentValueBombAux;  //Se establece la nueva medicion como
la ultima.
```

```
906
907   }
908
909
910 }
911
912
913
914 void LeerEnviarValorNivAuto() {
915
916   byte datos[12]; // Desde el byte 0 al 11
917
918   //Leemos el pulsador
919   bool sinoNivAuto = ReadDigitalValue(NivAuto);
920
921   #if DEBUG == true
922     Serial.println("Nivelacion Auto: ");
923     Serial.println(sinoNivAuto);
924   #endif
925
926   if(sinoNivAuto != lastValueNivAuto) {  //Si el valor es distinto, envia el nuevo
    valor y setea el valor actual como el ultimo.
927
928     datos[0] = idModulo[0];
929     datos[1] = idModulo[1];
930     datos[2] = idModulo[2];
931     datos[3] = idModulo[3];
932     datos[4] = idNivAuto;
933
934     if(sinoNivAuto==true){
935
936       datos[5] = 0;
937
938       #if NEXTION == true
939       myNextion.setComponentText("page2.t1","MANUAL");
940       #endif
941
942       #if OLED == true
943         oled.clearDisplay();
944         oled.setTextColor(WHITE);
945         oled.setCursor(0, 0);
946         oled.setTextSize(1);
947         oled.print("     NIVELACION");
948         oled.setCursor (5, 30);
949         oled.setTextSize(2);
950         oled.print("AUTOMATICA");
951         oled.display();
952         delay(500);
953         oled.clearDisplay();
954       #endif
955
956
957     } else {
958
959       datos[5] = 255;
960       #if NEXTION == true
961       myNextion.setComponentText("page2.t1","AUTO");
962       #endif
963
964       #if OLED == true
```

```
965         oled.clearDisplay();
966         oled.setTextColor(WHITE);
967         oled.setCursor(0, 0);
968         oled.setTextSize(1);
969         oled.print("    NIVELACION");
970         oled.setCursor (5, 30);
971         oled.setTextSize(2);
972         oled.print("MANUAL");
973         oled.display();
974         delay(500);
975         oled.clearDisplay();
976       #endif
977
978     }
979
980     datos[6] = 0;
981     datos[7] = 0;
982     datos[8] = 0;
983     datos[9] = 0;
984     datos[10] = 0;
985     datos[11] = 0;
986
987     radio.write(datos, sizeof(datos));  //Se envian los datos por RF
988
989     lastValueNivAuto = sinoNivAuto;  //Se establece la nueva medicion como la
    ultima.
990
991   }
992
993 }
994
995
996 void LeerEnviarValorSetaEmergencia() {
997
998   byte datos[12]; // Desde el byte 0 al 11
999
1000  //Leemos el interruptor
1001  bool sinosetaemergencia = ReadDigitalValue(SetaEmergencia);
1002
1003  #if DEBUG == true
1004    Serial.println("Seta de emergencia: ");
1005    Serial.println(sinosetaemergencia);
1006  #endif
1007
1008  if(sinosetaemergencia != lastValueSetaEmergencia) {  //Si el valor es distinto,
    envia el nuevo valor y setea el valor actual como el ultimo.
1009
1010    datos[0] = 255;
1011    datos[1] = 255;
1012    datos[2] = 255;
1013    datos[3] = 255;
1014
1015    if(sinosetaemergencia == false) {
1016      datos[4] = 0;
1017      #if NEXTION == true
1018      myNextion.sendCommand("page 2");
1019      //Al desactivar la SETA DE EMERGENCIA volvemos a las variables mas usadas
1020
1021      myNextion.setPictureValue("page2.p0",3);
1022      myNextion.setPictureValue("page2.p3",16);
```

```
1023        myNextion.setComponentText("page2.t2","0");
1024        myNextion.setComponentText("page2.t1","MANUAL");
1025        myNextion.setPictureValue("page2.p1",6);
1026
1027
1028        #endif
1029      } else {
1030        datos[4] = 255;
1031        #if NEXTION == true
1032        myNextion.sendCommand("page 3");
1033        #endif
1034        #if OLED == true
1035          oled.clearDisplay();
1036          oled.setTextColor(WHITE);
1037          oled.setCursor(0, 0);
1038          oled.setTextSize(1);
1039          oled.print("----------------------");
1040          oled.setCursor (10, 25);
1041          oled.setTextSize(2);
1042          oled.print(" WARNING");
1043          oled.display();
1044        #endif
1045
1046      }
1047
1048      datos[5] = 0;
1049      datos[6] = 0;
1050      datos[7] = 0;
1051      datos[8] = 0;
1052      datos[9] = 0;
1053      datos[10] = 0;
1054      datos[11] = 0;
1055
1056      radio.write(datos, sizeof(datos));  //Se envian los datos por RF
1057
1058      lastValueSetaEmergencia = sinosetaemergencia;  //Se establece la nueva medicion
     como la ultima.
1059
1060    }
1061
1062  }
1063
1064
1065  void LeerEnviarValorRF() {
1066
1067    byte datos[12]; // Desde el byte 0 al 11
1068
1069    //Leemos el interruptor
1070    bool sinoRF = ReadDigitalValue(InterruptorRF);
1071
1072    #if DEBUG == true
1073      Serial.println("RF :");
1074      Serial.println(sinoRF);
1075    #endif
1076
1077    if(sinoRF != lastValueRF) {  //Si el valor es distinto, envia el nuevo valor y
     setea el valor actual como el ultimo.
1078
1079      datos[0] = 255;
1080      datos[1] = 0;
```

```
1081      datos[2] = 0;
1082      datos[3] = 2;
1083
1084      if(sinoRF==true) {datos[4] = 0;
1085      #if NEXTION == true
1086      myNextion.setPictureValue("page2.p1",6);
1087      #endif
1088      }
1089      else {datos[4] = 255;
1090      #if NEXTION == true
1091      myNextion.setPictureValue("page2.p1",7);
1092      #endif
1093      }
1094      datos[5] = 0;
1095      datos[6] = 0;
1096      datos[7] = 0;
1097      datos[8] = 0;
1098      datos[9] = 0;
1099      datos[10] = 0;
1100      datos[11] = 0;
1101
1102      radio.write(datos, sizeof(datos));  //Se envian los datos por RF
1103
1104      lastValueRF = sinoRF;  //Se establece la nueva medicion como la ultima.
1105
1106      #if OLED == true
1107
1108        oled.clearDisplay();
1109        oled.setTextColor(WHITE);
1110        oled.setCursor(4,4);
1111        oled.setTextSize(4);
1112        oled.print(" UCO ");
1113        oled.display();
1114
1115      #endif
1116
1117    }
1118
1119  }
1120
1121  void LeerEnviarValorAutonomo() {
1122
1123    byte datos[12]; // Desde el byte 0 al 11
1124
1125    //Leemos el pulsador
1126    bool sinoAutonomo = ReadDigitalValue(Autonomo);
1127
1128    #if DEBUG == true
1129      Serial.println("Autonomo: ");
1130      Serial.println(sinoAutonomo);
1131    #endif
1132
1133    if(sinoAutonomo != lastValueAutonomo) {  //Si el valor es distinto, envia el
     nuevo valor y setea el valor actual como el ultimo.
1134
1135      datos[0] = idModulo[0];
1136      datos[1] = idModulo[1];
1137      datos[2] = idModulo[2];
1138      datos[3] = idModulo[3];
1139      datos[4] = idAutonomo;
```

```
1140
1141      if(sinoAutonomo == true){
1142
1143        datos[5] = 255;
1144        #if NEXTION == true
1145        myNextion.setPictureValue("page2.p2",8);
1146        #endif
1147
1148        #if OLED == true
1149          oled.clearDisplay();
1150          oled.setTextColor(WHITE);
1151          oled.setCursor(0, 0);
1152          oled.setTextSize(1);
1153          oled.print("      MODO");
1154          oled.setCursor (5, 30);
1155          oled.setTextSize(2);
1156          oled.print("AUTONOMO ");
1157          oled.display();
1158        #endif
1159
1160      } else {
1161
1162        datos[5] = 0;
1163        #if NEXTION == true
1164        myNextion.setPictureValue("page2.p2",12);
1165        #endif
1166
1167        #if OLED == true
1168          oled.clearDisplay();
1169          oled.setTextColor(WHITE);
1170          oled.setCursor(0, 0);
1171          oled.setTextSize(1);
1172          oled.print("      MODO");
1173          oled.setCursor (5, 30);
1174          oled.setTextSize(2);
1175          oled.print(" MANUAL ");
1176          oled.display();
1177          delay(500);
1178          oled.clearDisplay();
1179        #endif
1180
1181      }
1182
1183      datos[6] = 0;
1184      datos[7] = 0;
1185      datos[8] = 0;
1186      datos[9] = 0;
1187      datos[10] = 0;
1188      datos[11] = 0;
1189
1190      radio.write(datos, sizeof(datos));  //Se envian los datos por RF
1191
1192      lastValueAutonomo = sinoAutonomo;  //Se establece la nueva medicion como la
      ultima.
1193
1194    }
1195
1196  }
1197
1198
```

```
1199
1200
1201 int8_t ReadAnalogValue(uint8_t port, int portmin, int portmax, int8_t outmin,
     int8_t outmax, uint8_t steps) {
1202
1203   int valor = analogRead(port); //Leemos el puerto analogico
1204
1205   int8_t conv = map(valor, portmin, portmax, outmin, outmax);  //Convertimos el
     valor leido al rango que necesitamos
1206
1207
1208   if (conv < ValueZero && conv>-ValueZero){     //Si el valor se encuentra entre
     +/- ValueZero, el valor leido del puerto analógico se considera cero.
1209     conv = 0;
1210   }
1211
1212   conv = CalcularValorPasos(conv, steps);  //Convertimos el valor leido al salto
     más proximo según la configuración.
1213
1214   return conv;// En este tipo de funciones si retornamos
1215
1216 }
1217
1218 bool ReadDigitalValue(uint8_t port) {
1219
1220   int valor = digitalRead(port); //Leemos el puerto digital
1221
1222   if (valor>0) return true;
1223   else return false;
1224
1225 }
1226
1227 int8_t CalcularValorPasos(int8_t value, uint8_t steps) {
1228
1229   int8_t value1 = round(value / steps);
1230
1231   return (value1 * steps);
1232
1233 }
1234
1235 void EnviarEstoyVivo() {
1236
1237   if ((millis()-lastTimeEstoyVivo) >= EstoyVivoPeriod) {    //Comprobamos el tiempo
     para determinar si es necesario enviar la señal de ESTOY VIVO
1238
1239     byte datos[12] = {255, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0};
1240
1241     radio.write(datos, sizeof(datos));
1242
1243     lastTimeEstoyVivo = millis();
1244
1245   }
1246
1247 }
```

ANEXO N°2: Código de programación del receptor

```
1
2  #include <Arduino.h>
3  #include <SPI.h>
4  #include <nRF24L01.h>
5  #include <printf.h>
6  #include <RF24.h>
7  #include <RF24_config.h>
8  #include <mcp2515.h>
9
10
11 //CONFIGURACION
12
13 #define DEBUG true
14
15 const uint8_t CE_PIN = 9;   //Declaremos los pines CE y el CSN
16 const uint8_t CSN_PIN = 10;
17 const uint8_t PinCS_MCP2515 = 8;
18 const uint8_t PinEnable = 6;
19
20 byte RFPipe[5] ={0x60, 0x59, 0x70, 0x54, 0x01}; //Variable con la dirección del
   canal por donde se va a transmitir
21
22 const int TimeOutEstoyVivo = 1000;
23 const int periodNoErrors = 250;
24
25
26
27 const CAN_SPEED canspeed = CAN_500KBPS;
28
29 //--------------------------------
30
31
32 //DEFINICIONES
33
34 RF24 radio(CE_PIN, CSN_PIN);  //creamos el objeto radio (NRF24L01)
35 MCP2515 mcp2515(PinCS_MCP2515);
36
37 unsigned long LastTimeEstoyVivo = 0;
38
39
40 bool errorEstoyVivoTimeLimit = false;
41 bool errorSetaEmergencia = false;
42 bool errorControlDesactivado = false;
43 unsigned long LastTimeNoErrors = 0;
44
45
46 void SendCAN(byte * data);
47 void EnviarErrores();
48 void EnviarErrorEstoyVivo();
49 void EnviarErrorControlDesactivado();
50 void EnviarErrorSetaEmergencia();
51 void EnviarNoErrores();
52 bool CheckTimeEstoyVivo();
53 bool CheckEnable();
54 void LeerDatosRF();
55 void LeerDatosCAN();
56 void SplitULong(unsigned long value, uint8_t *b0, uint8_t *b1, uint8_t *b2, uint8_t
   *b3);
57 void EnviarIdPipe();
58
```

```
59
60 //----------------------------------
61
62
63 void setup() {
64
65   #if DEBUG == true
66     Serial.begin(115200);
67   #endif
68
69   //Inicializamos el módulo de RF
70   radio.begin();
71   radio.openReadingPipe(1, RFPipe);
72   radio.startListening();
73
74   //Inicializamos el modulo CAN
75   mcp2515.reset();
76   mcp2515.setBitrate(canspeed);
77   mcp2515.setNormalMode();
78
79   pinMode(PinEnable, INPUT);
80
81   LastTimeEstoyVivo = millis();
82
83 }
84
85
86 void loop() {
87
88
89   //Se leen y procesan los datos que existan disponibles por el canal de RF
90   LeerDatosRF();
91
92   //Leemos los datos disponibles en CAN y los procesamos
93   LeerDatosCAN();
94
95   //Se comprueba el tiempo desde la ultima señal EstoyVivo y en caso necesario envia
   un error.
96   if (!CheckTimeEstoyVivo()) errorEstoyVivoTimeLimit = true;
97   else errorEstoyVivoTimeLimit = false;
98
99   //Se envian los errores si los hay
100  EnviarErrores();
101
102
103 }
104
105
106 void LeerDatosRF() {
107
108   uint8_t datos[12];
109
110   if (radio.available() ) {           //Comprobamos si hay datos que leer
111
112     radio.read(datos,sizeof(datos));  //Leemos los datos disponibles
113
114     if (datos[0] != 255) {           //Si el primer dato es distinto a 255 se trata
   de una trama CAN y debe ser enviada a traves del módulo CAN.
115
116       //Solo enviamos ordenes si el modulo está habilitado
```

```
117       if (CheckEnable()) SendCAN(datos);
118 
119     }
120     else {
121 
122       //Si el primer datos es 255 se trata de unos datos internos y deben ser procesados según corresponda.
123 
124       if (datos[0] == 255 && datos [1] == 0 && datos[2] == 0 && datos[3] == 1) {
125           //Es la señal de Estoy Vivo
126           LastTimeEstoyVivo = millis(); //Establecemos la hora actual como la ultima a la que se recibio la señal
127 
128       }
129       else if (datos[0] == 255 && datos [1] == 0 && datos[2] == 0 && datos[3] == 2) {
130           //Activacion / Desactivacion control inalambrico
131           if (datos[4] == 255) errorControlDesactivado = true;
132           else errorControlDesactivado = false;
133       }
134       else if (datos[0] == 255 && datos [1] == 255 && datos[2] == 255 && datos[3] == 255) {
135           //Seta de emergencia
136           if (datos[4] == 255) errorSetaEmergencia = true;
137           else errorSetaEmergencia = false;
138       }
139 
140     }
141 
142   }
143 
144 }
145 
146 
147 void LeerDatosCAN() {
148 
149   struct can_frame canMsg;
150 
151   //Comprobamos si existen mensajes que leer
152   if (mcp2515.readMessage(&canMsg) == MCP2515::ERROR_OK) {
153 
154     //Comprobamos si es una solicitud de alguna trama
155     if(canMsg.can_id == 201) {
156 
157       byte array_id[4];
158 
159       array_id[0] = canMsg.data[0];
160       array_id[1] = canMsg.data[1];
161       array_id[2] = canMsg.data[2];
162       array_id[3] = canMsg.data[3];
163 
164       unsigned long id_solicitud =* (unsigned long*)(&array_id);
165 
166       switch (id_solicitud)
167       {
168          case 221:  //Es una solicitud para enviar el idPipe del receptor
169             EnviarIdPipe();
170             break;
171 
172          default:
```

```
            break;
      }

    }


  }


}


void EnviarErrores() {

  //Si hay error en la seta de emergencia (está presionada) se envia por CAN
  if (errorSetaEmergencia) {
    EnviarErrorSetaEmergencia();
  }

  //Si hay error en la activación del control inalambrico (está desactivado) se envia por CAN
  if (errorControlDesactivado) {
    EnviarErrorControlDesactivado();
  }

  //Si se ha superado el tiempo limite de espera para recibir una señal de Estoy Vivo, se envia por CAN
  if (errorEstoyVivoTimeLimit) {
    EnviarErrorEstoyVivo();
  }

  //Si no hay errores se envia por CAN para indicar que el modulo receptor funciona correctamente
  if (!errorControlDesactivado && !errorEstoyVivoTimeLimit && !errorSetaEmergencia) {

    //Comprobamos el tiempo desde el ultimo envio. Solo se envia cada cierto periodo para evitar saturar el canal.
    if ((millis() - LastTimeNoErrors) >= periodNoErrors) {
      EnviarNoErrores();
      LastTimeNoErrors = millis();
    }

  }


}



void SendCAN(byte * data) {

  struct can_frame canMsg;

  byte array_id[4];

  array_id[0] = data[3];
  array_id[1] = data[2];
  array_id[2] = data[1];
  array_id[3] = data[0];
```

```
228
229   canMsg.can_id =* (unsigned long*)(&array_id);
230
231   #if DEBUG == true
232     Serial.print("       ");
233     Serial.print(data[0]);
234     Serial.print("   ");
235     Serial.print(data[1]);
236     Serial.print("   ");
237     Serial.print(data[2]);
238     Serial.print("   ");
239     Serial.print(data[3]);
240     Serial.print("   ");
241     Serial.print(data[4]);
242     Serial.print("   ");
243     Serial.print(data[5]);
244     Serial.print("   ");
245     Serial.print(data[6]);
246     Serial.print("   ");
247     Serial.print(data[7]);
248     Serial.print("   ");
249     Serial.print(data[8]);
250     Serial.print("       ");
251     Serial.print(data[9]);
252     Serial.print("       ");
253     Serial.print(data[10]);
254     Serial.print("       ");
255     Serial.print(data[11]);
256     Serial.println("       ");
257   #endif
258
259   canMsg.can_dlc = 8;
260
261   canMsg.data[0] = data[4];
262   canMsg.data[1] = data[5];
263   canMsg.data[2] = data[6];
264   canMsg.data[3] = data[7];
265   canMsg.data[4] = data[8];
266   canMsg.data[5] = data[9];
267   canMsg.data[6] = data[10];
268   canMsg.data[7] = data[11];
269   mcp2515.sendMessage(&canMsg);
270
271
272 }
273
274 bool CheckTimeEstoyVivo() {
275
276     if ((millis() - LastTimeEstoyVivo)<= TimeOutEstoyVivo) {
277       return true;
278     }
279     else {
280       return false;
281     }
282
283 }
284
285 bool CheckEnable() {
286
287   if (digitalRead(PinEnable)) return true;
```

```
288   else return false;
289 
290 }
291 
292 void EnviarIdPipe() {
293 
294   //Trama CAN
295   struct can_frame canMsg;
296   canMsg.can_id = 221;
297   canMsg.can_dlc = 8;
298   canMsg.data[0] = RFPipe[0];
299   canMsg.data[1] = RFPipe[1];
300   canMsg.data[2] = RFPipe[2];
301   canMsg.data[3] = RFPipe[3];
302   canMsg.data[4] = RFPipe[4];
303   canMsg.data[5] = 0;
304   canMsg.data[6] = 0;
305   canMsg.data[7] = 0;
306 
307   mcp2515.sendMessage(&canMsg);
308 
309 }
310 
311 
312 void EnviarNoErrores() {
313 
314   //Trama CAN
315   struct can_frame canMsg;
316   canMsg.can_id = 11;
317   canMsg.can_dlc = 8;
318   canMsg.data[0] = 0;
319   canMsg.data[1] = 0;
320   canMsg.data[2] = 0;
321   canMsg.data[3] = 0;
322   canMsg.data[4] = 0;
323   canMsg.data[5] = 0;
324   canMsg.data[6] = 0;
325   canMsg.data[7] = 0;
326 
327   mcp2515.sendMessage(&canMsg);
328 
329 }
330 
331 void EnviarErrorSetaEmergencia() {
332 
333   //Trama CAN
334   struct can_frame canMsg;
335   canMsg.can_id = 11;
336   canMsg.can_dlc = 8;
337   canMsg.data[0] = 255;
338   canMsg.data[1] = 0;
339   canMsg.data[2] = 0;
340   canMsg.data[3] = 0;
341   canMsg.data[4] = 0;
342   canMsg.data[5] = 0;
343   canMsg.data[6] = 0;
344   canMsg.data[7] = 0;
345 
346   mcp2515.sendMessage(&canMsg);
347 
```

```
348 }
349
350 void EnviarErrorControlDesactivado() {
351
352   //Trama CAN
353   struct can_frame canMsg;
354   canMsg.can_id = 11;
355   canMsg.can_dlc = 8;
356   canMsg.data[0] = 2;
357   canMsg.data[1] = 255;
358   canMsg.data[2] = 0;
359   canMsg.data[3] = 0;
360   canMsg.data[4] = 0;
361   canMsg.data[5] = 0;
362   canMsg.data[6] = 0;
363   canMsg.data[7] = 0;
364
365   mcp2515.sendMessage(&canMsg);
366
367 }
368
369 void EnviarErrorEstoyVivo() {
370
371   //Bytes correspondientes al tiempo limite de espera.
372   uint8_t b0,b1,b2,b3;
373   SplitULong(TimeOutEstoyVivo, &b0, &b1, &b2, &b3);
374
375   //Trama CAN
376   struct can_frame canMsg;
377   canMsg.can_id = 11;
378   canMsg.can_dlc = 8;
379   canMsg.data[0] = 1;
380   canMsg.data[1] = b0;
381   canMsg.data[2] = b1;
382   canMsg.data[3] = b2;
383   canMsg.data[4] = b3;
384   canMsg.data[5] = 0;
385   canMsg.data[6] = 0;
386   canMsg.data[7] = 0;
387
388   mcp2515.sendMessage(&canMsg);
389
390 }
391
392
393 void SplitULong(unsigned long value, uint8_t *b0, uint8_t *b1, uint8_t *b2, uint8_t
    *b3) {
394
395     uint8_t len = sizeof(unsigned long);
396     uint8_t data[len];
397     memcpy(data, &value, len);
398
399     *b0 = data[0];
400     *b1 = data[1];
401     *b2 = data[2];
402     *b3 = data[3];
403
404 }
```

ANEXO N°3: Código de programación del robot simulador

```
#include <SPI.h>
#include <mcp2515.h>
struct can_frame canMsg;// Parece el identificador pero NOSE BIEN
MCP2515 mcp2515(10);
void setup() {
Serial.begin(115200);
mcp2515.reset();
mcp2515.setBitrate(CAN_500KBPS);
//mcp2515.setBitrate (CAN_250KBPS, MCP_16MHZ);
mcp2515.setNormalMode();
}

void loop() {
 if (mcp2515.readMessage(&canMsg) == MCP2515::ERROR_OK) {
    Serial.print(canMsg.can_id, DEC); // print ID
    Serial.print(" ");
//    Serial.print(canMsg.can_dlc, DEC); // print DLC
//    Serial.print(" ");

    for (int i = 0; i<canMsg.can_dlc; i++)  {  // print the data
      Serial.print(canMsg.data[i],DEC);
      Serial.print(" ");
    }
    Serial.print("      ");
 }
 else{
  Serial.print("     No llega trama CAN      ");
 }
 delay(100);
}
```

ANEXO N°4: Datasheet MCP2515

MCP2515

Stand-Alone CAN Controller With SPI™ Interface

Features

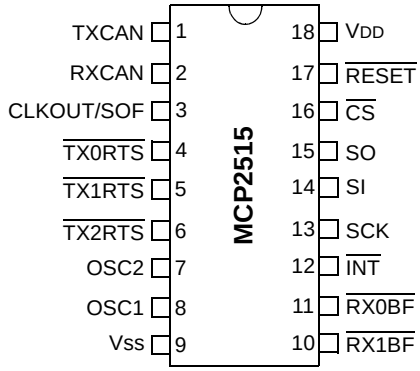
- Implements CAN V2.0B at 1 Mb/s:
  - 0 – 8 byte length in the data field
  - Standard and extended data and remote frames
- Receive buffers, masks and filters:
  - Two receive buffers with prioritized message storage
  - Six 29-bit filters
  - Two 29-bit masks
- Data byte filtering on the first two data bytes (applies to standard data frames)
- Three transmit buffers with prioritizaton and abort features
- High-speed SPI™ Interface (10 MHz):
  - SPI modes 0,0 and 1,1
- One-shot mode ensures message transmission is attempted only one time
- Clock out pin with programmable prescaler:
  - Can be used as a clock source for other device(s)
- Start-of-Frame (SOF) signal is available for monitoring the SOF signal:
  - Can be used for time-slot-based protocols and/or bus diagnostics to detect early bus degredation
- Interrupt output pin with selectable enables
- Buffer Full output pins configurable as:
  - Interrupt output for each receive buffer
  - General purpose output
- Request-to-Send (RTS) input pins individually configurable as:
  - Control pins to request transmission for each transmit buffer
  - General purpose inputs
- Low-power CMOS technology:
  - Operates from 2.7V – 5.5V
  - 5 mA active current (typical)
  - 1 µA standby current (typical) (Sleep mode)
- Temperature ranges supported:
  - Industrial (I): -40°C to +85°C
  - Extended (E): -40°C to +125°C

Description

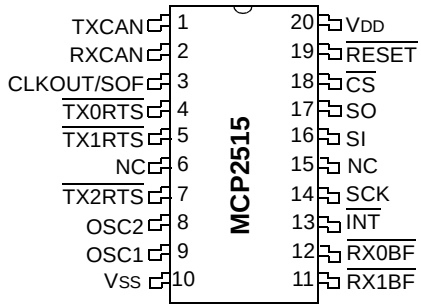
Microchip Technology's MCP2515 is a stand-alone Controller Area Network (CAN) controller that implements the CAN specification, version 2.0B. It is capable of transmitting and receiving both standard and extended data and remote frames. The MCP2515 has two acceptance masks and six acceptance filters that are used to filter out unwanted messages, thereby reducing the host MCUs overhead. The MCP2515 interfaces with microcontrollers (MCUs) via an industry standard Serial Peripheral Interface (SPI).

Package Types

18-Lead PDIP/SOIC



20-LEAD TSSOP



© 2005 Microchip Technology Inc.

NOTES:

 © 2005 Microchip Technology Inc.

1.0 DEVICE OVERVIEW

The MCP2515 is a stand-alone CAN controller developed to simplify applications that require interfacing with a CAN bus. A simple block diagram of the MCP2515 is shown in Figure 1-1. The device consists of three main blocks:

1. The CAN module, which includes the CAN protocol engine, masks, filters, transmit and receive buffers.
2. The control logic and registers that are used to configure the device and its operation.
3. The SPI protocol block.

An example system implementation using the device is shown in Figure 1-2.

1.1 CAN Module

The CAN module handles all functions for receiving and transmitting messages on the CAN bus. Messages are transmitted by first loading the appropriate message buffer and control registers. Transmission is initiated by using control register bits via the SPI interface or by using the transmit enable pins. Status and errors can be checked by reading the appropriate registers. Any message detected on the CAN bus is checked for errors and then matched against the user-defined filters to see if it should be moved into one of the two receive buffers.

1.2 Control Logic

The control logic block controls the setup and operation of the MCP2515 by interfacing to the other blocks in order to pass information and control.

Interrupt pins are provided to allow greater system flexibility. There is one multi-purpose interrupt pin (as well as specific interrupt pins) for each of the receive registers that can be used to indicate a valid message has been received and loaded into one of the receive buffers. Use of the specific interrupt pins is optional. The general purpose interrupt pin, as well as status registers (accessed via the SPI interface), can also be used to determine when a valid message has been received.

Additionally, there are three pins available to initiate immediate transmission of a message that has been loaded into one of the three transmit registers. Use of these pins is optional, as initiating message transmissions can also be accomplished by utilizing control registers, accessed via the SPI interface.

1.3 SPI Protocol Block

The MCU interfaces to the device via the SPI interface. Writing to, and reading from, all registers is accomplished using standard SPI read and write commands, in addition to specialized SPI commands.

FIGURE 1-1: BLOCK DIAGRAM

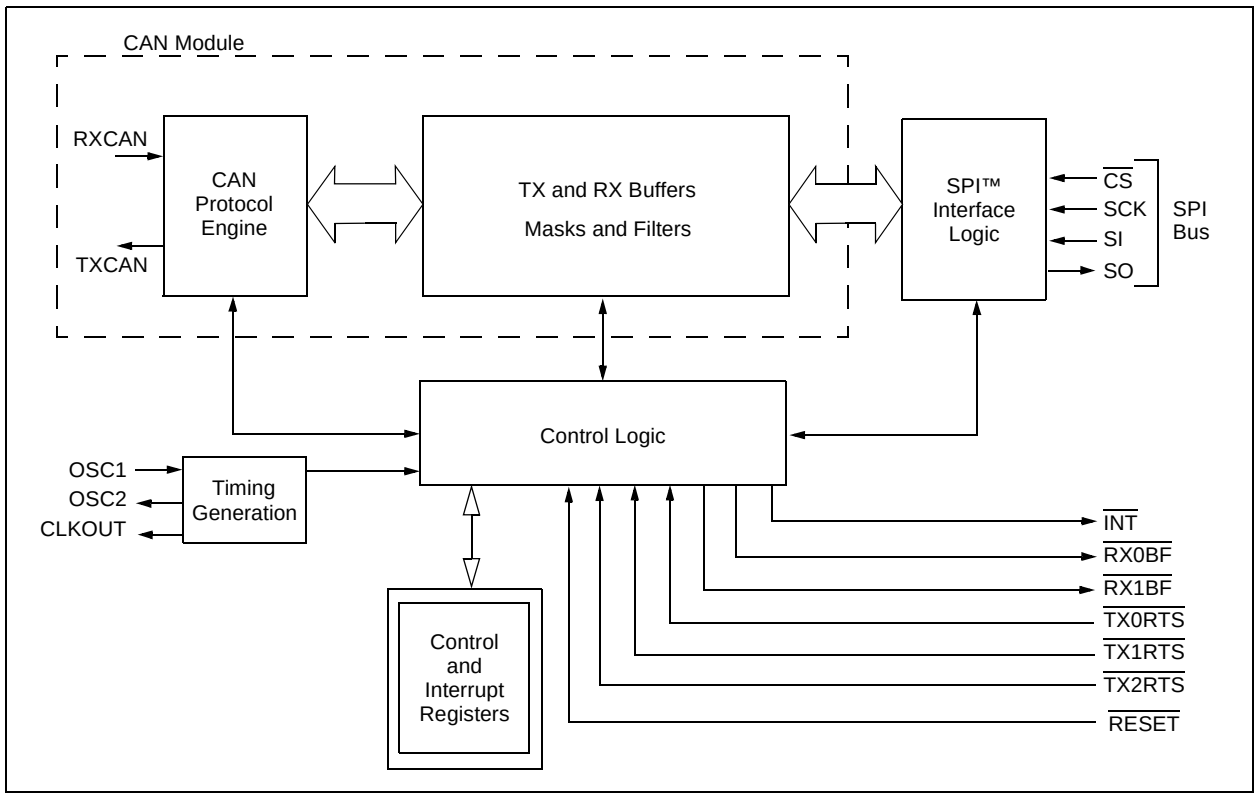


FIGURE 1-2: EXAMPLE SYSTEM IMPLEMENTATION

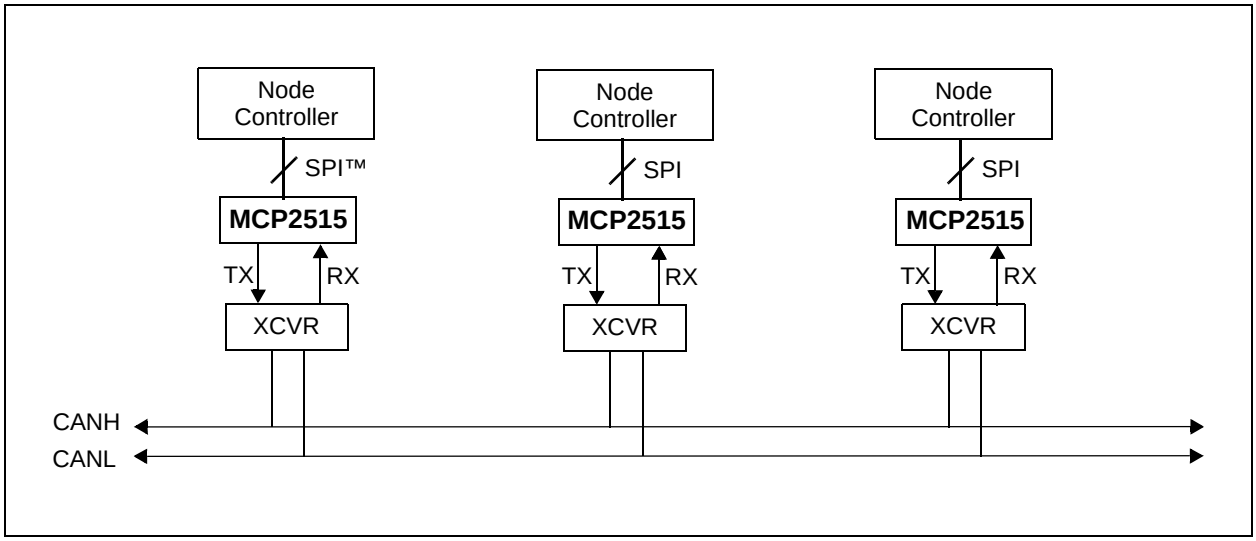


TABLE 1-1: PINOUT DESCRIPTION

Name	PDIP/SOIC Pin #	TSSOP Pin #	I/O/P Type	Description	Alternate Pin Function
TXCAN	1	1	O	Transmit output pin to CAN bus	—
RXCAN	2	2	I	Receive input pin from CAN bus	—
CLKOUT	3	3	O	Clock output pin with programmable prescaler	Start-of-Frame signal
$\overline{\text{TX0RTS}}$	4	4	I	Transmit buffer TXB0 request-to-send. 100 kΩ internal pull-up to VDD	General purpose digital input. 100 kΩ internal pull-up to VDD
$\overline{\text{TX1RTS}}$	5	5	I	Transmit buffer TXB1 request-to-send. 100 kΩ internal pull-up to VDD	General purpose digital input. 100 kΩ internal pull-up to VDD
$\overline{\text{TX2RTS}}$	6	7	I	Transmit buffer TXB2 request-to-send. 100 kΩ internal pull-up to VDD	General purpose digital input. 100 kΩ internal pull-up to VDD
OSC2	7	8	O	Oscillator output	—
OSC1	8	9	I	Oscillator input	External clock input
VSS	9	10	P	Ground reference for logic and I/O pins	—
$\overline{\text{RX1BF}}$	10	11	O	Receive buffer RXB1 interrupt pin or general purpose digital output	General purpose digital output
$\overline{\text{RX0BF}}$	11	12	O	Receive buffer RXB0 interrupt pin or general purpose digital output	General purpose digital output
$\overline{\text{INT}}$	12	13	O	Interrupt output pin	—
SCK	13	14	I	Clock input pin for SPI™ interface	—
SI	14	16	I	Data input pin for SPI interface	—
SO	15	17	O	Data output pin for SPI interface	—
$\overline{\text{CS}}$	16	18	I	Chip select input pin for SPI interface	—
$\overline{\text{RESET}}$	17	19	I	Active low device reset input	—
VDD	18	20	P	Positive supply for logic and I/O pins	—
NC	—	6,15	—	No internal connection	

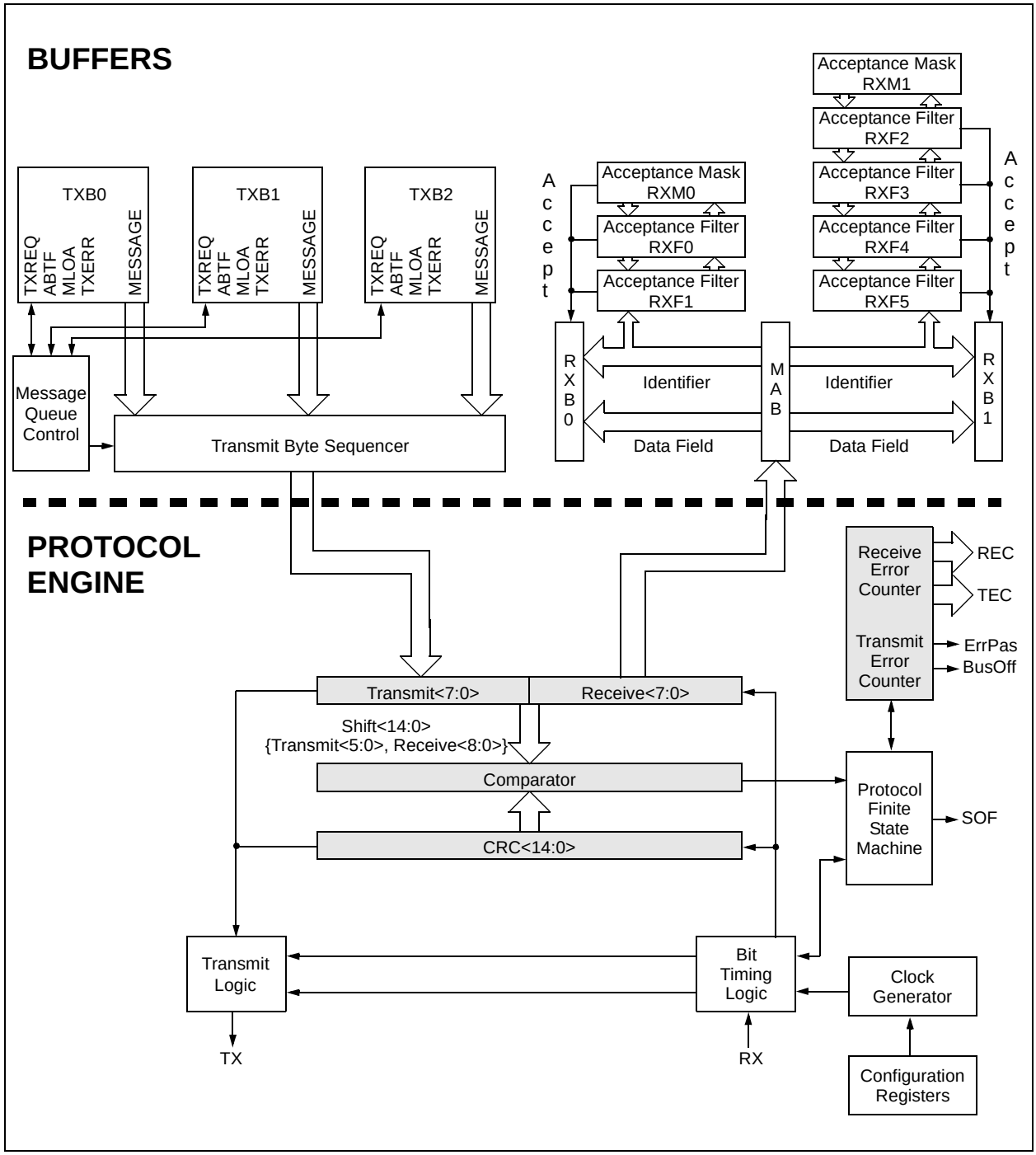
Note: Type Identification: I = Input; O = Output; P = Power

 © 2005 Microchip Technology Inc.

1.4 Transmit/Receive Buffers/Masks/Filters

The MCP2515 has three transmit and two receive buffers, two acceptance masks (one for each receive buffer) and a total of six acceptance filters. Figure 1-3 shows a block diagram of these buffers and their connection to the protocol engine.

FIGURE 1-3: CAN BUFFERS AND PROTOCOL ENGINE BLOCK DIAGRAM



1.5 CAN Protocol Engine

The CAN protocol engine combines several functional blocks, shown in Figure 1-4 and described below.

1.5.1 PROTOCOL FINITE STATE MACHINE

The heart of the engine is the Finite State Machine (FSM). The FSM is a sequencer that controls the sequential data stream between the TX/RX shift register, the CRC register and the bus line. The FSM also controls the Error Management Logic (EML) and the parallel data stream between the TX/RX shift registers and the buffers. The FSM ensures that the processes of reception, arbitration, transmission and error-signaling are performed according to the CAN protocol. The automatic retransmission of messages on the bus line is also handled by the FSM.

1.5.2 CYCLIC REDUNDANCY CHECK

The Cyclic Redundancy Check register generates the Cyclic Redundancy Check (CRC) code, which is transmitted after either the Control Field (for messages with 0 data bytes) or the Data Field and is used to check the CRC field of incoming messages.

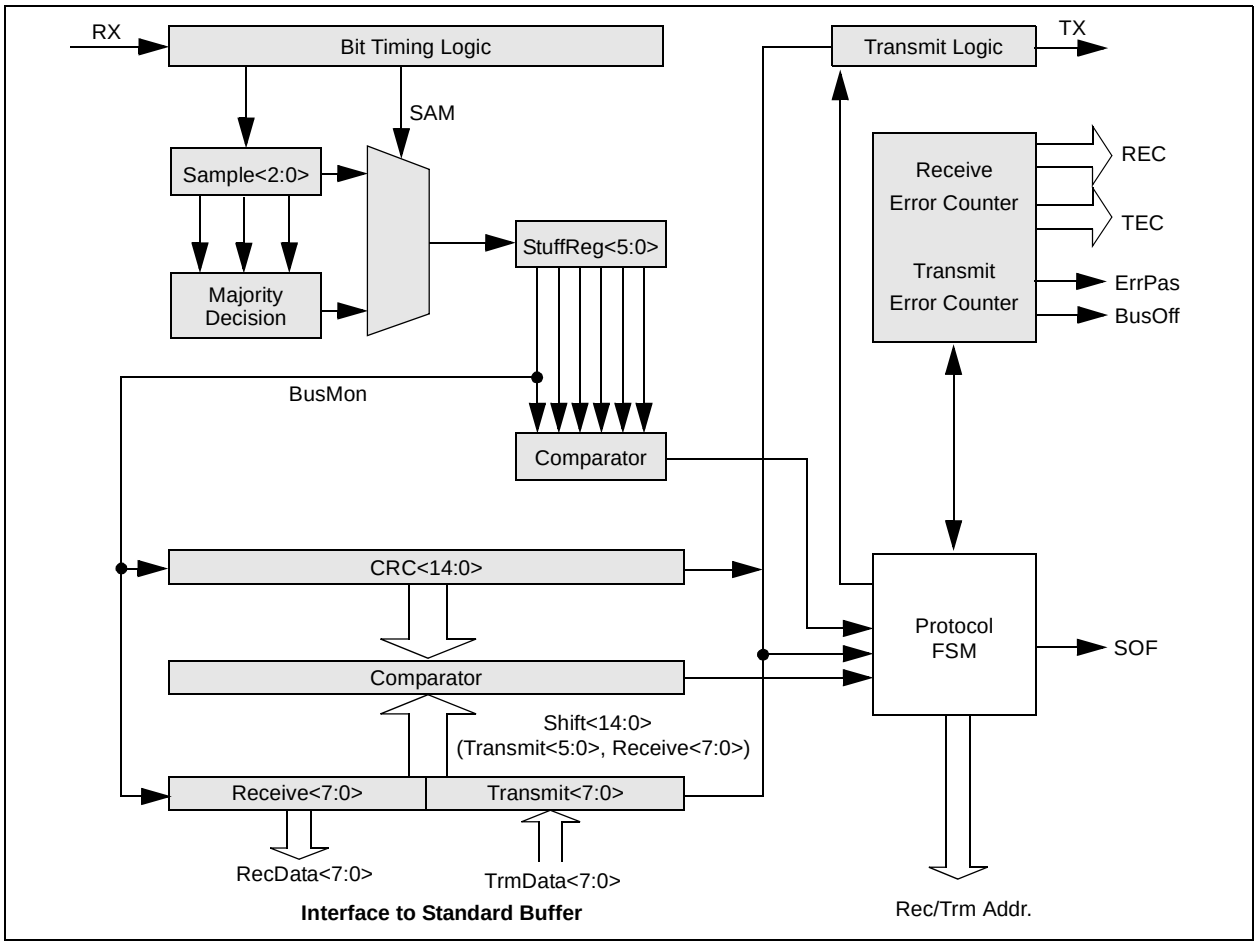
1.5.3 ERROR MANAGEMENT LOGIC

The Error Management Logic (EML) is responsible for the fault confinement of the CAN device. Its two counters, the Receive Error Counter (REC) and the Transmit Error Counter (TEC), are incremented and decremented by commands from the bit stream processor. Based on the values of the error counters, the CAN controller is set into the states error-active, error-passive or bus-off.

1.5.4 BIT TIMING LOGIC

The Bit Timing Logic (BTL) monitors the bus line input and handles the bus-related bit timing according to the CAN protocol. The BTL synchronizes on a recessive-to-dominant bus transition at Start-of-Frame (hard synchronization) and on any further recessive-to-dominant bus line transition if the CAN controller itself does not transmit a dominant bit (resynchronization). The BTL also provides programmable time segments to compensate for the propagation delay time, phase shifts and to define the position of the sample point within the bit time. The programming of the BTL depends on the baud rate and external physical delay times.

FIGURE 1-4: CAN PROTOCOL ENGINE BLOCK DIAGRAM



 © 2005 Microchip Technology Inc.

2.0 CAN MESSAGE FRAMES

The MCP2515 supports standard data frames, extended data frames and remote frames (standard and extended), as defined in the CAN 2.0B specification.

2.1 Standard Data Frame

The CAN standard data frame is shown in Figure 2-1. As with all other frames, the frame begins with a Start-Of-Frame (SOF) bit, which is of the dominant state and allows hard synchronization of all nodes.

The SOF is followed by the arbitration field, consisting of 12 bits: the 11-bit identifier and the Remote Transmission Request (RTR) bit. The RTR bit is used to distinguish a data frame (RTR bit dominant) from a remote frame (RTR bit recessive).

Following the arbitration field is the control field, consisting of six bits. The first bit of this field is the Identifier Extension (IDE) bit, which must be dominant to specify a standard frame. The following bit, Reserved Bit Zero (RB0), is reserved and is defined as a dominant bit by the CAN protocol. The remaining four bits of the control field are the Data Length Code (DLC), which specifies the number of bytes of data (0 – 8 bytes) contained in the message.

After the control field is the data field, which contains any data bytes that are being sent, and is of the length defined by the DLC (0 – 8 bytes).

The Cyclic Redundancy Check (CRC) field follows the data field and is used to detect transmission errors. The CRC field consists of a 15-bit CRC sequence, followed by the recessive CRC Delimiter bit.

The final field is the two-bit Acknowledge (ACK) field. During the ACK Slot bit, the transmitting node sends out a recessive bit. Any node that has received an error-free frame acknowledges the correct reception of the frame by sending back a dominant bit (regardless of whether the node is configured to accept that specific message or not). The recessive acknowledge delimiter completes the acknowledge field and may not be overwritten by a dominant bit.

2.2 Extended Data Frame

In the extended CAN data frame, shown in Figure 2-2, the SOF bit is followed by the arbitration field, which consists of 32 bits. The first 11 bits are the Most Significant bits (MSb) (Base-ID) of the 29-bit identifier. These 11 bits are followed by the Substitute Remote Request (SRR) bit, which is defined to be recessive. The SRR bit is followed by the IDE bit, which is recessive to denote an extended CAN frame.

It should be noted that if arbitration remains unresolved after transmission of the first 11 bits of the identifier, and one of the nodes involved in the arbitration is sending a standard CAN frame (11-bit identifier), the standard CAN frame will win arbitration due to the assertion of a dominant IDE bit. Also, the SRR bit in an extended CAN frame must be recessive to allow the assertion of a dominant RTR bit by a node that is sending a standard CAN remote frame.

The SRR and IDE bits are followed by the remaining 18 bits of the identifier (Extended ID) and the remote transmission request bit.

To enable standard and extended frames to be sent across a shared network, the 29-bit extended message identifier is split into 11-bit (most significant) and 18-bit (least significant) sections. This split ensures that the IDE bit can remain at the same bit position in both the standard and extended frames.

Following the arbitration field is the six-bit control field. The first two bits of this field are reserved and must be dominant. The remaining four bits of the control field are the DLC, which specifies the number of data bytes contained in the message.

The remaining portion of the frame (data field, CRC field, acknowledge field, end-of-frame and intermission) is constructed in the same way as a standard data frame (see **Section 2.1 “Standard Data Frame”**).

2.3 Remote Frame

Normally, data transmission is performed on an autonomous basis by the data source node (e.g., a sensor sending out a data frame). It is possible, however, for a destination node to request data from the source. To accomplish this, the destination node sends a remote frame with an identifier that matches the identifier of the required data frame. The appropriate data source node will then send a data frame in response to the remote frame request.

There are two differences between a remote frame (shown in Figure 2-3) and a data frame. First, the RTR bit is at the recessive state and, second, there is no data field. In the event of a data frame and a remote frame with the same identifier being transmitted at the same time, the data frame wins arbitration due to the dominant RTR bit following the identifier. In this way, the node that transmitted the remote frame receives the desired data immediately.

2.4 Error Frame

An error frame is generated by any node that detects a bus error. An error frame, shown in Figure 2-4, consists of two fields: an error flag field followed by an error delimiter field. There are two types of error flag fields. The type of error flag field sent depends upon the error status of the node that detects and generates the error flag field.

2.4.1 ACTIVE ERRORS

If an error-active node detects a bus error, the node interrupts transmission of the current message by generating an active error flag. The active error flag is composed of six consecutive dominant bits. This bit sequence actively violates the bit-stuffing rule. All other stations recognize the resulting bit-stuffing error and, in turn, generate error frames themselves, called error echo flags.

The error flag field, therefore, consists of between six and twelve consecutive dominant bits (generated by one or more nodes). The error delimiter field (eight recessive bits) completes the error frame. Upon completion of the error frame, bus activity returns to normal and the interrupted node attempts to resend the aborted message.

Note:	Error echo flags typically occur when a localized disturbance causes one or more (but not all) nodes to send an error flag. The remaining nodes generate error flags in response (echo) to the original error flag.

2.4.2 PASSIVE ERRORS

If an error-passive node detects a bus error, the node transmits an error-passive flag followed by the error delimiter field. The error-passive flag consists of six consecutive recessive bits. The error frame for an error-passive node consists of 14 recessive bits. From this it follows that, unless the bus error is detected by an error-active node or the transmitting node, the message will continue transmission because the error-passive flag does not interfere with the bus.

If the transmitting node generates an error-passive flag, it will cause other nodes to generate error frames due to the resulting bit-stuffing violation. After transmission of an error frame, an error-passive node must wait for six consecutive recessive bits on the bus before attempting to rejoin bus communications.

The error delimiter consists of eight recessive bits and allows the bus nodes to restart bus communications cleanly after an error has occurred.

2.5 Overload Frame

An overload frame, shown in Figure 2-5, has the same format as an active error frame. An overload frame, however, can only be generated during an interframe space. In this way, an overload frame can be differentiated from an error frame (an error frame is sent during the transmission of a message). The overload frame consists of two fields: an overload flag followed by an overload delimiter. The overload flag consists of six dominant bits followed by overload flags generated by other nodes (and, as for an active error flag, giving a maximum of twelve dominant bits). The overload delimiter consists of eight recessive bits. An overload frame can be generated by a node as a result of two conditions:

1. The node detects a dominant bit during the interframe space, an illegal condition. **Exception:** The dominant bit is detected during the third bit of IFS. In this case, the receivers will interpret this as a SOF.
2. Due to internal conditions, the node is not yet able to begin reception of the next message. A node may generate a maximum of two sequential overload frames to delay the start of the next message.

Note:	Case 2 should never occur with the MCP2515 due to very short internal delays.

2.6 Interframe Space

The interframe space separates a preceding frame (of any type) from a subsequent data or remote frame. The interframe space is composed of at least three recessive bits called the Intermission. This allows nodes time for internal processing before the start of the next message frame. After the intermission, the bus line remains in the recessive state (bus idle) until the next transmission starts.

 © 2005 Microchip Technology Inc.

FIGURE 2-1: STANDARD DATA FRAME

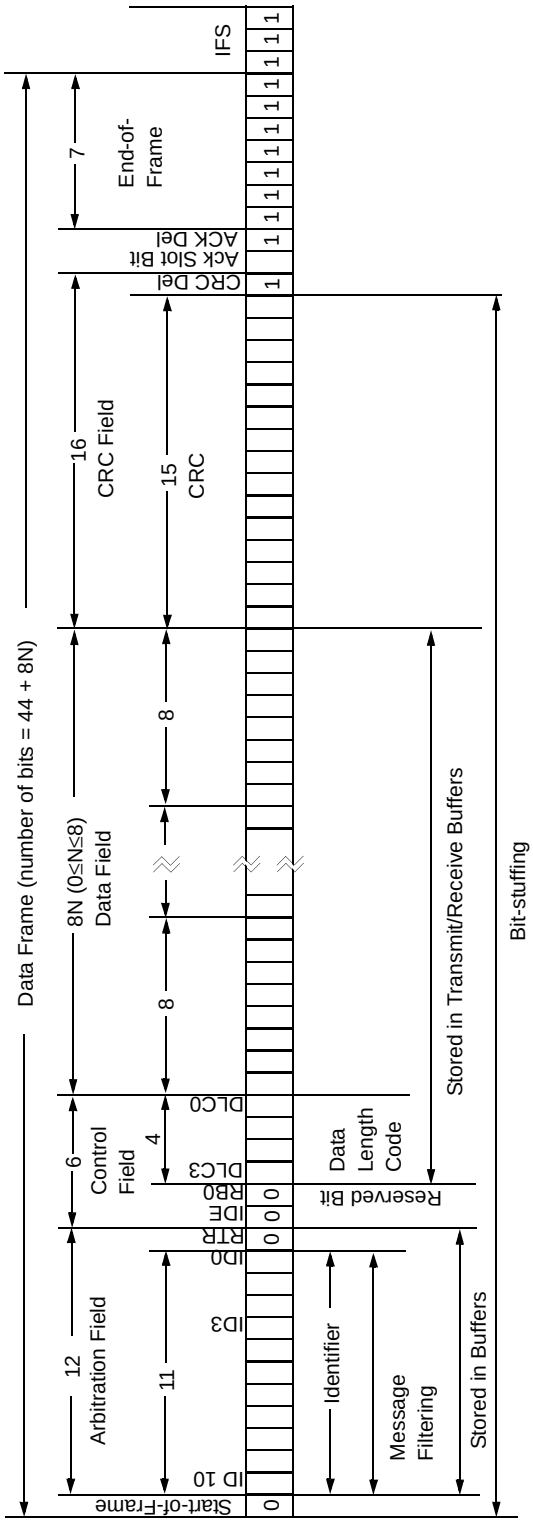
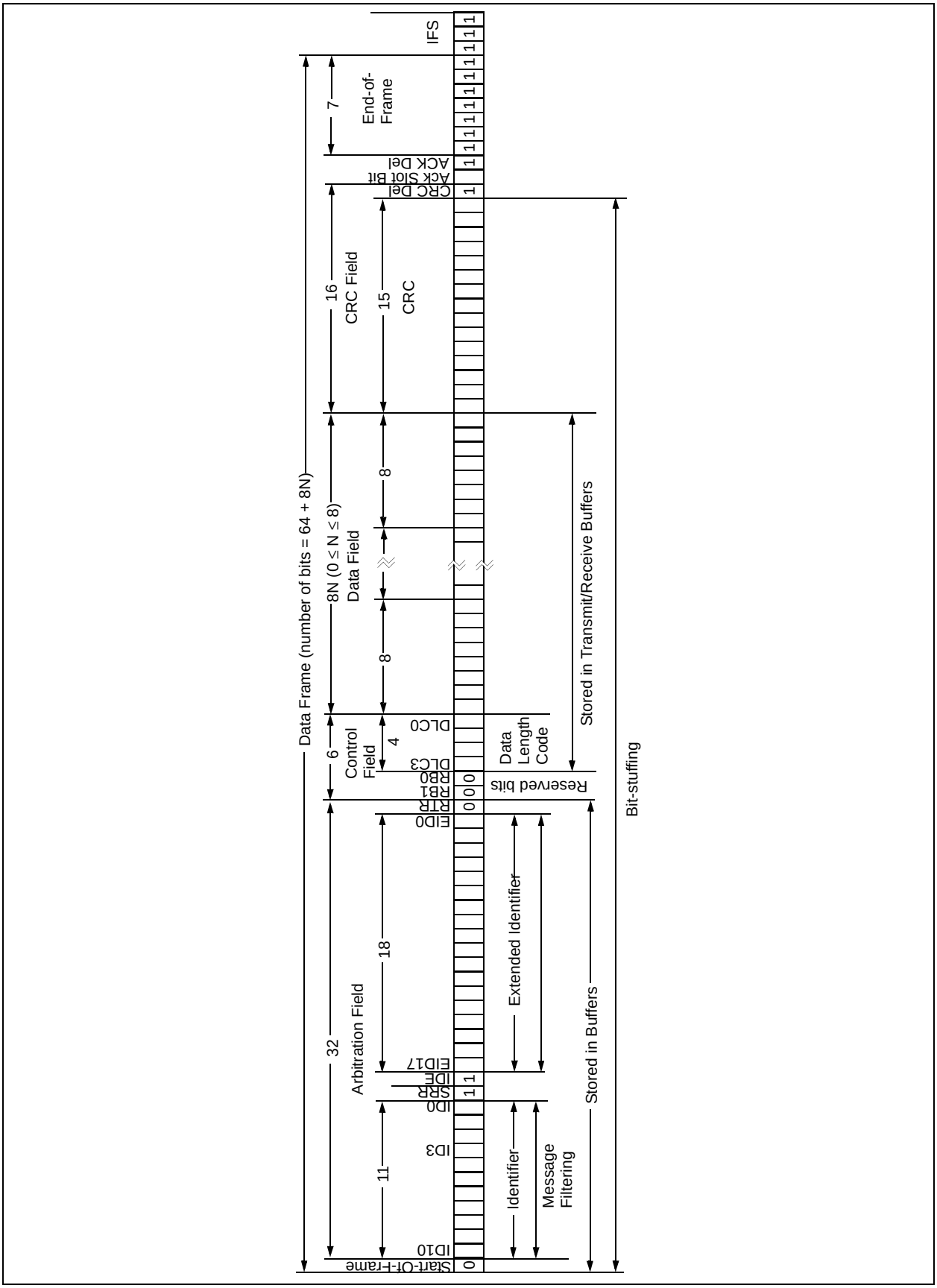


FIGURE 2-2: EXTENDED DATA FRAME



 © 2005 Microchip Technology Inc.

FIGURE 2-3: REMOTE FRAME

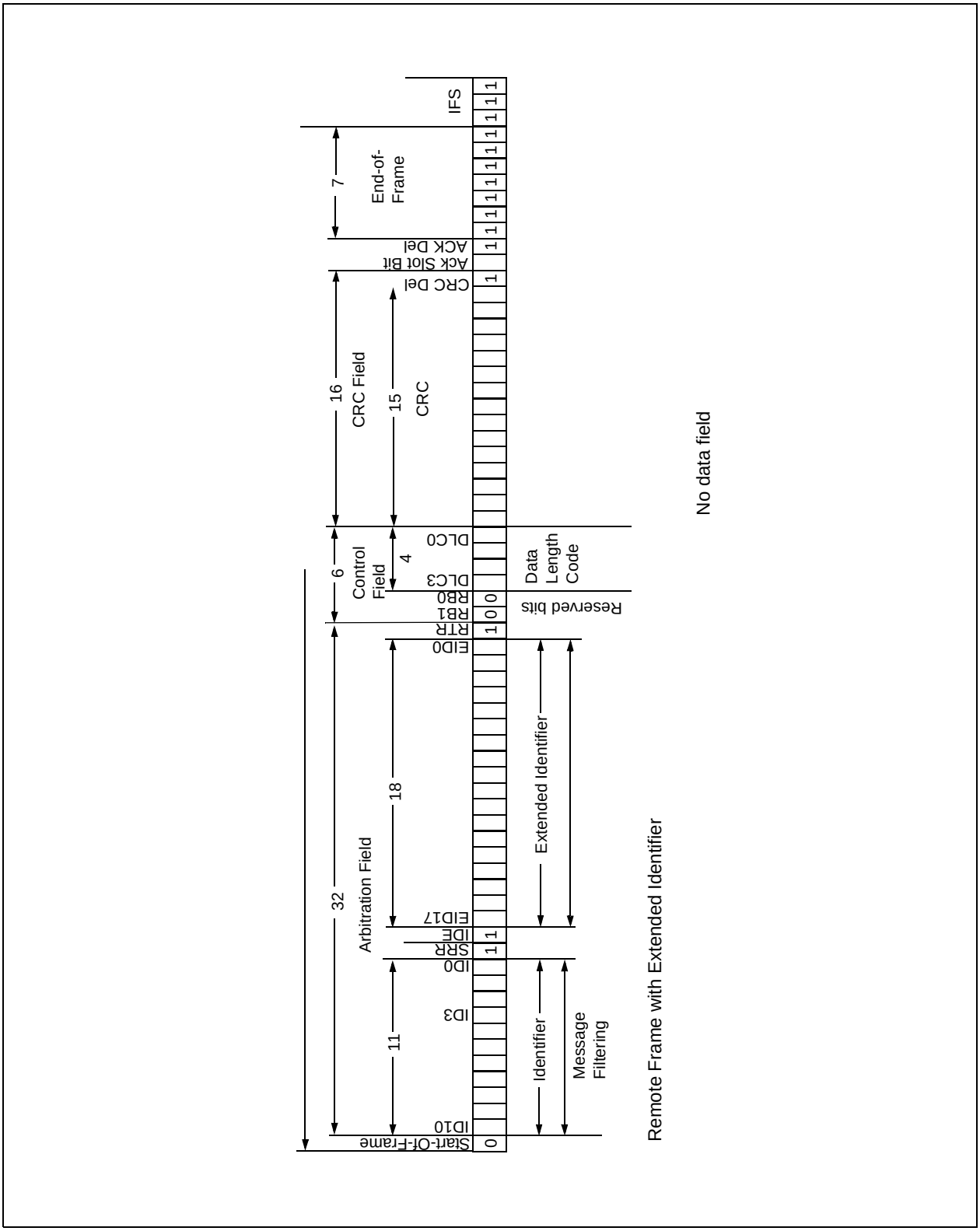
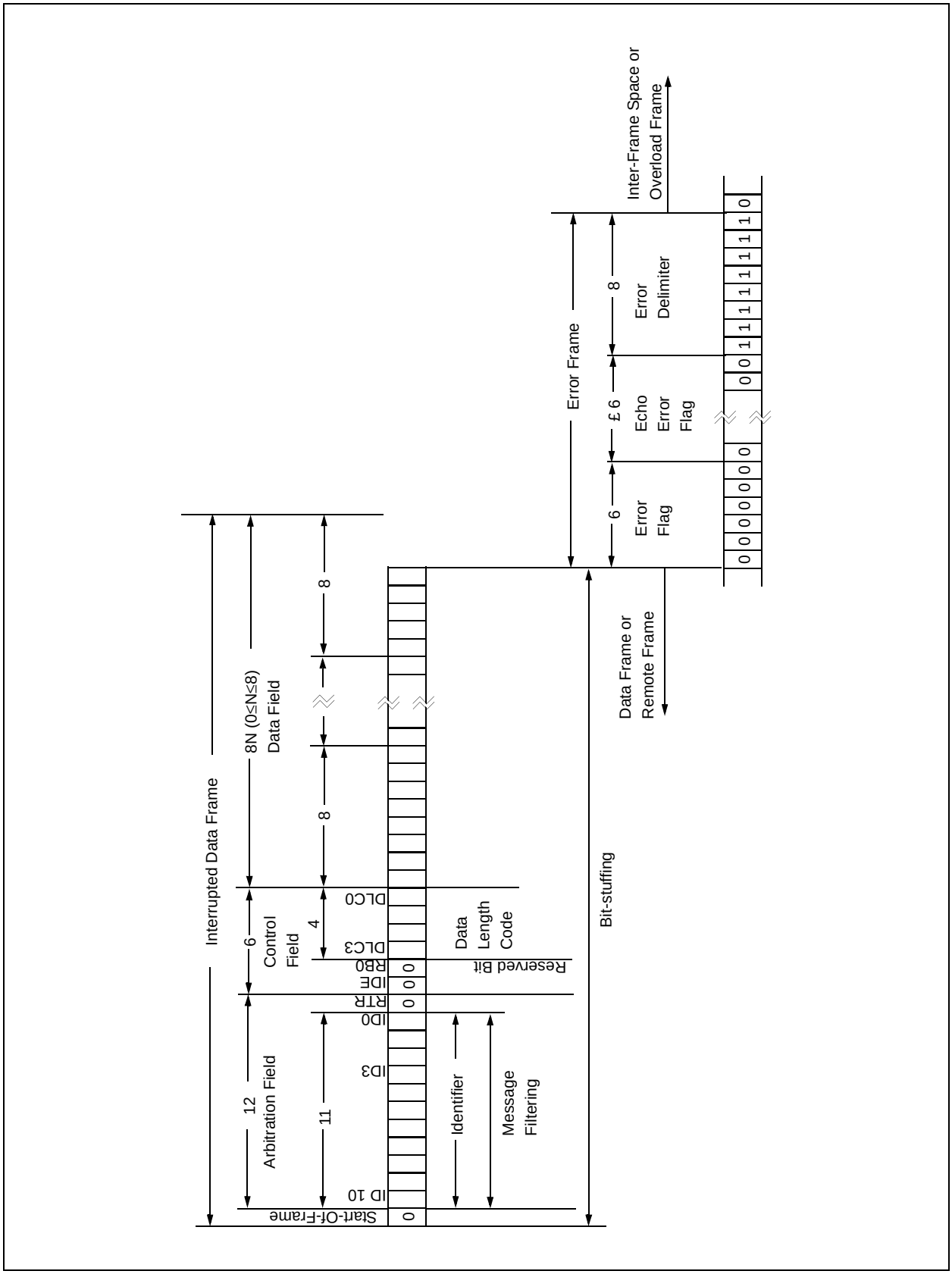
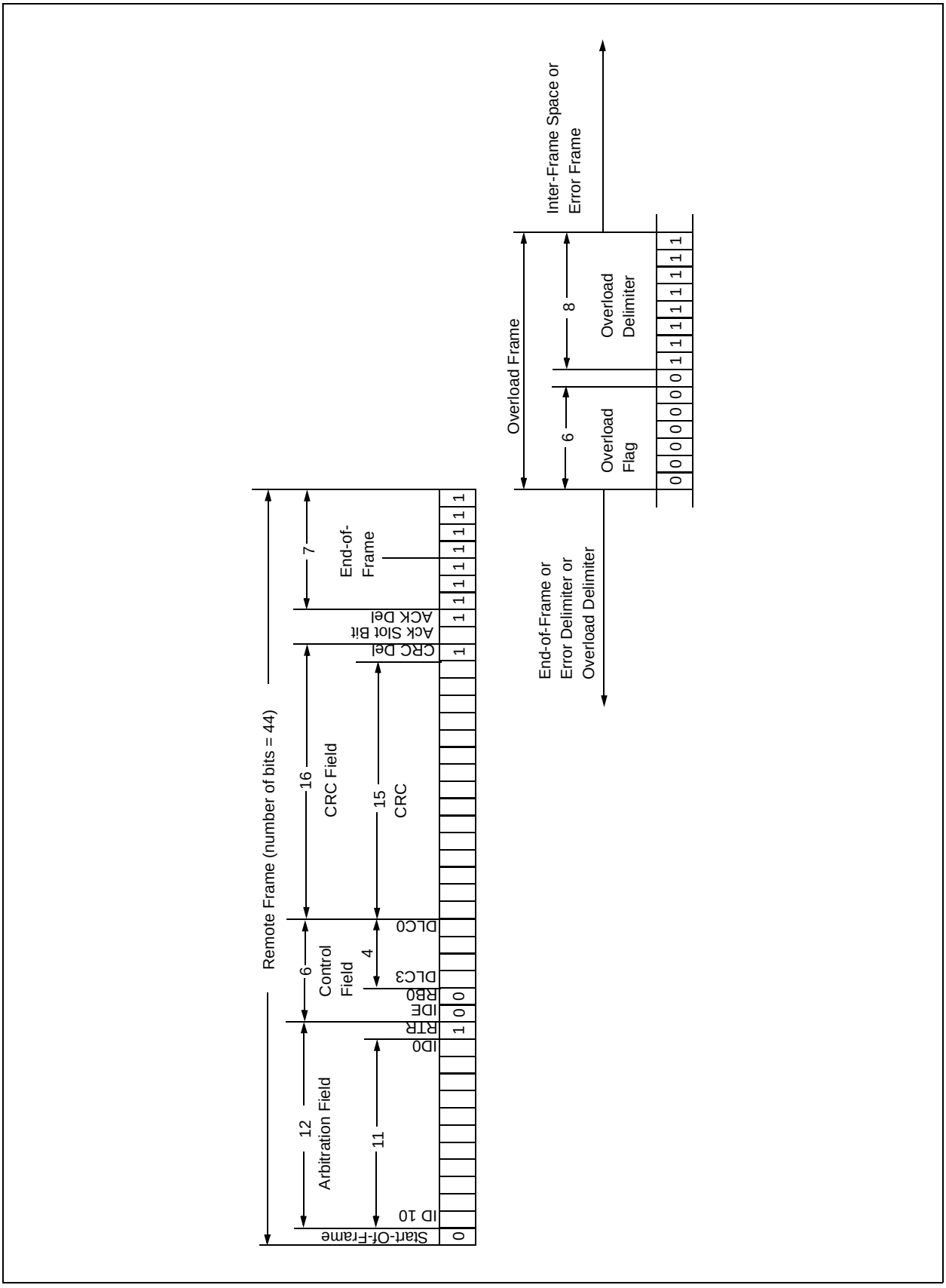


FIGURE 2-4: ACTIVE ERROR FRAME



 © 2005 Microchip Technology Inc.

FIGURE 2-5: OVERLOAD FRAME



NOTES:

 © 2005 Microchip Technology Inc.

3.0 MESSAGE TRANSMISSION

3.1 Transmit Buffers

The MCP2515 implements three transmit buffers. Each of these buffers occupies 14 bytes of SRAM and are mapped into the device memory map.

The first byte, TXBnCTRL, is a control register associated with the message buffer. The information in this register determines the conditions under which the message will be transmitted and indicates the status of the message transmission (see Register 3-1).

Five bytes are used to hold the standard and extended identifiers, as well as other message arbitration information (see Register 3-3 through Register 3-7). The last eight bytes are for the eight possible data bytes of the message to be transmitted (see Register 3-8).

At a minimum, the TXBnSIDH, TXBnSIDL and TXBnDLC registers must be loaded. If data bytes are present in the message, the TXBnDm registers must also be loaded. If the message is to use extended identifiers, the TXBnEIDm registers must also be loaded and the TXBnSIDL.EXIDE bit set.

Prior to sending the message, the MCU must initialize the CANINTE.TXInE bit to enable or disable the generation of an interrupt when the message is sent.

Note:	The TXBnCTRL.TXREQ bit must be clear (indicating the transmit buffer is not pending transmission) before writing to the transmit buffer.

3.2 Transmit Priority

Transmit priority is a prioritization within the MCP2515 of the pending transmittable messages. This is independent from, and not necessarily related to, any prioritization implicit in the message arbitration scheme built into the CAN protocol.

Prior to sending the SOF, the priority of all buffers that are queued for transmission is compared. The transmit buffer with the highest priority will be sent first. For example, if transmit buffer 0 has a higher priority setting than transmit buffer 1, buffer 0 will be sent first.

If two buffers have the same priority setting, the buffer with the highest buffer number will be sent first. For example, if transmit buffer 1 has the same priority setting as transmit buffer 0, buffer 1 will be sent first.

There are four levels of transmit priority. If TXBnCTRL.TXP<1:0> for a particular message buffer is set to 11, that buffer has the highest possible priority. If TXBnCTRL.TXP<1:0> for a particular message buffer is 00, that buffer has the lowest possible priority.

3.3 Initiating Transmission

In order to initiate message transmission, the TXBnCTRL.TXREQ bit must be set for each buffer to be transmitted. This can be accomplished by:

- Writing to the register via the SPI write command
- Sending the SPI RTS command
- Setting the $\overline{\text{TXnRTS}}$ pin low for the particular transmit buffer(s) that are to be transmitted

If transmission is initiated via the SPI interface, the TXREQ bit can be set at the same time as the TXP priority bits.

When TXBnCTRL.TXREQ is set, the TXBnCTRL.ABTF, TXBnCTRL.MLOA and TXBnCTRL.TXERR bits will be cleared automatically.

Note:	Setting the TXBnCTRL.TXREQ bit does not initiate a message transmission. It merely flags a message buffer as being ready for transmission. Transmission will start when the device detects that the bus is available.

Once the transmission has completed successfully, the TXBnCTRL.TXREQ bit will be cleared, the CANINTF.TXnIF bit will be set and an interrupt will be generated if the CANINTE.TXnIE bit is set.

If the message transmission fails, the TXBnCTRL.TXREQ will remain set. This indicates that the message is still pending for transmission and one of the following condition flags will be set:

- If the message started to transmit but encountered an error condition, the TXBnCTRL.TXERR and the CANINTF.MERRF bits will be set and an interrupt will be generated on the $\overline{\text{INT}}$ pin if the CANINTE.MERRE bit is set
- If the message is lost, arbitration at the TXBnCTRL.MLOA bit will be set

Note:	If One-shot mode is enabled (CANCTRL.OSM), the above conditions will still exist. However, the TXREQ bit will be cleared and the message will not attempt transmission a second time.

3.4 One-Shot Mode

One-shot mode ensures that a message will only attempt to transmit one time. Normally, if a CAN message loses arbitration, or is destroyed by an error frame, the message is retransmitted. With One-shot mode enabled, a message will only attempt to transmit one time, regardless of arbitration loss or error frame.

One-shot mode is required to maintain time slots in deterministic systems, such as TTCAN.

3.5 TXnRTS PINS

The $\overline{\text{TXnRTS}}$ pins are input pins that can be configured as:

- Request-to-send inputs, which provides an alternative means of initiating the transmission of a message from any of the transmit buffers
- Standard digital inputs

Configuration and control of these pins is accomplished using the TXRTSCTRL register (see Register 3-2). The TXRTSCTRL register can only be modified when the MCP2515 is in Configuration mode (see **Section 10.0 "Modes of Operation"**). If configured to operate as a request-to-send pin, the pin is mapped into the respective TXBnCTRL.TXREQ bit for the transmit buffer. The TXREQ bit is latched by the falling edge of the $\overline{\text{TXnRTS}}$ pin. The $\overline{\text{TXnRTS}}$ pins are designed to allow them to be tied directly to the $\overline{\text{RXnBF}}$ pins to automatically initiate a message transmission when the $\overline{\text{RXnBF}}$ pin goes low.

The $\overline{\text{TXnRTS}}$ pins have internal pull-up resistors of 100 kΩ (nominal).

3.6 Aborting Transmission

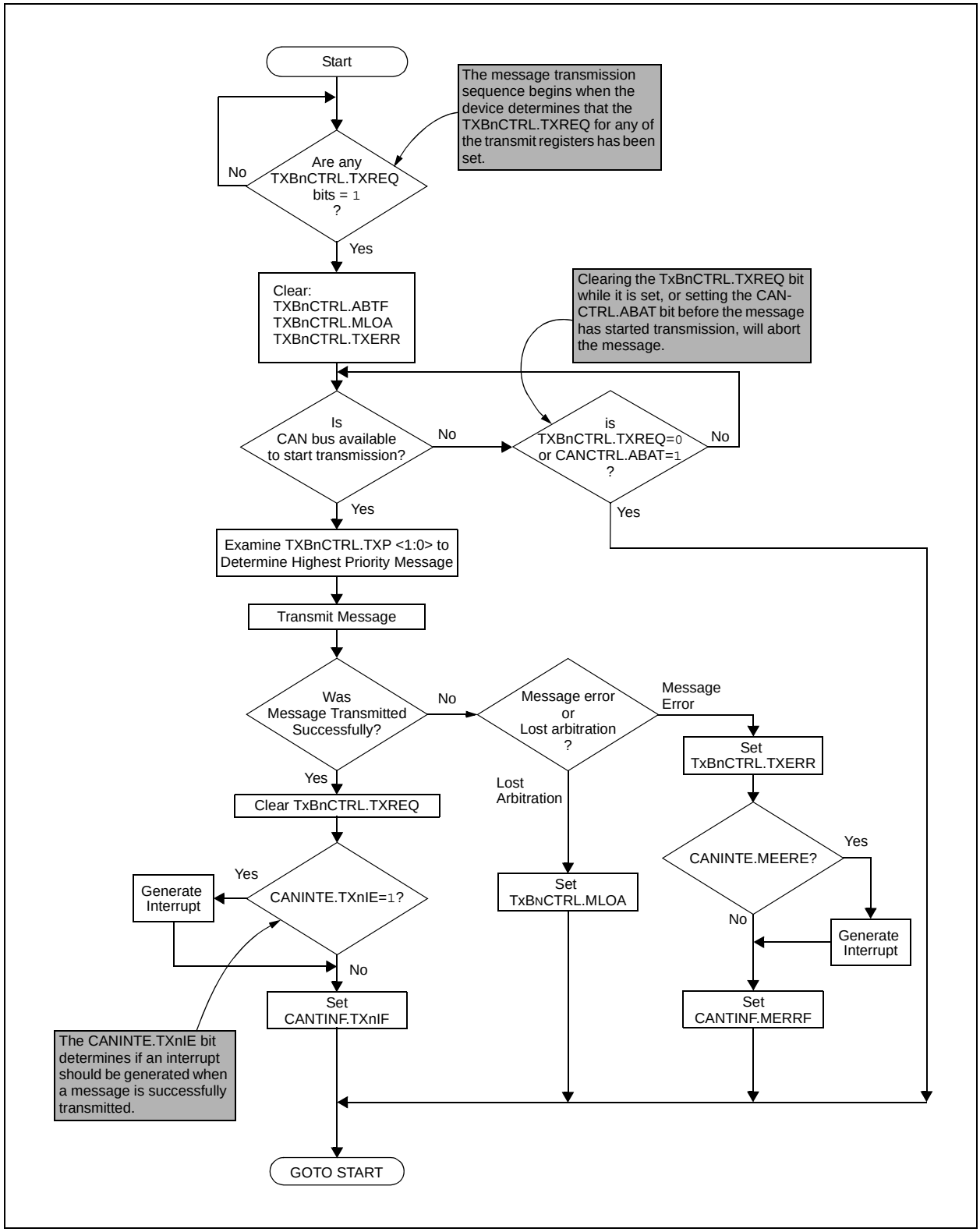
The MCU can request to abort a message in a specific message buffer by clearing the associated TXBnCTRL.TXREQ bit.

In addition, all pending messages can be requested to be aborted by setting the CANCTRL.ABAT bit. This bit MUST be reset (typically after the TXREQ bits have been verified to be cleared) to continue transmitting messages. The CANCTRL.ABTF flag will only be set if the abort was requested via the CANCTRL.ABAT bit. Aborting a message by resetting the TXREQ bit does NOT cause the ABTF bit to be set.

Note:	Messages that were transmitting when the abort was requested will continue to transmit. If the message does not successfully complete transmission (i.e., lost arbitration or was interrupted by an error frame), it will then be aborted.

 © 2005 Microchip Technology Inc.

FIGURE 3-1: TRANSMIT MESSAGE FLOWCHART



REGISTER 3-1: TXBnCTRL – TRANSMIT BUFFER n CONTROL REGISTER (ADDRESS: 30h, 40h, 50h)

U-0	R-0	R-0	R-0	R/W-0	U-0	R/W-0	R/W-0
—	ABTF	MLOA	TXERR	TXREQ	—	TXP1	TXP0
bit 7							bit 0

bit 7 **Unimplemented:** Read as '0'

bit 6 **ABTF**: Message Aborted Flag bit
1 = Message was aborted
0 = Message completed transmission successfully

bit 5 **MLOA**: Message Lost Arbitration bit
1 = Message lost arbitration while being sent
0 = Message did not lose arbitration while being sent

bit 4 **TXERR**: Transmission Error Detected bit
1 = A bus error occurred while the message was being transmitted
0 = No bus error occurred while the message was being transmitted

bit 3 **TXREQ**: Message Transmit Request bit
1 = Buffer is currently pending transmission
(MCU sets this bit to request message be transmitted - bit is automatically cleared when the message is sent)
0 = Buffer is not currently pending transmission
(MCU can clear this bit to request a message abort)

bit 2 **Unimplemented:** Read as '0'

bit 1-0 **TXP**: Transmit Buffer Priority <1:0> bits
11 = Highest Message Priority
10 = High Intermediate Message Priority
11 = Low Intermediate Message Priority
00 = Lowest Message Priority

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

 © 2005 Microchip Technology Inc.

REGISTER 3-2: TXRTSCTRL – TXnRTS PIN CONTROL AND STATUS REGISTER (ADDRESS: 0Dh)

U-0	U-0	R-x	R-x	R-x	R/W-0	R/W-0	R/W-0
—	—	B2RTS	B1RTS	B0RTS	B2RTSM	B1RTSM	B0RTSM
bit 7							bit 0

bit 7 **Unimplemented:** Read as '0'

bit 6 **Unimplemented:** Read as '0'

bit 5 **B2RTS**: $\overline{\text{TX2RTS}}$ Pin State bit
- Reads state of $\overline{\text{TX2RTS}}$ pin when in Digital Input mode
- Reads as '0' when pin is in 'Request-to-Send' mode

bit 4 **B1RTS**: $\overline{\text{TX1RTX}}$ Pin State bit
- Reads state of $\overline{\text{TX1RTS}}$ pin when in Digital Input mode
- Reads as '0' when pin is in 'Request-to-Send' mode

bit 3 **B0RTS**: $\overline{\text{TX0RTS}}$ Pin State bit
- Reads state of $\overline{\text{TX0RTS}}$ pin when in Digital Input mode
- Reads as '0' when pin is in 'Request-to-Send' mode

bit 2 **B2RTSM**: $\overline{\text{TX2RTS}}$ Pin mode bit
1 = Pin is used to request message transmission of TXB2 buffer (on falling edge)
0 = Digital input

bit 1 **B1RTSM**: $\overline{\text{TX1RTS}}$ Pin mode bit
1 = Pin is used to request message transmission of TXB1 buffer (on falling edge)
0 = Digital input

bit 0 **B0RTSM**: $\overline{\text{TX0RTS}}$ Pin mode bit
1 = Pin is used to request message transmission of TXB0 buffer (on falling edge)
0 = Digital input

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

REGISTER 3-3: TXBnSIDH – TRANSMIT BUFFER n STANDARD IDENTIFIER HIGH (ADDRESS: 31h, 41h, 51h)

R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
SID10	SID9	SID8	SID7	SID6	SID5	SID4	SID3
bit 7							bit 0

bit 7-0 **SID**: Standard Identifier bits <10:3>

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

REGISTER 3-4: TXBnSIDL – TRANSMIT BUFFER n STANDARD IDENTIFIER LOW (ADDRESS: 32h, 42h, 52h)

R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
SID2	SID1	SID0	—	EXIDE	—	EID17	EID16
bit 7							bit 0

bit 7-5 **SID**: Standard Identifier bits <2:0>

bit 4 **Unimplemented**: Reads as '0'

bit 3 **EXIDE**: Extended Identifier Enable bit
1 = Message will transmit extended identifier
0 = Message will transmit standard identifier

bit 2 **Unimplemented**: Reads as '0'

bit 1-0 **EID**: Extended Identifier bits <17:16>

Legend:		
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared x = Bit is unknown

REGISTER 3-5: TXBnEID8 – TRANSMIT BUFFER n EXTENDED IDENTIFIER HIGH (ADDRESS: 33h, 43h, 53h)

R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
EID15	EID14	EID13	EID12	EID11	EID10	EID9	EID8
bit 7							bit 0

bit 7-0 **EID**: Extended Identifier bits <15:8>

Legend:		
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared x = Bit is unknown

REGISTER 3-6: TXBnEID0 – TRANSMIT BUFFER n EXTENDED IDENTIFIER LOW (ADDRESS: 34h, 44h, 54h)

R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
EID7	EID6	EID5	EID4	EID3	EID2	EID1	EID0
bit 7							bit 0

bit 7-0 **EID**: Extended Identifier bits <7:0>

Legend:		
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared x = Bit is unknown

 © 2005 Microchip Technology Inc.

REGISTER 3-7: TXBnDLC - TRANSMIT BUFFER n DATA LENGTH CODE (ADDRESS: 35h, 45h, 55h)

R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
—	RTR	—	—	DLC3	DLC2	DLC1	DLC0
bit 7							bit 0

bit 7 **Unimplemented**: Reads as '0'

bit 6 **RTR**: Remote Transmission Request bit

1 = Transmitted Message will be a Remote Transmit Request
0 = Transmitted Message will be a Data Frame

bit 5-4 **Unimplemented**: Reads as '0'

bit 3-0 **DLC**: Data Length Code <3:0> bits

Sets the number of data bytes to be transmitted (0 to 8 bytes)

Note: It is possible to set the DLC to a value greater than 8, however only 8 bytes are transmitted

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

REGISTER 3-8: TXBnDm – TRANSMIT BUFFER n DATA BYTE m (ADDRESS: 36h - 3Dh, 46h - 4Dh, 56h - 5Dh)

R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
TXBnDm 7	TXBnDm 6	TXBnDm 5	TXBnDm 4	TXBnDm 3	TXBnDm 2	TXBnDm 1	TXBnDm 0
bit 7							bit 0

bit 7-0 **TXBnDM7:TXBnDM0**: Transmit Buffer n Data Field Bytes m

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

NOTES:

 © 2005 Microchip Technology Inc.

4.0 MESSAGE RECEPTION

4.1 Receive Message Buffering

The MCP2515 includes two full receive buffers with multiple acceptance filters for each. There is also a separate Message Assembly Buffer (MAB) that acts as a third receive buffer (see Figure 4-2).

4.1.1 MESSAGE ASSEMBLY BUFFER

Of the three receive buffers, the MAB is always committed to receiving the next message from the bus. The MAB assembles all messages received. These messages will be transferred to the RXBn buffers (See Register 4-4 to Register 4-9) only if the acceptance filter criteria is met.

4.1.2 RXB0 AND RXB1

The remaining two receive buffers, called RXB0 and RXB1, can receive a complete message from the protocol engine via the MAB. The MCU can access one buffer, while the other buffer is available for message reception, or for holding a previously received message.

Note:	The entire contents of the MAB is moved into the receive buffer once a message is accepted. This means that, regardless of the type of identifier (standard or extended) and the number of data bytes received, the entire receive buffer is overwritten with the MAB contents. Therefore, the contents of all registers in the buffer must be assumed to have been modified when any message is received.

4.1.3 RECEIVE FLAGS/INTERRUPTS

When a message is moved into either of the receive buffers, the appropriate CANINTF.RXnIF bit is set. This bit must be cleared by the MCU in order to allow a new message to be received into the buffer. This bit provides a positive lockout to ensure that the MCU has finished with the message before the MCP2515 attempts to load a new message into the receive buffer.

If the CANINTE.RXnIE bit is set, an interrupt will be generated on the $\overline{\text{INT}}$ pin to indicate that a valid message has been received. In addition, the associated RXnBF pin will drive low if configured as a receive buffer full pin. See **Section 4.4 "RX0BF and RX1BF Pins"** for details.

4.2 Receive Priority

RXB0, the higher priority buffer, has one mask and two message acceptance filters associated with it. The received message is applied to the mask and filters for RXB0 first.

RXB1 is the lower priority buffer, with one mask and four acceptance filters associated with it.

In addition to the message being applied to the RB0 mask and filters first, the lower number of acceptance filters makes the match on RXB0 more restrictive and implies a higher priority for that buffer.

When a message is received, bits <3:0> of the RXBnCTRL register will indicate the acceptance filter number that enabled reception and whether the received message is a remote transfer request.

4.2.1 ROLLOVER

Additionally, the RXB0CTRL register can be configured such that, if RXB0 contains a valid message and another valid message is received, an overflow error will not occur and the new message will be moved into RXB1, regardless of the acceptance criteria of RXB1.

4.2.2 RXM BITS

The RXBnCTRL.RXM bits set special receive modes. Normally, these bits are cleared to `00` to enable reception of all valid messages as determined by the appropriate acceptance filters. In this case, the determination of whether or not to receive standard or extended messages is determined by the RFXnSIDL.EXIDE bit in the acceptance filter register.

If the RXBnCTRL.RXM bits are set to `01` or `10`, the receiver will only accept messages with standard or extended identifiers, respectively. If an acceptance filter has the RFXnSIDL.EXIDE bit set such that it does not correspond with the RXBnCTRL.RXM mode, that acceptance filter is rendered useless. These two modes of RXBnCTRL.RXM bits can be used in systems where it is known that only standard or extended messages will be on the bus.

If the RXBnCTRL.RXM bits are set to `11`, the buffer will receive all messages, regardless of the values of the acceptance filters. Also, if a message has an error before the EOF, that portion of the message assembled in the MAB before the error frame will be loaded into the buffer. This mode has some value in debugging a CAN system and would not be used in an actual system environment.

4.3 Start-of-Frame Signal

If enabled, the Start-Of-Frame signal is generated on the SOF pin at the beginning of each CAN message detected on the RXCAN pin.

The RXCAN pin monitors an idle bus for a recessive-to-dominant edge. If the dominant condition remains until the sample point, the MCP2515 interprets this as a SOF and a SOF pulse is generated. If the dominant condition does not remain until the sample point, the MCP2515 interprets this as a glitch on the bus and no SOF signal is generated. Figure 4-1 illustrates SOF signalling and glitch-filtering.

As with One-shot mode, one use for SOF signaling is for TTCAN-type systems. In addition, by monitoring both the RXCAN pin and the SOF pin, a MCU can detect early physical bus problems by detecting small glitches before they affect the CAN communications.

4.4 RX0BF and RX1BF Pins

In addition to the $\overline{\text{INT}}$ pin, which provides an interrupt signal to the MCU for many different conditions, the receive buffer full pins ($\overline{\text{RX0BF}}$ and $\overline{\text{RX1BF}}$) can be used to indicate that a valid message has been loaded into RXB0 or RXB1, respectively. The pins have three different configurations (Register 4-1):

1. Disabled.
2. Buffer Full Interrupt.
3. Digital Output.

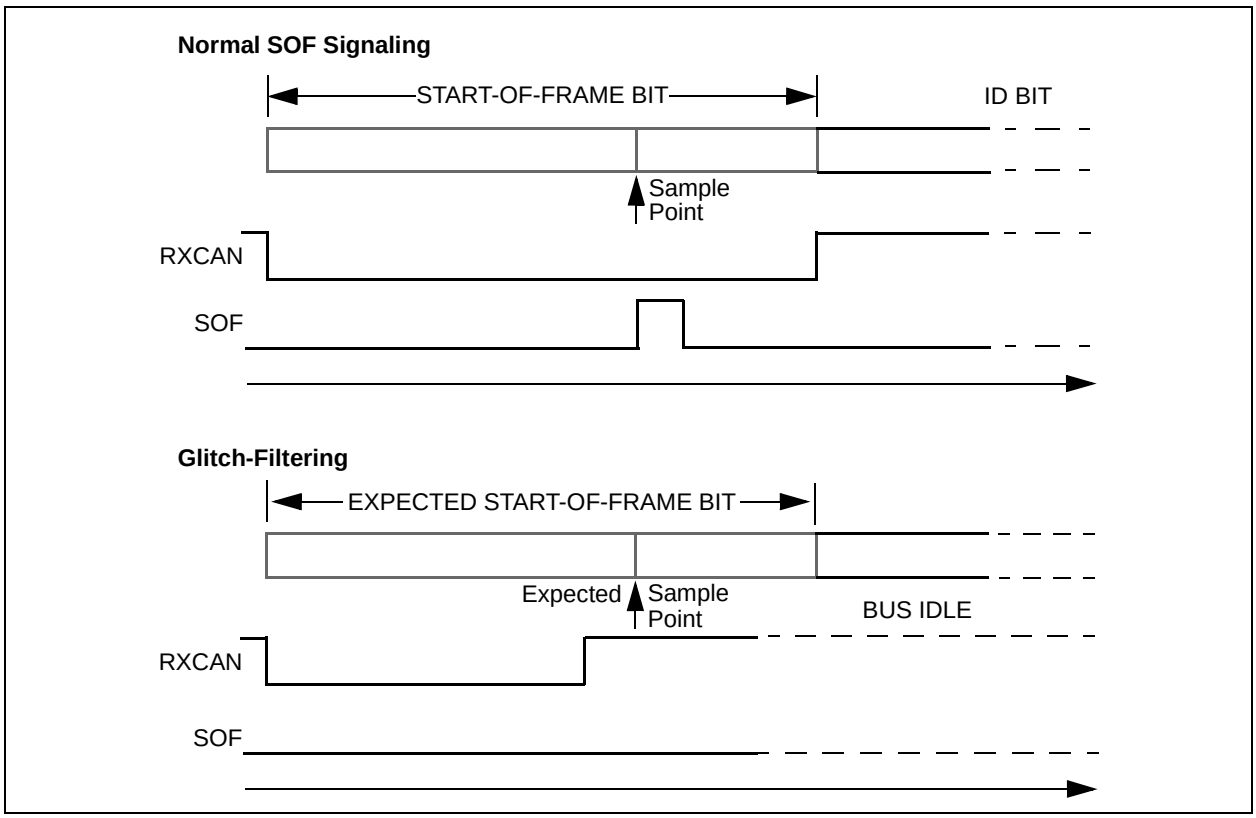
4.4.1 DISABLED

The $\overline{\text{RXBnBF}}$ pins can be disabled to the high-impedance state by clearing BFPCTRL.BnBFE.

4.4.2 CONFIGURED AS BUFFER FULL

The $\overline{\text{RXBnBF}}$ pins can be configured to act as either buffer full interrupt pins or as standard digital outputs. Configuration and status of these pins is available via the BFPCTRL register (Register 4-3). When set to operate in Interrupt mode (by setting BFPCTRL.BxBFE and BFPCTRL.BxBFM bits), these pins are active-low and are mapped to the CANINTF.RXnIF bit for each receive buffer. When this bit goes high for one of the receive buffers (indicating that a valid message has been loaded into the buffer), the corresponding $\overline{\text{RXBnBF}}$ pin will go low. When the CANINTF.RXnIF bit is cleared by the MCU, the corresponding interrupt pin will go to the logic-high state until the next message is loaded into the receive buffer.

FIGURE 4-1: START-OF-FRAME SIGNALING



 © 2005 Microchip Technology Inc.

4.4.3 CONFIGURED AS DIGITAL OUTPUT

When used as digital outputs, the BFPCTRL.BxBFM bit must be cleared and BFPCTRL.BnBFE must be set for the associated buffer. In this mode, the state of the pin is controlled by the BFPCTRL.BnBFS bits. Writing a '1' to the BnBFS bit will cause a high level to be driven on the associated buffer full pin, while a '0' will cause the pin to drive low. When using the pins in this mode, the state of the pin should be modified only by using the Bit Modify SPI command to prevent glitches from occurring on either of the buffer full pins.

TABLE 4-1: CONFIGURING RXNBF PINS

BnBFE	BnBFM	BnBFS	Pin Status
0	X	X	Disabled, high-impedance
1	1	X	Receive buffer interrupt
1	0	0	Digital output = 0
1	0	1	Digital output = 1

FIGURE 4-2: RECEIVE BUFFER BLOCK DIAGRAM

Note: Messages received in the MAB are intially applied to the mask and filters of RXB0. In addition, only one filter match occurs (e.g., if the message matches both RXF0 and RXF2, the match will be for RXF0 and the message will be moved into RXB0).

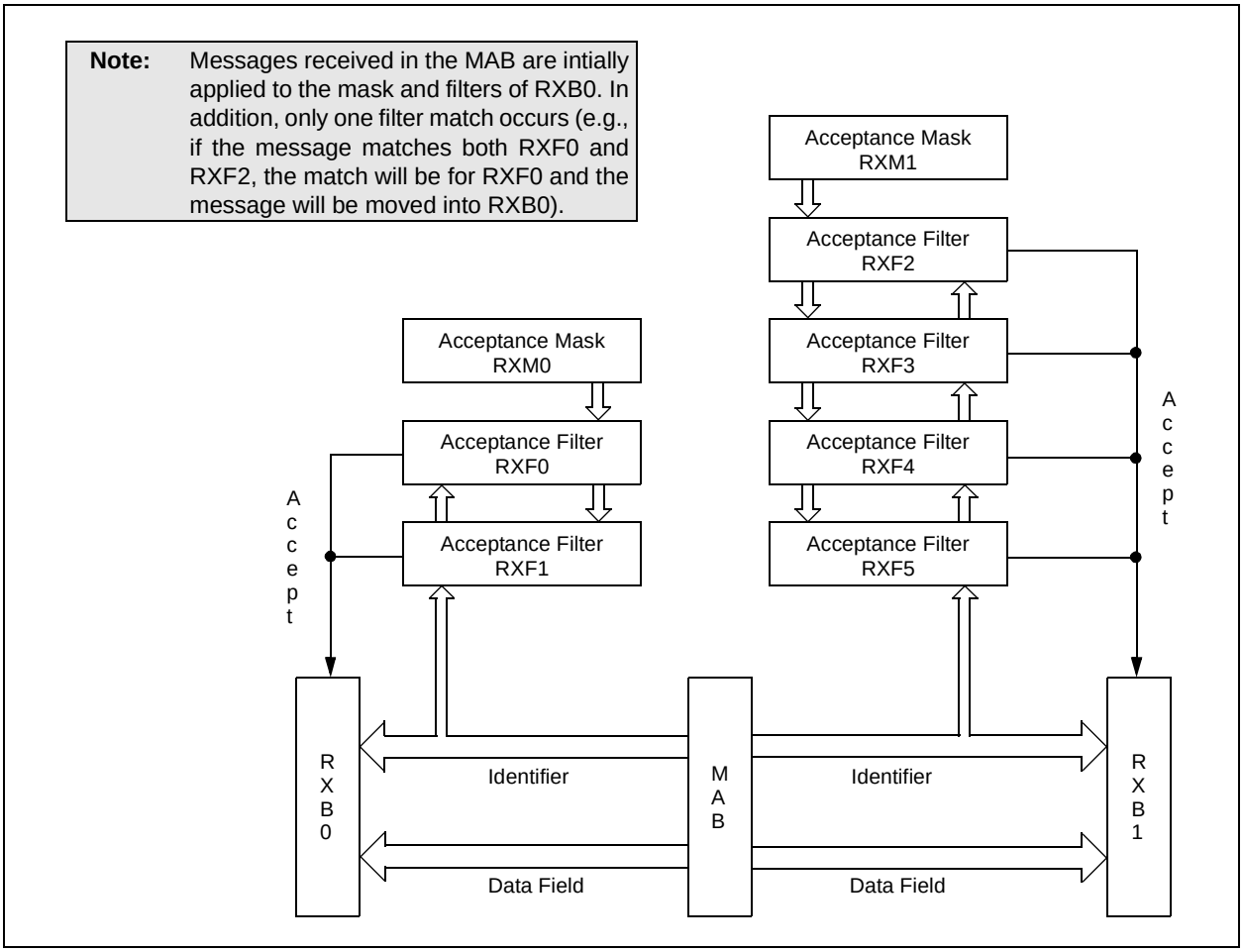
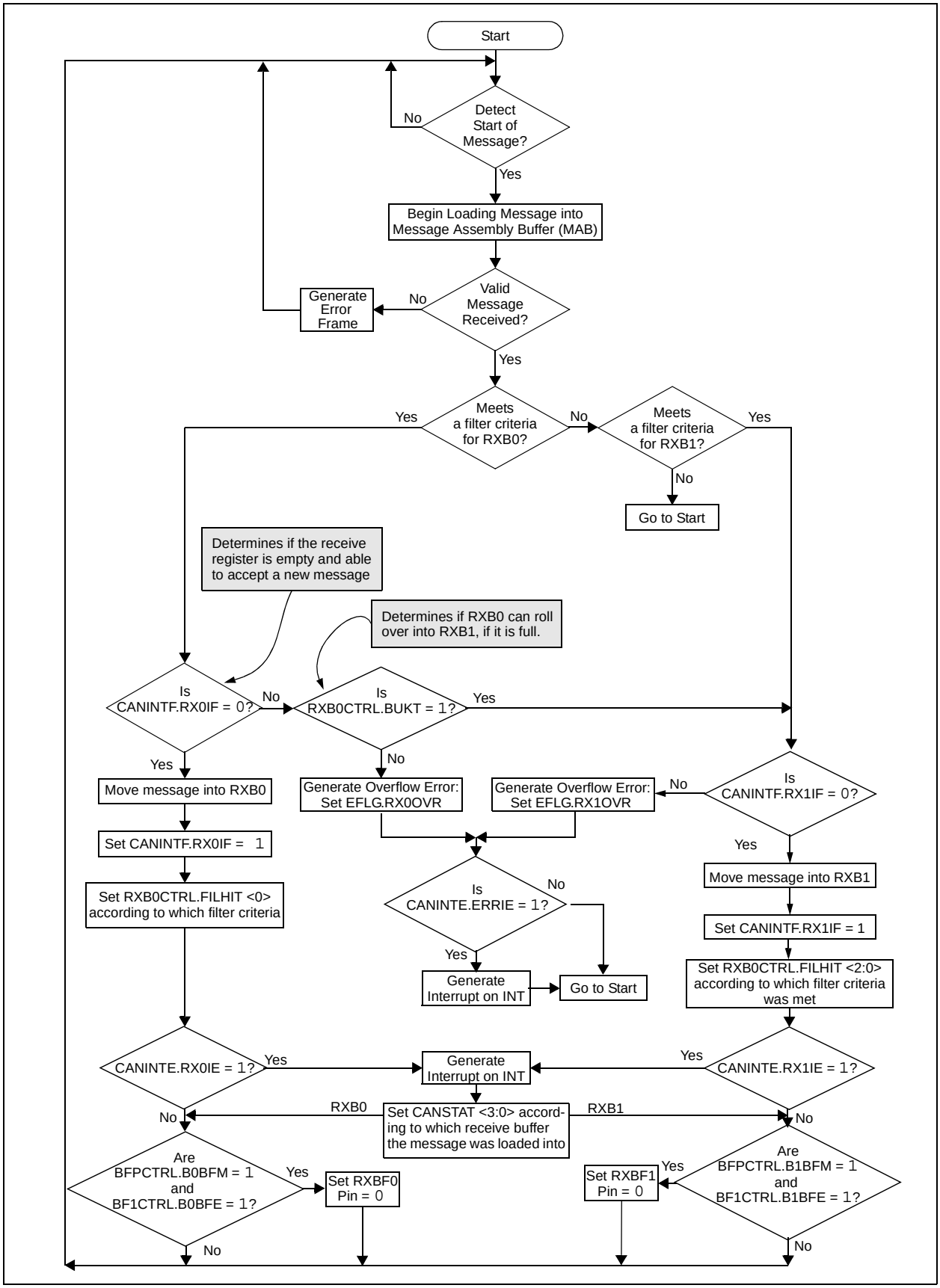


FIGURE 4-3: RECEIVE FLOW FLOWCHART



 © 2005 Microchip Technology Inc.

REGISTER 4-1: RXB0CTRL – RECEIVE BUFFER 0 CONTROL (ADDRESS: 60h)

U-0	R/W-0	R/W-0	U-0	R-0	R/W-0	R-0	R-0
—	RXM1	RXM0	—	RXRTR	BUKT	BUKT1	FILHIT0
bit 7							bit 0

bit 7 **Unimplemented:** Read as '0'

bit 6-5 **RXM**: Receive Buffer Operating Mode bits

11 = Turn mask/filters off; receive any message
10 = Receive only valid messages with extended identifiers that meet filter criteria
01 = Receive only valid messages with standard identifiers that meet filter criteria
00 = Receive all valid messages using either standard or extended identifiers that meet filter criteria

bit 4 **Unimplemented:** Read as '0'

bit 3 **RXRTR**: Received Remote Transfer Request bit

1 = Remote Transfer Request Received
0 = No Remote Transfer Request Received

bit 2 **BUKT**: Rollover Enable bit

1 = RXB0 message will rollover and be written to RXB1 if RXB0 is full
0 = Rollover disabled

bit 1 **BUKT1**: Read-only Copy of BUKT bit (used internally by the MCP2515)

bit 0 **FILHIT**: Filter Hit bit - indicates which acceptance filter enabled reception of message

1 = Acceptance Filter 1 (RXF1)
0 = Acceptance Filter 0 (RXF0)

Note: If a rollover from RXB0 to RXB1 occurs, the FILHIT bit will reflect the filter that accepted the message that rolled over.

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

REGISTER 4-2: RXB1CTRL – RECEIVE BUFFER 1 CONTROL (ADDRESS: 70h)

U-0	R/W-0	R/W-0	U-0	R-0	R-0	R-0	R-0
—	RXM1	RXM0	—	RXRTR	FILHIT2	FILHIT1	FILHIT0
bit 7							bit 0

bit 7 **Unimplemented:** Read as '0'

bit 6-5 **RXM**: Receive Buffer Operating Mode bits

11 = Turn mask/filters off; receive any message
10 = Receive only valid messages with extended identifiers that meet filter criteria
01 = Receive only valid messages with standard identifiers that meet filter criteria
00 = Receive all valid messages using either standard or extended identifiers that meet filter criteria

bit 4 **Unimplemented:** Read as '0'

bit 3 **RXRTR**: Received Remote Transfer Request bit

1 = Remote Transfer Request Received
0 = No Remote Transfer Request Received

bit 2-0 **FILHIT**: Filter Hit bits - indicates which acceptance filter enabled reception of message

101 = Acceptance Filter 5 (RXF5)
100 = Acceptance Filter 4 (RXF4)
011 = Acceptance Filter 3 (RXF3)
010 = Acceptance Filter 2 (RXF2)
001 = Acceptance Filter 1 (RXF1) (Only if BUKT bit set in RXB0CTRL)
000 = Acceptance Filter 0 (RXF0) (Only if BUKT bit set in RXB0CTRL)

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

 © 2005 Microchip Technology Inc.

REGISTER 4-3: BFPCTRL – RXnBF PIN CONTROL AND STATUS (ADDRESS: 0Ch)

U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
—	—	B1BFS	B0BFS	B1BFE	B0BFE	B1BFM	B0BFM
bit 7							bit 0

bit 7 **Unimplemented:** Read as '0'

bit 6 **Unimplemented:** Read as '0'

bit 5 **B1BFS**: $\overline{\text{RX1BF}}$ Pin State bit (Digital Output mode only)
- Reads as '0' when RX1BF is configured as interrupt pin

bit 4 **B0BFS**: $\overline{\text{RX0BF}}$ Pin State bit (Digital Output mode only)
- Reads as '0' when RX0BF is configured as interrupt pin

bit 3 **B1BFE**: $\overline{\text{RX1BF}}$ Pin Function Enable bit
1 = Pin function enabled, operation mode determined by B1BFM bit
0 = Pin function disabled, pin goes to high-impedance state

bit 2 **B0BFE**: $\overline{\text{RX0BF}}$ Pin Function Enable bit
1 = Pin function enabled, operation mode determined by B0BFM bit
0 = Pin function disabled, pin goes to high-impedance state

bit 1 **B1BFM**: $\overline{\text{RX1BF}}$ Pin Operation Mode bit
1 = Pin is used as interrupt when valid message loaded into RXB1
0 = Digital Output mode

bit 0 **B0BFM**: $\overline{\text{RX0BF}}$ Pin Operation Mode bit
1 = Pin is used as interrupt when valid message loaded into RXB0
0 = Digital Output mode

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

REGISTER 4-4: RXBnSIDH – RECEIVE BUFFER n STANDARD IDENTIFIER HIGH (ADDRESS: 61h, 71h)

R-x	R-x	R-x	R-x	R-x	R-x	R-x	R-x
SID10	SID9	SID8	SID7	SID6	SID5	SID4	SID3
bit 7							bit 0

bit 7-0 **SID**: Standard Identifier bits <10:3>
These bits contain the eight most significant bits of the Standard Identifier for the received message

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

REGISTER 4-5: RXBnSIDL – RECEIVE BUFFER n STANDARD IDENTIFIER LOW (ADDRESS: 62h, 72h)

R-x	R-x	R-x	R-x	R-x	U-0	R-x	R-x
SID2	SID1	SID0	SRR	IDE	—	EID17	EID16
bit 7							bit 0

bit 7-5 **SID**: Standard Identifier bits <2:0>
These bits contain the three least significant bits of the Standard Identifier for the received message

bit 4 **SRR:** Standard Frame Remote Transmit Request bit (valid only if IDE bit = '0')
1 = Standard Frame Remote Transmit Request Received
0 = Standard Data Frame Received

bit 3 **IDE:** Extended Identifier Flag bit
This bit indicates whether the received message was a Standard or an Extended Frame
1 = Received message was an Extended Frame
0 = Received message was a Standard Frame

bit 2 **Unimplemented:** Reads as '0'

bit 1-0 **EID**: Extended Identifier bits <17:16>
These bits contain the two most significant bits of the Extended Identifier for the received message

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

REGISTER 4-6: RXBnEID8 – RECEIVE BUFFER n EXTENDED IDENTIFIER HIGH (ADDRESS: 63h, 73h)

R-x	R-x	R-x	R-x	R-x	R-x	R-x	R-x
EID15	EID14	EID13	EID12	EID11	EID10	EID9	EID8
bit 7							bit 0

bit 7-0 **EID**: Extended Identifier bits <15:8>
These bits hold bits 15 through 8 of the Extended Identifier for the received message

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

 © 2005 Microchip Technology Inc.

REGISTER 4-7: RXBnEID0 – RECEIVE BUFFER n EXTENDED IDENTIFIER LOW (ADDRESS: 64h, 74h)

R-x	R-x	R-x	R-x	R-x	R-x	R-x	R-x
EID7	EID6	EID5	EID4	EID3	EID2	EID1	EID0
bit 7							bit 0

bit 7-0 **EID**: Extended Identifier bits <7:0>

These bits hold the least significant eight bits of the Extended Identifier for the received message

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

REGISTER 4-8: RXBnDLC – RECEIVE BUFFER n DATA LENGHT CODE (ADDRESS: 65h, 75h)

U-0	R-x	R-x	R-x	R-x	R-x	R-x	R-x
—	RTR	RB1	RB0	DLC3	DLC2	DLC1	DLC0
bit 7							bit 0

bit 7 **Unimplemented:** Reads as '0'

bit 6 **RTR**: Extended Frame Remote Transmission Request bit (valid only when RXBnSIDL.IDE = '1')

1 = Extended Frame Remote Transmit Request Received
0 = Extended Data Frame Received

bit 5 **RB1:** Reserved Bit 1

bit 4 **RB0:** Reserved Bit 0

bit 3-0 **DLC**: Data Length Code bits <3:0>

Indicates number of data bytes that were received

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

REGISTER 4-9: RXBnDM – RECEIVE BUFFER n DATA BYTE M (ADDRESS: 66h - 6Dh, 76h - 7Dh)

R-x	R-x	R-x	R-x	R-x	R-x	R-x	R-x
RBnDm7	RBnDm6	RBnDm5	RBnDm4	RBnDm3	RBnDm2	RBnDm1	RBnDm0
bit 7							bit 0

bit 7-0 **RBnDm7:RBnDm0**: Receive Buffer n Data Field Bytes m

Eight bytes containing the data bytes for the received message

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

4.5 Message Acceptance Filters and Masks

The message acceptance filters and masks are used to determine if a message in the message assembly buffer should be loaded into either of the receive buffers (see Figure 4-5). Once a valid message has been received into the MAB, the identifier fields of the message are compared to the filter values. If there is a match, that message will be loaded into the appropriate receive buffer.

4.5.1 DATA BYTE FILTERING

When receiving standard data frames (11-bit identifier), the MCP2515 automatically applies 16 bits of masks and filters normally associated with extended identifiers to the first 16 bits of the data field (data bytes 0 and 1). Figure 4-4 illustrates how masks and filters apply to extended and standard data frames.

Data byte filtering reduces the load on the MCU when implementing Higher Layer Protocols (HLPs) that filter on the first data byte (e.g., DeviceNet™).

4.5.2 FILTER MATCHING

The filter masks (see Register 4-14 through Register 4-17) are used to determine which bits in the identifier are examined with the filters. A truth table is shown in Table 4-2 that indicates how each bit in the identifier is compared to the masks and filters to determine if the message should be loaded into a receive buffer. The mask essentially determines which bits to apply the acceptance filters to. If any mask bit is set to a zero, that bit will automatically be accepted, regardless of the filter bit.

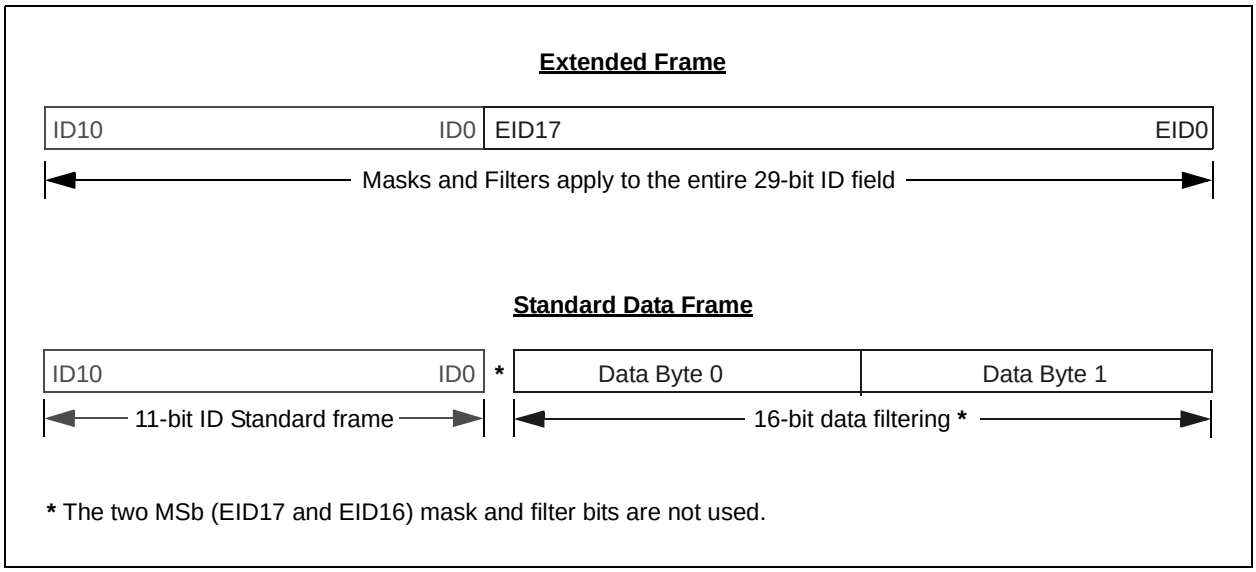
TABLE 4-2: FILTER/MASK TRUTH TABLE

Mask Bit n	Filter Bit n	Message Identifier bit	Accept or Reject bit n
0	x	x	Accept
1	0	0	Accept
1	0	1	Reject
1	1	0	Reject
1	1	1	Accept

Note: x = don't care

As shown in the receive buffers block diagram (Figure 4-2), acceptance filters RXF0 and RXF1 (and filter mask RXM0) are associated with RXB0. Filters RXF2, RXF3, RXF4, RXF5 and mask RXM1 are associated with RXB1.

FIGURE 4-4: MASKS AND FILTERS APPLY TO CAN FRAMES



* The two MSb (EID17 and EID16) mask and filter bits are not used.

 © 2005 Microchip Technology Inc.

4.5.3 FILHIT BITS

Filter matches on received messages can be determined by the FILHIT bits in the associated RXBnCTRL register. RXB0CTRL.FILHIT0 for buffer 0 and RXB1CTRL.FILHIT<2:0> for buffer 1.

The three FILHIT bits for receive buffer 1 (RXB1) are coded as follows:

- 101 = Acceptance Filter 5 (RXF5)
- 100 = Acceptance Filter 4 (RXF4)
- 011 = Acceptance Filter 3 (RXF3)
- 010 = Acceptance Filter 2 (RXF2)
- 001 = Acceptance Filter 1 (RXF1)
- 000 = Acceptance Filter 0 (RXF0)

Note:	000 and 001 can only occur if the BUKT bit in RXB0CTRL is set, allowing RXB0 messages to roll over into RXB1.

RXB0CTRL contains two copies of the BUKT bit and the FILHIT<0> bit.

The coding of the BUKT bit enables these three bits to be used similarly to the RXB1CTRL.FILHIT bits and to distinguish a hit on filter RXF0 and RXF1 in either RXB0 or after a roll over into RXB1.

- 111 = Acceptance Filter 1 (RXB1)
- 110 = Acceptance Filter 0 (RXB1)
- 001 = Acceptance Filter 1 (RXB0)
- 000 = Acceptance Filter 0 (RXB0)

If the BUKT bit is clear, there are six codes corresponding to the six filters. If the BUKT bit is set, there are six codes corresponding to the six filters, plus two additional codes corresponding to RXF0 and RXF1 filters that roll over into RXB1.

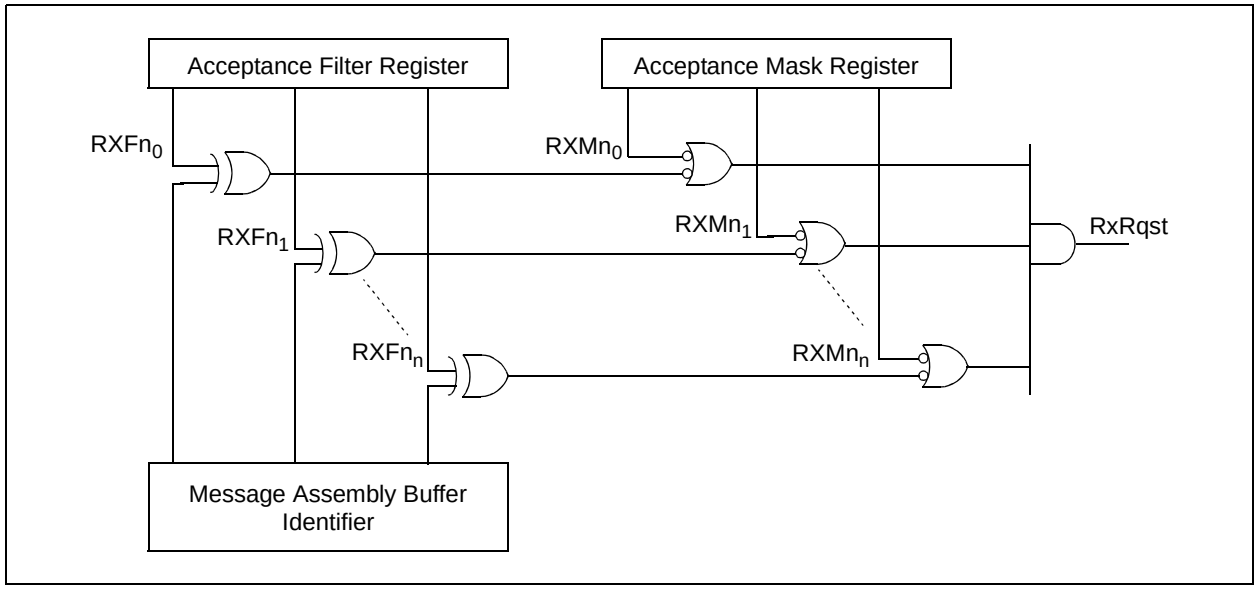
4.5.4 MULTIPLE FILTER MATCHES

If more than one acceptance filter matches, the FILHIT bits will encode the binary value of the lowest numbered filter that matched. For example, if filter RXF2 and filter RXF4 match, FILHIT will be loaded with the value for RXF2. This essentially prioritizes the acceptance filters with a lower-numbered filter having higher priority. Messages are compared to filters in ascending order of filter number. This also insures that the message will only be received into one buffer. This implies that RXB0 has a higher priority than RXB1.

4.5.5 CONFIGURING THE MASKS AND FILTERS

The mask and filter registers can only be modified when the MCP2515 is in Configuration mode (see **Section 10.0 “Modes of Operation”**).

FIGURE 4-5: MESSAGE ACCEPTANCE MASK AND FILTER OPERATION



REGISTER 4-10: RXFnSIDH – FILTER n STANDARD IDENTIFIER HIGH (ADDRESS: 00h, 04h, 08h, 10h, 14h, 18h)

R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
SID10	SID9	SID8	SID7	SID6	SID5	SID4	SID3
bit 7							bit 0

bit 7-0 **SID**: Standard Identifier Filter bits <10:3>

These bits hold the filter bits to be applied to bits <10:3> of the Standard Identifier portion of a received message

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

REGISTER 4-11: RXFnSIDL – FILTER n STANDARD IDENTIFIER LOW (ADDRESS: 01h, 05h, 09h, 11h, 15h, 19h)

R/W-x	R/W-x	R/W-x	U-0	R/W-x	U-0	R/W-x	R/W-x
SID2	SID1	SID0	—	EXIDE	—	EID17	EID16
bit 7							bit 0

bit 7-5 **SID**: Standard Identifier Filter bits <2:0>

These bits hold the filter bits to be applied to bits <2:0> of the Standard Identifier portion of a received message

bit 4 **Unimplemented**: Reads as '0'

bit 3 **EXIDE**: Extended Identifier Enable bit

1 = Filter is applied only to Extended Frames
0 = Filter is applied only to Standard Frames

bit 2 **Unimplemented**: Reads as '0'

bit 1-0 **EID**: Extended Identifier Filter bits <17:16>

These bits hold the filter bits to be applied to bits <17:16> of the Extended Identifier portion of a received message

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

 © 2005 Microchip Technology Inc.

REGISTER 4-12: RXFnEID8 – FILTER n EXTENDED IDENTIFIER HIGH (ADDRESS: 02h, 06h, 0Ah, 12h, 16h, 1Ah)

R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
EID15	EID14	EID13	EID12	EID11	EID10	EID9	EID8
bit 7							bit 0

bit 7-0 **EID**: Extended Identifier bits <15:8>

These bits hold the filter bits to be applied to bits <15:8> of the Extended Identifier portion of a received message

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

REGISTER 4-13: RXFnEID0 – FILTER n EXTENDED IDENTIFIER LOW (ADDRESS: 03h, 07h, 0Bh, 13h, 17h, 1Bh)

R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
EID7	EID6	EID5	EID4	EID3	EID2	EID1	EID0
bit 7							bit 0

bit 7-0 **EID**: Extended Identifier bits <7:0>

These bits hold the filter bits to be applied to the bits <7:0> of the Extended Identifier portion of a received message

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

REGISTER 4-14: RXMnSIDH – MASK n STANDARD IDENTIFIER HIGH (ADDRESS: 20h, 24h)

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
SID10	SID9	SID8	SID7	SID6	SID5	SID4	SID3
bit 7							bit 0

bit 7-0 **SID**: Standard Identifier Mask bits <10:3>

These bits hold the mask bits to be applied to bits <10:3> of the Standard Identifier portion of a received message

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

REGISTER 4-15: RXMnSIDL – MASK n STANDARD IDENTIFIER LOW (ADDRESS: 21h, 25h)

R/W-0	R/W-0	R/W-0	U-0	U-0	U-0	R/W-0	R/W-0
SID2	SID1	SID0	—	—	—	EID17	EID16
bit 7							bit 0

bit 7-5 **SID**: Standard Identifier Mask bits <2:0>
These bits hold the mask bits to be applied to bits<2:0> of the Standard Identifier portion of a received message

bit 4-2 **Unimplemented**: Reads as '0'

bit 1-0 **EID**: Extended Identifier Mask bits <17:16>
These bits hold the mask bits to be applied to bits <17:16> of the Extended Identifier portion of a received message

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

REGISTER 4-16: RXMnEID8 – MASK n EXTENDED IDENTIFIER HIGH (ADDRESS: 22h, 26h)

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
EID15	EID14	EID13	EID12	EID11	EID10	EID9	EID8
bit 7							bit 0

bit 7-0 **EID**: Extended Identifier bits <15:8>
These bits hold the filter bits to be applied to bits <15:8> of the Extended Identifier portion of a received message

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

REGISTER 4-17: RXMnEID0 – MASK n EXTENDED IDENTIFIER LOW (ADDRESS: 23h, 27h)

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
EID7	EID6	EID5	EID4	EID3	EID2	EID1	EID0
bit 7							bit 0

bit 7-0 **EID**: Extended Identifier Mask bits <7:0>
These bits hold the mask bits to be applied to the bits <7:0> of the Extended Identifier portion of a received message

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

 © 2005 Microchip Technology Inc.

5.0 BIT TIMING

All nodes on a given CAN bus must have the same nominal bit rate. The CAN protocol uses Non Return to Zero (NRZ) coding, which does not encode a clock within the data stream. Therefore, the receive clock must be recovered by the receiving nodes and synchronized to the transmitter's clock.

As oscillators and transmission times may vary from node to node, the receiver must have some type of Phase Lock Loop (PLL) synchronized to data transmission edges to synchronize and maintain the receiver clock. Since the data is NRZ-coded, it is necessary to include bit-stuffing to insure that an edge occurs at least every six bit times to maintain the Digital Phase Lock Loop (DPLL) synchronization.

The bit timing of the MCP2515 is implemented using a DPLL that is configured to synchronize to the incoming data, as well as provide the nominal timing for the transmitted data. The DPLL breaks each bit time into multiple segments made up of minimal periods of time, called the Time Quanta (TQ).

Bus timing functions executed within the bit time frame (such as synchronization to the local oscillator, network transmission delay compensation and sample point positioning) are defined by the programmable bit timing logic of the DPLL.

5.1 The CAN Bit TIme

All devices on the CAN bus must use the same bit rate. However, all devices are not required to have the same master oscillator clock frequency. For the different clock frequencies of the individual devices, the bit rate has to be adjusted by appropriately setting the baud rate prescaler and number of time quanta in each segment.

The CAN bit time is made up of non-overlapping segments. Each of these segments are made up of integer units called Time Quanta (TQ), explained later in this data sheet. The Nominal Bit Rate (NBR) is defined in the CAN specification as the number of bits per second transmitted by an ideal transmitter with no resynchronization. It can be described with the equation:

EQUATION 5-1:

$$NBR = f_{bit} = \frac{1}{t_{bit}}$$

Nominal Bit Time

The Nominal Bit Time (NBT) (t_{bit}) is made up of non-overlapping segments (Figure 5-1). Therefore, the NBT is the summation of the following segments:

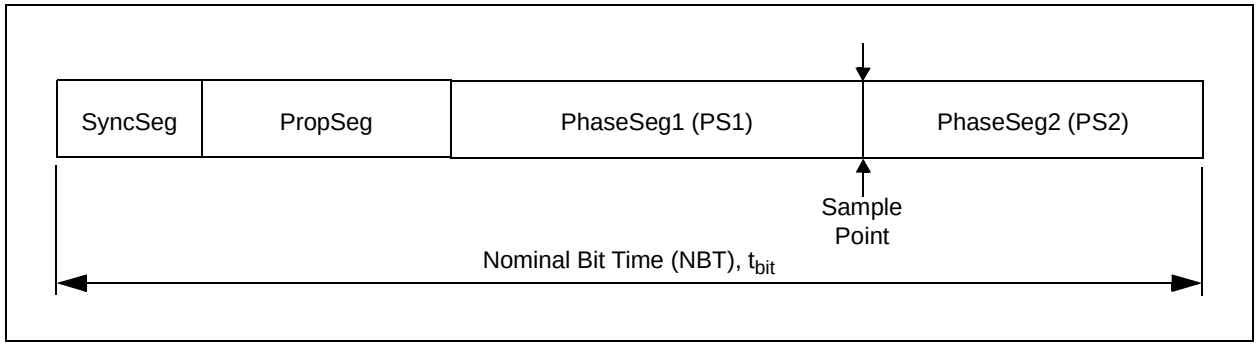
$$t_{bit} = t_{SyncSeg} + t_{PropSeg} + t_{PS1} + t_{PS2}$$

Associated with the NBT are the sample point, Synchronization Jump Width (SJW) and Information Processing Time (IPT), which are explained later.

SYNCHRONIZATION SEGMENT

The Synchronization Segment (SyncSeg) is the first segment in the NBT and is used to synchronize the nodes on the bus. Bit edges are expected to occur within the SyncSeg. This segment is fixed at 1 TQ.

FIGURE 5-1: CAN BIT TIME SEGMENTS



PROPAGATION SEGMENT

The Propagation Segment (PropSeg) exists to compensate for physical delays between nodes. The propagation delay is defined as twice the sum of the signal's propagation time on the bus line, including the delays associated with the bus driver. The PropSeg is programmable from 1 – 8 TQ.

PHASE SEGMENT 1 (PS1) AND PHASE SEGMENT 2 (PS2)

The two phase segments, PS1 and PS2, are used to compensate for edge phase errors on the bus. PS1 can be lengthened (or PS2 shortened) by resyncronization. PS1 is programmable from 1 – 8 TQ and PS2 is programmable from 2 – 8 TQ.

SAMPLE POINT

The sample point is the point in the bit time at which the logic level is read and interpreted. The sample point is located at the end of PS1. The exception to this rule is if the sample mode is configured to sample three times per bit. In this case, while the bit is still sampled at the end of PS1, two additional samples are taken at one-half TQ intervals prior to the end of PS1, with the value of the bit being determined by a majority decision.

INFORMATION PROCESSING TIME

The Information Processing Time (IPT) is the time required for the logic to determine the bit level of a sampled bit. The IPT begins at the sample point, is measured in TQ and is fixed at 2 TQ for the Microchip CAN module. Since PS2 also begins at the sample point and is the last segment in the bit time, it is required that the PS2 minimum is not less than the IPT.

Therefore:

$$PS2_{min} = IPT = 2TQ$$

SYNCHRONIZATION JUMP WIDTH

The Synchronization Jump Width (SJW) adjusts the bit clock as necessary by 1 – 4 TQ (as configured) to maintain synchronization with the transmitted message. Synchronization is covered in more detail later in this data sheet.

Time Quantum

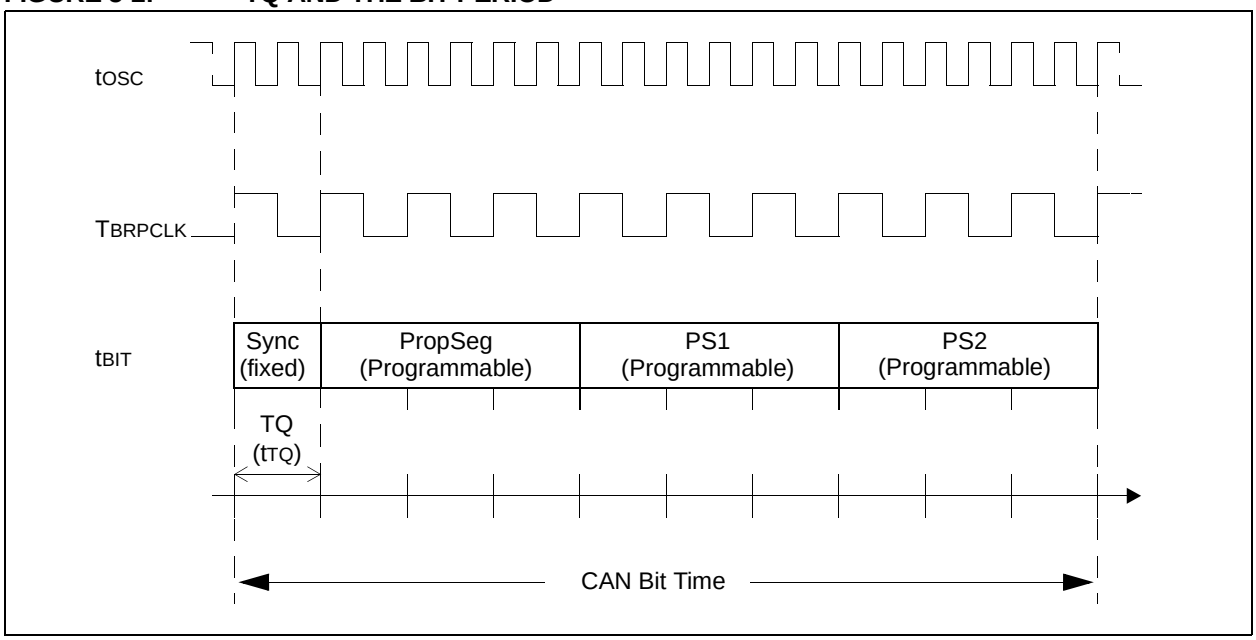
Each of the segments that make up a bit time are made up of integer units called Time Quanta (TQ). The length of each Time Quantum is based on the oscillator period (t_{OSC}). The base TQ equals twice the oscillator period. Figure 5-2 shows how the bit period is derived from T_{OSC} and TQ. The TQ length equals one TQ clock period (t_{BRPCLK}), which is programmable using a programmable prescaler, called the Baud Rate Prescaler (BRP). This is illustrated in the following equation:

EQUATION 5-2:

$$TQ = 2 \cdot BRP \cdot T_{OSC} = \frac{2 \cdot BRP}{F_{OSC}}$$

Where: BRP equals the configuration as shown in Register 5-1.

FIGURE 5-2: TQ AND THE BIT PERIOD



 © 2005 Microchip Technology Inc.

5.2 Synchronization

To compensate for phase shifts between the oscillator frequencies of each of the nodes on the bus, each CAN controller must be able to synchronize to the relevant signal edge of the incoming signal. Synchronization is the process by which the DPLL function is implemented.

When an edge in the transmitted data is detected, the logic will compare the location of the edge to the expected time (SyncSeg). The circuit will then adjust the values of PS1 and PS2 as necessary.

There are two mechanisms used for synchronization:

1. Hard synchronization.
2. Resynchronization.

5.2.1 HARD SYNCHRONIZATION

Hard synchronization is only performed when there is a recessive-to-dominant edge during a BUS IDLE condition, indicating the start of a message. After hard synchronization, the bit time counters are restarted with SyncSeg.

Hard synchronization forces the edge that has occurred to lie within the synchronization segment of the restarted bit time. Due to the rules of synchronization, if a hard synchronization occurs, there will not be a resynchronization within that bit time.

5.2.2 RESYNCHRONIZATION

As a result of resynchronization, PS1 may be lengthened or PS2 may be shortened. The amount of lengthening or shortening of the phase buffer segments has an upper-bound, given by the Synchronization Jump Width (SJW).

The value of the SJW will be added to PS1 or subtracted from PS2 (see Figure 5-3). The SJW represents the loop filtering of the DPLL. The SJW is programmable between 1 TQ and 4 TQ.

5.2.2.1 Phase Errors

The NRZ bit coding method does not encode a clock into the message. Clocking information will only be derived from recessive-to-dominant transitions. The property which states that only a fixed maximum number of successive bits have the same value (bit-stuffing) ensures resynchronization to the bit stream during a frame.

The phase error of an edge is given by the position of the edge relative to SyncSeg, measured in TQ. The phase error is defined in magnitude of TQ as follows:

- e = 0 if the edge lies within SYNCSEG.
- e > 0 if the edge lies before the SAMPLE POINT (TQ is added to PS1).
- e < 0 if the edge lies after the SAMPLE POINT of the previous bit (TQ is subtracted from PS2).

5.2.2.2 No Phase Error (e = 0)

If the magnitude of the phase error is less than or equal to the programmed value of the SJW, the effect of a resynchronization is the same as that of a hard synchronization.

5.2.2.3 Positive Phase Error (e > 0)

If the magnitude of the phase error is larger than the SJW and, if the phase error is positive, PS1 is lengthened by an amount equal to the SJW.

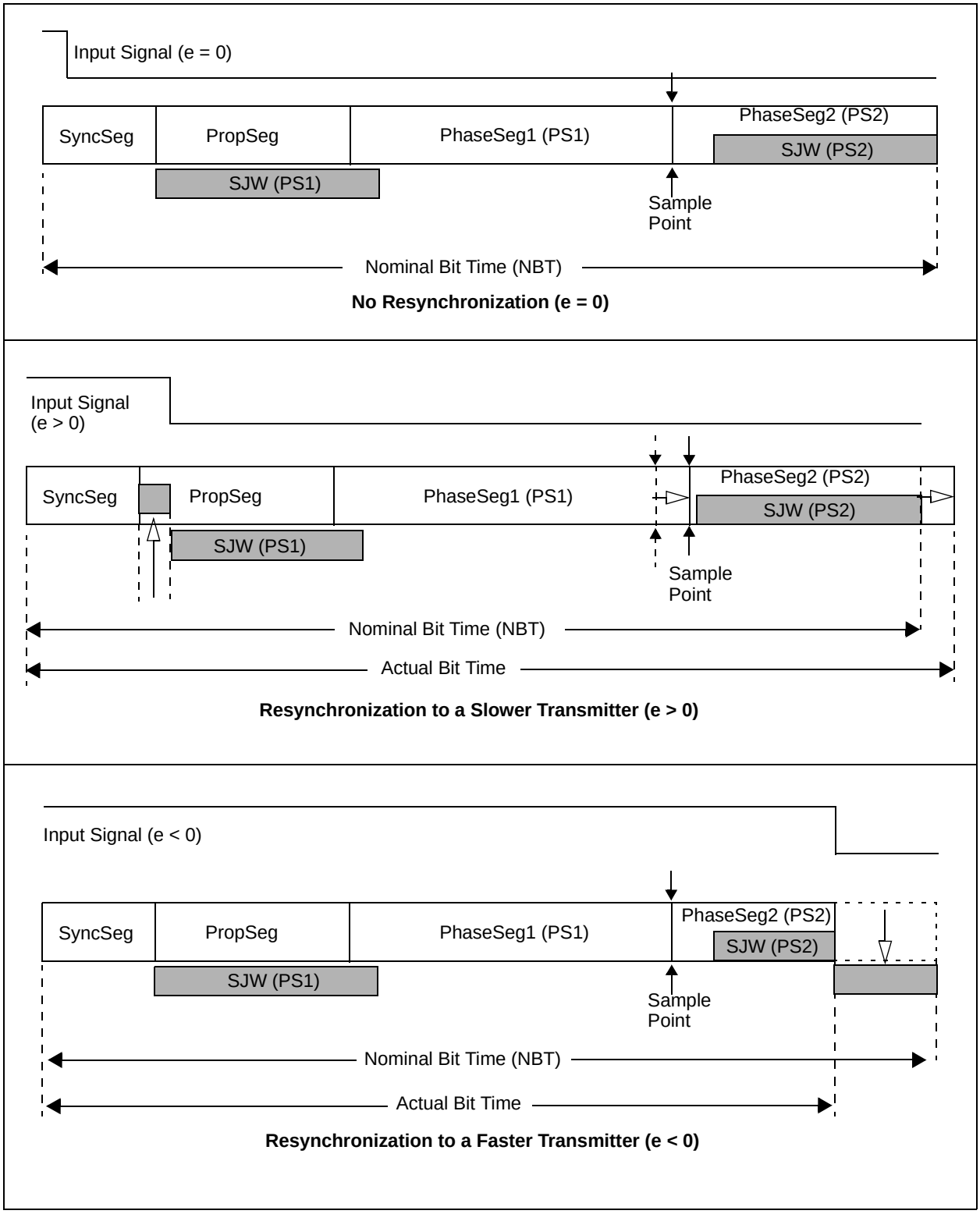
5.2.2.4 Negative Phase Error (e < 0)

If the magnitude of the phase error is larger than the resynchronization jump width and the phase error is negative, PS2 is shortened by an amount equal to the SJW.

5.2.3 SYNCHRONIZATION RULES

1. Only recessive-to-dominant edges will be used for synchronization.
2. Only one synchronization within one bit time is allowed.
3. An edge will be used for synchronization only if the value detected at the previous sample point (previously read bus value) differs from the bus value immediately after the edge.
4. A transmitting node will not resynchronize on a positive phase error (e > 0).
5. If the absolute magnitude of the phase error is greater than the SJW, the appropriate phase segment will adjust by an amount equal to the SJW.

FIGURE 5-3: SYNCHRONIZING THE BIT TIME



 © 2005 Microchip Technology Inc.

5.3 Programming Time Segments

Some requirements for programming of the time segments:

- PropSeg + PS1 >= PS2
- PropSeg + PS1 >= TDELAY
- PS2 > SJW

For example, assuming that a 125 kHz CAN baud rate with FOSC = 20 MHz is desired:

TOSC = 50 ns, choose BRP<5:0> = 04h, then TQ = 500 ns. To obtain 125 kHz, the bit time must be 16 TQ.

Typically, the sampling of the bit should take place at about 60-70% of the bit time, depending on the system parameters. Also, typically, the TDELAY is 1-2 TQ.

SyncSeg = 1 TQ and PropSeg = 2 TQ. So setting PS1 = 7 TQ would place the sample at 10 TQ after the transition. This would leave 6 TQ for PS2.

Since PS2 is 6, according to the rules, SJW could be a maximum of 4 TQ. However, a large SJW is typically only necessary when the clock generation of the different nodes is inaccurate or unstable, such as using ceramic resonators. So a SJW of 1 is usually enough.

5.4 Oscillator Tolerance

The bit timing requirements allow ceramic resonators to be used in applications with transmission rates of up to 125 kbit/sec as a rule of thumb. For the full bus speed range of the CAN protocol, a quartz oscillator is required. A maximum node-to-node oscillator variation of 1.7% is allowed.

5.5 Bit Timing Configuration Registers

The configuration registers (CNF1, CNF2, CNF3) control the bit timing for the CAN bus interface. These registers can only be modified when the MCP2515 is in Configuration mode (see **Section 10.0 “Modes of Operation”**).

5.5.1 CNF1

The BRP<5:0> bits control the baud rate prescaler. These bits set the length of TQ relative to the OSC1 input frequency, with the minimum TQ length being 2 TOSC (when BRP<5:0> = ‘b000000’). The SJW<1:0> bits select the SJW in terms of number of TQs.

5.5.2 CNF2

The PRSEG<2:0> bits set the length (in TQ’s) of the propagation segment. The PHSEG1<2:0> bits set the length (in TQ’s) of PS1.

The SAM bit controls how many times the RXCAN pin is sampled. Setting this bit to a ‘1’ causes the bus to be sampled three times: twice at TQ/2 before the sample point and once at the normal sample point (which is at the end of PS1). The value of the bus is determined to be the majority sampled. If the SAM bit is set to a ‘0’, the RXCAN pin is sampled only once at the sample point.

The BTLMODE bit controls how the length of PS2 is determined. If this bit is set to a ‘1’, the length of PS2 is determined by the PHSEG2<2:0> bits of CNF3 (see **Section 5.5.3 “CNF3”**). If the BTLMODE bit is set to a ‘0’, the length of PS2 is greater than that of PS1 and the information processing time (which is fixed at 2 TQ for the MCP2515).

5.5.3 CNF3

The PHSEG2<2:0> bits set the length (in TQ’s) of PS2, if the CNF2.BTLMODE bit is set to a ‘1’. If the BTLMODE bit is set to a ‘0’, the PHSEG2<2:0> bits have no effect.

REGISTER 5-1: CNF1 – CONFIGURATION 1 (ADDRESS: 2Ah)

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
SJW1	SJW0	BRP5	BRP4	BRP3	BRP2	BRP1	BRP0
bit 7							bit 0

bit 7-6 **SJW**: Synchronization Jump Width Length bits <1:0>

11 = Length = 4 x TQ
10 = Length = 3 x TQ
01 = Length = 2 x TQ
00 = Length = 1 x TQ

bit 5-0 **BRP**: Baud Rate Prescaler bits <5:0>

TQ = 2 x (BRP + 1)/FOSC

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

REGISTER 5-2: CNF2 – CONFIGURATION 1 (ADDRESS: 29h)

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
BTLMODE	SAM	PHSEG12	PHSEG11	PHSEG10	PRSEG2	PRSEG1	PRSEG0
bit 7							bit 0

bit 7 **BTLMODE**: PS2 Bit Time Length bit

1 = Length of PS2 determined by PHSEG22:PHSEG20 bits of CNF3
0 = Length of PS2 is the greater of PS1 and IPT (2 TQ)

bit 6 **SAM**: Sample Point Configuration bit

1 = Bus line is sampled three times at the sample point
0 = Bus line is sampled once at the sample point

bit 5-3 **PHSEG1**: PS1 Length bits<2:0>

(PHSEG1 + 1) x TQ

bit 2-0 **PRSEG**: Propagation Segment Length bits <2:0>

(PRSEG + 1) x TQ

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

 © 2005 Microchip Technology Inc.

REGISTER 5-3: CNF3 - CONFIGURATION 1 (ADDRESS: 28h)

R/W-0	R/W-0	U-0	U-0	U-0	R/W-0	R/W-0	R/W-0
SOF	WAKFIL	—	—	—	PHSEG22	PHSEG21	PHSEG20
bit 7							bit 0

bit 7 **SOF:** Start-of-Frame signal bit
If CANCTRL.CLKEN = 1:
1 = CLKOUT pin enabled for SOF signal
0 = CLKOUT pin enabled for clockout function
If CANCTRL.CLKEN = 0, Bit is don't care.

bit 6 **WAKFIL**: Wake-up Filter bit
1 = Wake-up filter enabled
0 = Wake-up filter disabled

bit 5-3 **Unimplemented**: Reads as '0'

bit 2-0 **PHSEG2**: PS2 Length bits<2:0>
(PHSEG2 + 1) x T_Q

Note: Minimum valid setting for PS2 is 2 T_Q

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

NOTES:

 © 2005 Microchip Technology Inc.

6.0 ERROR DETECTION

The CAN protocol provides sophisticated error detection mechanisms. The following errors can be detected.

6.1 CRC Error

With the Cyclic Redundancy Check (CRC), the transmitter calculates special check bits for the bit sequence from the start of a frame until the end of the data field. This CRC sequence is transmitted in the CRC Field. The receiving node also calculates the CRC sequence using the same formula and performs a comparison to the received sequence. If a mismatch is detected, a CRC error has occurred and an error frame is generated. The message is repeated.

6.2 Acknowledge Error

In the acknowledge field of a message, the transmitter checks if the acknowledge slot (which has been sent out as a recessive bit) contains a dominant bit. If not, no other node has received the frame correctly. An acknowledge error has occurred, an error frame is generated and the message will have to be repeated.

6.3 Form Error

If a node detects a dominant bit in one of the four segments (including end-of-frame, interframe space, acknowledge delimiter or CRC delimiter), a form error has occurred and an error frame is generated. The message is repeated.

6.4 Bit Error

A bit error occurs if a transmitter detects the opposite bit level to what it transmitted (i.e., transmitted a dominant and detected a recessive, or transmitted a recessive and detected a dominant).

Exception: In the case where the transmitter sends a recessive bit and a dominant bit is detected during the arbitration field and the acknowledge slot, no bit error is generated because normal arbitration is occurring.

6.5 Stuff Error

If, between the start-of-frame and the CRC delimiter, six consecutive bits with the same polarity are detected, the bit-stuffing rule has been violated. A stuff error occurs and an error frame is generated. The message is repeated.

6.6 Error States

Detected errors are made known to all other nodes via error frames. The transmission of the erroneous message is aborted and the frame is repeated as soon as possible. Furthermore, each CAN node is in one of the three error states according to the value of the internal error counters:

1. Error-active.
2. Error-passive.
3. Bus-off (transmitter only).

The error-active state is the usual state where the node can transmit messages and active error frames (made of dominant bits) without any restrictions.

In the error-passive state, messages and passive error frames (made of recessive bits) may be transmitted.

The bus-off state makes it temporarily impossible for the station to participate in the bus communication. During this state, messages can neither be received or transmitted. Only transmitters can go bus-off.

6.7 Error Modes and Error Counters

The MCP2515 contains two error counters: the Receive Error Counter (REC) (see Register 6-2) and the Transmit Error Counter (TEC) (see Register 6-1). The values of both counters can be read by the MCU. These counters are incremented/decremented in accordance with the CAN bus specification.

The MCP2515 is error-active if both error counters are below the error-passive limit of 128.

It is error-passive if at least one of the error counters equals or exceeds 128.

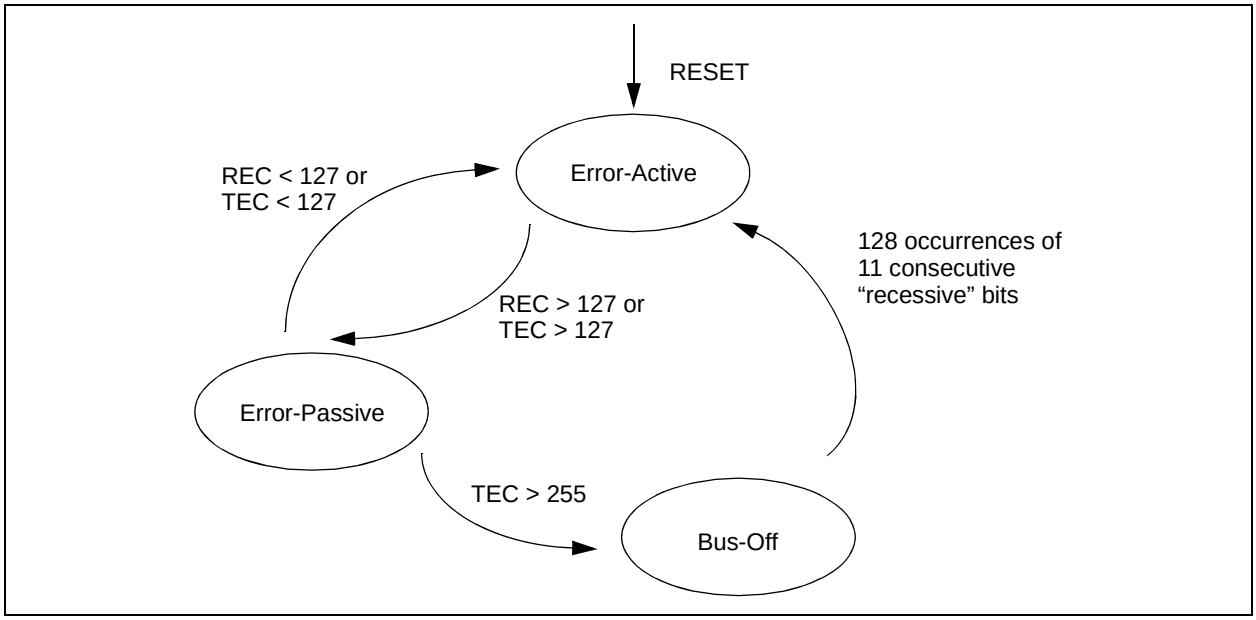
It goes to bus-off if the TEC exceeds the bus-off limit of 255. The device remains in this state until the bus-off recovery sequence is received. The bus-off recovery sequence consists of 128 occurrences and 11 consecutive recessive bits (see Figure 6-1).

Note:	The MCP2515, after going bus-off, will recover back to error-active without any intervention by the MCU if the bus remains idle for 128 x 11 bit times. If this is not desired, the error interrupt service routine should address this.

The Current Error mode of the MCP2515 can be read by the MCU via the EFLG register (see Register 6-3).

Additionally, there is an error state warning flag bit (EFLG:EWARN) which is set if at least one of the error counters equals or exceeds the error warning limit of 96. EWARN is reset if both error counters are less than the error warning limit.

FIGURE 6-1: ERROR MODES STATE DIAGRAM



REGISTER 6-1: TEC – TRANSMIT ERROR COUNTER (ADDRESS: 1Ch)

R-0	R-0	R-0	R-0	R-0	R-0	R-0	R-0
TEC7	TEC6	TEC5	TEC4	TEC3	TEC2	TEC1	TEC0
bit 7							bit 0

bit 7-0 **TEC**: Transmit Error Count bits <7:0>

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as ‘0’	
-n = Value at POR	‘1’ = Bit is set	‘0’ = Bit is cleared	x = Bit is unknown

REGISTER 6-2: REC – RECEIVER ERROR COUNTER (ADDRESS: 1Dh)

R-0	R-0	R-0	R-0	R-0	R-0	R-0	R-0
REC7	REC6	REC5	REC4	REC3	REC2	REC1	REC0
bit 7							bit 0

bit 7-0 **REC**: Receive Error Count bits <7:0>

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as ‘0’	
-n = Value at POR	‘1’ = Bit is set	‘0’ = Bit is cleared	x = Bit is unknown

 © 2005 Microchip Technology Inc.

REGISTER 6-3: EFLG – ERROR FLAG (ADDRESS: 2Dh)

R/W-0	R/W-0	R-0	R-0	R-0	R-0	R-0	R-0
RX1OVR	RX0OVR	TXBO	TXEP	RXEP	TXWAR	RXWAR	EWARN
bit 7							bit 0

bit 7 **RX1OVR**: Receive Buffer 1 Overflow Flag bit
- Set when a valid message is received for RXB1 and CANINTF.RX1IF = 1
- Must be reset by MCU

bit 6 **RX0OVR**: Receive Buffer 0 Overflow Flag bit
- Set when a valid message is received for RXB0 and CANINTF.RX0IF = 1
- Must be reset by MCU

bit 5 **TXBO**: Bus-Off Error Flag bit
- Bit set when TEC reaches 255
- Reset after a successful bus recovery sequence

bit 4 **TXEP**: Transmit Error-Passive Flag bit
- Set when TEC is equal to or greater than 128
- Reset when TEC is less than 128

bit 3 **RXEP**: Receive Error-Passive Flag bit
- Set when REC is equal to or greater than 128
- Reset when REC is less than 128

bit 2 **TXWAR**: Transmit Error Warning Flag bit
- Set when TEC is equal to or greater than 96
- Reset when TEC is less than 96

bit 1 **RXWAR**: Receive Error Warning Flag bit
- Set when REC is equal to or greater than 96
- Reset when REC is less than 96

bit 0 **EWARN**: Error Warning Flag bit
- Set when TEC or REC is equal to or greater than 96 (TXWAR or RXWAR = 1)
- Reset when both REC and TEC are less than 96

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

NOTES:

 © 2005 Microchip Technology Inc.

7.0 INTERRUPTS

The MCP2515 has eight sources of interrupts. The CANINTE register contains the individual interrupt enable bits for each interrupt source. The CANINTF register contains the corresponding interrupt flag bit for each interrupt source. When an interrupt occurs, the $\overline{INT}$ pin is driven low by the MCP2515 and will remain low until the interrupt is cleared by the MCU. An interrupt can not be cleared if the respective condition still prevails.

It is recommended that the bit modify command be used to reset flag bits in the CANINTF register rather than normal write operations. This is done to prevent unintentionally changing a flag that changes during the write command, potentially causing an interrupt to be missed.

It should be noted that the CANINTF flags are read/write and an interrupt can be generated by the MCU setting any of these bits, provided the associated CANINTE bit is also set.

7.1 Interrupt Code Bits

The source of a pending interrupt is indicated in the CANSTAT.ICOD (interrupt code) bits, as indicated in Register 10-2. In the event that multiple interrupts occur, the $\overline{INT}$ will remain low until all interrupts have been reset by the MCU. The CANSTAT.ICOD bits will reflect the code for the highest priority interrupt that is currently pending. Interrupts are internally prioritized such that the lower the ICOD value, the higher the interrupt priority. Once the highest priority interrupt condition has been cleared, the code for the next highest priority interrupt that is pending (if any) will be reflected by the ICOD bits (see Table 7-1). Only those interrupt sources that have their associated CANINTE enable bit set will be reflected in the ICOD bits.

TABLE 7-1: ICOD<2:0> DECODE

ICOD<2:0>	Boolean Expression
000	$\overline{ERR}$•$\overline{WAK}$•$\overline{TX0}$•$\overline{TX1}$•$\overline{TX2}$•$\overline{RX0}$•$\overline{RX1}$
001	ERR
010	$\overline{ERR}$•WAK
011	$\overline{ERR}$•$\overline{WAK}$•TX0
100	$\overline{ERR}$•$\overline{WAK}$•$\overline{TX0}$•TX1
101	$\overline{ERR}$•$\overline{WAK}$•$\overline{TX0}$•$\overline{TX1}$•TX2
110	$\overline{ERR}$•$\overline{WAK}$•$\overline{TX0}$•$\overline{TX1}$•$\overline{TX2}$•RX0
111	$\overline{ERR}$•$\overline{WAK}$•$\overline{TX0}$•$\overline{TX1}$•$\overline{TX2}$•$\overline{RX0}$•RX1

Note: $\overline{ERR}$ is associated with CANINTE,ERRIE.

7.2 Transmit Interrupt

When the transmit interrupt is enabled (CANINTE.TXnIE = 1), an interrupt will be generated on the $\overline{INT}$ pin once the associated transmit buffer becomes empty and is ready to be loaded with a new message. The CANINTF.TXnIF bit will be set to indicate the source of the interrupt. The interrupt is cleared by clearing the TXnIF bit.

7.3 Receive Interrupt

When the receive interrupt is enabled (CANINTE.RXnIE = 1), an interrupt will be generated on the $\overline{INT}$ pin once a message has been successfully received and loaded into the associated receive buffer. This interrupt is activated immediately after receiving the EOF field. The CANINTF.RXnIF bit will be set to indicate the source of the interrupt. The interrupt is cleared by clearing the RXnIF bit.

7.4 Message Error Interrupt

When an error occurs during the transmission or reception of a message, the message error flag (CANINTF.MERRF) will be set and, if the CANINTE.MERRE bit is set, an interrupt will be generated on the $\overline{INT}$ pin. This is intended to be used to facilitate baud rate determination when used in conjunction with Listen-only mode.

7.5 Bus Activity Wakeup Interrupt

When the MCP2515 is in Sleep mode and the bus activity wakeup interrupt is enabled (CANINTE.WAKIE = 1), an interrupt will be generated on the $\overline{INT}$ pin and the CANINTF.WAKIF bit will be set when activity is detected on the CAN bus. This interrupt causes the MCP2515 to exit Sleep mode. The interrupt is reset by clearing the WAKIF bit.

Note:	The MCP2515 wakes up into Listen-only mode.

7.6 Error Interrupt

When the error interrupt is enabled (CANINTE.ERRIE = 1), an interrupt is generated on the $\overline{INT}$ pin if an overflow condition occurs or if the error state of the transmitter or receiver has changed. The Error Flag (EFLG) register will indicate one of the following conditions.

7.6.1 RECEIVER OVERFLOW

An overflow condition occurs when the MAB has assembled a valid receive message (the message meets the criteria of the acceptance filters) and the receive buffer associated with the filter is not available for loading of a new message. The associated EFLG.RXnOVR bit will be set to indicate the overflow condition. This bit must be cleared by the MCU.

7.6.2 RECEIVER WARNING

The REC has reached the MCU warning limit of 96.

7.6.3 TRANSMITTER WARNING

The TEC has reached the MCU warning limit of 96.

7.6.4 RECEIVER ERROR-PASSIVE

The REC has exceeded the error-passive limit of 127 and the device has gone to error-passive state.

7.6.5 TRANSMITTER ERROR-PASSIVE

The TEC has exceeded the error- passive limit of 127 and the device has gone to error- passive state.

7.6.6 BUS-OFF

The TEC has exceeded 255 and the device has gone to bus-off state.

7.7 Interrupt Acknowledge

Interrupts are directly associated with one or more status flags in the CANINTF register. Interrupts are pending as long as one of the flags is set. Once an interrupt flag is set by the device, the flag can not be reset by the MCU until the interrupt condition is removed.

REGISTER 7-1: CANINTE – INTERRUPT ENABLE (ADDRESS: 2Bh)

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
MERRE	WAKIE	ERRIE	TX2IE	TX1IE	TX0IE	RX1IE	RX0IE
bit 7							bit 0

bit 7 **MERRE**: Message Error Interrupt Enable bit
1 = Interrupt on error during message reception or transmission
0 = Disabled

bit 6 **WAKIE**: Wakeup Interrupt Enable bit
1 = Interrupt on CAN bus activity
0 = Disabled

bit 5 **ERRIE**: Error Interrupt Enable bit (multiple sources in EFLG register)
1 = Interrupt on EFLG error condition change
0 = Disabled

bit 4 **TX2IE**: Transmit Buffer 2 Empty Interrupt Enable bit
1 = Interrupt on TXB2 becoming empty
0 = Disabled

bit 3 **TX1IE**: Transmit Buffer 1 Empty Interrupt Enable bit
1 = Interrupt on TXB1 becoming empty
0 = Disabled

bit 2 **TX0IE**: Transmit Buffer 0 Empty Interrupt Enable bit
1 = Interrupt on TXB0 becoming empty
0 = Disabled

bit 1 **RX1IE**: Receive Buffer 1 Full Interrupt Enable bit
1 = Interrupt when message received in RXB1
0 = Disabled

bit 0 **RX0IE**: Receive Buffer 0 Full Interrupt Enable bit
1 = Interrupt when message received in RXB0
0 = Disabled

Legend:		
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared x = Bit is unknown

 © 2005 Microchip Technology Inc.

REGISTER 7-2: CANINTF – INTERRUPT FLAG (ADDRESS: 2Ch)

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
MERRF	WAKIF	ERRIF	TX2IF	TX1IF	TX0IF	RX1IF	RX0IF
bit 7							bit 0

bit 7 **MERRF**: Message Error Interrupt Flag bit
1 = Interrupt pending (must be cleared by MCU to reset interrupt condition)
0 = No interrupt pending

bit 6 **WAKIF**: Wakeup Interrupt Flag bit
1 = Interrupt pending (must be cleared by MCU to reset interrupt condition)
0 = No interrupt pending

bit 5 **ERRIF**: Error Interrupt Flag bit (multiple sources in EFLG register)
1 = Interrupt pending (must be cleared by MCU to reset interrupt condition)
0 = No interrupt pending

bit 4 **TX2IF**: Transmit Buffer 2 Empty Interrupt Flag bit
1 = Interrupt pending (must be cleared by MCU to reset interrupt condition)
0 = No interrupt pending

bit 3 **TX1IF**: Transmit Buffer 1 Empty Interrupt Flag bit
1 = Interrupt pending (must be cleared by MCU to reset interrupt condition)
0 = No interrupt pending

bit 2 **TX0IF**: Transmit Buffer 0 Empty Interrupt Flag bit
1 = Interrupt pending (must be cleared by MCU to reset interrupt condition)
0 = No interrupt pending

bit 1 **RX1IF**: Receive Buffer 1 Full Interrupt Flag bit
1 = Interrupt pending (must be cleared by MCU to reset interrupt condition)
0 = No interrupt pending

bit 0 **RX0IF**: Receive Buffer 0 Full Interrupt Flag bit
1 = Interrupt pending (must be cleared by MCU to reset interrupt condition)
0 = No interrupt pending

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

NOTES:

 © 2005 Microchip Technology Inc.

8.0 OSCILLATOR

The MCP2515 is designed to be operated with a crystal or ceramic resonator connected to the OSC1 and OSC2 pins. The MCP2515 oscillator design requires the use of a parallel cut crystal. Use of a series cut crystal may give a frequency out of the crystal manufacturers specifications. A typical oscillator circuit is shown in Figure 8-1. The MCP2515 may also be driven by an external clock source connected to the OSC1 pin, as shown in Figure 8-2 and Figure 8-3.

8.1 Oscillator Startup Timer

The MCP2515 utilizes an Oscillator Startup Timer (OST) that holds the MCP2515 in reset to ensure that the oscillator has stabilized before the internal state machine begins to operate. The OST maintains reset for the first 128 OSC1 clock cycles after power-up or a wake-up from Sleep mode occurs. It should be noted that no SPI protocol operations should be attempted until after the OST has expired.

8.2 CLKOUT Pin

The CLKOUT pin is provided to the system designer for use as the main system clock or as a clock input for other devices in the system. The CLKOUT has an internal prescaler which can divide F_{OSC} by 1, 2, 4 and 8. The CLKOUT function is enabled and the prescaler is selected via the CANCNTRL register (see Register 10-1).

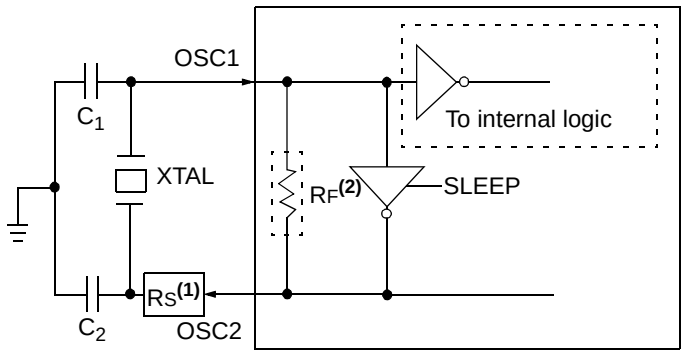
Note:	The maximum frequency on CLKOUT is specified as 25 MHz (See Table 13-5)

The CLKOUT pin will be active upon system reset and default to the slowest speed (divide by 8) so that it can be used as the MCU clock.

When Sleep mode is requested, the MCP2515 will drive sixteen additional clock cycles on the CLKOUT pin before entering Sleep mode. The idle state of the CLKOUT pin in Sleep mode is low. When the CLKOUT function is disabled (CANCNTRL.CLKEN = '0') the CLKOUT pin is in a high-impedance state.

The CLKOUT function is designed to ensure that $t_{hCLKOUT}$ and $t_{lCLKOUT}$ timings are preserved when the CLKOUT pin function is enabled, disabled or the prescaler value is changed.

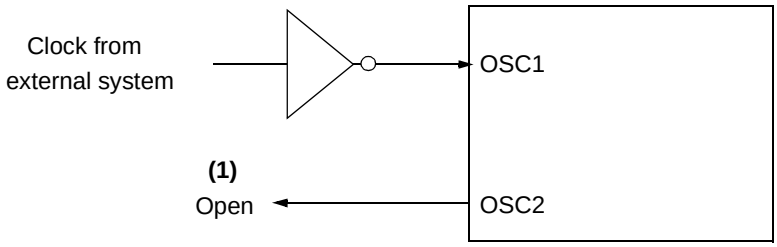
FIGURE 8-1: CRYSTAL/CERAMIC RESONATOR OPERATION



Note 1: A series resistor (RS) may be required for AT strip cut crystals.

2: The feedback resistor (RF), is typically in the range of 2 to 10 MΩ.

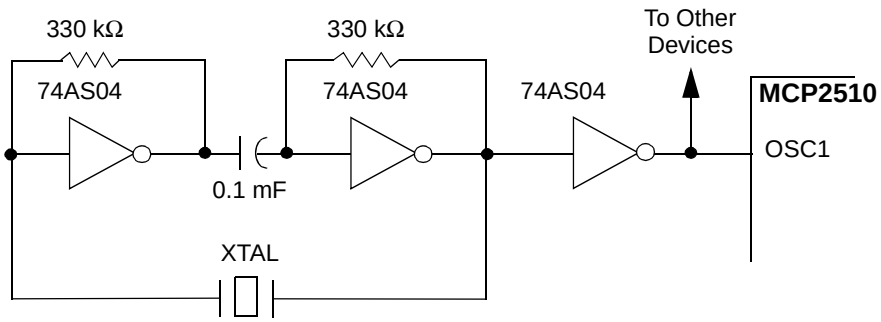
FIGURE 8-2: EXTERNAL CLOCK SOURCE



Note 1: A resistor to ground may be used to reduce system noise. This may increase system current.

2: Duty cycle restrictions must be observed (see Table 12-2).

FIGURE 8-3: EXTERNAL SERIES RESONANT CRYSTAL OSCILLATOR CIRCUIT(1)



Note 1: Duty cycle restrictions must be observed (see Table 12-2).

TABLE 8-1: CAPACITOR SELECTION FOR CERAMIC RESONATORS

Typical Capacitor Values Used:			
Mode	**Freq.**	**OSC1**	**OSC2**
HS	8.0 MHz	27 pF	27 pF
	16.0 MHz	22 pF	22 pF
Capacitor values are for design guidance only: These capacitors were tested with the resonators listed below for basic start-up and operation. **These values are not optimized.** Different capacitor values may be required to produce acceptable oscillator operation. The user should test the performance of the oscillator over the expected VDD and temperature range for the application. See the notes following Table 8-2 for additional information.			
Resonators Used:			
4.0 MHz			
8.0 MHz			
16.0 MHz			

TABLE 8-2: CAPACITOR SELECTION FOR CRYSTAL OSCILLATOR

Osc Type(1)(4)	**Crystal Freq.(2)**	**Typical Capacitor Values Tested:**	
		C1	**C2**
HS	4 MHz	27 pF	27 pF
	8 MHz	22 pF	22 pF
	20 MHz	15 pF	15 pF
Capacitor values are for design guidance only: These capacitors were tested with the crystals listed below for basic start-up and operation. **These values are not optimized.** Different capacitor values may be required to produce acceptable oscillator operation. The user should test the performance of the oscillator over the expected VDD and temperature range for the application. See the notes following this Table for additional information.			
Crystals Used(3):			
4.0 MHz			
8.0 MHz			
20.0 MHz			

Note 1: While higher capacitance increases the stability of the oscillator, it also increases the start-up time.

2: Since each resonator/crystal has its own characteristics, the user should consult the resonator/crystal manufacturer for appropriate values of external components.

3: RS may be required to avoid overdriving crystals with low drive level specification.

4: Always verify oscillator performance over the VDD and temperature range that is expected for the application.

 © 2005 Microchip Technology Inc.

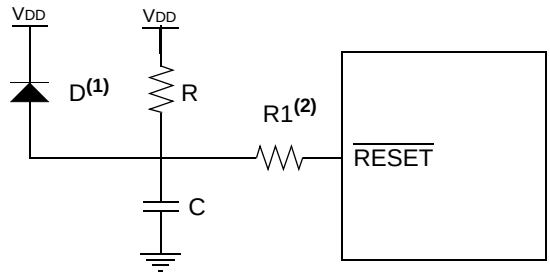
9.0 RESET

The MCP2515 differentiates between two resets:

1. Hardware Reset – Low on $\overline{\text{RESET}}$ pin.
2. SPI Reset – Reset via SPI command.

Both of these resets are functionally equivalent. It is important to provide one of these two resets after power-up to ensure that the logic and registers are in their default state. A hardware reset can be achieved automatically by placing an RC on the $\overline{\text{RESET}}$ pin. (see Figure 9-1). The values must be such that the device is held in reset for a minimum of 2 µs after V_{DD} reaches operating voltage, as indicated in the electrical specification (t_{RL}).

FIGURE 9-1: $\overline{\text{RESET}}$ PIN CONFIGURATION EXAMPLE



Note 1: The diode D helps discharge the capacitor quickly when V_{DD} powers down.

2: R1 = 1 kΩ to 10 kΩ will limit any current flowing into $\overline{\text{RESET}}$ from external capacitor C, in the event of $\overline{\text{RESET}}$ pin breakdown due to Electrostatic Discharge (ESD) or Electrical Overstress (EOS).

NOTES:

 © 2005 Microchip Technology Inc.

10.0 MODES OF OPERATION

The MCP2515 has five modes of operation. These modes are:

1. Configuration mode.
2. Normal mode.
3. Sleep mode.
4. Listen-only mode.
5. Loopback mode.

The operational mode is selected via the CANCTRL. REQOP bits (see Register 10-1).

When changing modes, the mode will not actually change until all pending message transmissions are complete. The requested mode must be verified by reading the CANSTAT.OPMODE bits (see Register 10-2).

10.1 Configuration Mode

The MCP2515 must be initialized before activation. This is only possible if the device is in the Configuration mode. Configuration mode is automatically selected after power-up, a reset or can be entered from any other mode by setting the CANTRL.REQOP bits to '100'. When Configuration mode is entered, all error counters are cleared. Configuration mode is the only mode where the following registers are modifiable:

- CNF1, CNF2, CNF3
- TXRTSCTRL
- Filter registers
- Mask registers

10.2 Sleep Mode

The MCP2515 has an internal Sleep mode that is used to minimize the current consumption of the device. The SPI interface remains active for reading even when the MCP2515 is in Sleep mode, allowing access to all registers.

To enter Sleep mode, the mode request bits are set in the CANCTRL register (REQOP<2:0>). The CANSTAT.OPMODE bits indicate operation mode. These bits should be read after sending the sleep command to the MCP2515. The MCP2515 is active and has not yet entered Sleep mode until these bits indicate that Sleep mode has been entered.

When in internal Sleep mode, the wake-up interrupt is still active (if enabled). This is done so that the MCU can also be placed into a Sleep mode and use the MCP2515 to wake it up upon detecting activity on the bus.

When in Sleep mode, the MCP2515 stops its internal oscillator. The MCP2515 will wake-up when bus activity occurs or when the MCU sets, via the SPI interface, the CANINTF.WAKIF bit to 'generate' a wake-up attempt (the CANINTE.WAKIE bit must also be set in order for the wake-up interrupt to occur).

The TXCAN pin will remain in the recessive state while the MCP2515 is in Sleep mode.

10.2.1 WAKE-UP FUNCTIONS

The device will monitor the RXCAN pin for activity while it is in Sleep mode. If the CANINTE.WAKIE bit is set, the device will wake up and generate an interrupt. Since the internal oscillator is shut down while in Sleep mode, it will take some amount of time for the oscillator to start up and the device to enable itself to receive messages. This Oscillator Start-up Timer (OST) is defined as 128 TOSC.

The device will ignore the message that caused the wake-up from Sleep mode, as well as any messages that occur while the device is 'waking up'. The device will wake up in Listen-only mode. The MCU must set Normal mode before the MCP2515 will be able to communicate on the bus.

The device can be programmed to apply a low-pass filter function to the RXCAN input line while in internal Sleep mode. This feature can be used to prevent the device from waking up due to short glitches on the CAN bus lines. The CNF3.WAKFIL bit enables or disables the filter.

10.3 Listen-only Mode

Listen-only mode provides a means for the MCP2515 to receive all messages (including messages with errors) by configuring the RXBnCTRL.RXM<1:0> bits. This mode can be used for bus monitor applications or for detecting the baud rate in 'hot plugging' situations.

For auto-baud detection, it is necessary that there are at least two other nodes that are communicating with each other. The baud rate can be detected empirically by testing different values until valid messages are received.

Listen-only mode is a silent mode, meaning no messages will be transmitted while in this mode (including error flags or acknowledge signals). The filters and masks can be used to allow only particular messages to be loaded into the receive registers, or the masks can be set to all zeros to allow a message with any identifier to pass. The error counters are reset and deactivated in this state. The Listen-only mode is activated by setting the mode request bits in the CANCTRL register.

10.4 Loopback Mode

Loopback mode will allow internal transmission of messages from the transmit buffers to the receive buffers without actually transmitting messages on the CAN bus. This mode can be used in system development and testing.

In this mode, the ACK bit is ignored and the device will allow incoming messages from itself just as if they were coming from another node. The Loopback mode is a silent mode, meaning no messages will be transmitted while in this state (including error flags or acknowledge signals). The TXCAN pin will be in a recessive state.

The filters and masks can be used to allow only particular messages to be loaded into the receive registers. The masks can be set to all zeros to provide a mode that accepts all messages. The Loopback mode is activated by setting the mode request bits in the CANCTRL register.

10.5 Normal Mode

Normal mode is the standard operating mode of the MCP2515. In this mode, the device actively monitors all bus messages and generates acknowledge bits, error frames, etc. This is also the only mode in which the MCP2515 will transmit messages over the CAN bus.

REGISTER 10-1: CANCTRL – CAN CONTROL REGISTER (ADDRESS: XFh)

R/W-1	R/W-0	R/W-0	R/W-0	R/W-0	R/W-1	R/W-1	R/W-1
REQOP2	REQOP1	REQOP0	ABAT	OSM	CLKEN	CLKPRE1	CLKPRE0
bit 7							bit 0

bit 7-5 **REQOP:** Request Operation Mode bits <2:0>
000 = Set Normal Operation mode
001 = Set Sleep mode
010 = Set Loopback mode
011 = Set Listen-only mode
100 = Set Configuration mode
All other values for REQOP bits are invalid and should not be used

Note: On power-up, REQOP = b'111'

bit 4 **ABAT:** Abort All Pending Transmissions bit
1 = Request abort of all pending transmit buffers
0 = Terminate request to abort all transmissions

bit 3 **OSM:** One Shot Mode bit
1 = Enabled. Message will only attempt to transmit one time
0 = Disabled. Messages will reattempt transmission, if required

bit 2 **CLKEN:** CLKOUT Pin Enable bit
1 = CLKOUT pin enabled
0 = CLKOUT pin disabled (Pin is in high-impedance state)

bit 1-0 **CLKPRE:** CLKOUT Pin Prescaler bits <1:0>
00 = F_{CLKOUT} = System Clock/1
01 = F_{CLKOUT} = System Clock/2
10 = F_{CLKOUT} = System Clock/4
11 = F_{CLKOUT} = System Clock/8

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

 © 2005 Microchip Technology Inc.

REGISTER 10-2: CANSTAT – CAN STATUS REGISTER (ADDRESS: XEh)

R-1	R-0	R-0	U-0	R-0	R-0	R-0	U-0
OPMOD2	OPMOD1	OPMOD0	—	ICOD2	ICOD1	ICOD0	—
bit 7							bit 0

bit 7-5 **OPMOD**: Operation Mode bits <2:0>
- 000 = Device is in the Normal operation mode
- 001 = Device is in Sleep mode
- 010 = Device is in Loopback mode
- 011 = Device is in Listen-only mode
- 100 = Device is in Configuration mode

bit 4 **Unimplemented:** Read as '0'

bit 3-1 **ICOD**: Interrupt Flag Code bits <2:0>
- 000 = No Interrupt
- 001 = Error Interrupt
- 010 = Wake-up Interrupt
- 011 = TXB0 Interrupt
- 100 = TXB1 Interrupt
- 101 = TXB2 Interrupt
- 110 = RXB0 Interrupt
- 111 = RXB1 Interrupt

bit 0 **Unimplemented:** Read as '0'

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

NOTES:

 © 2005 Microchip Technology Inc.

11.0 REGISTER MAP

The register map for the MCP2515 is shown in Table 11-1. Address locations for each register are determined by using the column (higher-order 4 bits) and row (lower-order 4 bits) values. The registers have been arranged to optimize the sequential reading and writing of data. Some specific control and status registers allow individual bit modification using the SPI Bit Modify command. The registers that allow this command are shown as shaded locations in Table 11-1. A summary of the MCP2515 control registers is shown in Table 11-2.

TABLE 11-1: CAN CONTROLLER REGISTER MAP

Lower Address Bits	Higher-Order Address Bits							
	0000 xxxx	0001 xxxx	0010 xxxx	0011 xxxx	0100 xxxx	0101 xxxx	0110 xxxx	0111 xxxx
0000	RXF0SIDH	RXF3SIDH	RXM0SIDH	TXB0CTRL	TXB1CTRL	TXB2CTRL	RXB0CTRL	RXB1CTRL
0001	RXF0SIDL	RXF3SIDL	RXM0SIDL	TXB0SIDH	TXB1SIDH	TXB2SIDH	RXB0SIDH	RXB1SIDH
0010	RXF0EID8	RXF3EID8	RXM0EID8	TXB0SIDL	TXB1SIDL	TXB2SIDL	RXB0SIDL	RXB1SIDL
0011	RXF0EID0	RXF3EID0	RXM0EID0	TXB0EID8	TXB1EID8	TXB2EID8	RXB0EID8	RXB1EID8
0100	RXF1SIDH	RXF4SIDH	RXM1SIDH	TXB0EID0	TXB1EID0	TXB2EID0	RXB0EID0	RXB1EID0
0101	RXF1SIDL	RXF4SIDL	RXM1SIDL	TXB0DLC	TXB1DLC	TXB2DLC	RXB0DLC	RXB1DLC
0110	RXF1EID8	RXF4EID8	RXM1EID8	TXB0D0	TXB1D0	TXB2D0	RXB0D0	RXB1D0
0111	RXF1EID0	RXF4EID0	RXM1EID0	TXB0D1	TXB1D1	TXB2D1	RXB0D1	RXB1D1
1000	RXF2SIDH	RXF5SIDH	CNF3	TXB0D2	TXB1D2	TXB2D2	RXB0D2	RXB1D2
1001	RXF2SIDL	RXF5SIDL	CNF2	TXB0D3	TXB1D3	TXB2D3	RXB0D3	RXB1D3
1010	RXF2EID8	RXF5EID8	CNF1	TXB0D4	TXB1D4	TXB2D4	RXB0D4	RXB1D4
1011	RXF2EID0	RXF5EID0	CANINTE	TXB0D5	TXB1D5	TXB2D5	RXB0D5	RXB1D5
1100	BFPCTRL	TEC	CANINTF	TXB0D6	TXB1D6	TXB2D6	RXB0D6	RXB1D6
1101	TXRTSCTRL	REC	EFLG	TXB0D7	TXB1D7	TXB2D7	RXB0D7	RXB1D7
1110	CANSTAT	CANSTAT	CANSTAT	CANSTAT	CANSTAT	CANSTAT	CANSTAT	CANSTAT
1111	CANCTRL	CANCTRL	CANCTRL	CANCTRL	CANCTRL	CANCTRL	CANCTRL	CANCTRL

Note: Shaded register locations indicate that these allow the user to manipulate individual bits using the Bit Modify command.

TABLE 11-2: CONTROL REGISTER SUMMARY

Register Name	Address (Hex)	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	POR/RST Value
BFPCTRL	0C	—	—	B1BFS	B0BFS	B1BFE	B0BFE	B1BFM	B0BFM	--00 0000
TXRTSCTRL	0D	—	—	B2RTS	B1RTS	B0RTS	B2RTSM	B1RTSM	B0RTSM	--xx x000
CANSTAT	xE	OPMOD2	OPMOD1	OPMOD0	—	ICOD2	ICOD1	ICOD0	—	100- 000-
CANCTRL	xF	REQOP2	REQOP1	REQOP0	ABAT	OSM	CLKEN	CLKPRE1	CLKPRE0	1110 0111
TEC	1C	Transmit Error Counter (TEC)								0000 0000
REC	1D	Receive Error Counter (REC)								0000 0000
CNF3	28	SOF	WAKFIL	—	—	—	PHSEG22	PHSEG21	PHSEG20	00-- -000
CNF2	29	BTLMODE	SAM	PHSEG12	PHSEG11	PHSEG10	PRSEG2	PRSEG1	PRSEG0	0000 0000
CNF1	2A	SJW1	SJW0	BRP5	BRP4	BRP3	BRP2	BRP1	BRP0	0000 0000
CANINTE	2B	MERRE	WAKIE	ERRIE	TX2IE	TX1IE	TX0IE	RX1IE	RX0IE	0000 0000
CANINTF	2C	MERRF	WAKIF	ERRIF	TX2IF	TX1IF	TX0IF	RX1IF	RX0IF	0000 0000
EFLG	2D	RX1OVR	RX0OVR	TXBO	TXEP	RXEP	TXWAR	RXWAR	EWARN	0000 0000
TXB0CTRL	30	—	ABTF	MLOA	TXERR	TXREQ	—	TXP1	TXP0	-000 0-00
TXB1CTRL	40	—	ABTF	MLOA	TXERR	TXREQ	—	TXP1	TXP0	-000 0-00
TXB2CTRL	50	—	ABTF	MLOA	TXERR	TXREQ	—	TXP1	TXP0	-000 0-00
RXB0CTRL	60	—	RXM1	RXM0	—	RXRTR	BUKT	BUKT	FILHIT0	-00- 0000
RXB1CTRL	70	—	RSM1	RXM0	—	RXRTR	FILHIT2	FILHIT1	FILHIT0	-00- 0000

NOTES:

 © 2005 Microchip Technology Inc.

12.0 SPI™ INTERFACE

12.1 Overview

The MCP2515 is designed to interface directly with the Serial Peripheral Interface (SPI) port available on many microcontrollers and supports Mode 0,0 and Mode 1,1. Commands and data are sent to the device via the SI pin, with data being clocked in on the rising edge of SCK. Data is driven out by the MCP2515 (on the SO line) on the falling edge of SCK. The $\overline{CS}$ pin must be held low while any operation is performed. Table 12-1 shows the instruction bytes for all operations. Refer to Figure 12-10 and Figure 12-11 for detailed input and output timing diagrams for both Mode 0,0 and Mode 1,1 operation.

Note:	The MCP2515 expects the first byte after lowering $\overline{CS}$ to be the instruction/command byte. This implies that $\overline{CS}$ must be raised and then lowered again to invoke another command.

12.2 Reset Instruction

The Reset instruction can be used to re-initialize the internal registers of the MCP2515 and set Configuration mode. This command provides the same functionality, via the SPI interface, as the $\overline{RESET}$ pin.

The Reset instruction is a single-byte instruction that requires selecting the device by pulling $\overline{CS}$ low, sending the instruction byte and then raising $\overline{CS}$. It is highly recommended that the reset command be sent (or the $\overline{RESET}$ pin be lowered) as part of the power-on initialization sequence.

12.3 Read Instruction

The Read instruction is started by lowering the $\overline{CS}$ pin. The Read instruction is then sent to the MCP2515 followed by the 8-bit address (A7 through A0). Next, the data stored in the register at the selected address will be shifted out on the SO pin.

The internal address pointer is automatically incremented to the next address once each byte of data is shifted out. Therefore, it is possible to read the next consecutive register address by continuing to provide clock pulses. Any number of consecutive register locations can be read sequentially using this method. The read operation is terminated by raising the $\overline{CS}$ pin (Figure 12-2).

12.4 Read RX Buffer Instruction

The Read RX Buffer instruction (Figure 12-3) provides a means to quickly address a receive buffer for reading. This instruction reduces the SPI overhead by one byte, the address byte. The command byte actually has four possible values that determine the address pointer location. Once the command byte is sent, the controller clocks out the data at the address location the same as the Read instruction (i.e., sequential reads are possible). This instruction further reduces the SPI overhead by automatically clearing the associated receive flag (CANINTF.RXnIF) when $\overline{CS}$ is raised at the end of the command.

12.5 Write Instruction

The Write instruction is started by lowering the $\overline{CS}$ pin. The Write instruction is then sent to the MCP2515 followed by the address and at least one byte of data.

It is possible to write to sequential registers by continuing to clock in data bytes, as long as $\overline{CS}$ is held low. Data will actually be written to the register on the rising edge of the SCK line for the D0 bit. If the $\overline{CS}$ line is brought high before eight bits are loaded, the write will be aborted for that data byte and previous bytes in the command will have been written. Refer to the timing diagram in Figure 12-4 for a more detailed illustration of the byte write sequence.

12.6 Load TX Buffer Instruction

The Load TX Buffer instruction (Figure 12-5) eliminates the eight-bit address required by a normal write command. The eight-bit instruction sets the address pointer to one of six addresses to quickly write to a transmit buffer that points to the “ID” or “data” address of any of the three transmit buffers.

12.7 Request-To-Send (RTS) Instruction

The RTS command can be used to initiate message transmission for one or more of the transmit buffers.

The MCP2515 is selected by lowering the $\overline{CS}$ pin. The RTS command byte is then sent. Shown in Figure 12-6, the last 3 bits of this command indicate which transmit buffer(s) are enabled to send.

This command will set the TxBnCTRL.TXREQ bit for the respective buffer(s). Any or all of the last three bits can be set in a single command. If the RTS command is sent with nnn = `000`, the command will be ignored.

12.8 Read Status Instruction

The Read Status instruction allows single instruction access to some of the often used status bits for message reception and transmission.

The MCP2515 is selected by lowering the $\overline{CS}$ pin and the read status command byte, shown in Figure 12-8, is sent to the MCP2515. Once the command byte is sent, the MCP2515 will return eight bits of data that contain the status.

If additional clocks are sent after the first eight bits are transmitted, the MCP2515 will continue to output the status bits as long as the $\overline{CS}$ pin is held low and clocks are provided on SCK.

Each status bit returned in this command may also be read by using the standard read command with the appropriate register address.

12.9 RX Status Instruction

The RX Status instruction (Figure 12-9) is used to quickly determine which filter matched the message and message type (standard, extended, remote). After the command byte is sent, the controller will return 8 bits of data that contain the status data. If more clocks are sent after the 8 bits are transmitted, the controller will continue to output the same status bits as long as the $\overline{CS}$ pin stays low and clocks are provided.

12.10 Bit Modify Instruction

The Bit Modify instruction provides a means for setting or clearing individual bits in specific status and control registers. This command is not available for all registers. See **Section 11.0 "Register Map"** to determine which registers allow the use of this command.

Note:	Executing the Bit Modify command on registers that are not bit-modifiable will force the mask to FFh. This will allow byte-writes to the registers, not bit modify.

The part is selected by lowering the $\overline{CS}$ pin and the Bit Modify command byte is then sent to the MCP2515. The command is followed by the address of the register, the mask byte and finally the data byte.

The mask byte determines which bits in the register will be allowed to change. A '1' in the mask byte will allow a bit in the register to change, while a '0' will not.

The data byte determines what value the modified bits in the register will be changed to. A '1' in the data byte will set the bit and a '0' will clear the bit, provided that the mask for that bit is set to a '1' (see Figure 12-7).

FIGURE 12-1: BIT MODIFY

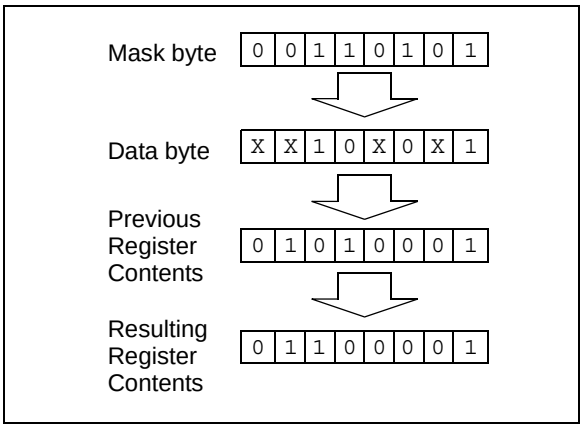


TABLE 12-1: SPI™ INSTRUCTION SET

Instruction Name	Instruction Format	Description
RESET	1100 0000	Resets internal registers to default state, set Configuration mode.
READ	0000 0011	Read data from register beginning at selected address.
Read RX Buffer	1001 0nm0	When reading a receive buffer, reduces the overhead of a normal read command by placing the address pointer at one of four locations, as indicated by 'n,m'. **Note:** The associated RX flag bit (CANINTF.RXnIF) will be cleared after bringing $\overline{CS}$ high.
WRITE	0000 0010	Write data to register beginning at selected address.
Load TX Buffer	0100 0abc	When loading a transmit buffer, reduces the overhead of a normal Write command by placing the address pointer at one of six locations as indicated by 'a,b,c'.
RTS (Message Request-To-Send)	1000 0nnn	Instructs controller to begin message transmission sequence for any of the transmit buffers. 1000 0nnn Request-to-send for TXB2 — Request-to-send for TXBO Request-to-send for TXB1
Read Status	1010 0000	Quick polling command that reads several status bits for transmit and receive functions.
RX Status	1011 0000	Quick polling command that indicates filter match and message type (standard, extended and/or remote) of received message.
Bit Modify	0000 0101	Allows the user to set or clear individual bits in a particular register. **Note:** Not all registers can be bit-modified with this command. Executing this command on registers that are not bit-modifiable will force the mask to FFh. See the register map in **Section 11.0 "Register Map"** for a list of the registers that apply.

 © 2005 Microchip Technology Inc.

FIGURE 12-2: READ INSTRUCTION

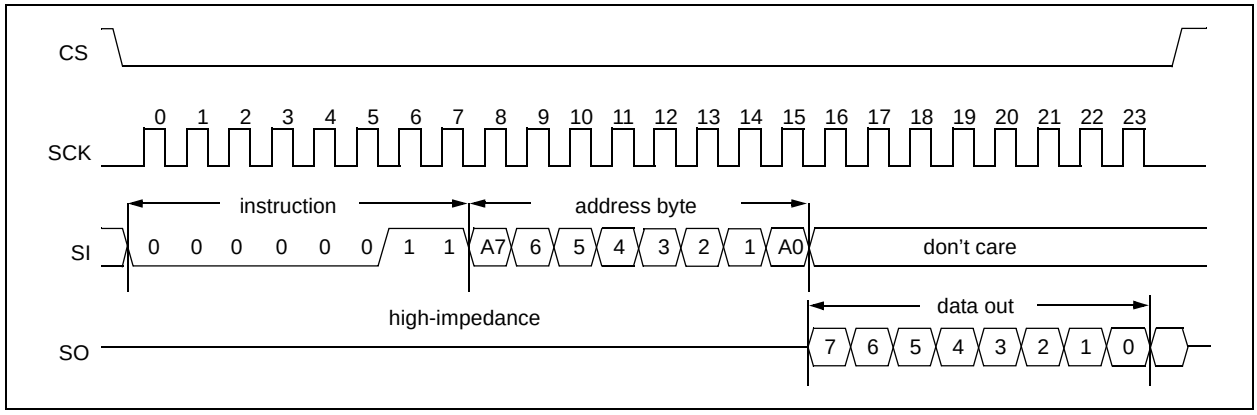
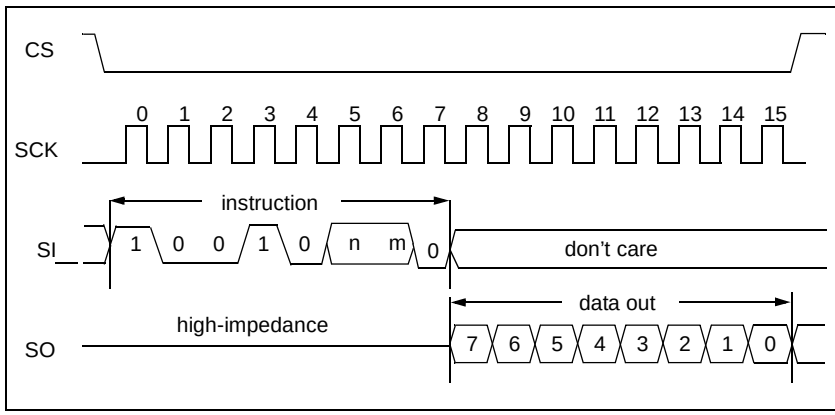


FIGURE 12-3: READ RX BUFFER INSTRUCTION



n	m	Address Points to	Address
0	0	Receive Buffer 0, Start at RXB0SIDH	0x61
0	1	Receive Buffer 0, Start at RXB0D0	0x66
1	0	Receive Buffer 1, Start at RXB1SIDH	0x71
1	1	Receive Buffer 1, Start at RXB1D0	0x76

FIGURE 12-4: BYTE WRITE INSTRUCTION

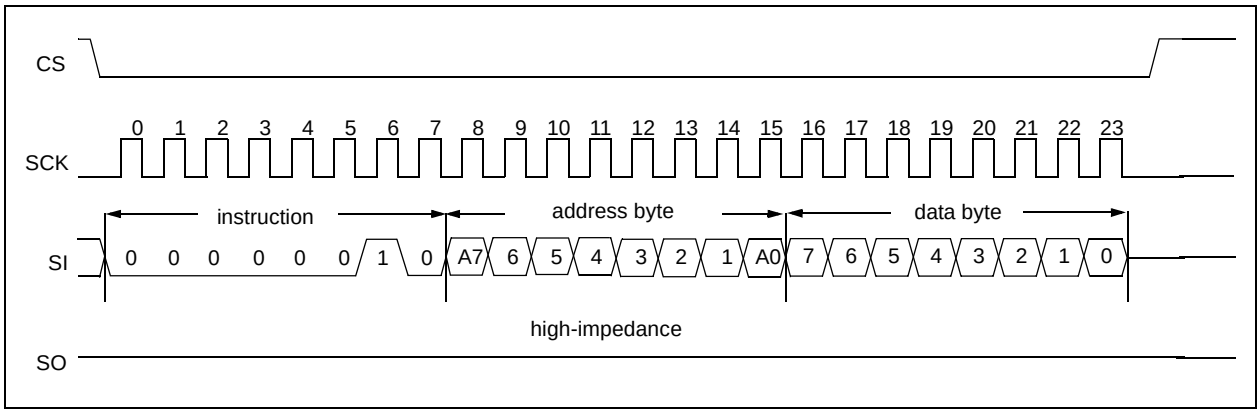
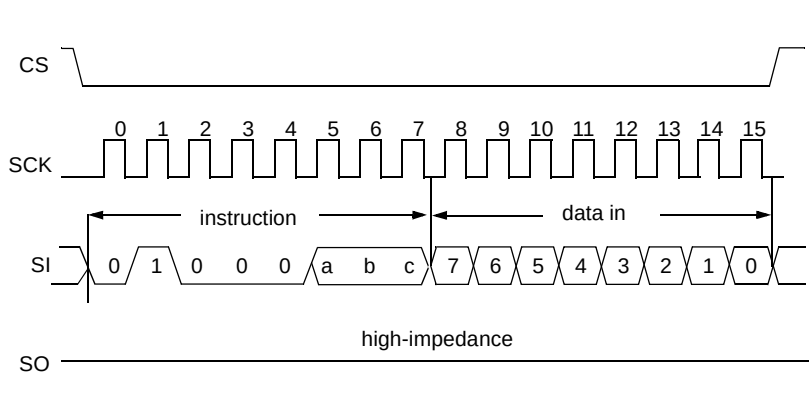


FIGURE 12-5: LOAD TX BUFFER



a	b	c	Address Points to	Addr
0	0	0	TX buffer 0, Start at TXB0SIDH	0x31
0	0	1	TX buffer 0, Start at TXB0D0	0x36
0	1	0	TX buffer 1, Start at TXB1SIDH	0x41
0	1	1	TX buffer 1, Start at TXB1D0	0x46
1	0	0	TX buffer 2, Start at TXB2SIDH	0x51
1	0	1	TX buffer 2, Start at TXB2D0	0x56

FIGURE 12-6: REQUEST-TO-SEND (RTS) INSTRUCTION

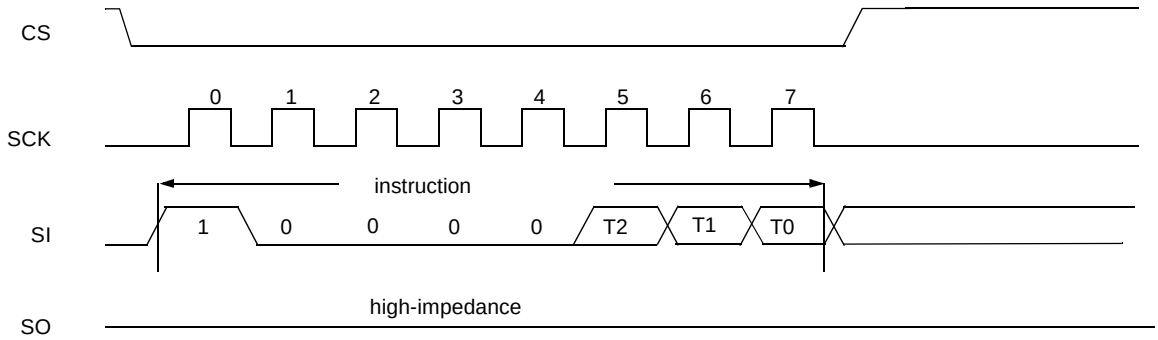
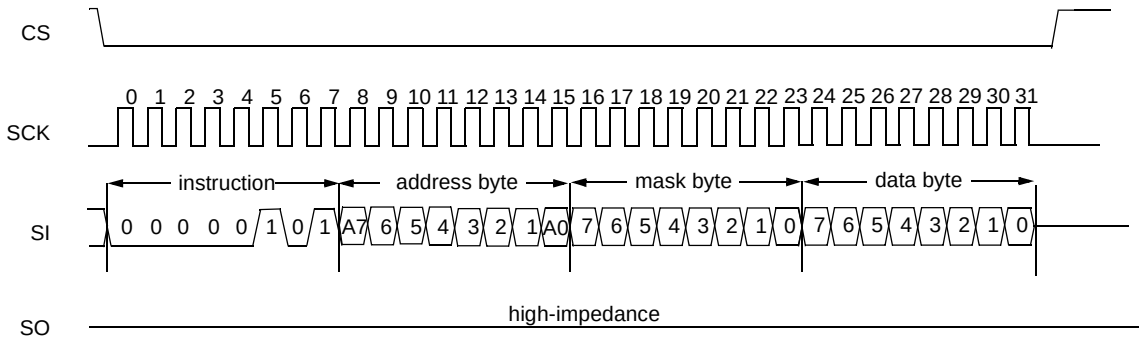


FIGURE 12-7: BIT MODIFY INSTRUCTION



Note: Not all registers can be accessed with this command. See the register map for a list of the registers that apply.

 © 2005 Microchip Technology Inc.

FIGURE 12-8: READ STATUS INSTRUCTION

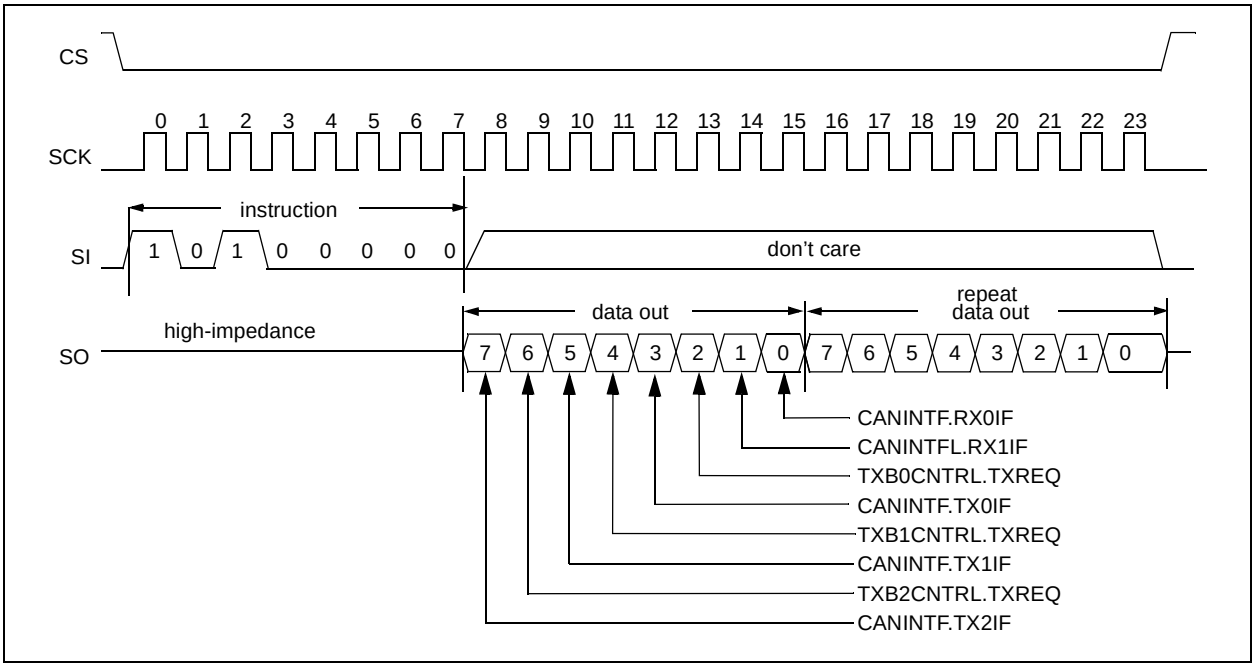
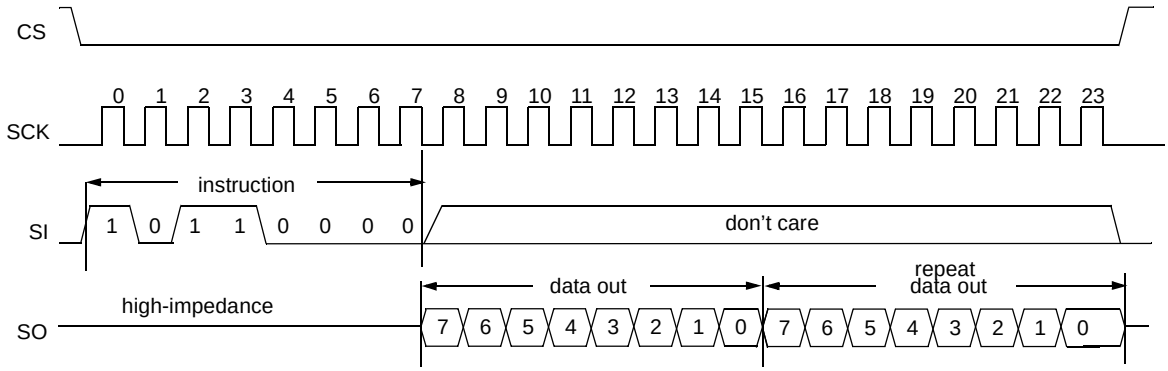


FIGURE 12-9: RX STATUS INSTRUCTION



7	6	Received Message
0	0	No RX message
0	1	Message in RXB0
1	0	Message in RXB1
1	1	Messages in both buffers*

CANINTF.RXnIF bits are mapped to bits 7 and 6.

4	3	Msg Type Received
0	0	Standard data frame
0	1	Standard remote frame
1	0	Extended data frame
1	1	Extended remote frame

The extended ID bit is mapped to bit 4. The RTR bit is mapped to bit 3.

2	1	0	Filter Match
0	0	0	RXF0
0	0	1	RXF1
0	1	0	RXF2
0	1	1	RXF3
1	0	0	RXF4
1	0	1	RXF5
1	1	0	RXF0 (rollover to RXB1)
1	1	1	RXF1 (rollover to RXB1)

* Buffer 0 has higher priority, therefore, RXB0 status is reflected in bits 4:0.

FIGURE 12-10: SPI™ INPUT TIMING

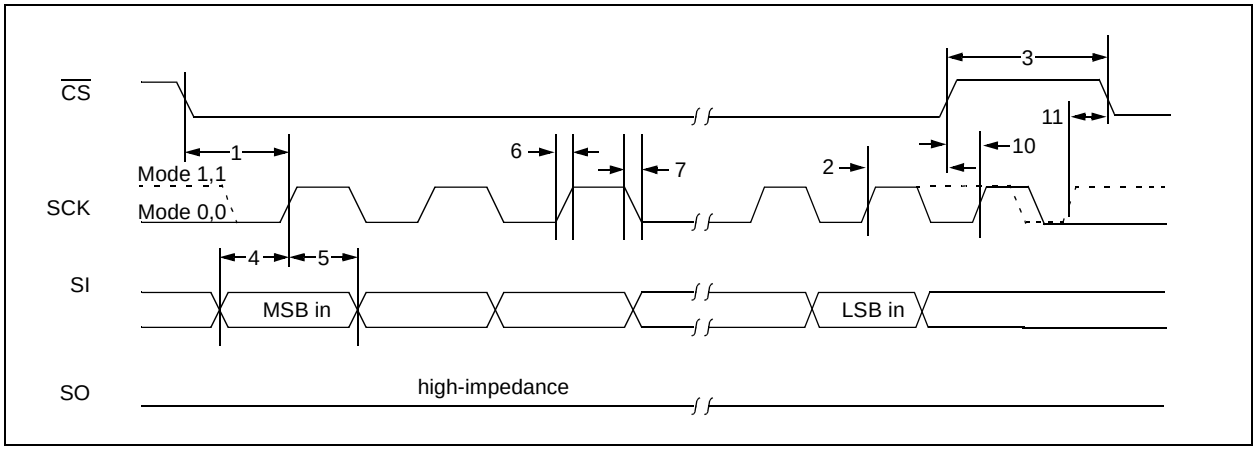
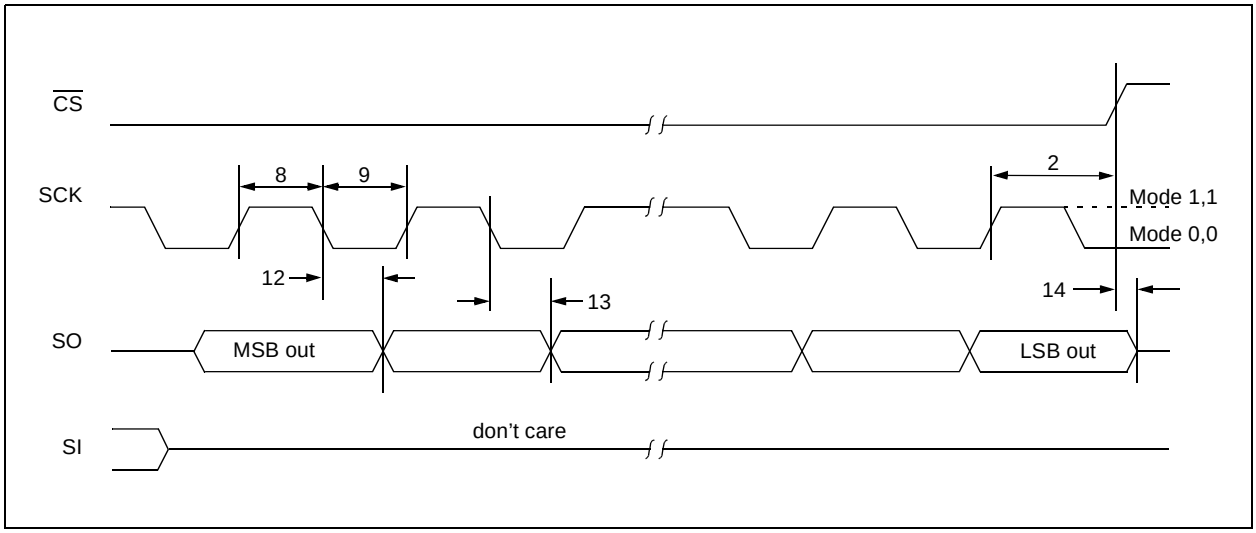


FIGURE 12-11: SPI™ OUTPUT TIMING



 © 2005 Microchip Technology Inc.

13.0 ELECTRICAL CHARACTERISTICS

13.1 Absolute Maximum Ratings †

V_{DD}.....7.0V
All inputs and outputs w.r.t. V_{SS} .....-0.6V to V_{DD} +1.0V
Storage temperature .....-65°C to +150°C
Ambient temp. with power applied .....-65°C to +125°C
Soldering temperature of leads (10 seconds) ..... +300°C

† **Notice:** Stresses above those listed under “Maximum Ratings” may cause permanent damage to the device. This is a stress rating only and functional operation of the device at those or any other conditions above those indicated in the operational listings of this specification is not implied. Exposure to maximum rating conditions for extended periods may affect device reliability.

TABLE 13-1: DC CHARACTERISTICS

DC Characteristics			Industrial (I): T_{AMB} = -40°C to +85°C V_{DD} = 2.7V to 5.5V Extended (E): T_{AMB} = -40°C to +125°C V_{DD} = 4.5V to 5.5V			
Param. No.	**Sym**	**Characteristic**	**Min**	**Max**	**Units**	**Conditions**
	V_{DD}	Supply Voltage	2.7	5.5	V	
	V_{RET}	Register Retention Voltage	2.4	—	V	
		High-Level Input Voltage				
	V_{IH}	RXCAN	2	V_{DD} + 1	V	
		SCK, $\overline{CS}$, SI, $\overline{TXnRTS}$ Pins	0.7 V_{DD}	V_{DD} + 1	V	
		OSC1	0.85 V_{DD}	V_{DD}	V	
		RESET	0.85 V_{DD}	V_{DD}	V	
		Low-Level Input Voltage				
	V_{IL}	RXCAN, $\overline{TXnRTS}$ Pins	-0.3	.15 V_{DD}	V	
		SCK, $\overline{CS}$, SI	-0.3	0.4	V	
		OSC1	V_{SS}	.3 V_{DD}	V	
		RESET	V_{SS}	.15 V_{DD}	V	
		Low-Level Output Voltage				
	V_{OL}	TXCAN	—	0.6	V	I_{OL} = +6.0 mA, V_{DD} = 4.5V
		$\overline{RXnBF}$ Pins	—	0.6	V	I_{OL} = +8.5 mA, V_{DD} = 4.5V
		SO, CLKOUT	—	0.6	V	I_{OL} = +2.1 mA, V_{DD} = 4.5V
		$\overline{INT}$	—	0.6	V	I_{OL} = +1.6 mA, V_{DD} = 4.5V
		High-Level Output Voltage			V	
	V_{OH}	TXCAN, $\overline{RXnBF}$ Pins	V_{DD} – 0.7	—	V	I_{OH} = -3.0 mA, V_{DD} = 4.5V
		SO, CLKOUT	V_{DD} – 0.5	—	V	I_{OH} = -400 µA, V_{DD} = 4.5V
		$\overline{INT}$	V_{DD} – 0.7	—	V	I_{OH} = -1.0 mA, V_{DD} = 4.5V
		Input Leakage Current				
	I_{LI}	All I/O except OSC1 and TXnRTS pins	-1	+1	µA	$\overline{CS}$ = $\overline{RESET}$ = V_{DD}, V_{IN} = V_{SS} to V_{DD}
		OSC1 Pin	-5	+5	µA	
	C_{INT}	Internal Capacitance (All Inputs and Outputs)	—	7	pF	T_{AMB} = 25°C, f_C = 1.0 MHz, V_{DD} = 0V (**Note 1**)
	I_{DD}	Operating Current	—	10	mA	V_{DD} = 5.5V, F_{OSC} = 25 MHz, F_{CLK} = 1 MHz, SO = Open
	I_{DDS}	Standby Current (Sleep mode)	—	5	µA	$\overline{CS}$, $\overline{TXnRTS}$ = V_{DD}, Inputs tied to V_{DD} or V_{SS}, -40°C TO +85°C
			—	8	µA	$\overline{CS}$, $\overline{TXnRTS}$ = V_{DD}, Inputs tied to V_{DD} or V_{SS}, -40°C TO +125°C

Note 1: This parameter is periodically sampled and not 100% tested.

 © 2005 Microchip Technology Inc.

TABLE 13-2: OSCILLATOR TIMING CHARACTERISTICS

Oscillator Timing Characteristics(Note)			Industrial (I): TAMB = -40°C to +85°C VDD = 2.7V to 5.5V Extended (E): TAMB = -40°C to +125°C VDD = 4.5V to 5.5V			
Param. No.	**Sym**	**Characteristic**	**Min**	**Max**	**Units**	**Conditions**
	FOSC	Clock-In Frequency	1 1	40 25	MHz MHz	4.5V to 5.5V 2.7V to 5.5V
	TOSC	Clock-In Period	25 40	1000 1000	ns ns	4.5V to 5.5V 2.7V to 5.5V
	TDUTY	Duty Cycle (External Clock Input)	0.45	0.55	—	TOSH/(TOSH + TOSL)

Note: This parameter is periodically sampled and not 100% tested.

TABLE 13-3: CAN INTERFACE AC CHARACTERISTICS

CAN Interface AC Characteristics			Industrial (I): TAMB = -40°C to +85°C VDD = 2.7V to 5.5V Extended (E): TAMB = -40°C to +125°C VDD = 4.5V to 5.5V			
Param. No.	**Sym**	**Characteristic**	**Min**	**Max**	**Units**	**Conditions**
	TWF	Wake-up Noise Filter	100	—	ns	

TABLE 13-4: RESET AC CHARACTERISTICS

RESET AC Characteristics			Industrial (I): TAMB = -40°C to +85°C VDD = 2.7V to 5.5V Extended (E): TAMB = -40°C to +125°C VDD = 4.5V to 5.5V			
Param. No.	**Sym**	**Characteristic**	**Min**	**Max**	**Units**	**Conditions**
	trl	RESET Pin Low Time	2	—	µs	

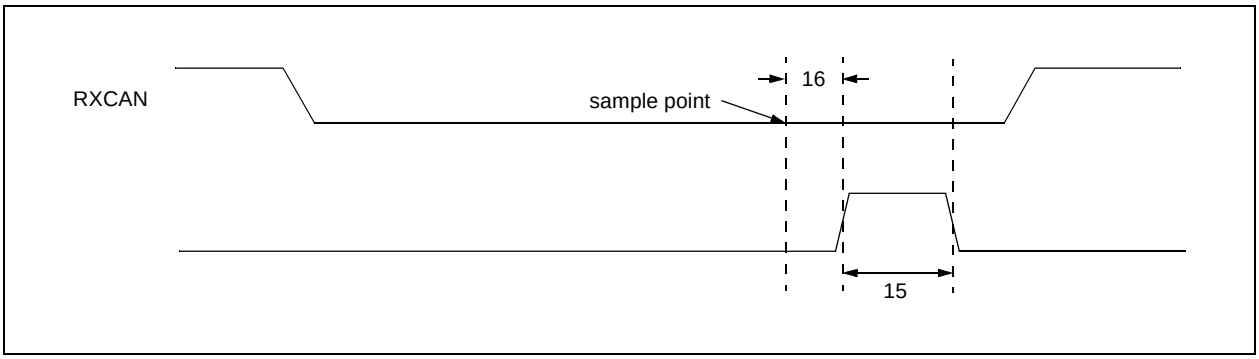
TABLE 13-5: CLKOUT PIN AC CHARACTERISTICS

CLKOUT Pin AC/DC Characteristics			Industrial (I): TAMB = -40°C to +85°C VDD = 2.7V to 5.5V Extended (E): TAMB = -40°C to +125°C VDD = 4.5V to 5.5V			
Param. No.	**Sym**	**Characteristic**	**Min**	**Max**	**Units**	**Conditions**
	$t_{hCLKOUT}$	CLKOUT Pin High Time	15	—	ns	TOSC = 40 ns **(Note 1)**
	$t_{lCLKOUT}$	CLKOUT Pin Low Time	15	—	ns	TOSC = 40 ns **(Note 1)**
	$t_{rCLKOUT}$	CLKOUT Pin Rise Time	—	5	ns	Measured from 0.3 VDD to 0.7 VDD **(Note 1)**
	$t_{fCLKOUT}$	CLKOUT Pin Fall Time	—	5	ns	Measured from 0.7 VDD to 0.3 VDD **(Note 1)**
	$t_{dCLKOUT}$	CLOCKOUT Propagation Delay	—	100	ns	**Note 1**
15	t_{hSOF}	Start-Of-Frame High Time	—	2 TOSC	ns	**Note 1**
16	t_{dSOF}	Start-Of-Frame Propagation Delay	—	2 TOSC + 0.5 TQ	ns	Measured from CAN bit sample point. Device is a receiver. CNF1.BRP<5:0> = 0 **(Note 2)**

Note 1: All CLKOUT mode functionality and output frequency is tested at device frequency limits, however, CLKOUT prescaler is set to divide by one. This parameter is periodically sampled and not 100% tested.

2: Design guidance only, not tested.

FIGURE 13-1: START-OF-FRAME PIN AC CHARACTERISTICS



 © 2005 Microchip Technology Inc.

TABLE 13-6: SPI™ INTERFACE AC CHARACTERISTICS

SPI™ Interface AC Characteristics			Industrial (I): TAMB = -40°C to +85°C VDD = 2.7V to 5.5V Extended (E): TAMB = -40°C to +125°C VDD = 4.5V to 5.5V			
Param. No.	**Sym**	**Characteristic**	**Min**	**Max**	**Units**	**Conditions**
	FCLK	Clock Frequency	—	10	MHz	
1	TCSS	$\overline{CS}$ Setup Time	50	—	ns	
2	TCSH	$\overline{CS}$ Hold Time	50	—	ns	
3	TCSD	$\overline{CS}$ Disable Time	50	—	ns	
4	TSU	Data Setup Time	10	—	ns	
5	THD	Data Hold Time	10	—	ns	
6	TR	CLK Rise Time	—	2	µs	**Note 1**
7	TF	CLK Fall Time	—	2	µs	**Note 1**
8	THI	Clock High Time	45	—	ns	
9	TLO	Clock Low Time	45	—	ns ns	
10	TCLD	Clock Delay Time	50	—	ns	
11	TCLE	Clock Enable Time	50	—	ns	
12	TV	Output Valid from Clock Low	—	45	ns	
13	THO	Output Hold Time	0	—	ns	
14	TDIS	Output Disable Time	—	100	ns	

Note 1: This parameter is not 100% tested.

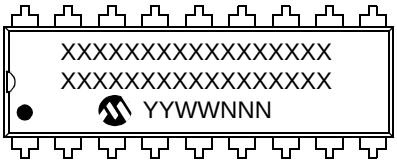
NOTES:

 © 2005 Microchip Technology Inc.

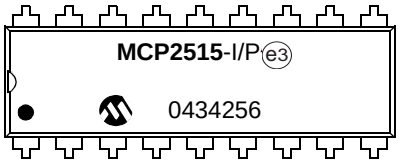
14.0 PACKAGING INFORMATION

14.1 Package Marking Information

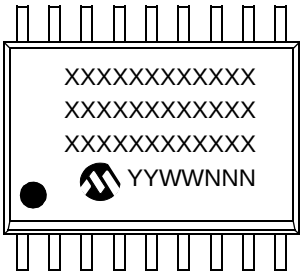
18-Lead PDIP (300 mil)



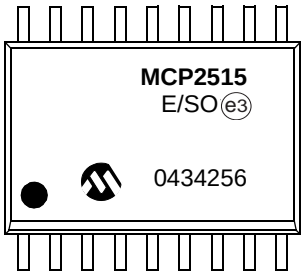
Example:



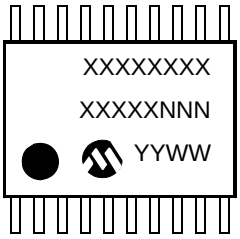
18-Lead SOIC (300 mil)



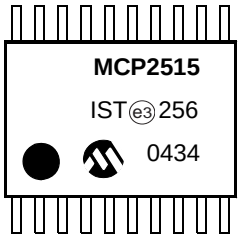
Example:



20-Lead TSSOP (4.4 mm)



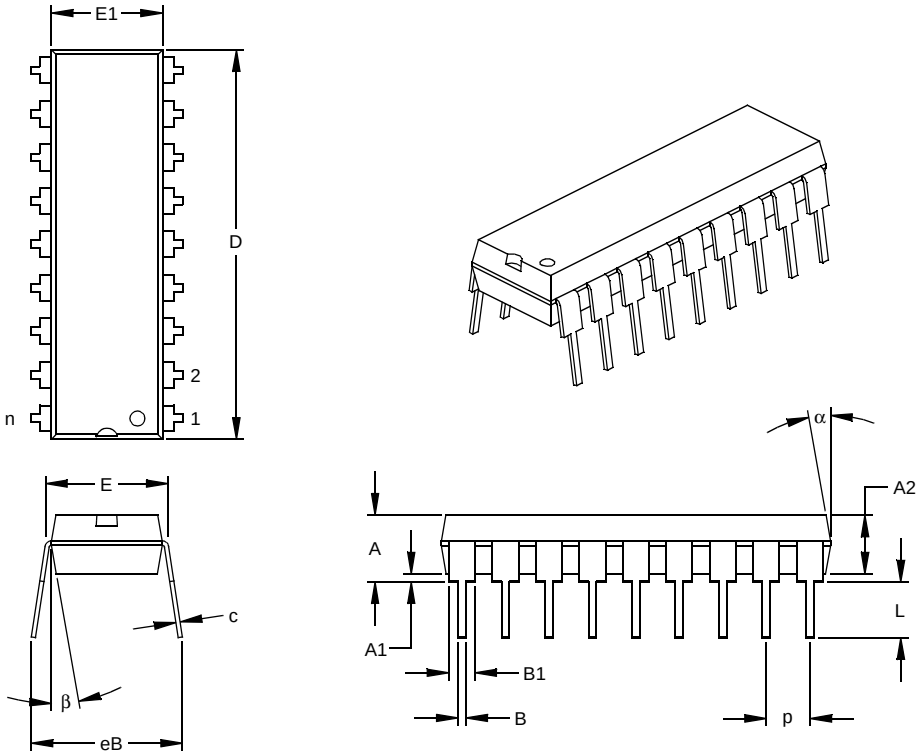
Example:



Legend:	XX...X	Customer-specific information
	Y	Year code (last digit of calendar year)
	YY	Year code (last 2 digits of calendar year)
	WW	Week code (week of January 1 is week ‘01’)
	NNN	Alphanumeric traceability code
	(e3)	Pb-free JEDEC designator for Matte Tin (Sn)
	*	This package is Pb-free. The Pb-free JEDEC designator ((e3)) can be found on the outer packaging for this package.

Note: In the event the full Microchip part number cannot be marked on one line, it will be carried over to the next line, thus limiting the number of available characters for customer-specific information.

18-Lead Plastic Dual In-line (P) – 300 mil (PDIP)



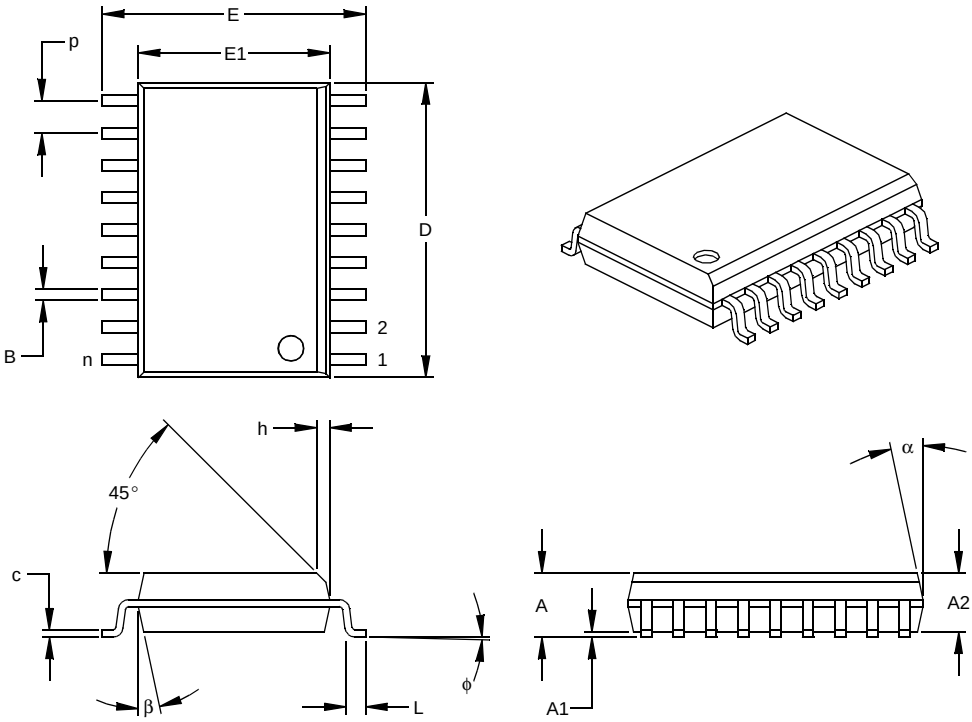
Units		INCHES*			MILLIMETERS		
Dimension Limits		MIN	NOM	MAX	MIN	NOM	MAX
Number of Pins	n		18			18	
Pitch	p		.100			2.54	
Top to Seating Plane	A	.140	.155	.170	3.56	3.94	4.32
Molded Package Thickness	A2	.115	.130	.145	2.92	3.30	3.68
Base to Seating Plane	A1	.015			0.38		
Shoulder to Shoulder Width	E	.300	.313	.325	7.62	7.94	8.26
Molded Package Width	E1	.240	.250	.260	6.10	6.35	6.60
Overall Length	D	.890	.898	.905	22.61	22.80	22.99
Tip to Seating Plane	L	.125	.130	.135	3.18	3.30	3.43
Lead Thickness	c	.008	.012	.015	0.20	0.29	0.38
Upper Lead Width	B1	.045	.058	.070	1.14	1.46	1.78
Lower Lead Width	B	.014	.018	.022	0.36	0.46	0.56
Overall Row Spacing §	eB	.310	.370	.430	7.87	9.40	10.92
Mold Draft Angle Top	α	5	10	15	5	10	15
Mold Draft Angle Bottom	β	5	10	15	5	10	15

* Controlling Parameter
§ Significant Characteristic

Notes:
Dimensions D and E1 do not include mold flash or protrusions. Mold flash or protrusions shall not exceed .010” (0.254mm) per side.
JEDEC Equivalent: MS-001
Drawing No. C04-007

 © 2005 Microchip Technology Inc.

18-Lead Plastic Small Outline (SO) – Wide, 300 mil (SOIC)

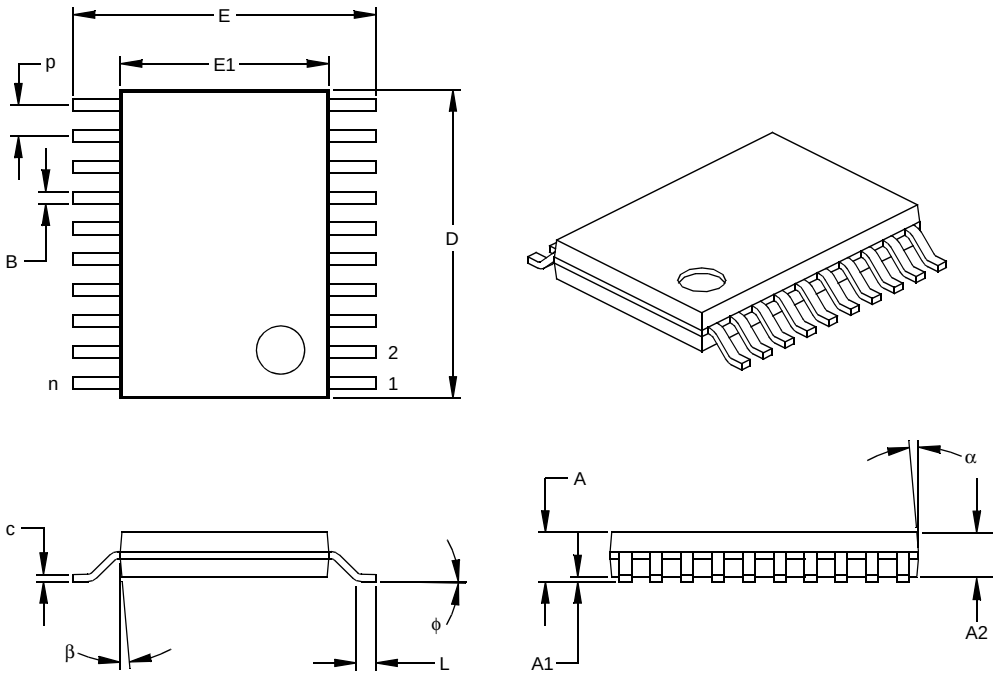


Units		INCHES*			MILLIMETERS		
Dimension Limits		MIN	NOM	MAX	MIN	NOM	MAX
Number of Pins	n		18			18	
Pitch	p		.050			1.27	
Overall Height	A	.093	.099	.104	2.36	2.50	2.64
Molded Package Thickness	A2	.088	.091	.094	2.24	2.31	2.39
Standoff §	A1	.004	.008	.012	0.10	0.20	0.30
Overall Width	E	.394	.407	.420	10.01	10.34	10.67
Molded Package Width	E1	.291	.295	.299	7.39	7.49	7.59
Overall Length	D	.446	.454	.462	11.33	11.53	11.73
Chamfer Distance	h	.010	.020	.029	0.25	0.50	0.74
Foot Length	L	.016	.033	.050	0.41	0.84	1.27
Foot Angle	ϕ	0	4	8	0	4	8
Lead Thickness	c	.009	.011	.012	0.23	0.27	0.30
Lead Width	B	.014	.017	.020	0.36	0.42	0.51
Mold Draft Angle Top	α	0	12	15	0	12	15
Mold Draft Angle Bottom	β	0	12	15	0	12	15

* Controlling Parameter
§ Significant Characteristic

Notes:
Dimensions D and E1 do not include mold flash or protrusions. Mold flash or protrusions shall not exceed .010” (0.254mm) per side.
JEDEC Equivalent: MS-013
Drawing No. C04-051

20-Lead Plastic Thin Shrink Small Outline (ST) – 4.4 mm (TSSOP)



Units		INCHES			MILLIMETERS*		
Dimension Limits		MIN	NOM	MAX	MIN	NOM	MAX
Number of Pins	n		20			20	
Pitch	p		.026			0.65	
Overall Height	A			.043			1.10
Molded Package Thickness	A2	.033	.035	.037	0.85	0.90	0.95
Standoff §	A1	.002	.004	.006	0.05	0.10	0.15
Overall Width	E	.246	.251	.256	6.25	6.38	6.50
Molded Package Width	E1	.169	.173	.177	4.30	4.40	4.50
Molded Package Length	D	.252	.256	.260	6.40	6.50	6.60
Foot Length	L	.020	.024	.028	0.50	0.60	0.70
Foot Angle	φ	0	4	8	0	4	8
Lead Thickness	c	.004	.006	.008	0.09	0.15	0.20
Lead Width	B	.007	.010	.012	0.19	0.25	0.30
Mold Draft Angle Top	α	0	5	10	0	5	10
Mold Draft Angle Bottom	β	0	5	10	0	5	10

* Controlling Parameter
§ Significant Characteristic

Notes:
Dimensions D and E1 do not include mold flash or protrusions. Mold flash or protrusions shall not exceed .005” (0.127mm) per side.
JEDEC Equivalent: MO-153
Drawing No. C04-088

 © 2005 Microchip Technology Inc.

APPENDIX A: REVISION HISTORY

Revision D (April 2005)

The following is the list of modifications:

1. Section 8.0. Added Table 8-1 and Table 8-2. Added note box following tables.
2. Section 11.0, Table 11-1. Changed address bits in column heading.
3. Modified Section 14.0 Packaging Information to reflect pb free device markings.
4. Appendix A Revision History: Rearranged order of importance.

Revision C (November 2004)

The following is the list of modifications:

1. New section 9.0 added.
2. Section 12, Heading 12.1: added notebox. Heading 12.6: Changed verbiage within paragraph.
3. Added Appendix A: Revision History.

Revision B (September 2003)

The following is the list of modifications:

1. Front page bullet: Standby current (typical) (Sleep Mode) changed from 10 µA to 1 µA
2. Section 8.2 CLKOUT Pin: Added notebox for maximum frequency on CLKOUT.
3. Section 12.0, Table 12-1:
   - Changed supply voltage minimum to 2.7V.
   - Internal Capacitance: Changed V_{DD} condition to 0V.
   - Standby Current (Sleep mode): Split specification into -40°C to +85°C and -40°C to +125°C.

Revision A (May 2003)

- Original Relase of this Document.

NOTES:

 © 2005 Microchip Technology Inc.

PRODUCT IDENTIFICATION SYSTEM

To order or obtain information, e.g., on pricing or delivery, refer to the factory or the listed sales office.

PART NO.	–	X	/XX
Device		Temperature Range	Package

Device	MCP2515:	CAN Controller w/ SPI™ Interface
	MCP2515T:	CAN Controller w/SPI Interface (Tape and Reel)
Temperature Range	I	= -40°C to +85°C (Industrial)
	E	= -40°C to +125°C (Extended)
Package	P	= Plastic DIP (300 mil Body), 18-Lead
	SO	= Plastic SOIC (300 mil Body), 18-Lead
	ST	= TSSOP, (4.4 mm Body), 20-Lead

Examples:

a) MCP2515-E/P: Extended Temperature, 18LD PDIP package.

b) MCP2515-I/P: Industrial Temperature, 18LD PDIP package.

c) MCP2515-E/SO: Extended Temperature, 18LD SOIC package.

d) MCP2515-I/SO: Industrial Temperature, 18LD SOIC package.

e) MCP2515T-I/SO: Tape and Reel, Industrial Temperature, 18LD SOIC package.

f) MCP2515-I/ST: Industrial Temperature, 20LD TSSOP package.

g) MCP2515T-I/ST: Tape and Reel, Industrial Temperature, 20LD TSSOP package.

NOTES:

 © 2005 Microchip Technology Inc.

Note the following details of the code protection feature on Microchip devices:

- Microchip products meet the specification contained in their particular Microchip Data Sheet.
- Microchip believes that its family of products is one of the most secure families of its kind on the market today, when used in the intended manner and under normal conditions.
- There are dishonest and possibly illegal methods used to breach the code protection feature. All of these methods, to our knowledge, require using the Microchip products in a manner outside the operating specifications contained in Microchip's Data Sheets. Most likely, the person doing so is engaged in theft of intellectual property.
- Microchip is willing to work with the customer who is concerned about the integrity of their code.
- Neither Microchip nor any other semiconductor manufacturer can guarantee the security of their code. Code protection does not mean that we are guaranteeing the product as "unbreakable."

Code protection is constantly evolving. We at Microchip are committed to continuously improving the code protection features of our products. Attempts to break Microchip's code protection feature may be a violation of the Digital Millennium Copyright Act. If such acts allow unauthorized access to your software or other copyrighted work, you may have a right to sue for relief under that Act.

Information contained in this publication regarding device applications and the like is provided only for your convenience and may be superseded by updates. It is your responsibility to ensure that your application meets with your specifications. MICROCHIP MAKES NO REPRESENTATIONS OR WARRANTIES OF ANY KIND WHETHER EXPRESS OR IMPLIED, WRITTEN OR ORAL, STATUTORY OR OTHERWISE, RELATED TO THE INFORMATION, INCLUDING BUT NOT LIMITED TO ITS CONDITION, QUALITY, PERFORMANCE, MERCHANTABILITY OR FITNESS FOR PURPOSE**.** Microchip disclaims all liability arising from this information and its use. Use of Microchip's products as critical components in life support systems is not authorized except with express written approval by Microchip. No licenses are conveyed, implicitly or otherwise, under any Microchip intellectual property rights.

Trademarks

The Microchip name and logo, the Microchip logo, Accuron, dsPIC, KEELOQ, microID, MPLAB, PIC, PICmicro, PICSTART, PRO MATE, PowerSmart, rfPIC, and SmartShunt are registered trademarks of Microchip Technology Incorporated in the U.S.A. and other countries.

AmpLab, FilterLab, Migratable Memory, MXDEV, MXLAB, PICMASTER, SEEVAL, SmartSensor and The Embedded Control Solutions Company are registered trademarks of Microchip Technology Incorporated in the U.S.A.

Analog-for-the-Digital Age, Application Maestro, dsPICDEM, dsPICDEM.net, dsPICworks, ECAN, ECONOMONITOR, FanSense, FlexROM, fuzzyLAB, In-Circuit Serial Programming, ICSP, ICEPIC, MPASM, MPLIB, MPLINK, MPSIM, PICkit, PICDEM, PICDEM.net, PICLAB, PICtail, PowerCal, PowerInfo, PowerMate, PowerTool, rfLAB, rfPICDEM, Select Mode, Smart Serial, SmartTel, Total Endurance and WiperLock are trademarks of Microchip Technology Incorporated in the U.S.A. and other countries.

SQTP is a service mark of Microchip Technology Incorporated in the U.S.A.

All other trademarks mentioned herein are property of their respective companies.

© 2005, Microchip Technology Incorporated, Printed in the U.S.A., All Rights Reserved.

Printed on recycled paper.

QUALITY MANAGEMENT SYSTEM
CERTIFIED BY DNV
ISO/TS 16949:2002

Microchip received ISO/TS-16949:2002 quality system certification for its worldwide headquarters, design and wafer fabrication facilities in Chandler and Tempe, Arizona and Mountain View, California in October 2003. The Company's quality system processes and procedures are for its PICmicro® 8-bit MCUs, KEELOQ® code hopping devices, Serial EEPROMs, microperipherals, nonvolatile memory and analog products. In addition, Microchip's quality system for the design and manufacture of development systems is ISO 9001:2000 certified.

WORLDWIDE SALES AND SERVICE

AMERICAS

Corporate Office
2355 West Chandler Blvd.
Chandler, AZ 85224-6199
Tel: 480-792-7200
Fax: 480-792-7277
Technical Support:
http://support.microchip.com
Web Address:
www.microchip.com

Atlanta
Alpharetta, GA
Tel: 770-640-0034
Fax: 770-640-0307

Boston
Westborough, MA
Tel: 774-760-0087
Fax: 774-760-0088

Chicago
Itasca, IL
Tel: 630-285-0071
Fax: 630-285-0075

Dallas
Addison, TX
Tel: 972-818-7423
Fax: 972-818-2924

Detroit
Farmington Hills, MI
Tel: 248-538-2250
Fax: 248-538-2260

Kokomo
Kokomo, IN
Tel: 765-864-8360
Fax: 765-864-8387

Los Angeles
Mission Viejo, CA
Tel: 949-462-9523
Fax: 949-462-9608

San Jose
Mountain View, CA
Tel: 650-215-1444
Fax: 650-961-0286

Toronto
Mississauga, Ontario,
Canada
Tel: 905-673-0699
Fax: 905-673-6509

ASIA/PACIFIC

Australia - Sydney
Tel: 61-2-9868-6733
Fax: 61-2-9868-6755

China - Beijing
Tel: 86-10-8528-2100
Fax: 86-10-8528-2104

China - Chengdu
Tel: 86-28-8676-6200
Fax: 86-28-8676-6599

China - Fuzhou
Tel: 86-591-8750-3506
Fax: 86-591-8750-3521

China - Hong Kong SAR
Tel: 852-2401-1200
Fax: 852-2401-3431

China - Shanghai
Tel: 86-21-5407-5533
Fax: 86-21-5407-5066

China - Shenyang
Tel: 86-24-2334-2829
Fax: 86-24-2334-2393

China - Shenzhen
Tel: 86-755-8203-2660
Fax: 86-755-8203-1760

China - Shunde
Tel: 86-757-2839-5507
Fax: 86-757-2839-5571

China - Qingdao
Tel: 86-532-502-7355
Fax: 86-532-502-7205

ASIA/PACIFIC

India - Bangalore
Tel: 91-80-2229-0061
Fax: 91-80-2229-0062

India - New Delhi
Tel: 91-11-5160-8631
Fax: 91-11-5160-8632

Japan - Kanagawa
Tel: 81-45-471- 6166
Fax: 81-45-471-6122

Korea - Seoul
Tel: 82-2-554-7200
Fax: 82-2-558-5932 or
82-2-558-5934

Singapore
Tel: 65-6334-8870
Fax: 65-6334-8850

Taiwan - Kaohsiung
Tel: 886-7-536-4818
Fax: 886-7-536-4803

Taiwan - Taipei
Tel: 886-2-2500-6610
Fax: 886-2-2508-0102

Taiwan - Hsinchu
Tel: 886-3-572-9526
Fax: 886-3-572-6459

EUROPE

Austria - Weis
Tel: 43-7242-2244-399
Fax: 43-7242-2244-393

Denmark - Ballerup
Tel: 45-4450-2828
Fax: 45-4485-2829

France - Massy
Tel: 33-1-69-53-63-20
Fax: 33-1-69-30-90-79

Germany - Ismaning
Tel: 49-89-627-144-0
Fax: 49-89-627-144-44

Italy - Milan
Tel: 39-0331-742611
Fax: 39-0331-466781

Netherlands - Drunen
Tel: 31-416-690399
Fax: 31-416-690340

England - Berkshire
Tel: 44-118-921-5869
Fax: 44-118-921-5820

03/01/05

© 2005 Microchip Technology Inc.

ANEXO N°5: Datasheet NRF24L01

Single chip 2.4 GHz Transceiver nRF24L01

FEATURES

- True single chip GFSK transceiver
- Complete OSI Link Layer in hardware
- Enhanced ShockBurst™
- Auto ACK & retransmit
- Address and CRC computation
- On the air data rate 1 or 2Mbps
- Digital interface (SPI) speed 0-8 Mbps
- 125 RF channel operation
- Short switching time enable frequency hopping
- Fully RF compatible with nRF24XX
- 5V tolerant signal input pads
- 20-pin package (QFN20 4x4mm)
- Uses ultra low cost +/- 60 ppm crystal
- Uses low cost chip inductors and 2-layer PCB
- Power supply range: 1.9 to 3.6 V

APPLICATIONS

- Wireless mouse, keyboard, joystick
- Keyless entry
- Wireless data communication
- Alarm and security systems
- Home automation
- Surveillance
- Automotive
- Telemetry
- Intelligent sports equipment
- Industrial sensors
- Toys

GENERAL DESCRIPTION

nRF24L01 is a single chip radio transceiver for the world wide 2.4 - 2.5 GHz ISM band. The transceiver consists of a fully integrated frequency synthesizer, a power amplifier, a crystal oscillator, a demodulator, modulator and Enhanced ShockBurst™ protocol engine. Output power, frequency channels, and protocol setup are easily programmable through a SPI interface. Current consumption is very low, only 9.0mA at an output power of -6dBm and 12.3mA in RX mode. Built-in Power Down and Standby modes makes power saving easily realizable.

QUICK REFERENCE DATA

Parameter	Value	Unit
Minimum supply voltage	1.9	V
Maximum output power	0	dBm
Maximum data rate	2000	kbps
Supply current in TX mode @ 0dBm output power	11.3	mA
Supply current in RX mode @ 2000 kbps	12.3	mA
Temperature range	-40 to +85	°C
Sensitivity @ 1000 kbps	-85	dBm
Supply current in Power Down mode	900	nA

Table 1 nRF24L01 quick reference data

Type Number	Description	Version
nRF24L01	20 pin QFN 4x4, RoHS & SS-00259 compliant	D
nRF24L01 IC	Bare Dice	D
nRF24L01-EVKIT	Evaluation kit (2 test PCB, 2 configuration PCB, SW)	1.0

Table 2 nRF24L01 ordering information

BLOCK DIAGRAM

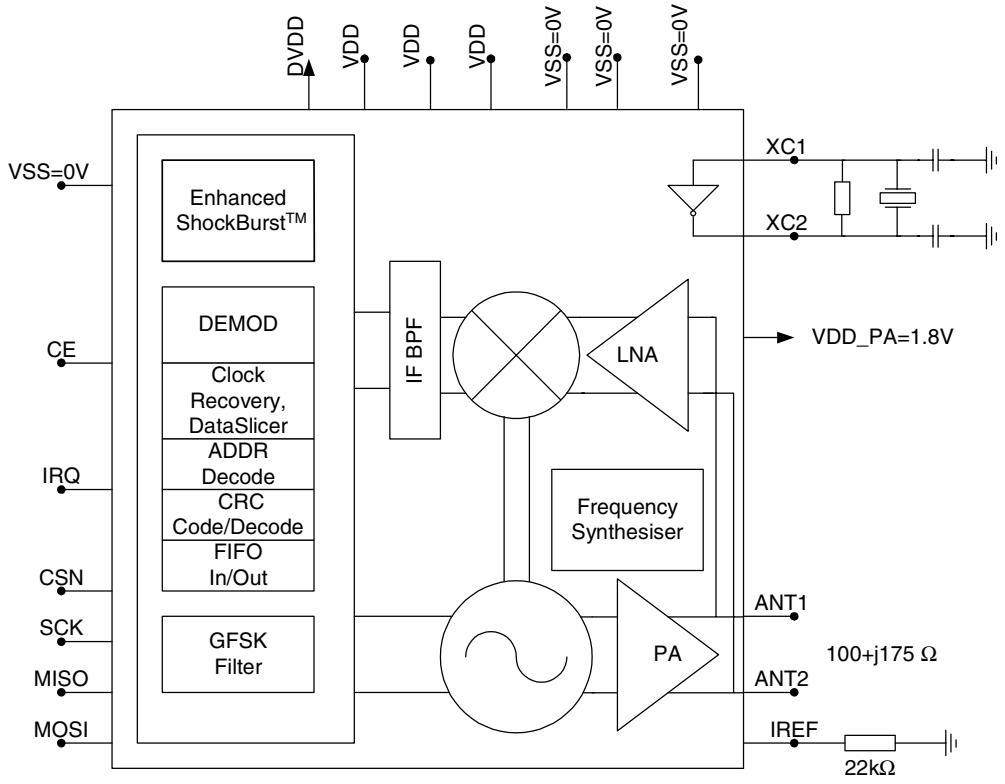


Figure 1 nRF24L01 with external components.

PIN FUNCTIONS

Pin	Name	Pin function	Description
1	CE	Digital Input	Chip Enable Activates RX or TX mode
2	CSN	Digital Input	SPI Chip Select
3	SCK	Digital Input	SPI Clock
4	MOSI	Digital Input	SPI Slave Data Input
5	MISO	Digital Output	SPI Slave Data Output, with tri-state option
6	IRQ	Digital Output	Maskable interrupt pin
7	VDD	Power	Power Supply (+3V DC)
8	VSS	Power	Ground (0V)
9	XC2	Analog Output	Crystal Pin 2
10	XC1	Analog Input	Crystal Pin 1
11	VDD_PA	Power Output	Power Supply (+1.8V) to Power Amplifier
12	ANT1	RF	Antenna interface 1
13	ANT2	RF	Antenna interface 2
14	VSS	Power	Ground (0V)
15	VDD	Power	Power Supply (+3V DC)
16	IREF	Analog Input	Reference current
17	VSS	Power	Ground (0V)
18	VDD	Power	Power Supply (+3V DC)
19	DVDD	Power Output	Positive Digital Supply output for de-coupling purposes
20	VSS	Power	Ground (0V)

Table 3 nRF24L01 pin function

PIN ASSIGNMENT

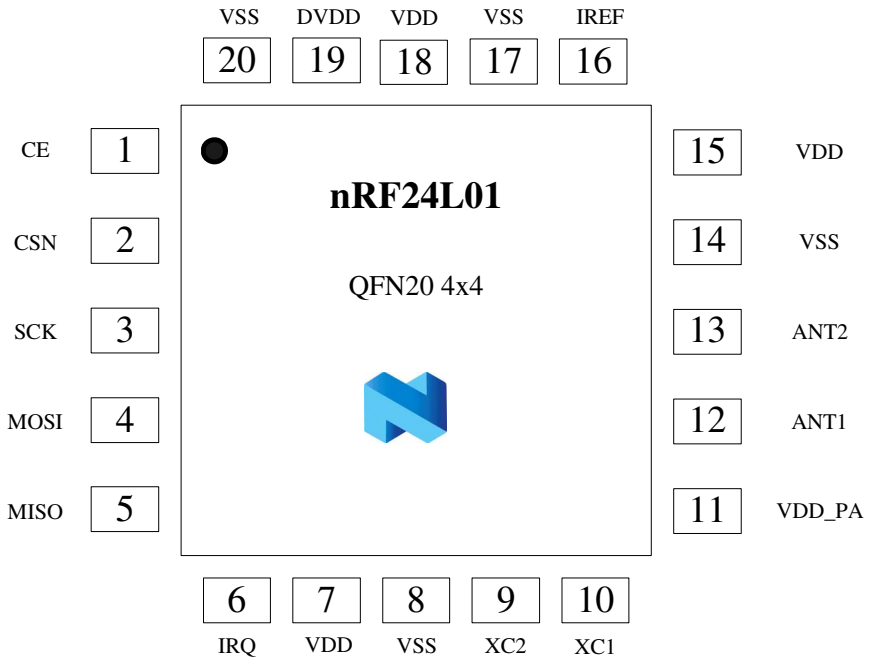


Figure 2 nRF24L01 pin assignment (top view) for a QFN20 4x4 package.

ELECTRICAL SPECIFICATIONS

Conditions: VDD = +3V, VSS = 0V, T_A = - 40ºC to + 85ºC

Symbol	Parameter (condition)	Notes	Min.	Typ.	Max.	Units
	Operating conditions					
VDD	Supply voltage		1.9	3.0	3.6	V
TEMP	Operating Temperature		-40	+27	+85	ºC
	Digital input pin					
V_{IH}	HIGH level input voltage	[1]	0.7VDD		5.25	V
V_{IL}	LOW level input voltage		VSS		0.3VDD	V
	Digital output pin					
V_{OH}	HIGH level output voltage (I_{OH}=-0.25mA)		VDD- 0.3		VDD	V
V_{OL}	LOW level output voltage (I_{OL}=0.25mA)		VSS		0.3	V
	General RF conditions					
f_{OP}	Operating frequency	[2]	2400		2525	MHz
f_{XTAL}	Crystal frequency			16		MHz
Δf_{1M}	Frequency deviation @ 1000kbps			±160		kHz
Δf_{2M}	Frequency deviation @ 2000kbps			±320		kHz
R_{GFSK}	Data rate ShockBurst™		>0		2000	kbps
$F_{CHANNEL}$	Channel spacing @ 1000kbps			1		MHz
$F_{CHANNEL}$	Channel spacing @ 2000kbps			2		MHz
	Transmitter operation					
P_{RF}	Maximum Output Power	[3]		0	+4	dBm
P_{RFC}	RF Power Control Range		16	18	20	dB
P_{RFCR}	RF Power Accuracy				±4	dB
P_{BW}	20dB Bandwidth for Modulated Carrier (2000kbps)			1800	2000	kHz
P_{RF1}	1st Adjacent Channel Transmit Power 2MHz				-20	dBm
P_{RF2}	2nd Adjacent Channel Transmit Power 4MHz				-50	dBm
I_{VDD}	Supply current @ 0dBm output power	[4]		11.3		mA
I_{VDD}	Supply current @ -18dBm output power			7.0		mA
I_{VDD}	Average Supply current @ -6dBm output power, Enhanced ShockBurst™	[5]		0.05		mA
I_{VDD}	Supply current in Standby-I mode	[6]		32		µA
I_{VDD}	Supply current in power down			900		nA

[1] All digital inputs handle up to 5.25V signal inputs. Keep in mind that the VDD of the nRF24L01 must match the V_{IH} of the driving device for output pins.
[2] Usable band is determined by local regulations
[3] Antenna load impedance = 15Ω+j88Ω
[4] Antenna load impedance = 15Ω+j88Ω. Effective data rate 1000kbps or 2000 kbps
[5] Antenna load impedance = 15Ω+j88Ω. Effective data rate 10kbps and full packets
[6] Given for a 12pF crystal. Current when using external clock is dependent on signal swing.

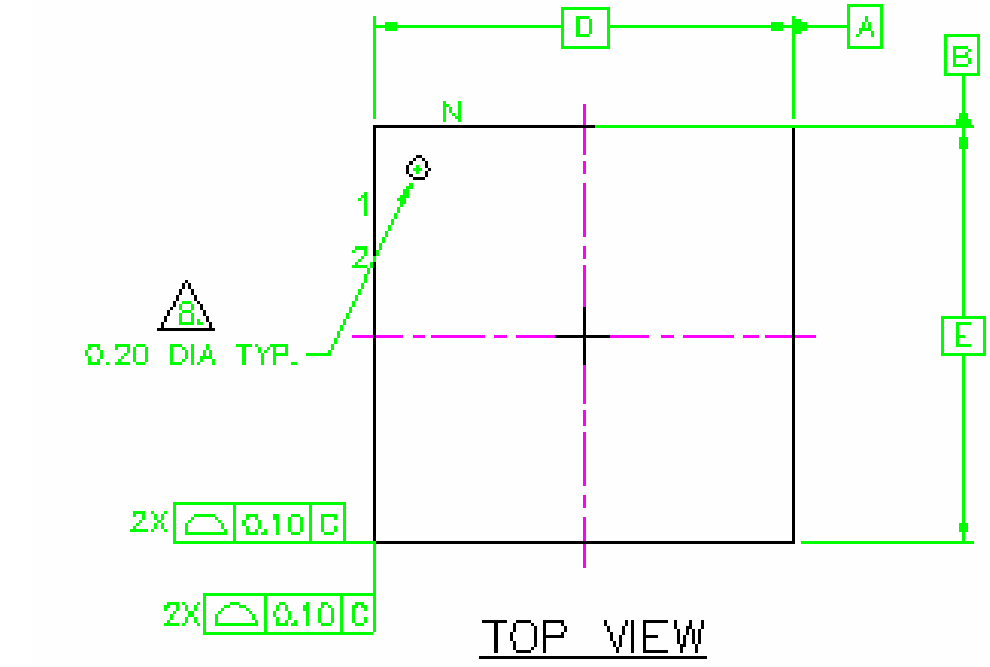
	Receiver operation					
I_{VDD}	Supply current one channel 2000kbps			12.3		mA
I_{VDD}	Supply current one channel 1000kbps			11.8		mA
RX_{SENS}	Sensitivity at 0.1%BER (@2000kbps)			-82		dBm
RX_{SENS}	Sensitivity at 0.1%BER (@1000kbps)			-85		dBm
C/I_{CO}	C/I Co-channel (@2000kbps)	[7]		7[8]/11[9]		dB
C/I_{1ST}	1st Adjacent Channel Selectivity C/I 2MHz			1/4		dB
C/I_{2ND}	2nd Adjacent Channel Selectivity C/I 4MHz			-21/-20		dB
C/I_{3RD}	3rd Adjacent Channel Selectivity C/I 6MHz			-27/-27		dB
C/I_{CO}	C/I Co-channel (@1000kbps)	[10]		9[11]/12[12]		dB
C/I_{1ST}	1st Adjacent Channel Selectivity C/I 1MHz			8/8		dB
C/I_{2ND}	2nd Adjacent Channel Selectivity C/I 2MHz			-22/-21		dB
C/I_{3RD}	3rd Adjacent Channel Selectivity C/I 3MHz			-30/-30		dB

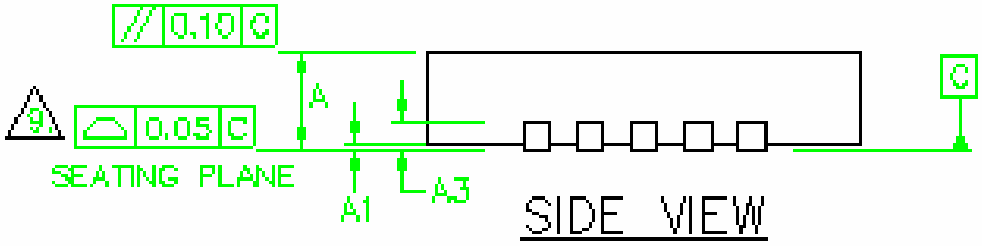
Table 4 nRF24L01 RF specifications

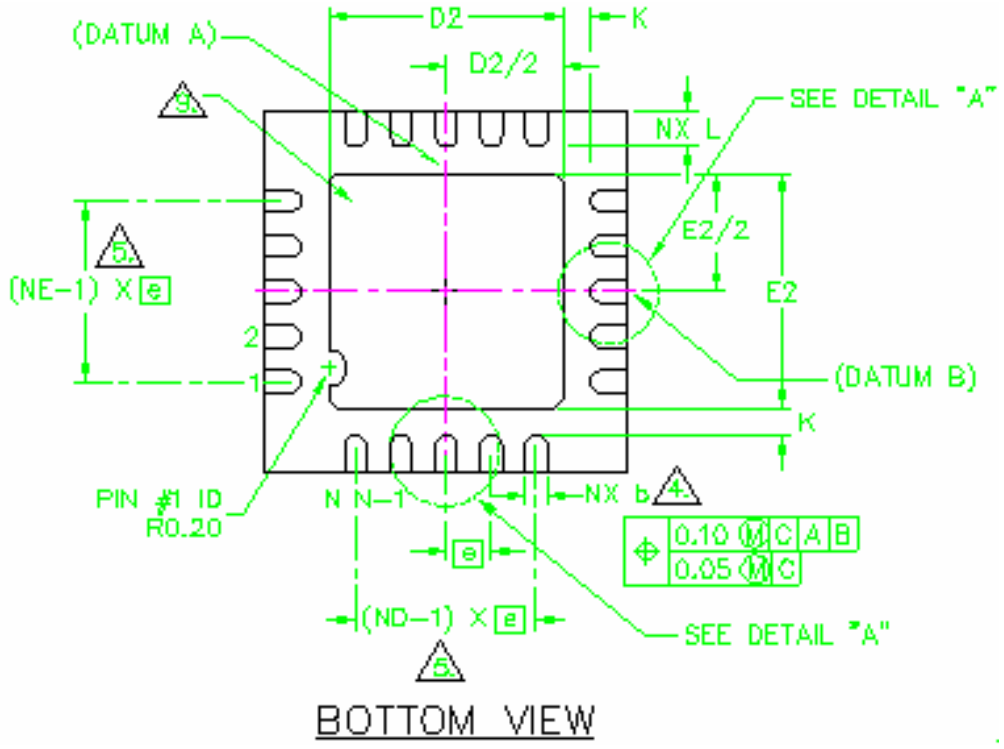
[7] Data rate is 2000kbps for the following C/I measurements
[8] According to ETSI EN 300 440-1 V1.3.1 (2001-09) page 27
[9] nRF24L01 equal modulation on interfering signal
[10] Data rate is 1000kbps for the following C/I measurements
[11] According to ETSI EN 300 440-1 V1.3.1 (2001-09) page 27
[12] nRF24L01 equal modulation on interfering signal

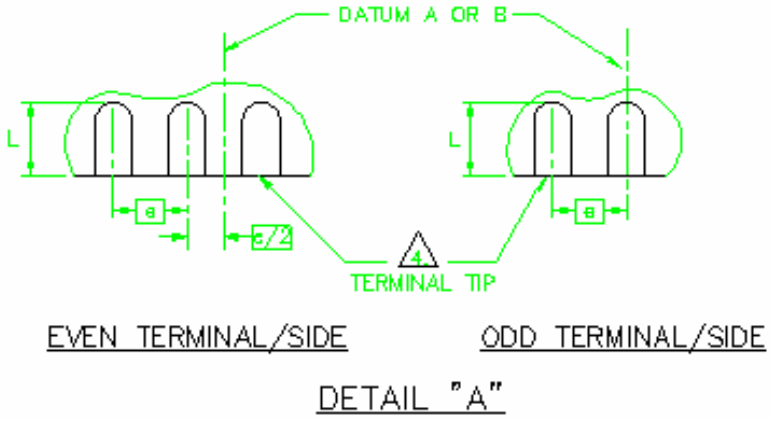
PACKAGE OUTLINE

nRF24L01 uses the QFN20 4x4 package, with matt tin plating.









Package Type		A	A1	A3	K	D/E	e	D2/E2	L	L1	b
Saw QFN20	**Min**	0.80	0.00					2.50	0.35		0.18
(4x4 mm)	**Typ.**	0.85	0.02	0.20	0.20	4.0 BSC[13]	0.5 BSC	2.60	0.40	0.15	0.25
	Max	0.95	0.05	REF.	min			2.70	0.45	max	0.30

Figure 3 nRF24L01 Package Outline.

[13] BSC: Basic Spacing between Centers, ref. JEDEC standard 95, page 4 17-11/A

Package marking:

n	R	F		*B*	*X*
2	4	L	0	1	
Y	*Y*	*W*	*W*	*L*	*L*

Abbreviations:

B – Build Code, i.e. unique code for production sites, package type and test platform
X – "X" grade, i.e. Engineering Samples (optional)
YY – 2 digit Year number
WW – 2 digit Week number
LL – 2 letter wafer lot number code

Absolute Maximum Ratings

Supply voltages
VDD ..... - 0.3V to + 3.6V
VSS 0V
Input voltage
V_I ..... - 0.3V to 5.25V
Output voltage
V_O ..... VSS to VDD

Total Power Dissipation
P_D (T_A=85°C) 60mW

Temperatures
Operating Temperature…. - 40°C to + 85°C
Storage Temperature……. - 40°C to + 125°C

Note: Stress exceeding one or more of the limiting values may cause permanent damage to the device.

ATTENTION!

Electrostatic Sensitive Device
Observe Precaution for handling.



Glossary of Terms

Term	Description
ACK	Acknowledgement
ART	Auto Re-Transmit
CE	Chip Enable
CLK	Clock
CRC	Cyclic Redundancy Check
CSN	Chip Select NOT
ESB	Enhanced ShockBurst™
GFSK	Gaussian Frequency Shift Keying
IRQ	Interrupt Request
ISM	Industrial-Scientific-Medical
LNA	Low Noise Amplifier
LSB	Least Significant Bit
LSByte	Least Significant Byte
Mbps	Megabit per second
MCU	Micro Controller Unit
MISO	Master In Slave Out
MOSI	Master Out Slave In
MSB	Most Significant Bit
MSByte	Most Significant Byte
PCB	Printed Circuit Board
PER	Packet Error Rate
PID	Packet Identity Bits
PLD	Payload
PRX	Primary RX
PTX	Primary TX
PWR_DWN	Power Down
PWR_UP	Power Up
RoHS	Restriction of use of Certain Hazardous Substances
RX	Receive
RX_DR	Receive Data Ready
SPI	Serial Peripheral Interface
TX	Transmit
TX_DS	Transmit Data Sent

Table 5 Glossary

FUNCTIONAL DESCRIPTION

Modes of operation

The nRF24L01 can be set in the following main modes depending on the level of the following primary I/Os and configuration registers:

Mode	PWR_UP register	PRIM_RX register	CE	FIFO state
RX mode	1	1	1	-
TX mode	1	0	1	Data in TX FIFO
TX mode	1	0	1➔0	Stays in TX mode until packet transmission is finished
Standby-II	1	0	1	TX FIFO empty
Standby-I	1	-	0	No ongoing packet transmission
Power Down	0	-	-	-

Table 6 nRF24L01 main modes

An overview of the nRF24L01 I/O pins in different modes is given in Table 7.

Pin functions in the different modes of nRF24L01

Pin Name	Direction	TX Mode	RX Mode	Standby Modes	Power Down
CE	Input	High Pulse >10µs	High	Low	-
CSN	Input	SPI Chip Select, active low			
SCK	Input	SPI Clock			
MOSI	Input	SPI Serial Input			
MISO	Tri-state Output	SPI Serial Output			
IRQ	Output	Interrupt, active low			

Table 7 Pin functions of the nRF24L01

Standby Modes

Standby-I mode is used to minimize average current consumption while maintaining short start up times. In this mode, part of the crystal oscillator is active. In Standby-II mode some extra clock buffers are active compared to Standby-I mode. Standby-II occurs when CE is held high on a PTX device with empty TX FIFO. The configuration word content is maintained during Standby modes. SPI interface may be activated. For start up time see Table 13.

Power Down Mode

In power down nRF24L01 is disabled with minimal current consumption. When entering this mode the device is not active, but all registers values available from the SPI interface are maintained during power down and the SPI interface may be activated (CSN=0). For start up time see Table 13. The power down is controlled by the PWR_UP bit in the CONFIG register.

Packet Handling Methods

nRF24L01 has the following Packet Handling Methods:

- ShockBurst™ (compatible with nRF2401, nRF24E1, nRF2402 and nRF24E2 with 1Mbps data rate, see page 26)
- Enhanced ShockBurst™

ShockBurst™

ShockBurst™ makes it possible to use the high data rate offered by nRF24L01 without the need of a costly, high-speed microcontroller (MCU) for data processing/clock recovery. By placing all high speed signal processing related to RF protocol on-chip, nRF24L01 offers the application microcontroller a simple SPI compatible interface, the data rate is decided by the interface-speed the micro controller itself sets up. By allowing the digital part of the application to run at low speed, while maximizing the data rate on the RF link, ShockBurst™ reduces the average current consumption in applications.
In ShockBurst™ RX, IRQ notifies the MCU when a valid address and payload is received respectively. The MCU can then clock out the received payload from an nRF24L01 RX FIFO.
In ShockBurst™ TX, nRF24L01 automatically generates preamble and CRC, see Table 12. IRQ notifies the MCU that the transmission is completed. All together, this means reduced memory demand in the MCU resulting in a low cost MCU, as well as reduced software development time. nRF24L01 has a three level deep RX FIFO (shared between 6 pipes) and a three level deep TX FIFO. The MCU can access the FIFOs at any time, in power down mode, in standby modes, and during RF packet transmission. This allows the slowest possible SPI interface compared to the average data-rate, and may enable usage of an MCU without hardware SPI.

Enhanced ShockBurst™

Enhanced ShockBurst™ is a packet handling method with functionality that makes bi-directional link protocol implementation easier and more efficient. In a typical bi-directional link, one will let the terminating part acknowledge received packets from the originating part in order to make it possible to detect data loss. Data loss can then be recovered by retransmission. The idea with Enhanced ShockBurst™ is to let nRF24L01 handle both acknowledgement of received packets and retransmissions of lost packets, without involvement from the microcontroller.

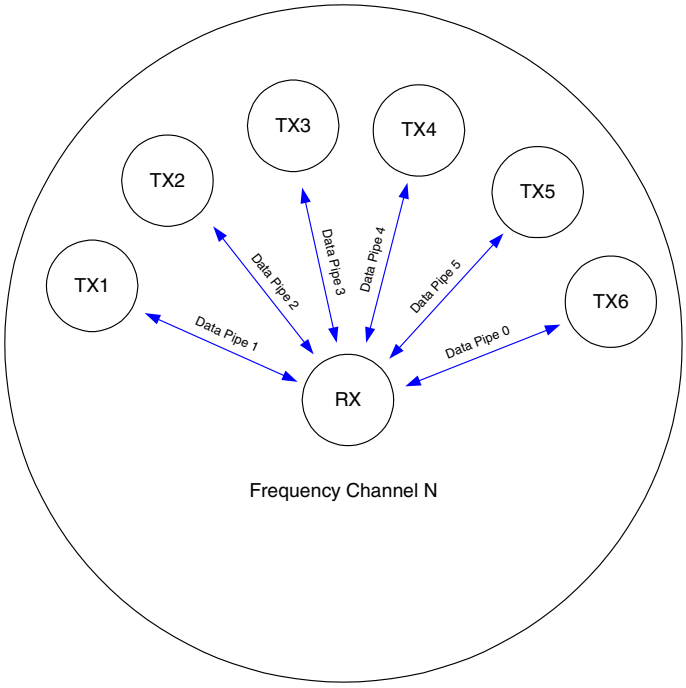


Figure 4: nRF24L01 in a star network configuration

An nRF24L01 configured as primary RX (PRX) will be able to receive data trough 6 different data pipes, see Figure 4. A data pipe will have a unique address but share the same frequency channel. This means that up to 6 different nRF24L01 configured as primary TX (PTX) can communicate with one nRF24L01 configured as PRX, and the nRF24L01 configured as PRX will be able to distinguish between them. Data pipe 0 has a unique 40 bit configurable address. Each of data pipe 1-5 has an 8 bit unique address and shares the 32 most significant address bits. All data pipes can perform full Enhanced ShockBurst™ functionality.
nRF24L01 will use the data pipe address when acknowledging a received packet. This means that nRF24L01 will transmit ACK with the same address as it receives payload at. In the PTX device data pipe 0 is used to received the acknowledgement, and therefore the receive address for data pipe 0 has to be equal to the transmit address to be able to receive the acknowledgement. See Figure 5 for addressing example.

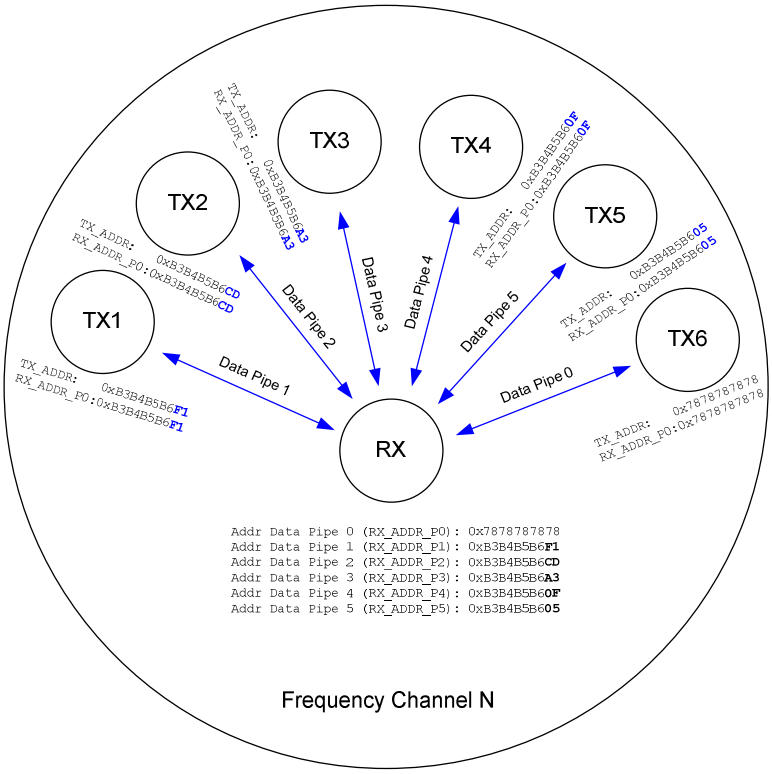


Figure 5: Example on how the acknowledgement addressing is done

An nRF24L01 configured as PTX with Enhanced ShockBurst™ enabled, will use the ShockBurst™ feature to send a packet whenever the microcontroller wants to. After the packet has been transmitted, nRF24L01 will switch on its receiver and expect an acknowledgement to arrive from the terminating part. If this acknowledgement fails to arrive, nRF24L01 will retransmit the same packet until it receives an acknowledgement or the number of retries exceeds the number of allowed retries given in the SETUP_RETR_ARC register. If the number of retries exceeds the number of allowed retries, this will be showed by the STATUS register bit MAX_RT which gives an interrupt.
Whenever an acknowledgement is received by an nRF24L01 it will consider the last transmitted packet as delivered. It will then be cleared from the TX FIFO, and the TX_DS IRQ source will be set high.

With Enhanced ShockBurst™ nRF24L01 offers the following benefits:

- Highly reduced current consumption due to short time on air and sharp timing when operating with acknowledgement traffic
- Lower system cost. Since the nRF24L01 handles all the high-speed link layer operations, like re-transmission of lost packet and generating acknowledgement to received packets, it is no need for hardware SPI on the system microcontroller to interface the nRF24L01. The interface can be done by using general purpose IO pins on a low cost microcontroller where the SPI is emulated in firmware. With the nRF24L01 this will be sufficient speed even when running a bi-directional link.
- Greatly reduced risk of “on-air” collisions due to short time on air
- Easier firmware development since the link layer is integrated on chip

Enhanced ShockBurst™ Transmitting Payload:

1. The configuration bit PRIM_RX has to be low.
2. When the application MCU has data to send, the address for receiving node (TX_ADDR) and payload data (TX_PLD) has to be clocked into nRF24L01 via the SPI interface. The width of TX-payload is counted from number of bytes written into the TX FIFO from the MCU. TX_PLD must be written continuously while holding CSN low. TX_ADDR does not have to be rewritten if it is unchanged from last transmit. If the PTX device shall receive acknowledge, data pipe 0 has to be configured to receive the acknowledgement. The receive address for data pipe 0 (RX_ADDR_P0) has to be equal to the transmit address (TX_ADDR) in the PTX device. For the example in Figure 5 the following address settings have to be performed for the TX5 device and the RX device:
   TX5 device: TX_ADDR = 0xB3B4B5B605
   TX5 device: RX_ADDR_P0 = 0xB3B4B5B605
   RX device: RX_ADDR_P5 = 0xB3B4B5B605
3. A high pulse on CE starts the transmission. The minimum pulse width on CE is 10 µs.
4. nRF24L01 ShockBurst™:
   - Radio is powered up
   - 16 MHz internal clock is started.
   - RF packet is completed (see the packet description)
   - Data is transmitted at high speed (1 Mbps or 2 Mbps configured by MCU).
5. If auto acknowledgement is activated (ENAA_P0=1) the radio goes into RX mode immediately. If a valid packet has been received in the valid acknowledgement time window, the transmission is considered a success. The TX_DS bit in the status register is set high and the payload is removed from TX FIFO. If a valid acknowledgement is not received in the specified time window, the payload is resent (if auto retransmit is enabled). If the auto retransmit counter (ARC_CNT) exceeds the programmed maximum limit (ARC), the MAX_RT bit in the status register is set high. The payload in TX FIFO is NOT removed. The IRQ pin will be active when MAX_RT or TX_DS is high. To turn off the IRQ pin, the interrupt source must be reset by writing to the status register (see Interrupt chapter). If no acknowledgement is received for a packet after the maximum number of retries, no further packets can be sent before the MAX_RX interrupt is cleared. The packet loss counter (PLOS_CNT) is incremented at each MAX_RT interrupt. I.e. ARC_CNT counts the number of retries that was required to get a single packet through. PLOS_CNT counts the number of packets that did not get through after maximum number of retries.
6. The device goes into Standby-I mode if CE is low. Otherwise next payload in TX FIFO will be sent. If TX FIFO is empty and CE is still high, the device will enter Standby-II mode.
7. If the device is in Standby-II mode, it will go to Standby-I mode immediately if CE is set low.

Enhanced ShockBurst™ Receive Payload:

1. RX is selected by setting the PRIM_RX bit in the configuration register to high. All data pipes that shall receive data must be enabled (EN_RXADDR register),



auto acknowledgement for all pipes running Enhanced ShockBurst™ has to be enabled (EN_AA register), and the correct payload widths must be set (RX_PW_Px registers). Addresses have to be set up as described in item 2 in the Enhanced ShockBurst™ transmit payload chapter above.

2. Active RX mode is started by setting CE high.
3. After 130µs nRF24L01 is monitoring the air for incoming communication.
4. When a valid packet has been received (matching address and correct CRC), the payload is stored in the RX-FIFO, and the RX_DR bit in status register is set high. The IRQ pin will be active when RX_DR is high. RX_P_NO in status register will indicate what data pipe the payload has been received in.
5. If auto acknowledgement is enabled, an acknowledgement is sent back.
6. MCU sets the CE pin low to enter Standby-I mode (low current mode).
7. MCU can clock out the payload data at a suitable rate via the SPI interface.
8. The device is now ready for entering TX or RX mode or power down mode.

Two way communication with payload in both directions

If payload shall be sent in both directions, the PRIM_RX register must be toggled by redefining the device from PRX to PTX or vice versa. The controlling processors must handle the synchronicity between a PTX and a PRX. Data buffering in both RX FIFO and TX FIFO simultaneously is possible, but restricted to data pipes 1 to 5. The third level in TX FIFO shall only be written in RX, TX or Standby-II mode if data is stored in RX FIFO

Auto Acknowledgement (RX)

The auto acknowledgement function reduces the load of the external microcontroller, and may remove the need for dedicated SPI hardware in a mouse/keyboard or comparable systems, and hence reduce cost and average current consumption. Auto acknowledgement can be configured individually for each data pipe via the SPI interface.

If auto acknowledgement is enabled and a valid packet (correct data pipe address and CRC) is received, the device will enter TX mode and send an acknowledgement packet. After the device has sent the acknowledgement packet, normal operation resumes, and the mode is determined by the PRIM_RX register and CE pin.

Auto Re-Transmission (ART) (TX)

An auto retransmission function is available. It will be used at the TX side in an auto acknowledgement system. In the SETUP_RETR register it will be possible to state how many times the data in the data register will be resent if data is not acknowledged. After each sending, the device will enter RX mode and wait a specified time period for acknowledgement. When the acknowledgement packet is received, the device will return to normal transmit function. If there is no more unsent data in the TX FIFO and the CE pin is low, the device will go into Standby-I mode. If the acknowledgement is not received, the device will go back to TX mode and resend the data. This will continue until acknowledgment is received, or a time out occurs

(i.e. the maximum number of sending is reached). The only way to reset this is to set the PWR_UP bit low or let the auto retransmission finish. A packet loss counter will be incremented each time a packet does not succeed to reach the destination before time out. (Time out is indicated by the MAX_RT interrupt.) The packet loss counter is reset when writing to the RF channel register.

Packet Identity (PID) and CRC used by Enhanced ShockBurst™

Each packet contains a two bit wide PID field to detect if the received packet is new or resent. The PID will prevent that the PRX device presents the same payload more than once to the microcontroller. This PID field is incremented at the TX side for each new packet received via the SPI interface. The PID and CRC field is used by the PRX device to determine whether a packet is resent or new. When several data is lost on the link, the PID fields may in some cases become equal to last received PID. If a packet has the same PID as the previous packet, nRF24L01 will compare the CRC sums from both packets. If they also are equal, the last received packet is considered as a copy of the previous and is discarded.

1: PRX device:
The PRX device compares the received PID with the last PID. If the PID fields are different, the packet is considered to be new. If the PID is equal to last received PID, the received packet might be the same as last time. The receiver must check if the CRC is equal to the previous CRC. If the CRC is equal to the previous one, the packet is probably the same, and will be discarded.

2: PTX device:
The transmitter increments the PID field each time it sends a new packet.

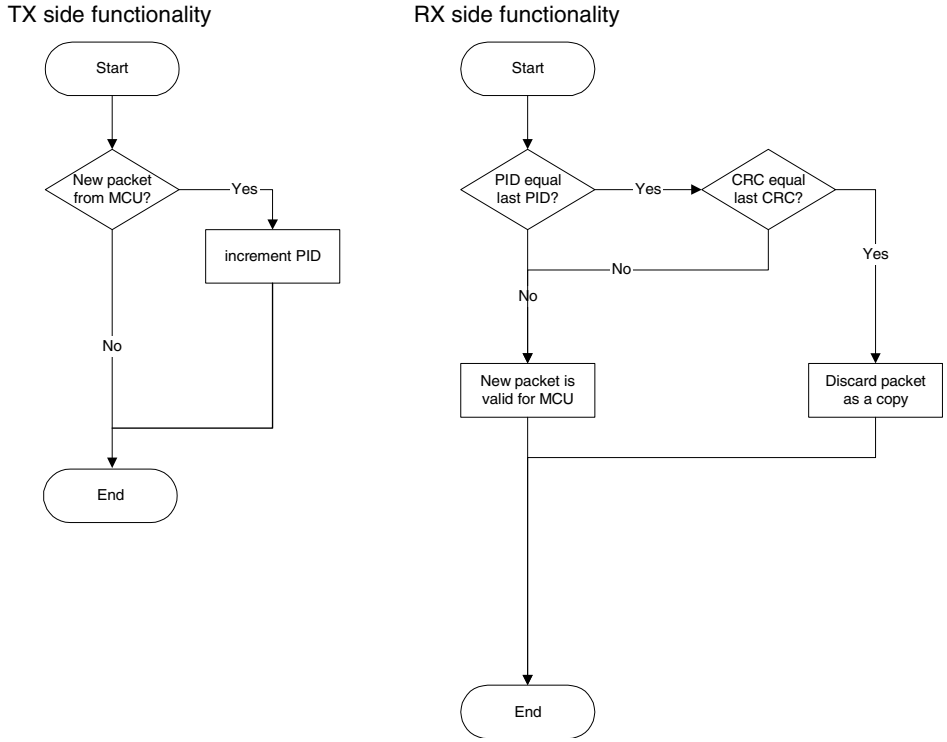


Figure 6 PID generation/detection

The length of the CRC is configurable through the SPI interface. It is important to notice that the CRC is calculated over the whole packet including address, PID and payload. No packet is accepted as correct if the CRC fails. This is an extra requirement for packet acceptance that is not illustrated in the figure above.

Stationary Disturbance Detection – CD

Carrier Detect (CD) is set high when an in-band RF signal is detected in RX mode, otherwise CD is low. The internal CD signal is filtered before presented to CD register. The internal CD signal must be high for at least 128µs.
In Enhanced ShockBurst™ it is recommended to use the Carrier Detect functionality only when the PTX device does not succeed to get packets through, as indicated by the MAX_RT interrupt for single packets and by the packet loss counter (PLOS_CNT) if several packets are lost. If the PLOS_CNT in the PTX device indicates to high rate of packet losses, the device can be configured to a PRX device for a short time (T_{stbt2a} + CD-filter delay = 130µs+128µs = 258µs) to check CD. If CD was high (jam situation), the frequency channel should be changed. If CD was low (out of range), it may continue on the same frequency channel, but perform other adjustments. (A dummy write to the RF_CH will clear the PLOS_CNT.)

Data Pipes

nRF24L01 configured as PRX can receive data addressed to 6 different data pipes in one physical frequency channel. Each data pipe has its own unique address and can be configured to have individual behavior.

The data pipes are enabled with the bits in the EN_RXADDR register. By default only data pipe 0 and 1 are enabled.
The address for each data pipe is configured in the RX_ADDR_Px registers. Always ensure that none of the data pipes have the exact same address.

Data pipe 0 has a unique 40 bit configurable address. Data pipes 1-5 share the 32 most significant address bits and have only the LSByte unique for each data pipe. Figure 7 shows an example of how data pipes 0-5 are addressed. All pipes can have up to 40 bit address, but for pipe 1-5 only the LSByte is different, and the LSByte must be unique for all pipes.

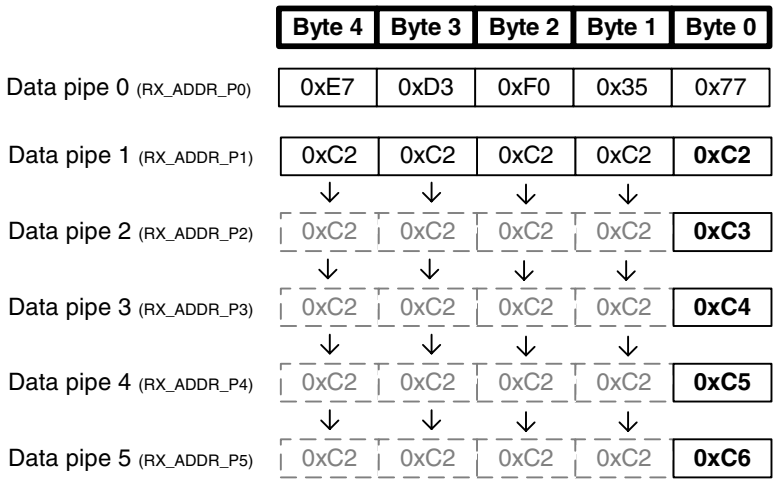


Figure 7: Addressing data pipes 0-5

When a packet has been received at one of the data pipes and the data pipe is setup to generate acknowledgement, nRF24L01 will generate an acknowledgement with an address that equals the data pipe address where the packet was received.

Some configuration settings are common to all data pipes and some are individual. The following settings are common to all data pipes:

- CRC enabled/disabled (CRC always enabled when ESB is enabled)
- CRC encoding scheme
- RX address width
- Frequency channel
- RF data rate
- LNA gain
- RF output power

DEVICE CONFIGURATION

All configuration of nRF24L01 is defined by values in some configuration registers. All these registers are writable via the SPI interface.

SPI Interface

The SPI interface is a standard SPI interface with a maximum data rate of 10Mbps. Most registers are readable.

SPI Instruction Set

The available commands to be used on the SPI interface are shown below. Whenever CSN is set low the interface expects an instruction. Every new instruction must be started by a high to low transition on CSN.

In parallel to the SPI instruction word applied on the MOSI pin, the STATUS register is shifted serially out on the MISO pin.

The serial shifting SPI commands is on the format:

<**Instruction word**: MSBit to LSBit (one byte)>

<**Data bytes**: LSByte to MSByte, MSBit in each byte first>

See Figure 8 and Figure 9.

Instruction Name	Instruction Format [binary]	# Data Bytes	Operation
R_REGISTER	000A AAAA	1 to 5 LSByte first	Read registers. AAAAA = 5 bit Memory Map Address
W_REGISTER	001A AAAA	1 to 5 LSByte first	Write registers. AAAAA = 5 bit Memory Map Address *Executable in power down or standby modes only.*
R_RX_PAYLOAD	0110 0001	1 to 32 LSByte first	Read RX-payload: 1 – 32 bytes. A read operation will always start at byte 0. Payload will be deleted from FIFO after it is read. Used in RX mode.
W_TX_PAYLOAD	1010 0000	1 to 32 LSByte first	Used in TX mode. Write TX-payload: 1 – 32 bytes. A write operation will always start at byte 0.
FLUSH_TX	1110 0001	0	Flush TX FIFO, used in TX mode
FLUSH_RX	1110 0010	0	Flush RX FIFO, used in RX mode Should not be executed during transmission of acknowledge, i.e. acknowledge package will not be completed.
REUSE_TX_PL	1110 0011	0	Used for a PTX device Reuse last sent payload. Packets will be repeatedly resent as long as CE is high. TX payload reuse is active until W_TX_PAYLOAD or FLUSH TX is executed. TX payload reuse must not be activated or deactivated during package transmission
NOP	1111 1111	0	No Operation. Might be used to read the STATUS register

Table 8 Instruction set for the nRF24L01 SPI interface.

The W_REGISTER and R_REGISTER may operate on single or multi-byte registers. When accessing multi-byte registers one will read or write MSBit of LSByte first. The

writing can be terminated before all bytes in a multi-byte register has been written. In this case the unwritten MSByte(s) will remain unchanged. E.g. the LSByte of RX_ADDR_P0 can be modified by writing only one byte to the RX_ADDR_P0 register. The content of the status register will always be read to MISO after a high to low transition on CSN.

Interrupt

The nRF24L01 has an active low interrupt pin (IRQ). The interrupt pin is activated when TX_DS, RX_DR or MAX_RT is set high in status register. When MCU writes '1' to the interrupt source, the IRQ pin will go inactive. The interrupt mask part of the CONFIG register is used to mask out the interrupt sources that are allowed to set the IRQ pin low. By setting one of the MASK bits high, the corresponding interrupt source will be disabled. By default all interrupt sources are enabled.

SPI Timing

The interface supports SPI. SPI operation and timing is given in Figure 8 to Figure 10 and in Table 9 and Table 10. The device must be in one of the standby modes or power down mode before writing to the configuration registers. In Figure 8 to Figure 10 the following notations are used:
Cn – SPI Instruction Bit
Sn – Status Register Bit
Dn – Data Bit (note: LSByte to MSByte, MSBit in each byte first)

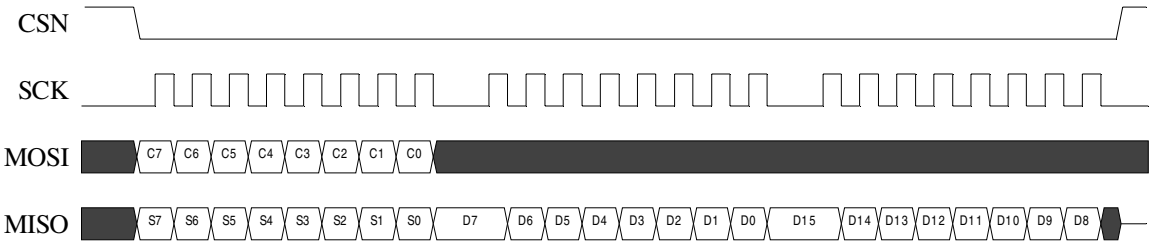


Figure 8 SPI read operation.

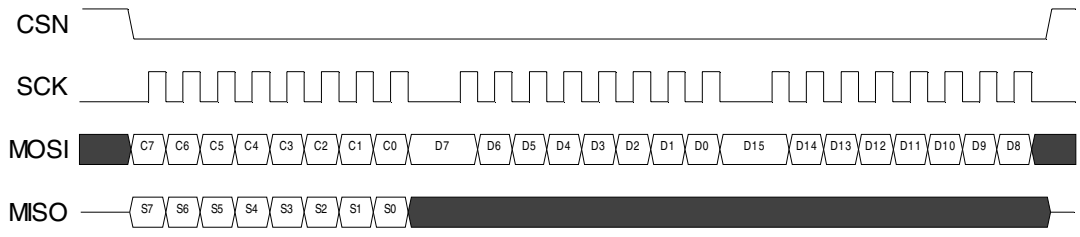


Figure 9 SPI write operation.

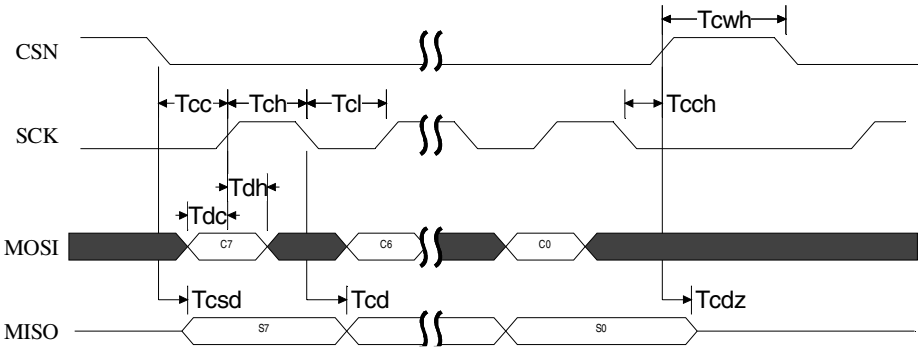


Figure 10 SPI NOP timing diagram.

PARAMETER	SYMBOL	MIN	MAX	UNITS
Data to SCK Setup	Tdc	2		ns
SCK to Data Hold	Tdh	2		ns
CSN to Data Valid	Tcsd		38	ns
SCK to Data Valid	Tcd		55	ns
SCK Low Time	Tcl	40		ns
SCK High Time	Tch	40		ns
SCK Frequency	Fsck	0	8	MHz
SCK Rise and Fall	Tr,Tf		100	ns
CSN to SCK Setup	Tcc	2		ns
SCK to CSN Hold	Tcch	2		ns
CSN Inactive time	Tcwh	50		ns
CSN to Output High Z	Tcdz		38	ns

Table 9 SPI timing parameters (C_{Load} = 5pF).

PARAMETER	SYMBOL	MIN	MAX	UNITS
Data to SCK Setup	Tdc	2		ns
SCK to Data Hold	Tdh	2		ns
CSN to Data Valid	Tcsd		42	ns
SCK to Data Valid	Tcd		58	ns
SCK Low Time	Tcl	40		ns
SCK High Time	Tch	40		ns
SCK Frequency	Fsck	0	8	MHz
SCK Rise and Fall	Tr,Tf		100	ns
CSN to SCK Setup	Tcc	2		ns
SCK to CSN Hold	Tcch	2		ns
CSN Inactive time	Tcwh	50		ns
CSN to Output High Z	Tcdz		42	ns

Table 10 SPI timing parameters (C_{Load} = 10pF).

MEMORY MAP

All undefined bits in the table below are redundant. They will be read out as '0'.

Address (Hex)	Mnemonic	Bit	Reset Value	Type	Description
00	CONFIG				Configuration Register
	Reserved	7	0	R/W	Only '0' allowed
	MASK_RX_DR	6	0	R/W	Mask interrupt caused by RX_DR 1: Interrupt not reflected on the IRQ pin 0: Reflect RX_DR as active low interrupt on the IRQ pin
	MASK_TX_DS	5	0	R/W	Mask interrupt caused by TX_DS 1: Interrupt not reflected on the IRQ pin 0: Reflect TX_DS as active low interrupt on the IRQ pin
	MASK_MAX_RT	4	0	R/W	Mask interrupt caused by MAX_RT 1: Interrupt not reflected on the IRQ pin 0: Reflect MAX_RT as active low interrupt on the IRQ pin
	EN_CRC	3	1	R/W	Enable CRC. Forced high if one of the bits in the EN_AA is high
	CRCO	2	0	R/W	CRC encoding scheme '0' - 1 byte '1' – 2 bytes
	PWR_UP	1	0	R/W	1: POWER UP, 0:POWER DOWN
	PRIM_RX	0	0	R/W	1: PRX, 0: PTX
01	EN_AA Enhanced ShockBurst™				Enable 'Auto Acknowledgment' Function Disable this functionality to be compatible with nRF2401, see page 26
	Reserved	7:6	00	R/W	Only '00' allowed
	ENAA_P5	5	1	R/W	Enable auto ack. data pipe 5
	ENAA_P4	4	1	R/W	Enable auto ack. data pipe 4
	ENAA_P3	3	1	R/W	Enable auto ack. data pipe 3
	ENAA_P2	2	1	R/W	Enable auto ack. data pipe 2
	ENAA_P1	1	1	R/W	Enable auto ack. data pipe 1
	ENAA_P0	0	1	R/W	Enable auto ack. data pipe 0
02	EN_RXADDR				Enabled RX Addresses
	Reserved	7:6	00	R/W	Only '00' allowed
	ERX _P5	5	0	R/W	Enable data pipe 5.
	ERX _P4	4	0	R/W	Enable data pipe 4.
	ERX _P3	3	0	R/W	Enable data pipe 3.
	ERX _P2	2	0	R/W	Enable data pipe 2.
	ERX _P1	1	1	R/W	Enable data pipe 1.
	ERX _P0	0	1	R/W	Enable data pipe 0.
03	SETUP_AW				Setup of Address Widths (common for all data pipes)
	Reserved	7:2	000000	R/W	Only '000000' allowed
	AW	1:0	11	R/W	RX/TX Address field width '00' - Illegal '01' - 3 bytes '10' - 4 bytes '11' – 5 bytes LSByte will be used if address width below 5 bytes
04	SETUP_RETR				Setup of Automatic Retransmission
	ARD	7:4	0000	R/W	Auto Re-transmit Delay '0000' – Wait 250+86uS

Address (Hex)	Mnemonic	Bit	Reset Value	Type	Description
					‘0001’ – Wait 500+86uS ‘0010’ – Wait 750+86uS …….. ‘1111’ – Wait 4000+86uS (Delay defined from end of transmission to start of next transmission)[14]
	ARC	3:0	0011	R/W	Auto Retransmit Count ‘0000’ –Re-Transmit disabled ‘0001’ – Up to 1 Re-Transmit on fail of AA …… ‘1111’ – Up to 15 Re-Transmit on fail of AA
05	RF_CH				RF Channel
	Reserved	7	0	R/W	Only '0' allowed
	RF_CH	6:0	0000010	R/W	Sets the frequency channel nRF24L01 operates on
06	RF_SETUP				RF Setup Register
	Reserved	7:5	000	R/W	Only '000' allowed
	PLL_LOCK	4	0	R/W	Force PLL lock signal. Only used in test
	RF_DR	3	1	R/W	Data Rate ‘0’ – 1 Mbps ‘1’ – 2 Mbps
	RF_PWR	2:1	11	R/W	Set RF output power in TX mode '00' – -18 dBm '01' – -12 dBm '10' – -6 dBm '11' – 0 dBm
	LNA_HCURR	0	1	R/W	Setup LNA gain
07	STATUS				Status Register (In parallel to the SPI instruction word applied on the MOSI pin, the STATUS register is shifted serially out on the MISO pin)
	Reserved	7	0	R/W	Only '0' allowed
	RX_DR	6	0	R/W	Data Ready RX FIFO interrupt. Set high when new data arrives RX FIFO[15]. Write 1 to clear bit.
	TX_DS	5	0	R/W	Data Sent TX FIFO interrupt. Set high when packet sent on TX. If AUTO_ACK is activated, this bit will be set high only when ACK is received. Write 1 to clear bit.
	MAX_RT	4	0	R/W	Maximum number of TX retries interrupt Write 1 to clear bit. If MAX_RT is set it must be cleared to enable further communication.
	RX_P_NO	3:1	111	R	Data pipe number for the payload

[14] Accurate formula for delay from start of transmission, to start of re-transmission:
TRD (us) = 250us * (ARD+1) + 4us *(AW + PW + CRCW) +138,5us.
TRD= total retransmit delay, AW=Address Width (#bytes), PW=Payload Width(#bytes)
, CRCW= CRC Width (#bytes)

[15] The Data Ready interrupt is set by a new packet arrival event. The procedure for handling this interrupt should be: 1) read payload via SPI, 2) clear RX_DR interrupt, 3) read FIFO_STATUS to check if there are more payloads available in RX FIFO, 4) if there are more data in RX FIFO, repeat from 1).

Address (Hex)	Mnemonic	Bit	Reset Value	Type	Description
					available for reading from RX_FIFO 000-101: Data Pipe Number 110: Not Used 111: RX FIFO Empty
	TX_FULL	0	0	R	TX FIFO full flag. 1: TX FIFO full. 0: Available locations in TX FIFO.
08	OBSERVE_TX				Transmit observe register
	PLOS_CNT	7:4	0	R	Count lost packets. The counter is overflow protected to 15, and discontinue at max until reset. The counter is reset by writing to RF_CH. See page 14 and 17.
	ARC_CNT	3:0	0	R	Count resent packets. The counter is reset when transmission of a new packet starts. See page 14.
09	CD				
	Reserved	7:1	000000	R	
	CD	0	0	R	Carrier Detect. See page 17.
0A	RX_ADDR_P0	39:0	0xE7E7E7E7E7	R/W	Receive address data pipe 0. 5 Bytes maximum length. (LSByte is written first. Write the number of bytes defined by SETUP_AW)
0B	RX_ADDR_P1	39:0	0xC2C2C2C2C2	R/W	Receive address data pipe 1. 5 Bytes maximum length. (LSByte is written first. Write the number of bytes defined by SETUP_AW)
0C	RX_ADDR_P2	7:0	0xC3	R/W	Receive address data pipe 2. Only LSB. MSBytes will be equal to RX_ADDR_P1[39:8]
0D	RX_ADDR_P3	7:0	0xC4	R/W	Receive address data pipe 3. Only LSB. MSBytes will be equal to RX_ADDR_P1[39:8]
0E	RX_ADDR_P4	7:0	0xC5	R/W	Receive address data pipe 4. Only LSB. MSBytes will be equal to RX_ADDR_P1[39:8]
0F	RX_ADDR_P5	7:0	0xC6	R/W	Receive address data pipe 5. Only LSB. MSBytes will be equal to RX_ADDR_P1[39:8]
10	TX_ADDR	39:0	0xE7E7E7E7E7	R/W	Transmit address. Used for a PTX device only. (LSByte is written first) Set RX_ADDR_P0 equal to this address to handle automatic acknowledge if this is a PTX device with Enhanced ShockBurst™ enabled. See page 14.
11	RX_PW_P0				
	Reserved	7:6	00	R/W	Only '00' allowed
	RX_PW_P0	5:0	0	R/W	Number of bytes in RX payload in data pipe 0 (1 to 32 bytes). 0 Pipe not used 1 = 1 byte … 32 = 32 bytes
12	RX_PW_P1				
	Reserved	7:6	00	R/W	Only '00' allowed
	RX_PW_P1	5:0	0	R/W	Number of bytes in RX payload in data pipe 1 (1 to 32 bytes). 0 Pipe not used

Address (Hex)	Mnemonic	Bit	Reset Value	Type	Description
					1 = 1 byte … 32 = 32 bytes
13	RX_PW_P2				
	Reserved	7:6	00	R/W	Only '00' allowed
	RX_PW_P2	5:0	0	R/W	Number of bytes in RX payload in data pipe 2 (1 to 32 bytes). 0 Pipe not used 1 = 1 byte … 32 = 32 bytes
14	RX_PW_P3				
	Reserved	7:6	00	R/W	Only '00' allowed
	RX_PW_P3	5:0	0	R/W	Number of bytes in RX payload in data pipe 3 (1 to 32 bytes). 0 Pipe not used 1 = 1 byte … 32 = 32 bytes
15	RX_PW_P4				
	Reserved	7:6	00	R/W	Only '00' allowed
	RX_PW_P4	5:0	0	R/W	Number of bytes in RX payload in data pipe 4 (1 to 32 bytes). 0 Pipe not used 1 = 1 byte … 32 = 32 bytes
16	RX_PW_P5				
	Reserved	7:6	00	R/W	Only '00' allowed
	RX_PW_P5	5:0	0	R/W	Number of bytes in RX payload in data pipe 5 (1 to 32 bytes). 0 Pipe not used 1 = 1 byte … 32 = 32 bytes
17	FIFO_STATUS				FIFO Status Register
	Reserved	7	0	R/W	Only '0' allowed
	TX_REUSE	6	0	R	Reuse last sent data packet if set high. The packet will be repeatedly resent as long as CE is high. TX_REUSE is set by the SPI instruction REUSE_TX_PL, and is reset by the SPI instructions W_TX_PAYLOAD or FLUSH TX
	TX_FULL	5	0	R	TX FIFO full flag. 1: TX FIFO full. 0: Available locations in TX FIFO.
	TX_EMPTY	4	1	R	TX FIFO empty flag. 1: TX FIFO empty. 0: Data in TX FIFO.
	Reserved	3:2	00	R/W	Only '00' allowed
	RX_FULL	1	0	R	RX FIFO full flag. 1: RX FIFO full. 0: Available locations in RX FIFO.
	RX_EMPTY	0	1	R	RX FIFO empty flag. 1: RX FIFO empty. 0: Data in RX FIFO.
N/A	TX_PLD	255:0	X	W	Written by separate SPI command TX data payload register 1 - 32 bytes. This register is implemented as a FIFO with 3 levels. Used in TX mode only
N/A	RX_PLD	255:0	X	R	Written by separate SPI command RX data payload register. 1 - 32 bytes. This register is implemented as a FIFO

Address (Hex)	Mnemonic	Bit	Reset Value	Type	Description
					with 3 levels. All receive channels share the same FIFO

Table 11 Memory map of nRF24L01

Configuration for compatibility with nRF24XX

How to setup nRF24L01 to receive from an nRF2401/nRF2402/nRF24E1/nRF24E2:

- Use same CRC configuration as the nRF2401/nRF2402/nRF24E1/nRF24E2 uses
- Set the PRIM_RX bit to 1
- Disable auto acknowledgement on the data pipe that will be addressed
- Use the same address width as the PTX device
- Use the same frequency channel as the PTX device
- Select data rate 1Mbit/s on both nRF24L01 and nRF2401/nRF2402/nRF24E1/nRF24E2
- Set correct payload width on the data pipe that will be addressed
- Set PWR_UP and CE high

How to setup nRF24L01 to transmit to an nRF2401/nRF24E1:

- Use same CRC configuration as the nRF2401/nRF2402/nRF24E1/nRF24E2 uses
- Set the PRIM_RX bit to 0
- Set the Auto Retransmit Count to 0 to disable the auto retransmit functionality
- Use the same address width as the nRF2401/nRF2402/nRF24E1/nRF24E2 uses
- Use the same frequency channel as the nRF2401/nRF2402/nRF24E1/nRF24E2 uses
- Select data rate 1Mbit/s on both nRF24L01 and nRF2401/nRF2402/nRF24E1/nRF24E2
- Set PWR_UP high
- Clock in a payload that has the same length as the nRF2401/nRF2402/nRF24E1/nRF24E2 is configured to receive
- Pulse CE to send the packet

PACKET DESCRIPTION

An Enhanced ShockBurst™ packet with payload (1-32 bytes).



A ShockBurst™ packet compatible to nRF2401/nRF2402/nRF24E1/nRF24E2 devices.



Preamble	• Preamble is used to detect 0 and 1 levels. It is stripped off (RX) and added (TX) by nRF24L01.
Address	• The address field contains the receiver address. • The address can be 3, 4 or 5 bytes wide • The address fields can be individually configured for all RX channels and the TX channel • Address is automatically removed from received packets.[16]
Flags	• PID: Packet Identification. 2 bits that is incremented for each new payload • 7 bits reserved for packet compatibility with future products • Not used when compatible to nRF2401/nRF24E1
Payload	• 1 - 32 bytes wide.
CRC	• The CRC is optional. • 0-2 bytes wide CRC • The polynomial for 8 bits CRC check is $X^8 + X^2 + X + 1$ • The polynomial for 16 bits CRC check is $X^{16} + X^{12} + X^5 + 1$.

Table 12 Data packet description

[16] Suggested use of addresses. In general more bits in the address gives less false detection, which in the end may give lower data Packet-Error-Rate (PER).

A. The address made by (5, 4, or 3) equal bytes are not recommended because it in general will make the packet-error-rate increase.
B. Addresses where the level shift only one time (i.e. 000FFFFFFF) could often be detected in noise that may give a false detection, which again may give raised packet-error-rate.
C. Addresses as a continuation of the preamble (hi-low toggling) will raise the Packet-Error-Rate (PER).

IMPORTANT TIMING DATA

The following timing applies for operation of nRF24L01.

nRF24L01 Timing Information

nRF24L01 timing	Max.	Min.	Name
Power Down ➔ Standby mode	1.5ms		Tpd2stby
Standby modes ➔ TX/RX mode	130µs		Tstby2a
Minimum CE high		10µs	Thce
Delay from CE pos. edge to CSN low		4µs	Tpece2csn

Table 13 Operational timing of nRF24L01

When the nRF24L01 is in power down it must always settle in Standby for 1.5ms before it can enter one of the TX or RX modes. Note that the configuration word will be lost if VDD is turned off and that the device then must be configured before going to one of the TX or RX mode.

Enhanced ShockBurst™ timing

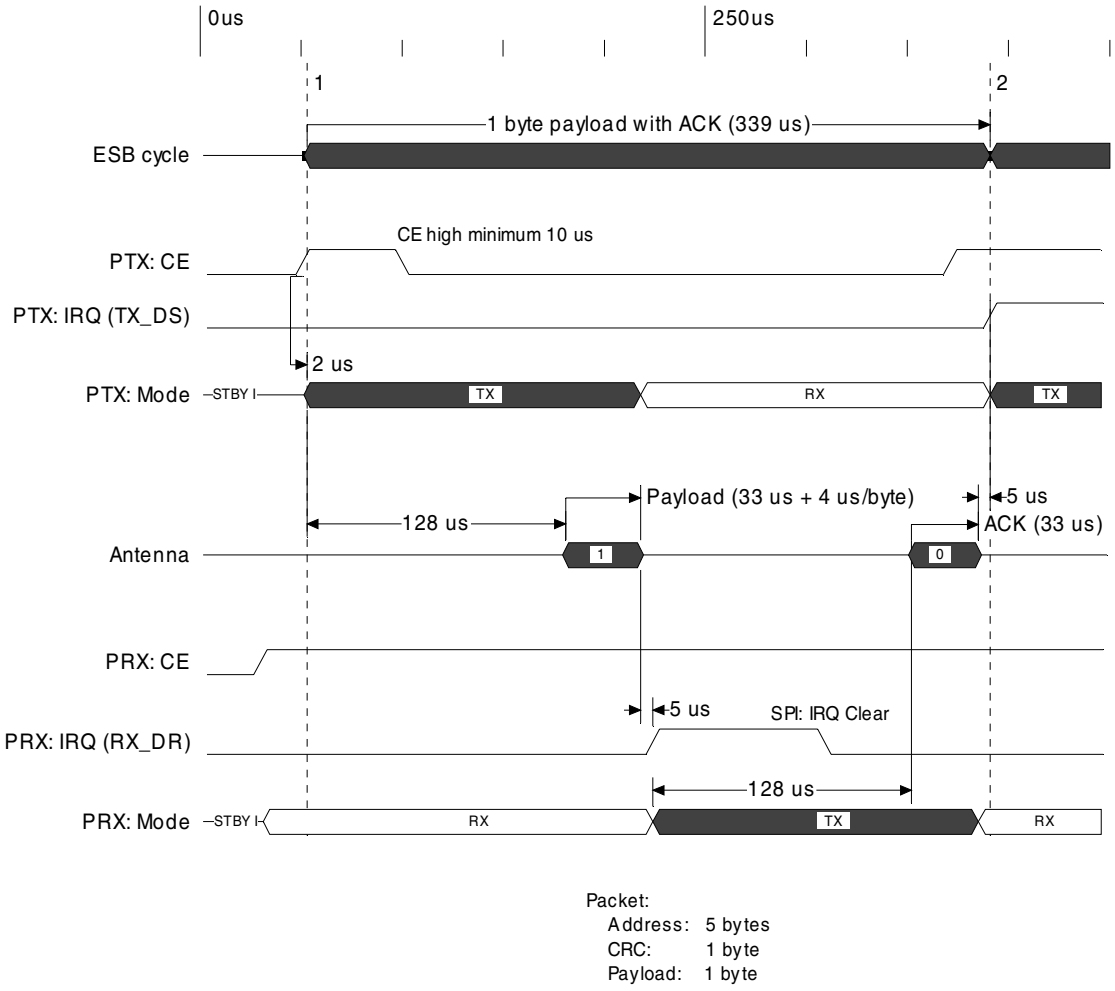


Figure 11 Timing of Enhanced ShockBurst™ for one packet upload (2Mbps).

In Figure 11 the sending of one packet and the acknowledgement of this packet is shown. The loading of payload to the PTX device is not shown in the figure. The PRX device is turned into RX mode (CE=1), and the PTX device is set into TX mode (CE=1 for minimum 10 µs). After 130 µs the transmission starts and is finished after another 37 µs (1 byte payload). The transmission ends, and the PTX device is automatically turned around to RX mode to wait for the acknowledgement from the PRX device. After the PTX device has received the acknowledgement it gives an interrupt to the MCU (IRQ (TX_DS) =>TX-data sent). After the PRX device has received the packet it gives an interrupt to the MCU (IRQ (RX_DR) =>RX-data ready).

PERIPHERAL RF INFORMATION

Antenna output

The ANT1 & ANT2 output pins provide a balanced RF output to the antenna. The pins must have a DC path to VDD, either via a RF choke or via the center point in a dipole antenna. A load of 15Ω+j88Ω (simulated values) is recommended for maximum output power (0dBm). Lower load impedance (for instance 50 Ω) can be obtained by fitting a simple matching network between the load and ANT1 and ANT2.

Output Power adjustment

SPI RF-SETUP (RF_PWR)	RF output power	DC current consumption
11	0 dBm	11.3 mA
10	-6 dBm	9.0 mA
01	-12 dBm	7.5 mA
00	-18 dBm	7.0 mA

Conditions: VDD = 3.0V, VSS = 0V, T_A = 27ºC, Load impedance = 15Ω+j88Ω.

Table 14 RF output power setting for the nRF24L01.

Crystal Specification

Frequency accuracy includes initial accuracy (tolerance) and stability over temperature and aging.

Frequency	C_L	ESR max	C_{0max}	Frequency accuracy
16MHz	8 – 16 pF	100 Ω	7.0pF	±60ppm

Table 15 Crystal specification of the nRF24L01

To achieve a crystal oscillator solution with low power consumption and fast start-up time, it is recommended to specify the crystal with a low value of crystal load capacitance. Specifying a lower value of crystal parallel equivalent capacitance, C_0 will also work, but this can increase the price of the crystal itself. Typically C_0=1.5pF at a crystal specified for C_{0max}=7.0pF.

The crystal load capacitance, C_L, is given by:

$C_L = \frac{C_1' \cdot C_2'}{C_1' + C_2'}$, where $C_1' = C_1 + C_{PCB1} + C_{I1}$ and $C_2' = C_2 + C_{PCB2} + C_{I2}$

C_1 and C_2 are SMD capacitors as shown in the application schematics. C_{PCB1} and C_{PCB2} are the layout parasitic on the circuit board. C_{I1} and C_{I2} are the capacitance seen into the XC1 and XC2 pin respectively; the value is typical 1pF.

nRF24L01 sharing crystal with a micro controller.

When using a micro controller to drive the crystal reference input XC1 of the nRF24L01 transceiver some rules must be followed.

Crystal parameters:

When the micro controller drives the nRF24L01 clock input, the requirement of load capacitance C_L is set by the micro controller only. The frequency accuracy of ±60 ppm is still required to get a functional radio link. The nRF24L01 will load the crystal by 0.5pF at XC1 in addition to the PBC routing.

Input crystal amplitude & Current consumption

The input signal should not have amplitudes exceeding any rail voltage, but any DC-voltage within this is OK. Exceeding rail voltage will excite the ESD structure and the radio performance is degraded below specification. If testing the nRF24L01 with a RF source with no DC offset as the reference source, the input signal will go below the ground level, which is not acceptable.

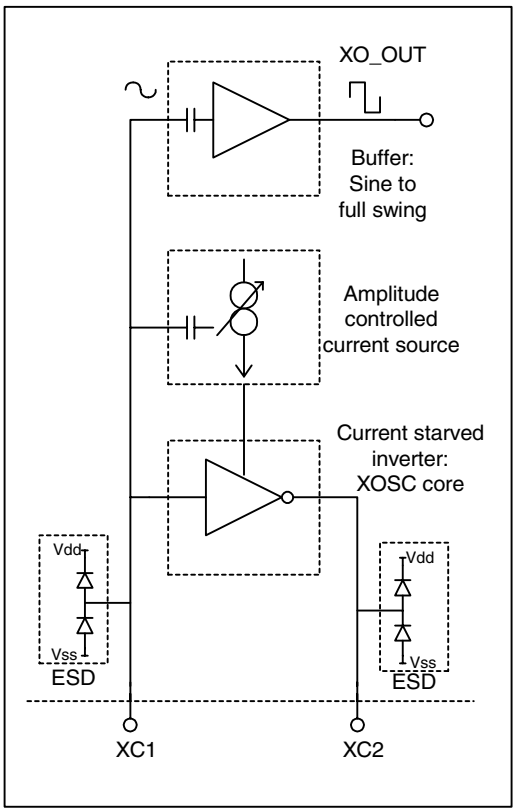


Figure 12 Principle of crystal oscillator

The nRF24L01 crystal oscillator is amplitude regulated. To achieve low current consumption and also good signal-to-noise ratio when using an external clock, it is recommended to use an input signal larger than 0.4 V-peak. When clocked externally, XC2 is not used and can be left as an open pin.

PCB layout and de-coupling guidelines

A well-designed PCB is necessary to achieve good RF performance. Keep in mind that a poor layout may lead to loss of performance, or even functionality, if due care is not taken. A fully qualified RF-layout for the nRF24L01 and its surrounding components, including matching networks, can be downloaded from **www.nordicsemi.no**.

A PCB with a minimum of two layers including a ground plane is recommended for optimum performance. The nRF24L01 DC supply voltage should be de-coupled as close as possible to the VDD pins with high performance RF capacitors, see Table 16. It is preferable to mount a large surface mount capacitor (e.g. 4.7µF tantalum) in parallel with the smaller value capacitors. The nRF24L01 supply voltage should be filtered and routed separately from the supply voltages of any digital circuitry.

Long power supply lines on the PCB should be avoided. All device grounds, VDD connections and VDD bypass capacitors must be connected as close as possible to the nRF24L01 IC. For a PCB with a topside RF ground plane, the VSS pins should be connected directly to the ground plane. For a PCB with a bottom ground plane, the best technique is to have via holes as close as possible to the VSS pads. At least one via hole should be used for each VSS pin.

Full swing digital data or control signals should not be routed close to the crystal or the power supply lines.

APPLICATION EXAMPLE

nRF24L01 with single ended matching network crystal, bias resistor, and decoupling capacitors.

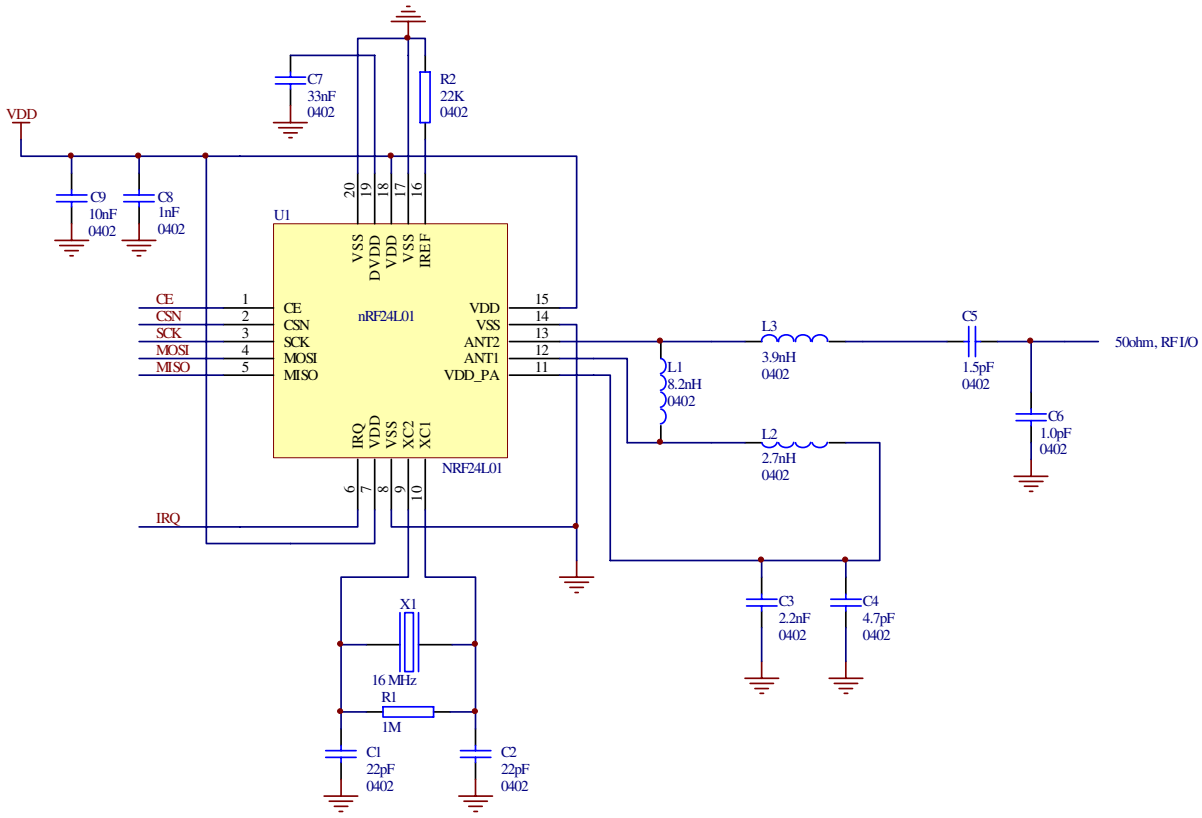


Figure 13 nRF24L01 schematic for RF layouts with single ended 50Ω RF output.

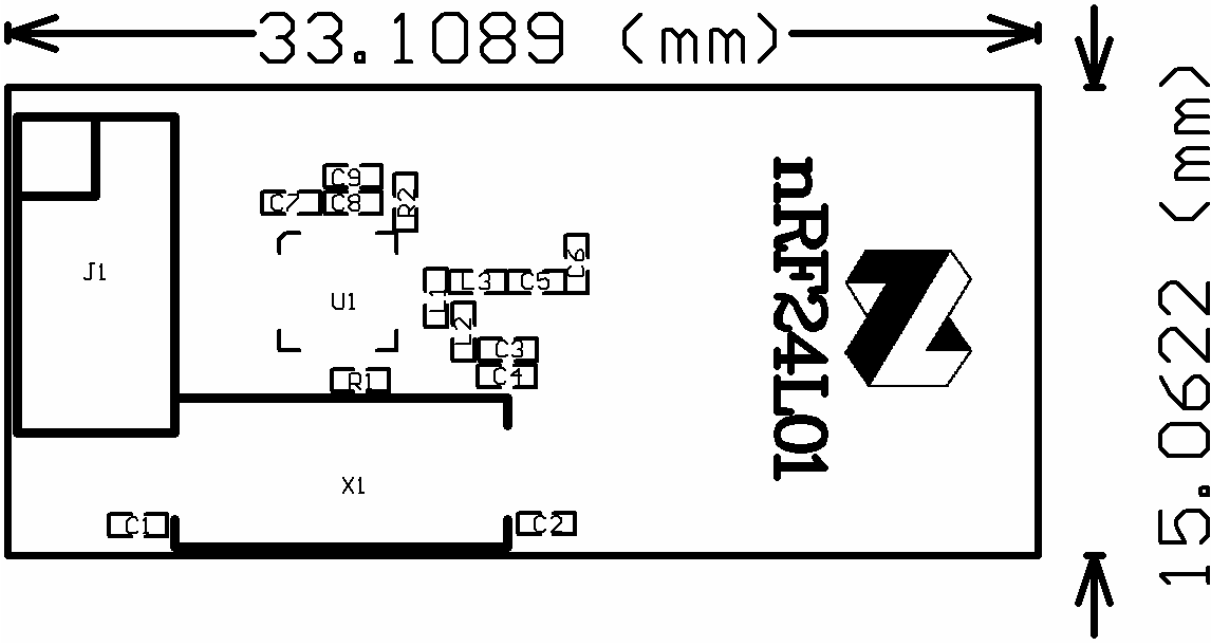
Part	Designator	Footprint	Description
22pF[17]	C1	0402	NPO, +/- 2%, 50V
22pF[17]	C2	0402	NPO, +/- 2%, 50V
2.2nF	C3	0402	X7R, +/- 10%, 50V
4.7pF	C4	0402	NPO, +/- 0.25 pF, 50V
1.5pF	C5	0402	NPO, +/- 0.1 pF, 50V
1,0pF	C6	0402	NPO, +/- 0.1 pF, 50V
33nF	C7	0402	X7R, +/- 10%, 50V
1nF	C8	0402	X7R, +/- 10%, 50V
10nF	C9	0402	X7R, +/- 10%, 50V
8,2nH	L1	0402	chip inductor +/- 5%
2.7nH	L2	0402	chip inductor +/- 5%
3,9nH	L3	0402	chip inductor +/- 5%
1M	R1	0402	+/-10%
22K	R2	0402	+/- 1 %
nRF24L01	U1	QFN20 4x4	
16MHz	X1		+/-60ppm, C_L=12pF[17]

Table 16 Recommended components (BOM) in nRF24L01 with antenna matching network

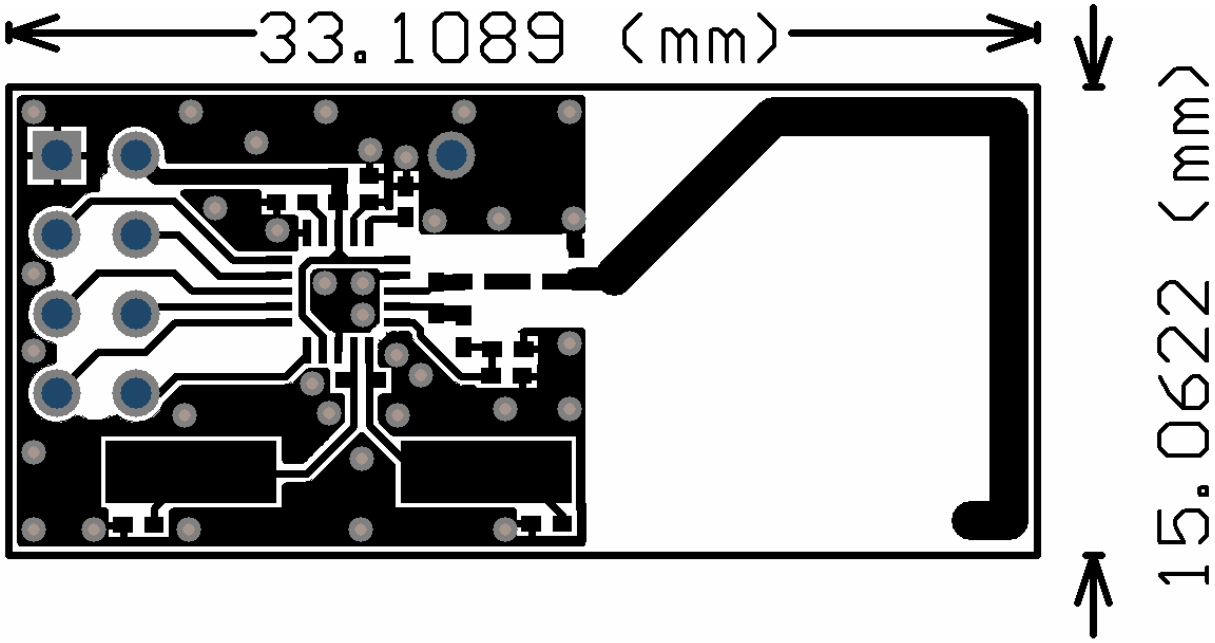
[17] C1 and C2 must have values that match the crystals load capacitance, C_L.

PCB layout examples

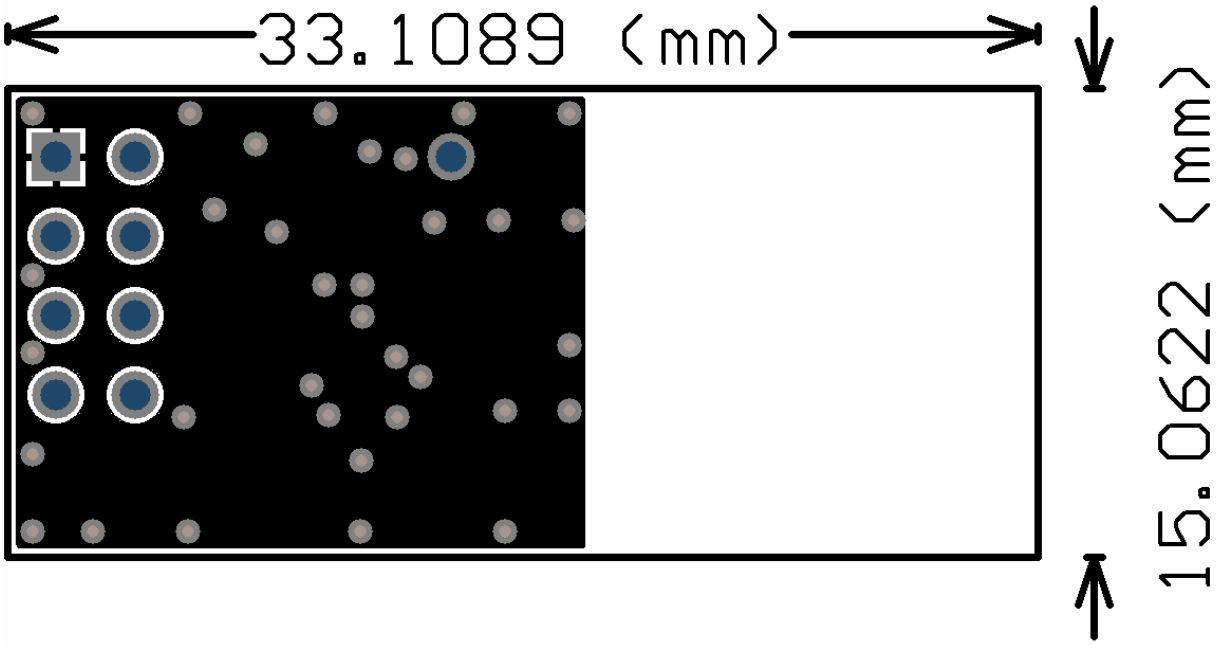
Figure 14 shows a PCB layout example for the application schematic in Figure 13. A double-sided FR-4 board of 1.6mm thickness is used. This PCB has a ground plane on the bottom layer. Additionally, there are ground areas on the component side of the board to ensure sufficient grounding of critical components. A large number of via holes connect the top layer ground areas to the bottom layer ground plane.



Top overlay



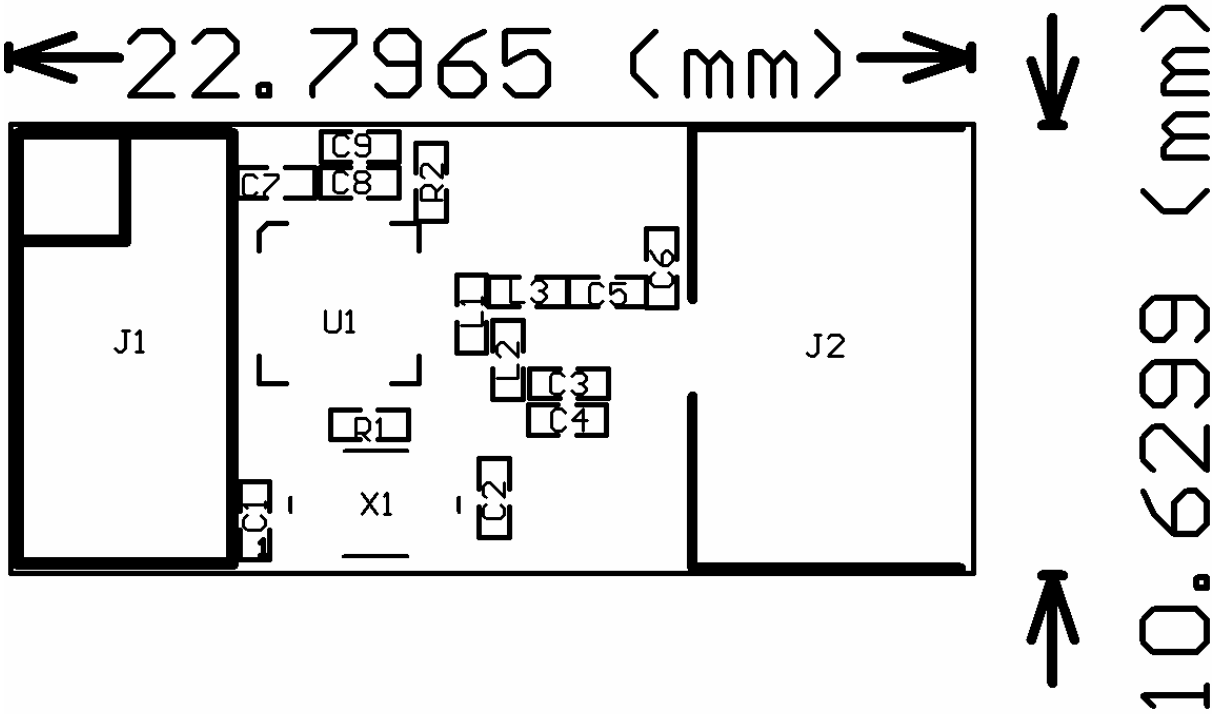
Top layer



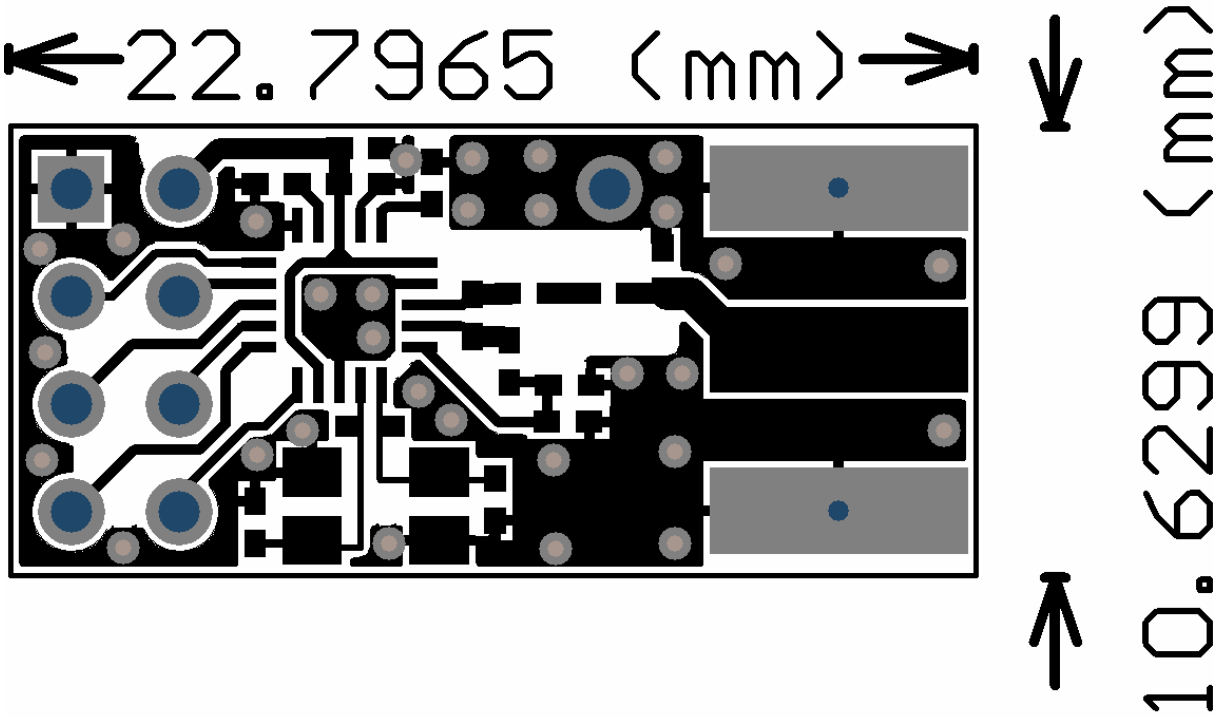
Bottom layer

Figure 14 nRF24L01 RF layout with single ended connection to PCB antenna and 0603 size passive components

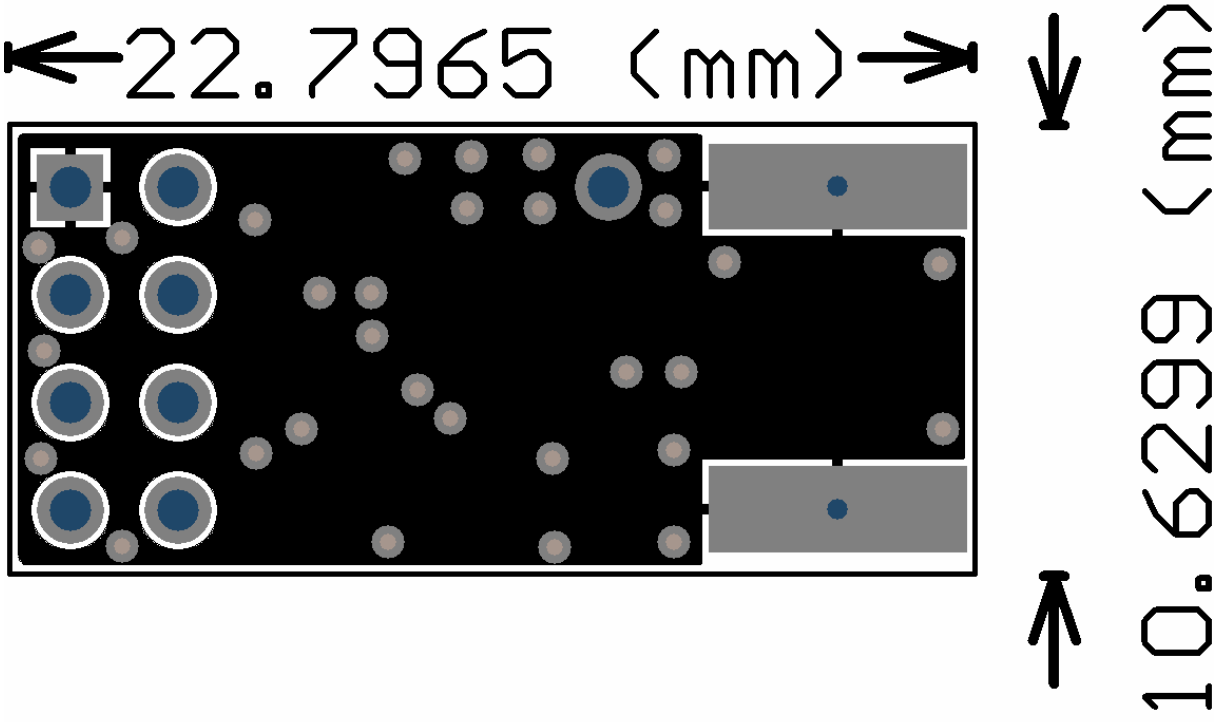
The nest figure (Figure 15) is for the SMA output to have a board for direct measurements at a 50Ω SMA connector.



Top Overlay



Top Layer



Bottom Layer

Figure 15 Module with OFM crystal and SMA connector

DEFINITIONS

Data sheet status	
Objective product specification	This data sheet contains target specifications for product development.
Preliminary product specification	This data sheet contains preliminary data; supplementary data may be published from Nordic Semiconductor ASA later.
Product specification	This data sheet contains final product specifications. Nordic Semiconductor ASA reserves the right to make changes at any time without notice in order to improve design and supply the best possible product.
Limiting values	
Stress above one or more of the limiting values may cause permanent damage to the device. These are stress ratings only and operation of the device at these or at any other conditions above those given in the Specifications sections of the specification is not implied. Exposure to limiting values for extended periods may affect device reliability.	
Application information	
Where application information is given, it is advisory and does not form part of the specification.	

Table 17. Definitions

Nordic Semiconductor ASA reserves the right to make changes without further notice to the product to improve reliability, function or design. Nordic Semiconductor ASA does not assume any liability arising out of the application or use of any product or circuits described herein.

LIFE SUPPORT APPLICATIONS

These products are not designed for use in life support appliances, devices, or systems where malfunction of these products can reasonably be expected to result in personal injury. Nordic Semiconductor ASA customers using or selling these products for use in such applications do so at their own risk and agree to fully indemnify Nordic Semiconductor ASA for any damages resulting from such improper use or sale.

Preliminary Product Specification: Revision Date: 08.03.2006.

Data sheet order code: 080306-nRF24L01

All rights reserved ®. Reproduction in whole or in part is prohibited without the prior written permission of the copyright holder.

YOUR NOTES

Nordic Semiconductor ASA – World Wide Distributors

For Your nearest dealer, please see http://www.nordicsemi.no

Main Office:
Vestre Rosten 81, N-7075 Tiller, Norway
Phone: +47 72 89 89 00, Fax: +47 72 89 89 89

Visit the Nordic Semiconductor ASA website at http://www.nordicsemi.no